SearCam: 스마트폰 몰래카메라 탐지 앱 개발기

기획부터 출시까지, AI 시대의 모바일 앱 개발 완전 가이드

true

2026

차 례

1 SearCam: 스마트폰 몰래카메라 탐지 앱 개발기 1

1.1 이 책에 대하여 1
1.2 대상 독자 2
1.3 사전 지식 3
1.4 이 책의 구성 3
1.4.1 Part 1: 기획과 설계 (Why & What) . . . 3
1.4.2 Part 2: 아키텍처와 설계 (How) 4
1.4.3 Part 3: 구현 (Build) 4
1.4.4 Part 4: 품질과 배포 (Ship) 4
1.4.5 Part 5: 성장 (Grow) 4
1.4.6 부록 4
1.5 실습 환경 5
1.6 표기 규칙 5
1.7 소스 코드 6
1.8 저자 6
1.9 피드백 6
1.10 감사의 말 7

2 Ch01: 문제 발견과 시장 분석 9

2.1 도입 9
2.2 1.1 문제의 심각성 파악 10
2.2.1 데이터부터 시작하라 10

2.2.2 스마트폰의 한계를 먼저 인정하라 11
2.3 1.2 경쟁 앱 해부하기 12
2.3.1 5개 앱을 직접 써보며 배운 것 12
2.3.2 공통 패턴: “위험!” 만 외치는 앱들 12
2.4 1.3 시장 기회 정의 13
2.4.1 TAM / SAM / SOM 분석 13
2.5 1.4 차별화 전략 14
2.6 실습 과제 14
2.7 핵심 정리 15
2.8 다음 장 예고 16
2.9 도입 19
2.10 2.1 PRD란 무엇인가 19
2.10.1 문서 하나가 팀 전체를 정렬한다 19
2.10.2 SearCam PRD의 핵심 구성 20
2.11 2.2 Executive Summary — 30초 안에 설명하기 21
2.11.1 좋은 Executive Summary의 조건 21
2.11.2 제품 포지셔닝: 솔직함이 차별점이다 . . 22
2.12 2.3 핵심 기능 우선순위 결정 23
2.12.1 기능 아이디어는 무한하다, 시간은 유한하다 23
2.12.2 30초 원칙 — 거스를 수 없는 제약 24
2.13 2.4 타겟 사용자와 페르소나 25
2.13.1 페르소나는 실제 사람처럼 구체적이어야 한다 25
2.13.2 페르소나에서 기능이 나온다 26
2.14 2.5 사용자 스토리 작성법 27
2.14.1 형식: As a / I want / So that 27
2.14.2 SearCam의 7개 Epic과 주요 스토리 . . 28
2.14.3 Acceptance Criteria가 핵심이다 30
2.15 2.6 탐지 정확도 목표 설정 31
2.15.1 목표 수치 설정의 함정 31
2.15.2 탐지 레이어별 성능 목표 31
2.16 2.7 비기능 요구사항 — 보이지 않는 품질 33
2.16.1 성능 목표 33

2.16.2 보안 요구사항 33
2.16.3 접근성 목표 33
2.17 2.8 제외 범위 — “만들지 않을 것” 목록 34
2.18 2.9 PRD 작성 체크리스트 35
2.19 마무리 35

3 Ch03: 기술 요구사항과 아키텍처 결정 — 도구를 고를 때 이유가 있어야 한다 37
3.1 도입 38
3.2 3.1 개발 언어: Kotlin을 선택한 이유 38
3.2.1 Java 대신 Kotlin — 2024년 기준 논쟁의 여지가 없다 38
3.3 3.2 UI 프레임워크: Jetpack Compose를 선택한 이유 41
3.3.1 XML vs Compose — 레거시냐 미래냐 . 41
3.4 3.3 의존성 주입: Hilt를 선택한 이유 44
3.4.1 DI가 필요한 이유를 먼저 이해한다 . . . 44
3.5 3.4 로컬 DB: Room을 선택한 이유 47
3.5.1 SQLite를 직접 쓰지 않는 이유 47
3.6 3.5 카메라: CameraX를 선택한 이유 48
3.6.1 Camera2 API vs CameraX 48
3.7 3.6 Android 버전 전략: API 26+를 선택한 이유 . 51
3.7.1 최소 버전 결정은 트레이드오프다 51
3.8 3.7 아키텍처: Clean Architecture를 선택한 이유 53
3.8.1 아키텍처는 “어떻게” 가 아니라 “왜” 다 . 53
3.9 3.8 비동기 처리: Coroutines + Flow를 선택한 이유 55
3.9.1 스마트폰의 메인 스레드는 UI 전용이다 . 55
3.10 3.9 기술 스택 최종 결정표 57
3.11 마무리 58

4 Ch04: 시스템 아키텍처 설계 — 층을 나누고 방향을 정한다 61
4.1 도입 62
4.2 4.1 전체 아키텍처 개요 62

4.2.1 3계층: Presentation → Domain → Data 62
4.3 4.2 Presentation 계층 — UI와 상태 관리 64
4.3.1 단방향 데이터 흐름 64
4.3.2 Presentation 계층 구성요소 68
4.4 4.3 Domain 계층 — 비즈니스 로직의 집 69
4.4.1 Domain은 Android를 모른다 69
4.5 4.4 탐지 3중 레이어 — 핵심 알고리즘 설계 . . . 73
4.5.1 단일 센서의 한계: 왜 3중 레이어인가 . . 73
4.5.2 가중치 설계 이유 75
4.5.3 교차 검증 알고리즘 78
4.6 4.5 의존성 역전 원칙 — Domain이 Data를 모르는 이유 80
4.6.1 인터페이스가 방향을 역전시킨다 80
4.7 4.6 Hilt DI 설계 — 의존성의 지도 83
4.7.1 DI 모듈 구조 83
4.8 4.7 패키지 구조 — 파일의 주소 86
4.8.1 기능별 vs 레이어별 패키징 86
4.9 4.8 모듈 의존성 그래프 92
4.9.1 어떤 모듈이 누구를 아는가 92
4.10 4.9 데이터 흐름: 스캔 시작부터 결과까지 93
4.11 4.10 아키텍처 체크리스트 96
4.12 마무리 97

5 Ch05: UI/UX 설계 — Jetpack Compose로 불안을 안심으로 99
5.1 도입 100
5.2 5.1 왜 Jetpack Compose인가 100
5.2.1 UI를 “어떻게” 그릴지가 아닌 “무엇을” 그릴지로 100
5.2.2 선언형 UI가 불안을 다루는 앱에 적합한 이유 101
5.3 5.2 4대 디자인 원칙과 그 이유 102
5.3.1 원칙 1: 30초 스캔 103
5.3.2 원칙 2: 공포 방지 103

5.3.3 원칙 3: 거짓 안심 방지 104
5.4 5.3 화면 구조와 네비게이션 104
5.4.1 Bottom Navigation 설계 105
5.5 5.4 색상 코드 시스템 — 위험도를 색으로 말하다 . 106
5.5.1 5단계 위험도 색상 스펙트럼 106
5.5.2 브랜드 색상과의 분리 107
5.5.3 게이지 그라데이션 구현 108
5.6 5.5 홈 화면 — 원클릭의 실현 110
5.6.1 Quick Scan 버튼의 물리적 크기 111
5.6.2 레이더 펄스 애니메이션 (Idle 상태) . . . 112
5.7 5.6 스캔 30초 UX 흐름 114
5.7.1 스캔 진행 화면 구조 114
5.7.2 UX 설계 원칙: 정보의 밀도 115
5.7.3 시간 압박을 줄이는 심리적 설계 116
5.8 5.7 결과 화면 — 불안에서 안심으로의 전환 . . . 117
5.8.1 점진적 정보 공개 118
5.9 5.8 다크 테마 설계 119
5.9.1 다크 테마 색상 스펙트럼 119
5.9.2 Material 3 Dynamic Color 비채택 이유 . 120
5.10 5.9 타이포그래피 — 숫자가 주인공 121
5.10.1 텍스트 스케일 121
5.11 5.10 접근성 고려사항 122
5.12 5.11 온보딩 — 신뢰를 먼저 쌓다 123
5.13 5.12 체크리스트 화면 — 앱 없이도 쓸 수 있는 가이드 124
5.14 정리: UX가 기술보다 먼저다 125
5.15 체크리스트 125

6 Ch06: 보안 설계 — 탐지 앱이 탐지 당하면 안 된다 127
6.1 도입 128
6.2 6.1 3대 핵심 원칙 128
6.3 6.2 보안 계층 구조 (Defense in Depth) 129
6.4 6.3 위협 모델링 — 적을 먼저 알아야 한다 131
6.4.1 SearCam STRIDE 위협 매트릭스 131

6.5 6.4 최소 권한 원칙 — 필요한 것만 요청한다 . . . 133
6.5.1 요청하는 권한 (7개만) 133
6.5.2 의도적으로 요청하지 않는 권한 134
6.6 6.5 네트워크 스캔 범위 제한 — 울타리를 쳐라 . . 134
6.6.1 제한 1: RFC 1918 사설 주소만 대상 . . . 135
6.6.2 제한 2: 카메라 관련 포트만 (6개) 136
6.6.3 제한 3: 동시 연결 5개 제한 136
6.6.4 제한 4: 포트당 2초 타임아웃 137
6.7 6.6 카메라 프레임 메모리 전용 처리 137
6.7.1 프레임 생명주기 138
6.7.2 검증: 카메라 프레임이 정말 저장 안 되나? 140
6.8 6.7 SQLCipher + Android Keystore 암호화 . . . 140
6.8.1 암호화 스펙 140
6.8.2 Android Keystore를 쓰는 이유 141
6.8.3 MAC 주소 SHA-256 해시 처리 144
6.9 6.8 네트워크 통신 보안 — 단 하나의 예외 144
6.9.1 Certificate Pinning (인증서 고정) 145
6.10 6.9 코드 보안 — 릴리스 빌드는 다르다 146
6.10.1 ProGuard/R8 난독화 146
6.10.2 디버그 vs 릴리스 분리 147
6.10.3 경로 순회(Path Traversal) 방지 148
6.11 6.10 루팅 기기 대응 149
6.12 6.11 릴리스 전 보안 체크리스트 150
6.12.1 데이터 보호 150
6.12.2 권한 151
6.12.3 네트워크 151
6.12.4 코드 151
6.12.5 검증 도구 151
6.13 6.12 보안과 기능성의 균형 152
6.14 정리: 탐지 앱이 탐지당하지 않는 방법 153
6.15 보안 원칙 요약 153

7 Ch07: 테스트 전략 — 버그보다 테스트가 먼저 155

7.1 도입 156
7.2 7.1 테스트 철학 — “동작한다” 가 아닌 “정확하게 동작한다” 156
7.2.1 오탐과 미탐 중 어느 것이 더 나쁜가? . . 156
7.2.2 테스트 피라미드 157
7.3 7.2 TDD 사이클 — RED-GREEN-IMPROVE . . . 158
7.3.1 자동차 안전벨트처럼 158
7.3.2 Step 1: RED — 실패하는 테스트 먼저 . . 159
7.3.3 Step 2: GREEN — 최소 구현 162
7.3.4 Step 3: IMPROVE — 리팩토링 164
7.4 7.3 MockK — Kotlin 스러운 모킹 165
7.4.1 MockK로 의존성 격리 165
7.5 7.4 Turbine — Flow 테스트의 해결사 169
7.6 7.5 Coroutine 테스트 — 시간을 제어하다 172
7.7 7.6 에러 코드 체계 테스트 174
7.7.1 에러 코드별 테스트 175
7.8 7.7 탐지 정확도 검증 — 알려진 카메라로 테스트 . 178
7.8.1 테스트 대상 카메라 (실기기) 178
7.8.2 OUI 데이터베이스 정확도 테스트 179
7.8.3 CrossValidator 교차 검증 테스트 181
7.9 7.8 Room DB 통합 테스트 184
7.10 7.9 Compose UI 테스트 188
7.11 7.10 커버리지 목표와 측정 192
7.11.1 레이어별 커버리지 목표 192
7.11.2 커버리지 측정 명령 193
7.11.3 커버리지가 목표 미달 시 193
7.12 7.11 TDD 실전 패턴과 함정 194
7.12.1 함정 1: “이건 단순하니까 테스트 안 해도 돼” 194
7.12.2 함정 2: 테스트가 구현 세부사항에 의존 . 195
7.12.3 함정 3: 비결정적 테스트 (Flaky Tests) . 196
7.13 7.12 E2E 테스트 — 실제 사용자처럼 테스트하기 . 197
7.13.1 Quick Scan 전체 플로우 197

7.14 정리: 테스트가 없는 탐지 앱은 장난감 199
7.15 테스트 체크리스트 199

8 Ch08: 데이터베이스 설계 — Room + SQLCipher로 스캔 이력 보호 201
8.1 도입 202
8.2 8.1 데이터베이스 전체 구조 202
8.2.1 테이블 목록 204
8.3 8.2 엔티티 설계 204
8.3.1 ScanReportEntity — 스캔 세션의 마스터 레코드 204
8.3.2 DeviceEntity — 발견된 네트워크 기기 . 207
8.3.3 RiskPointEntity — 위험 포인트의 통합 저장 209
8.3.4 ChecklistEntity — 육안 점검 기록 . . . 213
8.4 8.3 TypeConverter — Room이 모르는 타입을 가르치기 215
8.4.1 TypeConverter 등록 219
8.5 8.4 SQLCipher + Android Keystore — 데이터 암호화 220
8.5.1 암호화 적용 전후 비교 224
8.6 8.5 DAO 설계 — 필요한 것만, 정확하게 225
8.6.1 ReportDao 225
8.7 8.6 마이그레이션 전략 — 데이터는 사용자의 것 . 228
8.7.1 원칙 228
8.7.2 버전 계획 228
8.7.3 마이그레이션 예시 229
8.8 8.7 데이터 보존 정책 230
8.9 핵심 정리 232
8.10 다음 장 예고 233
8.11 도입 238
8.12 9.1 전체 데이터 흐름 개요 238
8.13 9.2 Flow vs LiveData — SearCam의 선택 . . . 240

8.14 9.3 StateFlow + SharedFlow — 두 종류의 Hot Flow 242
8.14.1 StateFlow — UI 상태 관리 242
8.14.2 SharedFlow — 일회성 이벤트 처리 . . . 243
8.14.3 왜 구분하는가 244
8.15 9.4 ViewModel → UseCase → Repository 체인 . 245
8.15.1 1단계: ViewModel이 UseCase를 실행 . 245
8.15.2 2단계: UseCase가 비즈니스 로직을 처리 246
8.15.3 3단계: Repository가 데이터 소스를 추상화 248
8.16 9.5 병렬 레이어 실행 — coroutineScope + async 250
8.16.1 SupervisorJob으로 격리된 실패 251
8.16.2 withTimeout으로 레이어별 제한 시간 . . 253
8.17 9.6 Flow 연산자 실전 패턴 254
8.17.1 map — 타입 변환 254
8.17.2 combine — 여러 Flow 합류 255
8.17.3 conflate — 밀린 이벤트 건너뛰기 256
8.17.4 sample — 주기적 샘플링 256
8.17.5 catch — 스트림 에러 처리 257
8.17.6 flowOn — 디스패처 전환 258
8.18 9.7 스캔 취소 — Job.cancel() 258
8.19 9.8 Compose에서 Flow 구독 261
8.20 9.9 전체 데이터 흐름 요약 다이어그램 263
8.21 핵심 정리 265
8.22 다음 장 예고 266
8.23 도입 271
8.24 10.1 Gradle Version Catalog 도입 배경 271
8.24.1 라이브러리 지옥에서 탈출하기 271
8.25 10.2 실제 build.gradle.kts 전체 구성 274
8.26 10.3 Hilt DI 설계 — 왜 Hilt를 선택했는가 277
8.26.1 의존성 주입이 뭐길래 277
8.26.2 Koin 대신 Hilt를 선택한 이유 278
8.26.3 SearCam의 DI 모듈 구조 278
8.27 10.4 Clean Architecture 패키지 구조 결정 과정 . 282

8.27.1 처음 실수: 타입별 분류 282
8.27.2 수정 방향: 레이어별 + 기능별 혼합 . . . 283
8.27.3 도메인 모델 예시 — 불변성 준수 285
8.28 10.5 AndroidManifest 권한 전략 — 최소 권한 원칙 286
8.28.1 왜 권한을 최소화해야 하는가 286
8.28.2 런타임 권한 요청 패턴 288
8.29 실습 289
8.30 핵심 정리 290
8.31 다음 장 예고 291
8.32 도입 294
8.33 11.1 Wi-Fi 스캔이 탐지의 핵심인 이유 294
8.33.1 가중치 50%의 근거 294
8.33.2 탐지 가능 범위와 한계 295
8.34 11.2 ARP 테이블 파싱 원리 296
8.34.1 ARP가 무엇인가 296
8.34.2 ARP 파싱 구현 296
8.35 11.3 mDNS(NsdManager)와 SSDP 멀티캐스트 동작 299
8.35.1 mDNS로 스마트 기기 탐색하기 299
8.35.2 SSDP 멀티캐스트로 UPnP 기기 탐색 . . 302
8.36 11.4 OUI 데이터베이스로 제조사 식별하기 304
8.36.1 OUI란 무엇인가 304
8.37 11.5 포트 스캐닝 — 카메라 서비스 찾기 307
8.37.1 카메라가 여는 포트 307
8.38 11.6 Android 12+ 스캔 쓰로틀링 해결 전략 . . . 310
8.38.1 쓰로틀링 문제 310
8.39 11.7 WifiScanRepository 구현 — 전체 흐름 . . . 312
8.40 실습 315
8.41 핵심 정리 315
8.42 11.8 코드 리뷰 개선 사항 — 실전 배포 전 발견한 버그들 316
8.42.1 개선 1: `cachedDevices`에 `@Volatile` 추가 — 스레드 가시성 보장 317

8.42.2 개선 2: SSDP 소켓 리소스 누수 수정 — try-finally 보장 317
8.42.3 개선 3: 호스트명 입력 Sanitization — 제어 문자 차단 319
8.42.4 개선 4: PortScanner에 사설 IP 범위 가드 — 인터넷 스캔 방지 320
8.43 다음 장 예고 321
8.44 도입 325
8.45 12.1 빛의 역반사(Retroreflection) 원리 325
8.45.1 왜 렌즈만 특별하게 반사하는가 325
8.45.2 역반사의 특성 326
8.46 12.2 CameraX ImageAnalysis 프레임 처리 . . . 327
8.46.1 CameraX 설정 327
8.47 12.3 5단계 분석 파이프라인 330
8.47.1 파이프라인 개요 330
8.48 12.4 플래시 OFF 동적 검증 — 오탐 방지 핵심 . . 335
8.49 12.5 IR 감지 — 전면 카메라의 적외선 필터 활용 . 338
8.49.1 스마트폰 카메라와 IR 필터 338
8.50 12.6 실전 오탐 사례와 보정 방법 342
8.50.1 오탐 사례 1: 야간 조명의 광원 342
8.50.2 오탐 사례 2: 금속 장식품과 액자 유리 . . 343
8.50.3 오탐 사례 3: 안경 렌즈 343
8.50.4 오탐 보정 결과 344
8.51 실습 344
8.52 핵심 정리 345
8.53 12.7 코드 리뷰 개선 사항 — 멀티스레드 안전과 불변성 346
8.53.1 개선 1: stabilityTracker를 ConcurrentHashMap으로 교체 346
8.53.2 개선 2: Cluster.contrastRatio를 val로 — 생성 후 불변 보장 347
8.53.3 개선 3: calculateRiskScore()를 listOfNotNull + sumOf 패턴으로 . 348

8.54 다음 장 예고 350
8.55 도입 353
8.56 13.1 전자기장(EMF) 탐지의 원리와 한계 353
8.56.1 어떻게 탐지하는가 353
8.56.2 현실적인 한계 354
8.57 13.2 SensorManager TYPE_MAGNETIC_FIELD 20Hz 수집 355
8.57.1 Android 자력계 API 355
8.58 13.3 이동 평균(Moving Average) 노이즈 필터 구현 359
8.58.1 왜 노이즈 필터가 필요한가 359
8.58.2 이동 평균의 트레이드오프 362
8.59 13.4 캘리브레이션 — 배경 기준선 설정 362
8.59.1 왜 캘리브레이션이 필요한가 362
8.59.2 캘리브레이션 결과 활용 365
8.60 13.5 EMF 단독으로 탐지 불가한 이유 — 보조 레이어 역할 367
8.60.1 일상적인 EMF 발생원 367
8.60.2 올바른 사용 방법 — 교차 검증 보강재 . . 368
8.60.3 UI에서의 투명한 표시 369
8.61 13.6 전체 EMF 레이어 통합 370
8.62 실습 371
8.63 핵심 정리 372
8.64 13.7 코드 리뷰 개선 사항 — 명확한 상수와 스레드 안전 373
8.64.1 개선 1: `SENSOR_DELAY_FASTEST` → 명시적 20Hz 상수 373
8.64.2 개선 2: `NoiseFilter` 스파이크 상수를 `companion object`으로 이동 374
8.65 다음 장 예고 375
8.66 도입 379
8.67 14.1 세 레이어를 어떻게 합치는가 — CrossValidator 설계 379
8.67.1 기본 가중치 모델 379

8.67.2 CrossValidator 핵심 설계 380
8.68 14.2 가중치 동적 조정 — Wi-Fi 없을 때 385
8.68.1 레이어가 비활성화되는 상황 385
8.68.2 동적 조정 예시 386
8.69 14.3 보정 계수 설계 근거 387
8.69.1 왜 ×0.7, ×1.2, ×1.5인가 387
8.69.2 상한선 100점의 의미 388
8.70 14.4 위험도 0~100 스코어 산출 알고리즘 — 전체 흐름 389
8.71 14.5 실제 탐지 시나리오 3가지 391
8.71.1 시나리오 1: 숙박시설 (모텔/에어비앤비) . 391
8.71.2 시나리오 2: 공중화장실 393
8.71.3 시나리오 3: 탈의실 394
8.72 14.6 교차 검증 결과 ViewModel 연동 396
8.73 실습 399
8.74 핵심 정리 399
8.75 14.7 코드 리뷰 개선 사항 — 단일 진실 출처와 중복 제거 400
8.75.1 개선 1: 가중치 상수를 `LayerType` 열거형으로 통합 — 단일 진실 출처 400
8.75.2 개선 2: `CalculateRiskUseCase.invoke()`가 `invokeWithCorrection()`에 위임 — 중복 로직 제거 402
8.75.3 개선 3: `RunFullScanUseCase`에 `AllLayerResults` 네임드 data class 도입 403
8.76 다음 장 예고 405
8.77 도입 409
8.78 15.1 선언형 UI의 핵심 개념 409
8.78.1 상태가 UI를 만든다 409
8.78.2 Recomposition과 성능 411

8.79 15.2 ScanViewModel — StateFlow와 sealed class UiState 412
8.79.1 sealed class로 UI 상태 모델링 412
8.79.2 StateFlow와 SharedFlow의 역할 분리 . 414
8.79.3 스캔 취소와 Job 관리 415
8.80 15.3 HomeScreen — 레이더 펄스 애니메이션 구현 417
8.80.1 InfiniteTransition으로 무한 반복 애니메이션 417
8.80.2 HomeScreen 전체 구조 — Scaffold와 상태 분리 420
8.81 15.4 RiskGauge — Canvas 애니메이션으로 그린 위험도 게이지 423
8.81.1 Canvas는 마우스 없는 그림판 423
8.81.2 각도 계산 — 왜 135도에서 시작하는가 . . 427
8.81.3 Animatable vs animateFloatAsState . . 427
8.82 15.5 Navigation Graph 구성 428
8.82.1 NavHost로 화면 연결하기 428
8.83 15.6 hiltViewModel()로 DI 연결 432
8.83.1 Compose와 Hilt의 만남 432
8.83.2 ViewModel 범위 선택 433
8.84 15.7 DisposableEffect로 카메라 생명주기 관리 . 434
8.84.1 카메라는 열면 반드시 닫아야 한다 434
8.84.2 CameraX와 Compose의 통합 패턴 . . . 436
8.85 15.8 스캔 결과 화면 — 상태별 UI 분기 438
8.85.1 when 표현식으로 완전한 분기 438
8.86 15.9 Composable 함수 설계 원칙 440
8.86.1 상태 끌어올리기 (State Hoisting) 440
8.86.2 컴포저블 분리 기준 442
8.87 실습 443
8.88 핵심 정리 443
8.89 15.9 코드 리뷰 개선 사항 — `LifecycleOwner` 전파 아키텍처 445

8.89.1 문제: 카메라 바인딩에 `Lifecycle-Owner`가 필요한데 어디서 제공해야 하나 445
8.89.2 해결: `LifecycleOwner`를 Compose Screen에서 UseCase까지 파라미터로 전달 446
8.89.3 왜 `LifecycleOwner`를 Repository 인터페이스에 두는가 449
8.90 다음 장 예고 450
8.91 도입 454
8.92 16.1 Room 아키텍처 — 세 계층의 역할 454
8.92.1 추상화의 사다리 454
8.93 16.2 Entity 설계 — 테이블 구조 정의 455
8.93.1 ScanReportEntity — 복잡한 객체를 테이블에 담기 455
8.93.2 인덱스 추가 — 조회 성능 최적화 458
8.94 16.3 TypeConverters — Gson 없이 직접 변환 . . 459
8.94.1 Room이 저장할 수 없는 타입들 459
8.94.2 List ↔ JSON 변환 460
8.94.3 문자열 배열 파서 — 이스케이프 처리 . . 462
8.94.4 enum TypeConverter — 안전한 역직렬화 463
8.94.5 AppDatabase에 TypeConverter 등록 . 464
8.95 16.4 DAO — SQL을 코드로 466
8.95.1 ReportDao — Flow로 실시간 업데이트 . 466
8.96 16.5 Repository 구현체 — Domain ↔ Data 변환 469
8.96.1 Mapper 패턴 — 계층 간 변환 책임 분리 . 469
8.96.2 Mapper 클래스 구현 472
8.97 16.6 SQLCipher 통합 — 데이터베이스 암호화 . . 474
8.97.1 왜 암호화가 필요한가 474
8.97.2 Android Keystore + SQLCipher 통합 . . 475
8.97.3 SQLCipher 의존성 추가 478
8.98 16.7 Migration 전략 — 데이터를 잃지 않고 스키마 바꾸기 479

8.98.1 fallbackToDestructiveMigration은 개발 중에만 479
8.98.2 Migration 작성 방법 480
8.98.3 MigrationTestHelper로 Migration 검증 481
8.99 16.8 DatabaseModule과 Hilt 등록 483
8.99.1 싱글톤으로 DB 인스턴스 관리 483
8.100실습 485
8.101핵심 정리 485
8.102다음 장 예고 487
8.103도입 491
8.104 17.1 Android PdfDocument API 개요 491
8.104.1 외부 라이브러리 없이 PDF 만들기 491
8.105 17.2 PdfGenerator 설계 — 3페이지 구조 492
8.105.1 페이지별 책임 분리 492
8.105.2 generate() — 메인 진입점과 Path Traversal 방어 494
8.106 17.3 Path Traversal 방어 — canonicalPath 검증 497
8.106.1 경로 조작 공격이란 497
8.107 17.4 페이지 1 — 종합 위험도 시각화 498
8.107.1 drawOverviewPage — 첫인상이 전부다 498
8.107.2 위험도 점수 대형 표시와 색상 코드 . . . 500
8.108 17.5 페이지 2 — 기기 목록 테이블 501
8.108.1 테이블 헤더와 행 렌더링 501
8.108.2 페이지 오버플로우 방지 503
8.109 17.6 페이지 3 — 발견 사항과 권고 사항 504
8.109.1 심각도 순 정렬과 구조화된 출력 504
8.109.2 위험 등급별 권고 사항 텍스트 506
8.110 17.7 드로잉 유틸리티 — buildPaint와 parseColor 507
8.110.1 Paint 팩토리 함수 507
8.110.2 안전한 색상 파싱 508
8.111 17.8 ExportReportUseCase — 도메인 레이어의 흐름 조율 509
8.111.1 UseCase는 오케스트라 지휘자 509

8.111.2 ViewModel에서 UseCase 호출 511
8.112 17.9 FileProvider로 PDF 안전하게 공유 513
8.112.1 다른 앱이 파일을 읽게 허용하기 513
8.112.2 Compose에서 공유 버튼 연결 516
8.113 17.10 전체 흐름 요약 517
8.114 실습 519
8.115 핵심 정리 520
8.116 다음 장 예고 521
8.117 도입 525
8.118 18.1 테스트 피라미드: 무엇을 얼마나 테스트할까 . 525
8.118.1 비율이 곧 전략이다 525
8.119 18.2 테스트 의존성 설정 527
8.119.1 도구함 준비하기 527
8.120 18.3 MockK로 의존성 격리하기 529
8.120.1 가짜 협력자를 만드는 법 529
8.120.2 Repository 모킹 529
8.121 18.4 Turbine으로 Flow 테스트하기 533
8.121.1 파이프를 검사하는 방법 533
8.122 18.5 Coroutine 테스트: runTest와 TestDispatcher 536
8.122.1 시간을 제어하는 법 536
8.123 18.6 PortScanner 단위 테스트 539
8.123.1 isPrivateIp() 경계값 테스트 539
8.124 18.7 NoiseFilter 이동 평균 테스트 542
8.124.1 노이즈 속에서 신호 찾기 542
8.125 18.8 RiskCalculator 점수 검증 테스트 546
8.125.1 위험도 공식을 믿을 수 있는가 546
8.126 18.9 ViewModel 테스트: StateFlow와 UI State . 551
8.126.1 화면과 비즈니스 로직 사이의 계약 551
8.127 18.10 테스트 커버리지 측정 554
8.127.1 커버리지는 목표가 아니라 척도다 554
8.128 정리 556
8.129 18.9 CrossValidatorImpl 테스트 — 가중치 재분배 검증 557

8.130 18.10 CalculateRiskUseCase 테스트 — 보정 계수 검증 560
8.131 18.11 RunQuickScanUseCase 테스트 — Turbine + MockK Flow 검증 563
8.132 18.12 단위 테스트 56개 전체 목록 566
8.133 도입 572
8.134 19.1 전체 파이프라인 설계 572
8.134.1 두 개의 파이프라인 572
8.135 19.2 PR 파이프라인: 빠른 피드백 573
8.135.1 코드를 제출하기 전 검문소 573
8.136 19.3 Lint 설정: 코드 스타일 강제하기 578
8.136.1 두 명의 코드 심사관 578
8.137 19.4 릴리즈 파이프라인: 태그 하나로 배포까지 . 580
8.137.1 키 하나로 열리는 자동화 580
8.138 19.5 APK 서명 자동화: Keystore 시크릿 관리 . . 585
8.138.1 열쇠를 안전하게 보관하는 법 585
8.139 19.6 ProGuard/R8 난독화 588
8.139.1 코드를 읽을 수 없게 만드는 이유 588
8.140 19.7 자동 버전 관리: Git Tag 기반 591
8.140.1 버전 번호를 수동으로 바꾸지 마라 591
8.141 19.8 품질 게이트: 통과하지 않으면 머지 불가 . . 593
8.141.1 자동 검문소 593
8.142 19.9 Nightly Build: 야간 전체 검증 594
8.142.1 자는 동안 일하는 파이프라인 594
8.143 19.10 Firebase 환경 분리 596
8.143.1 개발/스테이징/프로덕션을 완전히 격리하기 596
8.144 정리 598

9 Ch20: 릴리즈와 배포 전략 599
9.1 도입 599
9.2 20.1 Google Play Console 초기 설정 600
9.2.1 첫 등록 전에 준비할 것 600
9.3 20.2 보안 앱 심사 대비 601

9.3.1 심사관의 눈으로 보기 601
9.4 20.3 단계적 출시 (Staged Rollout) 604
9.4.1 전쟁 전 정찰하기 604
9.5 20.4 버전 관리: versionCode와 versionName . 606
9.5.1 버전이 꼬이면 배포가 막힌다 606
9.6 20.5 릴리즈 노트 작성 609
9.6.1 사용자와의 대화 609
9.7 20.6 앱 스토어 최적화 (ASO) 611
9.7.1 검색에서 발견되는 앱 만들기 611
9.8 20.7 앱 서명 관리: Play App Signing 613
9.8.1 열쇠를 Google에게도 맡기는 이유 . . . 613
9.9 20.8 긴급 대응: 핫픽스 배포 614
9.9.1 불이 났을 때 빨리 끄는 법 614
9.10 정리 615

10 Ch21: 모니터링과 성능 최적화 617
10.1 도입 618
10.2 21.1 모니터링 스택 구성 618
10.2.1 무엇을 볼 것인가 618
10.3 21.2 Firebase Crashlytics 설정 620
10.3.1 문제가 생기면 바로 알아야 한다 620
10.4 21.3 Firebase Performance로 성능 추적 623
10.4.1 숫자로 말하는 성능 623
10.5 21.4 Android Profiler로 병목 찾기 626
10.5.1 진단 없는 처방은 없다 626
10.6 21.5 배터리 최적화: 20Hz 센서와 병렬화 628
10.6.1 배터리를 아끼는 세 가지 원칙 628
10.7 21.6 메모리 관리: CameraX 프레임 즉시 close . 633
10.7.1 메모리 누수는 시한폭탄이다 633
10.8 21.7 ANR 방지 전략 637
10.8.1 “앱이 응답하지 않습니다” 를 막아라 . . . 637
10.9 21.8 Firebase Remote Config: 긴급 파라미터 조정 641
10.9.1 앱 업데이트 없이 동작을 바꾸는 법 . . . 641

10.10 21.9 성능 KPI 대시보드 643
10.10.1 한눈에 보는 앱 건강 상태 643
10.11 21.10 Battery Historian로 배터리 분석 644
10.11.1 배터리를 어디서 쓰는지 알아야 아낀다 . 644
10.12 정리 646

11 Ch22: 법적 준수와 개인정보 보호 647
11.1 도입 648
11.2 22.1 개인정보 보호법 준수 설계 648
11.2.1 "수집하지 않는다" 는 것을 증명하는 법 . 648
11.2.2 로컬 처리 원칙: Privacy by Design . . . 650
11.3 22.2 권한 최소화 원칙 652
11.3.1 위치 권한의 역설 652
11.3.2 권한 거부 시 대체 경로 653
11.4 22.3 네트워크 스캔의 합법성 확보 654
11.4.1 포트 스캔은 해킹인가? 654
11.4.2 스캔 전 사용자 고지 655
11.5 22.4 개인정보 처리방침 작성 656
11.5.1 Play Store에 처리방침이 필요한 이유 . . 656
11.6 22.5 면책 설계 658
11.6.1 "100% 보장하지 않는다" 를 법적으로 설계하기 658
11.6.2 이용약관의 탐지 한계 명시 659
11.7 22.6 GDPR 대비 660
11.7.1 유럽 시장 진출을 위한 사전 준비 660
11.8 22.7 오픈소스 라이선스 관리 662
11.8.1 iText AGPL 문제 662
11.9 22.8 컴플라이언스 유지 체계 663
11.10 마무리 664

12 Ch23: 수익화 전략과 GTM 665
12.1 도입 665
12.2 23.1 핵심 원칙: "안전은 유료가 아닙니다" . . . 666
12.3 23.2 무료 vs 프리미엄 기능 설계 667

12.3.1 기능 경계선 그리기 667
12.3.2 프리미엄 전환 트리거 설계 668
12.4 23.3 가격 전략 669
12.4.1 경쟁사 분석에서 찾은 포지셔닝 669
12.4.2 가격 구조 670
12.4.3 론칭 가격 실험 계획 670
12.5 23.4 광고 전략 671
12.5.1 광고는 최소화, 탐지는 방해 안 한다 . . . 671
12.5.2 AdMob 설정 671
12.5.3 광고 수익 추정 672
12.6 23.5 12개월 재무 모델 672
12.6.1 사용자 성장 시나리오 672
12.6.2 손익분기점 분석 673
12.7 23.6 B2B 확장: 숙박업소 파트너십 673
12.7.1 “SearCam 인증 숙소” 프로그램 673
12.7.2 API 라이선스 674
12.8 23.7 GTM(Go-to-Market) 전략 675
12.8.1 초기 사용자 획득: 0에서 1,000까지 . . 675
12.8.2 성장 채널: 1,000명에서 10,000명까지 . 676
12.8.3 KPI 설정 678
12.9 23.8 Phase별 수익 목표 678
12.10 마무리 679

13 Ch24: 회고와 다음 단계 681
13.1 도입 681
13.2 24.1 잘된 것들 682
13.2.1 Clean Architecture 선택 682
13.2.2 불변성(Immutability) 원칙 준수 683
13.2.3 OUI 데이터베이스 전략 684
13.2.4 면책 설계의 일관성 684
13.3 24.2 아쉬운 것들 685
13.3.1 테스트 커버리지 부족 685
13.3.2 Retroreflection 알고리즘의 오탐률 . . . 685

13.3.3 기기 다양성 테스트 부족 686
13.3.4 백그라운드 스캔 부재 687
13.4 24.3 기술 부채 현황 687
13.4.1 부채 목록 687
13.4.2 의도적 기술 부채 vs 비의도적 기술 부채 . 688
13.5 24.4 다음 버전 로드맵 688
13.5.1 Phase 2: iOS 포팅 + 기능 강화 (12~18개월)688
13.5.2 Phase 3: ML 고도화 + 플랫폼 (24~36개월)689
13.5.3 RTSP 스트림 분석의 법적 고려사항 . . . 690
13.6 24.5 독자에게 전하는 메시지 691
13.6.1 완벽한 계획보다 빠른 시작 691
13.6.2 문제에 집착하라 692
13.6.3 솔직함이 경쟁력이다 692
13.6.4 1인 개발의 한계를 받아들여라 693
13.6.5 마지막으로 693

14 Appendix A: OUI 데이터베이스 구조 695
14.1 A.1 OUI(Organizationally Unique Identifier) 구조 696
14.2 A.2 카메라 제조사 OUI 목록 697
14.2.1 IP 카메라 제조사 (전문 보안/감시 카메라) 697
14.2.2 스마트 카메라 / IoT 카메라 제조사 . . . 698
14.2.3 초소형/위장 카메라 (Wi-Fi 모듈 기반) . . 699
14.3 A.3 안전 기기 화이트리스트 700
14.4 A.4 oui.json 파일 포맷 701
14.5 A.5 OuiDatabase.kt 동작 방식 703
14.5.1 스캔 파이프라인에서의 OUI 활용 705
14.6 A.6 OUI DB 업데이트 방식 705

15 Appendix B: 체크리스트 설계 707
15.1 B.1 체크리스트가 필요한 이유 708
15.2 B.2 체크리스트 설계 원칙 708
15.2.1 항목 구조 708
15.2.2 소요 시간 기준 709
15.3 B.3 숙박시설 체크리스트 (20개 항목) 709

15.3.1 시선 방향 점검 (침대/소파를 향한 기기) . 709
15.3.2 벽면/천장 점검 710
15.3.3 욕실 점검 711
15.3.4 기타 점검 712
15.4 B.4 화장실/탈의실 체크리스트 (10개 항목) . . . 713
15.5 B.5 시각적 검사 vs 기술적 검사 714
15.6 B.6 체크리스트 UI 설계 715
15.6.1 Compose 컴포넌트 구조 715
15.6.2 우선순위 표시 UI 규칙 717
15.6.3 빠른 점검 모드 전환 717
15.7 B.7 체크리스트 결과 리포트 718

16 Appendix C: 에러 코드 레퍼런스 721
16.1 C.1 에러 코드 체계 721
16.2 C.2 E1xxx - 센서 오류 722
16.2.1 E1002 캘리브레이션 안내 화면 725
16.3 C.3 E2xxx - 네트워크 오류 725
16.3.1 E2002 대체 전략 727
16.4 C.4 E3xxx - 권한 오류 728
16.4.1 권한 거부 시 대체 모드 729
16.5 C.5 에러 처리 구현 패턴 730
16.5.1 sealed class 기반 결과 처리 730
16.5.2 ViewModel에서 에러 상태 처리 731
16.6 C.6 에러 코드 전체 요약표 732

17 Appendix D: 주요 API 레퍼런스 735
17.1 D.1 UseCase 레이어 736
17.1.1 스캔 관련 UseCase 736
17.1.2 렌즈 감지 UseCase 737
17.1.3 자기장 UseCase 738
17.1.4 리포트 UseCase 739
17.2 D.2 Repository 인터페이스 740
17.2.1 WifiScanRepository 740
17.2.2 CameraRepository 741

17.2.3 EmfRepository 741
17.2.4 ReportRepository 742
17.2.5 OuiRepository 743
17.3 D.3 주요 Domain 모델 744
17.3.1 ScanResult 744
17.3.2 RiskLevel 745
17.3.3 DetectedDevice 745
17.4 D.4 Constants 주요 값 746
17.4.1 네트워크 스캔 746
17.4.2 EMF 센서 747
17.4.3 위험도 임계값 747
17.4.4 카메라 설정 748
17.4.5 OUI 데이터베이스 748
17.5 D.5 탐지 레이어 가중치 749

18 Appendix E: 버전 이력 751
18.1 E.1 버전 정책 751
18.2 E.2 v1.0.0 – Phase 1 MVP (2026년 04월) . . . 752
18.2.1 신규 기능 752
18.3 E.3 Phase별 구현 계획 756
18.3.1 Phase 2: iOS 포팅 + 기능 강화 (v2.0.0, 예정 2026년 Q4) 756
18.3.2 Phase 3: 플랫폼 확장 (v3.0.0, 예정 2027년) 756
18.4 E.4 보안 수정 이력 757
18.5 E.5 알려진 제한 사항 758
18.6 E.6 개발 환경 이력 759

제 1 장

SearCam: 스마트폰 몰래카메라 탐지 앱 개발기

기획부터 출시까지, AI 시대의 모바일 앱 개발 완전 가이드

1.1 이 책에 대하여

이 책은 SearCam 프로젝트의 기획부터 출시까지 전 과정을 기록한 기술 서적이다.

SearCam(Search + Camera)은 스마트폰만으로 몰래카메라를 탐

지하는 1차 스크리닝 앱이다. Wi-Fi 네트워크 스캔, 플래시 Retroreflection 렌즈 감지, 자기장(EMF) 센서를 교차 검증하여 숨겨진 카메라를 찾는다.

29개의 기획 문서(PRD, TRD, TDD, 시스템 아키텍처, 보안 설계 등)를 기반으로 실제 상용 앱이 탄생하기까지의 의사결정, 기술 선택, 실패와 해결을 솔직하게 풀어냈다.

1.2 대상 독자

이 책은 다음 독자를 위해 쓰였다:

- **주니어 모바일 개발자 (1~3년 경력)**: 앱 하나를 처음부터 끝까지 만드는 전체 흐름을 경험하고 싶은 분
- **사이드 프로젝트 개발자**: 기획서 작성법, 수익 모델 설계, GTM 전략의 실전 사례가 필요한 분
- **스타트업 창업자/PM**: 기술 문서 체계와 아키텍처 의사결정 과정을 참고하고 싶은 분
- **CS/SE 전공 대학생**: 소프트웨어 공학 이론의 실전 적용 사례를 찾는 분
- **시니어 개발자**: 모바일 센서 활용, 보안/프라이버시 설계 패턴에 관심 있는 분

1.3 사전 지식

이 책을 읽기 위해 다음 지식이 필요하다:

- **필수**: Kotlin/Android 기초 문법 (변수, 함수, 클래스)
- **필수**: Git 기본 사용법 (commit, branch, merge)
- **필수**: REST API 개념 (HTTP 메서드, JSON)
- **권장**: Clean Architecture 용어 이해
- **권장**: Gradle 빌드 시스템 기초

1.4 이 책의 구성

총 5부 24챕터와 부록 5개로 구성되어 있다.

1.4.1 Part 1: 기획과 설계 (Why & What)

왜 만드는가, 누구를 위해 만드는가, 무엇을 만드는가. 시장 분석, 사용자 리서치, 제품 요구사항, 수익 모델을 다룬다.

1.4.2 Part 2: 아키텍처와 설계 (How)

어떻게 만들 것인가. 기술 요구사항, 시스템 아키텍처, API/DB 설계, 보안/프라이버시 설계를 다룬다.

1.4.3 Part 3: 구현 (Build)

설계를 코드로 옮기는 과정. Wi-Fi 스캔, 렌즈 감지, EMF 감지, 교차 검증 엔진, UI/UX 구현을 다룬다.

1.4.4 Part 4: 품질과 배포 (Ship)

완성된 코드를 제품으로 만드는 과정. 테스트 전략, CI/CD, Play Store 배포, 모니터링을 다룬다.

1.4.5 Part 5: 성장 (Grow)

출시 이후의 과정. 마케팅, 데이터 분석, 접근성/국제화, 파트너십 전략을 다룬다.

1.4.6 부록

에러 처리, 성능 최적화, OUI 데이터베이스, 리스크 매트릭스, 법적 준수 가이드를 수록한다.

1.5 실습 환경

각 챕터에는 실습이 포함되어 있다. 실습을 따라하려면 다음 환경이 필요하다:

항목	요구 사양
OS	macOS 13+, Windows 10+, Ubuntu 22.04+
IDE	Android Studio Hedgehog 이상
JDK	JDK 17
Android SDK	API 34 (Android 14)
Kotlin	1.9+
Git	2.30+
테스트 기기	Android 10 이상 실기기 (에뮬레이터로는 센서 테스트 불가)

1.6 표기 규칙

이 책에서 사용하는 표기 규칙은 다음과 같다:

- 고정폭 글꼴: 코드, 파일명, 명령어
- **굵은 글씨**: 중요한 개념, 처음 등장하는 용어

- 인용 블록: 핵심 원칙, 설계 결정의 이유
- [TIP] 블록: 실전 팁, 시간을 절약하는 방법
- [WARNING] 블록: 흔한 실수, 주의 사항

1.7 소스 코드

이 책에서 다루는 모든 소스 코드는 다음 저장소에서 확인할 수 있다:

- 기획 문서: `docs/` 디렉토리 (29개 문서)
- 챕터 원고: `docs/book/chapters/` 디렉토리

1.8 저자

[저자 정보 추가 예정]

1.9 피드백

이 책에 대한 의견, 오류 신고, 제안은 다음 채널로 보내주세요:

- GitHub Issues: [저장소 URL 추가 예정]
- Email: [이메일 추가 예정]

모든 피드백은 다음 판에 반영됩니다.

1.10 감사의 말

[감사의 말 추가 예정]

제 2 장

Ch01: 문제 발견과 시장 분석

이 장에서 배울 것: 좋은 앱은 좋은 문제에서 시작한다. 몰래카메라 탐지라는 문제를 어떻게 발견하고, 시장을 분석하고, 경쟁 앱을 해부했는지 보여줍니다.

2.1 도입

새벽 2시, 출장으로 처음 묵는 모텔 방. 에어컨 리모컨 위 작은 구멍이 눈에 밟힌다. 설마 카메라일까? 스마트폰을 들고 검색해 보니 관련 앱이 있다. 설치해서 돌려봐도 "위험!" 알림만 뜨고 이유는 없다. 믿어야 할지 말아야 할지 모르는 채로 잠에 든다.

이 경험이 SearCam의 시작입니다.

좋은 제품은 대부분 "내가 불편했던 것" 에서 출발합니다. 하지만 불편함을 느끼는 것과 그것을 제품으로 만드는 것 사이에는 큰 차이가 있습니다. 이 장에서는 SearCam이 어떻게 막연한 불편함을 구체적인 시장 기회로 전환했는지 보여줍니다.

2.2 1.1 문제의 심각성 파악

2.2.1 데이터부터 시작하라

감정이 아닌 데이터로 문제를 정의해야 합니다. SearCam 기획 단계에서 수집한 수치입니다.

지표	수치	출처
국내 몰카 범죄 발생 건수 (2023)	연 6,000건+	경찰청 통계
피해자 중 숙박시설 관련 비율	약 32%	여성가족부
스마트폰으로 탐지 가능한 카메라 비율	60~70%	기술 분석
기존 탐지 앱 평균 오탐률	55~70%	경쟁사 리뷰 분석

숫자가 말해주는 것: 문제는 실재하고, 기존 솔루션은 불충분하다.

2.2.2 스마트폰의 한계를 먼저 인정하라

많은 앱이 "완벽한 탐지" 를 주장합니다. 하지만 스마트폰은 전문 탐지 장비가 아닙니다.

전문 장비가 할 수 있는 것:

- RF 신호 감지 (모든 무선 카메라)
- Non-Linear Junction Detection
- 열화상 분석

스마트폰이 할 수 있는 것:

- Wi-Fi 네트워크 스캔 (같은 네트워크 한정)
- 카메라 렌즈 역반사 감지 (플래시 활용)
- 자기장 변화 감지
- RF 신호 분석 (하드웨어 없음)
- LTE/5G 카메라 탐지
- 전원 꺼진 카메라

이 한계를 인정하는 것이 SearCam의 핵심 차별점입니다. **솔직함이 신뢰를 만듭니다.**

2.3 1.2 경쟁 앱 해부하기

2.3.1 5개 앱을 직접 써보며 배운 것

기획 단계에서 주요 경쟁 앱 5개를 직접 설치하고 실제 카메라 앞에서 테스트했습니다.

앱	다운로드	평점	핵심 약점
Hidden Camera Detector	10M+	3.6	오탐 55%, 이유 없는 "위험!"
CamX	1M+	4.2	모드 수동 전환 필요
FindSpy	500K+	4.4	단일 센서만 사용
Peek	1M+	4.4	핵심 기능 유료
몰카 탐지기	100K+	4.2	Android 전용, 교차 검증 없음

2.3.2 공통 패턴: "위험!"만 외치는 앱들

모든 앱이 공유하는 문제가 있었습니다.

기존 앱의 결과 화면:

```
┌──────────────────────────┐
│                          │
│  ⚠  위험!                │
│                          │
│  [확인]                  │
└──────────────────────────┘
```

← 왜 위험한지 이유가 없음

← 오탐인지 진짜 탐지인지 구분 불가

← 다음 행동 지침 없음

SearCam의 결과 화면이 달라야 하는 이유가 여기서 나왔습니다.

2.4 1.3 시장 기회 정의

2.4.1 TAM / SAM / SOM 분석

```
TAM (Total Addressable Market)
├── 전 세계 스마트폰 사용자 중 연 1회 이상 숙박 이용자
├── 약 15억 명
└── 시장 규모: ~$2.85B

SAM (Serviceable Addressable Market)
├── 한국 내 숙박/원룸 거주자 + 여행자
├── 약 1,500만 명
└── 시장 규모: ~$285M

SOM (Serviceable Obtainable Market) — 1년 목표
├── 목표 다운로드: 50,000건
```

```
└──── 목표 수익: 월 ₩250,000 (광고 + 프리미엄)
```

2.5 1.4 차별화 전략

경쟁에서 이기는 방법은 두 가지입니다. 더 잘하거나, 다르거나.

SearCam의 선택: **다르게 접근하기 (Blue Ocean)**

기존 앱이 강조하는 것	SearCam이 강조하는 것
"100% 탐지!"	"한계를 솔직히 알려드립니다"
각 센서 독립 작동	3개 센서 교차 검증
기술 기능 나열	사용자 행동 가이드
유료 핵심 기능	핵심 완전 무료
단순 "위험!" 알림	왜 위험한지 근거 설명

2.6 실습 과제

실습 1-1: 지금 쓰는 스마트폰에 몰카 탐지 앱 2개를 설치하고, 실제 웹캠 앞에서 테스트해보세요. 각 앱의 탐지 결과와 오탐 여부를 표로 정리해보세요.

실습 1-2: 당신이 해결하고 싶은 문제를 하나 골라 TAM/SAM/SOM을 계산해보세요. 데이터는 통계청, 여성가족부, 경찰청 공개 자료를 활용하세요.

2.7 핵심 정리

개념	요점
문제 정의	감정이 아닌 데이터로 시작
한계 인정	불가능한 것을 약속하지 않는 것이 신뢰의 시작
경쟁 분석	직접 써봐야 약점이 보인다
차별화	더 잘하기 어려우면 다르게 접근

- ☒ 문제의 규모를 데이터로 검증하라
- ☒ 기술적 한계를 먼저 파악하라
- ☒ 경쟁 앱을 실제로 사용해보라
- ☒ “우리가 최고” 라는 주장은 데이터 없이 하지 마라

2.8 다음 장 예고

문제를 정의했으니 이제 “누가” 이 앱을 쓸지 구체적으로 그려볼 차례입니다. Ch02에서는 사용자 인터뷰, 페르소나 설계, Customer Journey Map을 다룹니다.

— 참고 자료: docs/project-plan.md, docs/11-competitor-analysis.md

—

Ch02: 제품 요구사항 정의 — PRD를 쓴다는 것의 의미

—

> **이 장에서 배울 것:** 좋은 앱은 좋은 문서에서 시작합니다.

SearCam 의

2.9 도입

음식점을 열기 전에 메뉴판을 먼저 설계합니다. 어떤 음식을 팔지, 가격은 얼마인지, 어떤 손님을 대상으로 할지. 이것 없이 주방을 먼저 꾸리는 식당은 없습니다.

PRD(Product Requirements Document)는 소프트웨어의 메뉴판입니다. 무엇을 만들지, 누구를 위해 만드는지, 성공의 기준은 무엇인지를 명문화합니다. PRD 없이 코딩을 시작하면 결국 "내가 필요한 것" 을 만들게 됩니다. 사용자가 필요한 것이 아니라.

SearCam PRD 작성 과정에서 가장 많이 싸운 질문은 하나였습니다: **"이 기능, 정말 필요한가?"**

2.10 2.1 PRD란 무엇인가

2.10.1 문서 하나가 팀 전체를 정렬한다

PRD는 개발자, 디자이너, 기획자가 서로 다른 것을 만드는 상황을 방지합니다. "30초 안에 결과를 보여준다" 는 한 줄이 없으면, 어떤 개발자는 3분짜리 정밀 스캔을 기본으로 만들 수 있습니다. 어떤 디자이너는 복잡한 설정 화면을 홈에 배치할 수 있습니다.

PRD는 팀이 아닌 혼자 만드는 프로젝트에도 필요합니다. 자신과

의 약속이기 때문입니다. “나는 이것을 만든다. 저것은 만들지 않는다.”

```
PRD 없이 개발할 때:
  1주차: "Wi-Fi 스캔 기능 추가"
  2주차: "아, IR 감지도 재미있겠는데..."
  3주차: "UI를 완전히 바꿔볼까?"
  4주차: 아무것도 완성 안 됨

PRD 있을 때:
  1주차: Wi-Fi 스캔 (Layer 1) → 체크
 2주차: 렌즈 감지 (Layer 2) → 체크
 3주차: EMF 센서 (Layer 3) → 체크
 4주차: MVP 출시
```

2.10.2 SearCam PRD의 핵심 구성

섹션	내용	목적
Executive Summary	한 문단으로 제품 설명	엘리베이터 피치
문제 정의	데이터 기반 문제 증명	Why this product
비전 및 목표	측정 가능한 성공 기준	완료 기준
타겟 사용자	구체적인 페르소나	누구를 위한 제품인가
사용자 스토리	기능 요구사항	무엇을 만들 것인가

섹션	내용	목적
비기능 요구사항	성능, 보안, 접근성	얼마나 잘 만들 것인가
제외 범위	만들지 않을 것	범위 방어

2.11 2.2 Executive Summary — 30초 안에 설명하기

2.11.1 좋은 Executive Summary의 조건

PRD의 첫 문단은 투자자, 팀원, 새로운 개발자 모두가 읽습니다. 이 문단만 읽고도 제품을 이해할 수 있어야 합니다.

SearCam Executive Summary:

> SearCam(Search + Camera)은 **스마트폰만으로 몰래 카메라를 탐지하는 1차 스크리닝 앱**이다. 숙소, 화장실, 탈의실 등에서 일반인이 **30초 안에** 숨겨진 카메라 여부를 점검할 수 있도록 한다. Wi-Fi 네트워크 스캔, 플래시 Retroreflection 렌즈 감지, 자기장(EMF) 센서를 교차 검증하여 **종합 탐지율 70~75%, 오탐율 5~10%**를 목표로 한다.

이 두 문장에 담긴 것들을 분해해 봅니다.

요소	내용	역할
“스마트폰만으로”	추가 장비 불필요	접근성
“1차 스크리닝”	전문 장비 대체 아님	포지셔닝
“30초 안에”	구체적 성능 목표	측정 가능한 약속
“3가지 센서”	기술 차별점	경쟁 우위
“탐지율 70~75%”	성능 목표 수치	기대값 관리

2.11.2 제품 포지셔닝: 솔직함이 차별점이다

대부분의 앱이 “완벽한 탐지” 를 주장합니다. SearCam은 다른 방향을 선택했습니다.

```
전문 탐지 장비 = 정밀 건강검진
  - RF 스캐너: 50만~300만원
  - NLJD: 200만~1,000만원
  - 열화상 카메라: 100만~500만원
  - 전문가 운용 필수

SearCam = 자가 건강체크
  - 비용: 무료
  - 장비: 스마트폰
  - 전문 지식: 불필요
```

- 한계: 솔직하게 안내

"없는 것보다 훨씬 나은 1차 스크리닝 도구" — 이 포지셔닝이 기존 앱과 SearCam을 구분하는 핵심입니다.

2.12 2.3 핵심 기능 우선순위 결정

2.12.1 기능 아이디어는 무한하다, 시간은 유한하다

PRD 작성 초기에 기능 아이디어가 30개 이상 나왔습니다. 이것을 어떻게 줄였을까요?

MoSCoW 방법론을 사용했습니다. Must have / Should have / Could have / Won't have.

우선순위	기능	이유
Must	Quick Scan (Wi-Fi 30초)	핵심 가치 제안
Must	렌즈 찾기 (Retroreflection)	기존 앱 없는 차별점
Must	종합 위험도 (0~100)	결과 이해 가능성
Must	스캔 결과 근거 설명	신뢰 구축 핵심
Should	EMF 자기장 스캔	3-Layer 완성
Should	스캔 리포트 저장	기록 관리
Should	육안 점검 체크리스트	센서 보완

우선순위	기능	이유
Could	PDF 내보내기	B2B 확장
Could	IR 야간 카메라 감지	정밀도 향상
Won’t (Phase 1)	112 신고 연동	Phase 2
Won’t (Phase 1)	커뮤니티 위험 장소 맵	Phase 3
Won’t (Phase 1)	ML 기반 자기장 분류	Phase 3

2.12.2 30초 원칙 — 거스를 수 없는 제약

30초는 임의로 정한 숫자가 아닙니다. 사용 시나리오에서 역산한 값입니다.

```
사용 시나리오: 호텔 체크인
  └─── 체크인 카운터에서 엘리베이터까지: 3분
    └─── 방 문 열고 짐 내려놓기: 1분
      └─── "앱 켜고 스캔": 30초 이내 완료 목표
        └─── 결과 확인 → 안심 or 추가 조치
```

30초를 넘기면 사용자는 스캔을 중단합니다. 짐을 풀고 싶으니까요. 이 제약이 전체 기술 설계를 결정했습니다:

- Wi-Fi ARP 조회: 병렬 처리
- 포트 스캔: 의심 기기만 선택적 실행
- OUI 매칭: 인메모리 해시맵

- UI 업데이트: 실시간 스트리밍 (완료 대기 없음)

2.13 2.4 타겟 사용자와 페르소나

2.13.1 페르소나는 실제 사람처럼 구체적이어야 한다

"20~40대 여성" 은 페르소나가 아닙니다. SearCam은 5개의 구체적인 페르소나를 정의했습니다.

페르소나 1: 여행자 김수진 (28세, 마케팅 대리)

> 에어비앤비를 자주 쓰는 솔로 여행자. 체크인할 때마다 뉴스에서 본 몰카 사건이 떠오른다. 방을 둘러봐도 어디를 봐야 할지 모른다.

핵심 니즈: **30초 안에, 버튼 하나로, 안전 확인**

페르소나 2: 1인 가구 이하은 (24세, 대학원생)

> 공중화장실, 탈의실에서 불안감이 크다. Wi-Fi가 없는 환경이 많다. 양면 거울인지 확인하고 싶은데 방법을 모른다.

핵심 니즈: **Wi-Fi 없이도 동작하는 렌즈 + EMF 스캔**

페르소나 3: 출장 직장인 박준호 (35세, 영업 과장)

주 2~3회 지방 출장. 회사 기밀 문서를 다룬다. 점검 결과를 리포트로 남기고 싶다.

핵심 니즈: **빠른 스캔 + PDF 리포트**

페르소나 4: 피트니스 이용자 최민서 (31세, 프리랜서 디자이너)

매일 헬스장 탈의실 사용. Wi-Fi가 없다. "왜 위험한지" 설명이 필요하다.

핵심 니즈: **오탐 판별 근거, Wi-Fi 없이도 동작**

페르소나 5: 숙박업소 관리자 정태영 (45세)

게스트하우스 10객실 운영. 전문 업체 의뢰 비용이 부담. 정기 점검 기록 관리가 필요하다.

핵심 니즈: **객실별 점검 리포트, 이력 관리**

2.13.2 페르소나에서 기능이 나온다

페르소나를 먼저 정의하면 기능의 우선순위가 자연스럽게 결정됩니다.

페르소나	핵심 니즈	파생 기능
김수진	30초 빠른 스캔	Quick Scan, 원클릭 UX
이하은	오프라인 탐지	Layer 2+3, 체크리스트
박준호	리포트 관리	PDF 내보내기, 이력 조회

페르소나	핵심 니즈	파생 기능
최민서	근거 설명	근거 기반 결과 화면
정태영	정기 점검	위험도 필터, 날짜별 이력

2.14 2.5 사용자 스토리 작성법

2.14.1 형식: As a / I want / So that

사용자 스토리는 기능 명세서와 다릅니다. 기능 명세서가 "무엇을 만드는가"를 설명한다면, 사용자 스토리는 "왜 만드는가"를 설명합니다.

형식:

```
As a [사용자 유형],
I want [원하는 것],
So that [얻고자 하는 가치].
```

나쁜 예:

```
"Wi-Fi 스캔 기능을 추가한다"
→ 왜 추가하는지, 누구를 위한지 없음
```

좋은 예:

"As a 여행자,

I want 앱을 열고 버튼 하나로 Wi-Fi 스캔을 시작하기를,

So that 숙소 체크인 직후 30초 안에 안전을 확인할 수 있다."

2.14.2 SearCam의 7개 Epic과 주요 스토리

SearCam PRD는 7개의 Epic으로 구성됩니다.

Epic 1: 빠른 안전 점검 (Quick Scan)

스토리 ID	제목	핵심 AC
US-01	원클릭 빠른 스캔	30초 이내 결과
US-02	Quick Scan 결과 확인	0~100 위험도 + 5단계 색상
US-03	Wi-Fi 미연결 시 대응	Layer 2+3 자동 전환

Epic 2: 정밀 통합 스캔 (Full Scan)

스토리 ID	제목	핵심 AC
US-04	3-Layer 통합 스캔	3개 Layer 자동 실행, 3분 이내
US-05	교차 검증 결과	오탐 자동 판별

스토리 ID	제목	핵심 AC
US-06	근거 기반 결과 설명	Layer별 탐지 근거 표시

Epic 3: 렌즈 감지 (Retroreflection)

US-07: 플래시 렌즈 찾기, US-08: IR 야간 카메라 감지, US-09: 렌즈 동적 검증

Epic 4: 자기장(EMF) 감지

US-10: 자동 캘리브레이션, US-11: 실시간 자기장 표시, US-12: EMF 감도 조정

Epic 5: 리포트 및 기록

US-13: 스캔 리포트 저장, US-14: PDF 내보내기, US-15: 리포트 이력 조회

Epic 6: 체크리스트 (육안 점검)

US-16: 숙소 육안 점검 가이드, US-17: 화장실 육안 점검 가이드

Epic 7: 온보딩 및 권한

US-18: 최초 사용 온보딩, US-19: 런타임 권한 요청, US-20: 앱 한계 안내

2.14.3 Acceptance Criteria가 핵심이다

사용자 스토리에서 가장 중요한 부분은 Acceptance Criteria(AC) 입니다. 언제 "완료" 인지를 정의하기 때문입니다.

```
US-07: 플래시 렌즈 찾기 — 예시

As a 피트니스 이용자,
I want Wi-Fi 없이 플래시를 켜고 카메라 렌즈를 찾을 수 있기를,
So that 탈의실처럼 네트워크가 없는 환경에서도 점검할 수 있다.

Acceptance Criteria:
  [ ] 후면 카메라 + 플래시 ON 상태에서 실시간 프레임 분석이 동작한다
  [ ] 렌즈 반사 의심 포인트를 화면에 빨간 원 + 펄스 애니메이션으로 표시한
  [ ] 사용자 가이드("벽면을 천천히 비춰주세요")를 화면 상단에 표시한다
  [ ] 진동 알림으로 의심 포인트 감지를 즉시 알린다
```

AC 없는 스토리는 개발자가 임의로 "완료" 를 선언합니다. AC 있는 스토리는 테스트 케이스가 됩니다.

2.15 2.6 탐지 정확도 목표 설정

2.15.1 목표 수치 설정의 함정

"탐지율 100%, 오탐율 0%" 는 목표가 아닙니다. 달성 불가능한 수치는 팀을 지치게 합니다.

SearCam의 정확도 목표는 현실적 데이터에서 출발했습니다.

```
경쟁 앱 벤치마크:
  Hidden Camera Detector:  탐지율 ~65%, 오탐율 ~55%
   CamX:                   탐지율 ~70%, 오탐율 ~35%
   FindSpy:                탐지율 ~60%, 오탐율 ~45%


SearCam 목표:
   탐지율: 70~75%  (기존 앱 최상위 수준)
   오탐율: 5~10%   (기존 앱 대비 대폭 개선)
```

오탐율을 35%→10%로 줄이는 것이 핵심입니다. 이것이 3-Layer 교차 검증 엔진의 존재 이유입니다.

2.15.2 탐지 레이어별 성능 목표

레이어	방식	탐지 가능 대상	탐지율 목표	오탐율 목표
Layer 1	Wi-Fi 스캔	동일 네트워크 연결 카메라	85%+	15% 이하
Layer 2	Retroreflection	렌즈 노출 카메라 (전원 무관)	60%+	20% 이하
Layer 3	EMF 감지	전원 공급 중 카메라	50%+	30% 이하
교차 검증	**3-Layer 합산**	**위 대상 합산**	**70~75%**	**5~10%**

교차 검증이 오탐율을 낮추는 원리:

단일 레이어 양성 → 오탐 가능성 높음 → 낮은 위험도
복수 레이어 양성 → 실제 위협 가능성 높음 → 높은 위험도

예시:

```
EMF만 양성 → "가전제품일 가능성이 높습니다" (위험도 낮게 유지)
Wi-Fi + EMF 양성 → 위험도 상향 보정
Wi-Fi + Retroreflection + EMF 양성 → 최고 위험도
```

2.16 2.7 비기능 요구사항 — 보이지 않는 품질

2.16.1 성능 목표

항목	목표	최대 허용
앱 Cold Start	1.5초 이내	3.0초
Quick Scan 완료	30초 이내	45초
Full Scan 완료	3분 이내	5분
크래시율	0.3% 미만	0.5%
렌즈 감지 프레임 처리	30ms/프레임	66ms/프레임

2.16.2 보안 요구사항

SearCam은 보안 앱입니다. 보안 앱이 안전하지 않으면 신뢰를 잃습니다.

- 카메라 프레임 데이터: 메모리에서만 처리, 저장 금지
- 위치 정보: 리포트 저장 시 선택적 포함, GPS 상시 추적 금지
- 네트워크 스캔: 연결된 Wi-Fi 서브넷 내로 제한 (/24 기준)
- 로컬 처리 원칙: 탐지 데이터를 외부 서버로 전송하지 않음

2.16.3 접근성 목표

항목	기준
최소 글자 크기	14sp 이상
색상 대비	WCAG AA (4.5:1 이상)
터치 타겟	48dp × 48dp 이상
색각 이상	색상만으로 구분되는 정보 없음 (아이콘 + 텍스트 병용)

2.17 2.8 제외 범위 — “만들지 않을 것” 목록

PRD에서 가장 중요하지만 가장 자주 빠지는 섹션입니다. 무엇을 만들지 않는지를 명확히 해야 범위가 통제됩니다.

제외 항목	이유	대안
iOS 버전	Phase 2로 연기	Android MVP 집중
LTE/5G 카메라 탐지	하드웨어 한계 (RF 수신기 없음)	한계로 명시
ML 기반 자기장 분류	Phase 3	규칙 기반으로 대체
클라우드 동기화	프라이버시 위험	로컬 저장
커뮤니티 위험 장소 맵	Phase 3	로컬 리포트로 대체
112 신고 연동	Phase 2	전화 앱 연결로 대체
기업용 대시보드	Phase 3	미지원
전원 꺼진 카메라 탐지	물리적 불가능	한계로 명시

2.18 2.9 PRD 작성 체크리스트

PRD를 작성했다면 아래 항목을 확인합니다.

- ☐ Executive Summary: 두 문단 이내로 제품 설명 가능
- ☐ 문제 정의: 데이터와 수치로 문제 증명
- ☐ 비전: 측정 가능한 성공 기준 존재
- ☐ 페르소나: 이름, 나이, 상황, 페인포인트, 니즈 포함
- ☐ 사용자 스토리: As a / I want / So that 형식
- ☐ Acceptance Criteria: 각 스토리에 최소 3개 이상
- ☐ 성능 목표: 구체적 수치 (초, %, MB)
- ☐ 제외 범위: "만들지 않을 것" 명시
- ☐ 보안/프라이버시 요구사항 포함

2.19 마무리

PRD는 완성본이 아닙니다. 살아있는 문서입니다. 개발하면서 배우는 것들이 PRD를 업데이트합니다.

SearCam PRD가 가르쳐준 가장 중요한 교훈: **"어떤 기능을 추가할지"보다 "어떤 기능을 빼야 할지"를 결정하는 것이 더 어렵다.**

30초 원칙 하나를 지키기 위해 얼마나 많은 기능을 Phase 2, 3으로 밀었는지 생각해보면, PRD의 진짜 가치는 “할 일 목록” 이 아니라 “하지 않을 일 목록” 에 있다는 것을 알 수 있습니다.

다음 장에서는 이 PRD를 기반으로 기술 요구사항(TRD)을 작성하고, 어떤 기술 스택을 선택했는지 — 그리고 왜 그 선택을 했는지 — 살펴봅니다.

제 3 장

Ch03: 기술 요구사항과 아키텍처 결정 — 도구를 고를 때 이유가 있어야 한다

이 장에서 배울 것: 기술 스택은 유행을 따르는 것이 아닙니다. SearCam이 Kotlin, Jetpack Compose, Hilt, Room, CameraX를 선택한 이유, Android API 26+를 최소 버전으로 정한 이유, Clean Architecture를 선택한 이유를 하나씩 논증합니다.

3.1 도입

목수는 못을 박기 위해 드라이버를 쓰지 않습니다. 당연한 말이지만, 소프트웨어 세계에서는 이 실수가 자주 일어납니다. "요즘 다들 쓰니까" 라는 이유로 기술을 선택하고, 나중에 그 선택이 프로젝트 내내 발목을 잡습니다.

SearCam의 기술 스택을 결정할 때 모든 선택에 "왜?" 를 물었습니다. 이 장에서는 그 대화를 재현합니다.

3.2 3.1 개발 언어: Kotlin을 선택한 이유

3.2.1 Java 대신 Kotlin — 2024년 기준 논쟁의 여지가 없다

결론부터: Android 앱에서 Kotlin은 사실상 표준입니다. Google이 2017년 Android 공식 언어로 채택한 이후, 새 Jetpack 라이브러리는 Kotlin 우선으로 설계됩니다.

하지만 "다들 쓰니까" 가 아니라 기술적 이유를 살펴봅니다.

Null Safety

SearCam에서 가장 자주 발생할 수 있는 버그는 무엇일까요? 센서 데이터입니다. 자력계가 없는 기기, 플래시가 없는 기기, Wi-Fi가 꺼진 기기 — 언제나 null이 가능합니다.

```
// Java 방식: NullPointerException 지뢰밭
SensorManager sensorManager = context.getSystemService(Con
Sensor magnetometer = sensorManager.getDefaultSensor(Senso
// magnetometer가 null이면? 바로 크래시

// Kotlin 방식: 컴파일 타임에 null 처리 강제
val magnetometer: Sensor? = sensorManager.getDefaultSensor
    ?: run {
        // null인 경우 명시적으로 처리
        notifyLayerUnavailable(DetectionLayer.EMF)
        return@run null
    }
```

Coroutines + Flow

SearCam의 핵심은 비동기 처리입니다. Wi-Fi 스캔, 카메라 프레임 분석, 자기장 측정 — 모두 동시에 일어납니다.

```
// Kotlin Coroutines로 비동기를 동기처럼 작성
suspend fun runQuickScan(): ScanResult {
    return coroutineScope {
        val networkResult = async { scanNetwork() }
        val emfResult = async { measureEMF() }
```

를 때 이유가 있어야 한다

```
        ScanResult(
            network = networkResult.await(),
            emf = emfResult.await()
        )
    }
}

// Kotlin Flow로 실시간 데이터 스트리밍
fun observeMagneticField(): Flow<MagneticReading> = flow {
    sensorManager.registerListener(...)
    // 20Hz로 지속 방출
}
```

Kotlin 없이 이 코드를 Java로 작성하면 스레드 관리, 콜백 지옥, 메모리 누수 위험이 몇 배로 늘어납니다.

Data Class

```
// 보일러플레이트 없이 불변 데이터 모델
data class MagneticReading(
    val x: Float,
    val y: Float,
    val z: Float,
    val magnitude: Float,
```

```
    val timestamp: Long
) {
    fun delta(baseline: MagneticReading): Float =
        abs(magnitude - baseline.magnitude)
}
```

Java라면 getter, setter, equals(), hashCode(), toString()을 직접 작성해야 합니다.

3.3 3.2 UI 프레임워크: Jetpack Compose를 선택한 이유

3.3.1 XML vs Compose — 레거시냐 미래냐

2024년 기준, 신규 Android 프로젝트에서 XML View System을 선택하는 것은 새 건물을 지으면서 1980년대 설계도를 쓰는 것과 같습니다.

Compose가 SearCam에 적합한 이유:

첫째, **실시간 데이터 표시**에 강합니다. SearCam의 핵심 화면은 자기장 실시간 그래프, 렌즈 감지 포인트 오버레이, 스캔 진행률 — 모두 매 프레임 변하는 데이터입니다.

를 때 이유가 있어야 한다

```
// Compose: 상태가 바뀌면 UI가 자동으로 재구성됨
@Composable
fun RiskGauge(riskScore: Int) {
    val animatedScore by animateIntAsState(
        targetValue = riskScore,
        animationSpec = tween(durationMillis = 500)
    )

    Canvas(modifier = Modifier.size(200.dp)) {
        // 위험도에 따라 색상과 각도가 자동으로 변함
        drawArc(
            color = riskColor(animatedScore),
            startAngle = 135f,
            sweepAngle = (animatedScore / 100f) * 270f,
            useCenter = false
        )
    }
}
```

XML로 같은 것을 구현하면 Canvas 커스텀 뷰, 애니메이션 처리, 상태 관리를 모두 수동으로 연결해야 합니다.

둘째, **선언형 패러다임**이 상태 관리를 단순화합니다.

```
// ViewModel의 상태가 변하면 Compose가 알아서 재구성
@Composable
fun ScanScreen(viewModel: ScanViewModel = hiltViewModel())
    val uiState by viewModel.uiState.collectAsStateWithLif

    when (uiState) {
        is ScanUiState.Idle -> IdleContent(onStartScan = v
        is ScanUiState.Scanning -> ScanningContent(progres
        is ScanUiState.Complete -> ResultContent(result =
        is ScanUiState.Error -> ErrorContent(message = uiS
    }
}
```

XML 방식이라면 각 상태마다 View의 visibility를 수동으로 토글해야 합니다. 상태가 4개면 분기가 기하급수로 늘어납니다.

셋째, **Google의 공식 지원 방향**입니다. 새 Jetpack 컴포넌트는 Compose 우선으로 설계되고, Material3도 Compose에서 가장 잘 동작합니다.

를 때 이유가 있어야 한다

3.4 3.3 의존성 주입: Hilt를 선택한 이유

3.4.1 DI가 필요한 이유를 먼저 이해한다

의존성 주입(Dependency Injection)은 커피 자판기와 바리스타의 차이와 같습니다. 자판기는 내부에 모든 재료가 고정되어 있어 교체가 불가능합니다. 바리스타는 외부에서 재료를 받아 음료를 만들기 때문에 원두를 바꿔도 됩니다.

SearCam에서 DI가 필요한 이유:

```
// DI 없이: WifiScanViewModel이 직접 의존성을 생성
class WifiScanViewModel : ViewModel() {
    private val scanner = WifiScanner(context) // context를
    private val repository = WifiScanRepositoryImpl(scanne
    // 테스트 시 실제 Wi-Fi 스캐너가 동작함 → 테스트 불가
}

// DI 있이: 외부에서 주입받음
@HiltViewModel
class WifiScanViewModel @Inject constructor(
    private val runQuickScanUseCase: RunQuickScanUseCase
) : ViewModel() {
    // 테스트 시 FakeQuickScanUseCase를 주입하면 됨 → 테스트 가
}
```

Hilt vs Koin vs Dagger

항목	Hilt	Koin	Dagger
컴파일 타임 검증	O	X	O
학습 난이도	중간	낮음	높음
Android 통합	최상 (Google 공식)	보통	좋음
보일러플레이트	낮음	매우 낮음	높음
성능	최상	런타임 오버헤드	최상

Koin이 편하지만, 런타임 의존성 해결은 컴파일 타임에 잡을 수 있는 오류를 앱 크래시로 바꿉니다. SearCam 같은 보안 앱은 배포 후 크래시가 용납되지 않습니다. Hilt가 정답입니다.

```
// Hilt 모듈 예시: 센서 바인딩
@Module
@InstallIn(SingletonComponent::class)
object SensorModule {

    @Provides
    @Singleton
    fun provideWifiScanner(
        @ApplicationContext context: Context
    ): WifiScanner = WifiScanner(context)
```

를 때 이유가 있어야 한다

```
    @Provides
    @Singleton
    fun provideMagneticSensor(
        @ApplicationContext context: Context
    ): MagneticSensor = MagneticSensor(context)
}

// Repository 인터페이스와 구현체 바인딩
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    abstract fun bindWifiScanRepository(
        impl: WifiScanRepositoryImpl
    ): WifiScanRepository
}
```

3.5 3.4 로컬 DB: Room을 선택한 이유

3.5.1 SQLite를 직접 쓰지 않는 이유

SQLite 위에 Room을 사용하는 것은 원자재보다 가공품을 쓰는 것과 같습니다. Room이 제공하는 것:

- SQL 쿼리를 Kotlin 메서드로 변환 (컴파일 타임 검증)
- Flow 반환 지원 (실시간 데이터 변화 감지)
- Migration 지원 (버전 업그레이드 관리)

```
// Room DAO: 스캔 리포트 저장 및 조회
@Dao
interface ReportDao {

    @Insert
    suspend fun insert(report: ScanReportEntity): Long

    @Query("SELECT * FROM scan_reports ORDER BY scan_time
    fun getRecentReports(limit: Int = 10): Flow<List<ScanR

    @Query("SELECT * FROM scan_reports WHERE risk_level =
    fun getReportsByRiskLevel(level: String): Flow<List<Sc

    @Delete
```

```
    suspend fun delete(report: ScanReportEntity)
}
```

Flow 반환형은 Compose와 완벽하게 연동됩니다. DB에 새 리포트가 저장되면 UI가 자동으로 업데이트됩니다.

SearCam의 데이터 모델

테이블	역할	관계
scan_reports	스캔 결과 헤더 (날짜, 위험도, 위치)	1:N
network_devices	발견된 네트워크 기기 목록	N:1 → scan_reports
risk_points	렌즈/IR 의심 포인트	N:1 → scan_reports
checklist_items	육안 점검 체크리스트 항목	N:1 → scan_reports

3.6 3.5 카메라: CameraX를 선택한 이유

3.6.1 Camera2 API vs CameraX

Camera2 API는 강력하지만 복잡합니다. CameraX는 Camera2를 추상화하여 개발자 친화적 API를 제공합니다.

SearCam에서 카메라가 하는 일: 1. **렌즈 감지**: 실시간 프레임을 분석하여 밝은 반사 포인트 추출 2. **IR 감지**: 전면 카메라로 IR LED 발광 분석

```
// CameraX ImageAnalysis: 실시간 프레임 처리
class LensDetector @Inject constructor(
    private val retroreflectionAnalyzer: RetroreflectionAn
) {
    fun startAnalysis(
        lifecycleOwner: LifecycleOwner,
        previewView: PreviewView
    ): Flow<List<LensPoint>> = callbackFlow {

        val imageAnalyzer = ImageAnalysis.Builder()
            .setTargetResolution(Size(1280, 720))
            .setBackpressureStrategy(ImageAnalysis.STRATEG
            .build()
            .also { analysis ->
                analysis.setAnalyzer(Dispatchers.Default.a
                    val points = retroreflectionAnalyzer.a
                    trySend(points)
                    imageProxy.close()
                }
            }

        // Preview + ImageAnalysis 동시 실행
```

를 때 이유가 있어야 한다

```
        val cameraProvider = ProcessCameraProvider.getInst
        cameraProvider.bindToLifecycle(
            lifecycleOwner,
            CameraSelector.DEFAULT_BACK_CAMERA,
            Preview.Builder().build().apply { setSurfacePr
            imageAnalyzer
        )

        awaitClose { cameraProvider.unbindAll() }
    }
}
```

CameraX의 핵심 장점: Lifecycle과 자동 연동됩니다. Activity/Fragment가 파괴되면 카메라가 자동으로 해제됩니다. Camera2로 직접 구현하면 이것을 수동으로 관리해야 합니다.

3.7 3.6 Android 버전 전략: API 26+를 선택한 이유

3.7.1 최소 버전 결정은 트레이드오프다

최소 지원 버전을 낮추면 더 많은 사용자를 커버하지만, 새 API를 쓰기 어려워집니다. 높이면 최신 기능을 쓸 수 있지만 일부 사용자를 잃습니다.

```
Android 버전별 시장 점유율 (2024 기준):
  API 26+ (Android 8.0+): ~95%의 기기 커버
 API 29+ (Android 10+): ~88%의 기기 커버
 API 33+ (Android 13+): ~60%의 기기 커버
```

SearCam이 API 26을 선택한 이유:

이유	설명
시장 커버리지	95%+ 기기 지원 (보급형 포함)
Wi-Fi 스캔 API	ACCESS_FINE_LOCATION 필수 (API 26+부터 강제)
CameraX 지원	API 21+부터 지원, API 26에서 안정적
Foreground Service	API 26부터 의무화 — 백그라운드 스캔 정책
타겟 사용자	보급형 기기 사용자 포함 (사회적 취약계층 포함)

보안 앱으로서의 의미

를 때 이유가 있어야 한다

몰래카메라 피해자는 비싼 최신 기기를 쓰지 않을 수도 있습니다. SearCam이 보안 도구로서 의미를 갖기 위해서는 보급형 기기에서도 동작해야 합니다. API 26 선택은 기술적 결정인 동시에 제품 철학의 반영입니다.

API 버전별 조건부 코드 처리

```
// API 레벨에 따른 조건부 처리
fun requestNotificationPermission() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAM
        // Android 13+ (API 33): POST_NOTIFICATIONS 런타임
        requestPermissionLauncher.launch(Manifest.permissi
    }
    // Android 12 이하: 권한 불필요 (알림 자동 허용)
}

fun enableHighSamplingRate() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        // Android 12+ (API 31): HIGH_SAMPLING_RATE_SENSOR
        // 20Hz+ 자력계 샘플링을 위해 필요
    }
}
```

3.8 3.7 아키텍처: Clean Architecture를 선택한 이유

3.8.1 아키텍처는 "어떻게"가 아니라 "왜"다

Clean Architecture를 선택한 이유를 한 문장으로: **변화에 유연하게 대응하기 위해서.**

SearCam은 Phase 1에서 Android만 지원하지만, Phase 2에서 iOS로 확장합니다. Phase 3에서 클라우드 백엔드가 추가됩니다. 이 변화가 올 때 코드 전체를 다시 짜지 않으려면 처음부터 계층을 분리해야 합니다.

레이어별 변화 시나리오:

변화	영향 레이어	Clean Architecture 덕분에
Room → SQLite 직접 사용	Data만	Domain, UI 변경 없음
Wi-Fi 스캔 → Bluetooth 추가	Data만	새 Repository 구현 추가
Compose → XML (가정)	UI만	Domain, Data 변경 없음
오프라인 → 클라우드 동기화	Data만	Domain 인터페이스 유지
탐지 알고리즘 개선	Domain + Data	UI 변경 없음

Clean Architecture가 없다면:

```
// 나쁜 예: UI가 DB에 직접 접근
```

를 때 이유가 있어야 한다

```
@Composable
fun ScanResultScreen() {
    val database = AppDatabase.getInstance(LocalContext.cu
    val reports = database.reportDao().getAll() // UI가 Da
    // 나중에 DB를 바꾸면 화면 코드도 바꿔야 함
    // 테스트 시 실제 DB가 필요함
}

// 좋은 예: UI는 ViewModel만 알고, ViewModel은 UseCase만 알고
@Composable
fun ScanResultScreen(viewModel: ScanViewModel = hiltViewMo
    val state by viewModel.uiState.collectAsStateWithLifec
    // DB가 바뀌어도 이 화면은 전혀 변하지 않음
}
```

3.9 3.8 비동기 처리: Coroutines + Flow를 선택한 이유

3.9.1 스마트폰의 메인 스레드는 UI 전용이다

Android의 황금률: 메인 스레드(UI 스레드)에서 블로킹 작업을 하면 ANR(앱 응답 없음) 다이얼로그가 뜹니다.

SearCam에서 메인 스레드에서 하면 안 되는 작업들: - Wi-Fi ARP 테이블 읽기 (파일 I/O) - TCP 포트 스캔 (네트워크) - Room DB 쿼리 (디스크 I/O) - 이미지 프레임 분석 (CPU 집약)

Coroutines가 이것을 우아하게 해결합니다:

```
class RunQuickScanUseCase @Inject constructor(
    private val wifiRepo: WifiScanRepository,
    private val magneticRepo: MagneticRepository,
    private val calculateRisk: CalculateRiskUseCase
) {
    suspend operator fun invoke(): ScanResult = withContex
        // IO 스레드에서 병렬 실행
        val networkDeferred = async { wifiRepo.scan() }
        val emfDeferred = async(Dispatchers.Default) {
            magneticRepo.calibrateAndMeasure(durationSecon
        }
```

를 때 이유가 있어야 한다

```
        val networkResult = networkDeferred.await()
        val emfResult = emfDeferred.await()

        calculateRisk(networkResult, emfResult)
    }
}
```

Flow가 실시간 데이터를 처리합니다:

```
// 자기장 센서 데이터를 20Hz로 Flow 방출
fun observeMagneticField(): Flow<MagneticReading> = callba
    val listener = object : SensorEventListener {
        override fun onSensorChanged(event: SensorEvent) {
            trySend(MagneticReading(event.values[0], event
        }
        override fun onAccuracyChanged(sensor: Sensor, acc
    }

    sensorManager.registerListener(listener, magnetometer,
    awaitClose { sensorManager.unregisterListener(listener
}

// ViewModel에서 Flow를 StateFlow로 변환
```

```
val magneticState = magneticRepo.observeMagneticField()
    .map { reading -> MagneticUiState(reading, riskLevel(r
    .stateIn(viewModelScope, SharingStarted.WhileSubscribe
```

3.10 3.9 기술 스택 최종 결정표

범주	선택	버전	선택 이유
언어	Kotlin	2.0+	Null Safety, Coroutines, Data Class
UI	Jetpack Compose	BOM 2024.x	선언형, 실시간 상태 관리, 미래 표준
DI	Hilt	2.51+	컴파일 타임 검증, Android 공식 지원
로컬 DB	Room	2.6.x	Flow 지원, 타입 안전 쿼리
카메라	CameraX	1.3.x	Lifecycle 자동 관리, 추상화
비동기	Coroutines + Flow	1.8.x	구조적 동시성, 리액티브 스트림

를 때 이유가 있어야 한다

범주	선택	버전	선택 이유
아키텍처	Clean Architecture + MVVM	-	변화 유연성, 테스트 가능성
그래프	MPAndroidChart	3.1.x	실시간 자기장 그래프
로깅	Timber	5.x	릴리즈 빌드 자동 제거
최소 SDK	API 26 (Android 8.0)	-	95%+ 기기 커버, 핵심 API 지원
타겟 SDK	API 34 (Android 14)	-	Play Store 정책, 최신 API

3.11 마무리

기술 선택에는 정답이 없습니다. 하지만 이유 없는 선택은 있습니다. SearCam의 모든 기술 결정에는 "왜?" 가 있었고, 그 이유는 제품의 요구사항에서 출발했습니다.

30초 안에 결과를 보여줘야 하니 비동기 처리가 필수고, 비동기 처리엔 Coroutines가 최선입니다. 실시간 센서 데이터를 표시해야 하니 Compose가 적합합니다. 보안 앱으로서 테스트 가능성이 중요하니 DI와 Clean Architecture가 필요합니다.

다음 장에서는 이 기술 스택으로 실제 시스템 아키텍처를 어떻게 설계했는지 — 패키지 구조, 의존성 방향, 탐지 레이어 설계 — 자세히 살펴봅니다.

를 때 이유가 있어야 한다

제 4 장

Ch04: 시스템 아키텍처 설계 — 층을 나누고 방향을 정한다

이 장에서 배울 것: 아키텍처는 설계도입니다. SearCam의 Clean Architecture 3계층 구조, 탐지 3중 레이어(Wi-Fi 50% + 렌즈 35% + EMF 15%), 의존성 역전 원칙, Hilt DI 설계, 그리고 실제 패키지 구조를 다이어그램과 함께 설명합니다.

4.1 도입

도시를 설계할 때 주거지역, 상업지역, 공업지역을 나눕니다. 공장이 주택가 한가운데 있으면 소음과 오염이 생깁니다. 가게가 없는 주거지역은 불편합니다. 구역을 나누고 도로로 연결하는 것, 그것이 도시 설계의 핵심입니다.

소프트웨어 아키텍처도 같습니다. "어떤 코드가 어디에 있어야 하는가" 를 정하는 것이 아키텍처입니다. SearCam은 Clean Architecture를 채택하여 UI, 비즈니스 로직, 데이터를 엄격하게 분리했습니다.

4.2 4.1 전체 아키텍처 개요

4.2.1 3계층: Presentation → Domain → Data

```
┌──────────────────────────────────────────────────────────────────────────
│          Presentation Layer                    │
│  ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────
│  │HomeScreen│  │ScanScreen│  │LensScreen│  │ReportScr
│  └──────────┬───────────┘  └──────────┬───────────┘  └──────────┬
│     │          │          │          │        │
│  ┌──────▼──────────┐  ┌──────▼──────────┐  ┌──────▼
```

```
│  │HomeVM    │  │ScanVM    │  │LensVM    │  │ReportVM  │   │
│  └──────────┴──────────┘  └──────────┴──────────┘  └──────────┴
├──────────────┼──────────────────────────┼──────────────────────
│    │     Domain Layer        │        │      │
│  ┌─────────▼──────────────────────────▼──────────────────────
│  │         UseCases                        │  │
│  │ RunQuickScan / RunFullScan / CalculateRisk / Export  │
│  └──────────────────────────────────────────────┬──────────────
│                │                     │
│  ┌──────────────────────────────────────────▼──────────────
│  │      Repository Interfaces                 │  │
│  │ WifiScanRepo / MagneticRepo / LensDetectionRepo / ..│
│  └──────────────────────────────────────────────┬──────────────
├──────────────────────────────────────────────────┼──────────
│          Data Layer                          │
│  ┌──────────────────────────────────────────▼──────────────
│  │      Repository Implementations            │  │
│  └────────┬──────────────────────┬──────────────────────┴──
│    │       │       │       │            │
│  ┌─────▼─────┐  ┌─────▼─────┐  ┌─────▼─────┐  ┌──
│  │Sensor │  │Analysis│  │ Local │  │ PDF  │       │
│  │Module │  │Module  │  │ DB    │  │Module │       │
```

```
│   └──────────────────┘   └──────────────────┘   └──────────────────┘   └──
└──────────────────────────────────────────────────────────────────────────
```

핵심 규칙: 의존성은 항상 아래 방향

```
Presentation → Domain ← Data
              (의존성 방향)
```

Presentation은 Domain을 알지만, Domain은 Presentation을 모릅니다. Data는 Domain의 인터페이스를 구현하지만, Domain은 Data의 구체적인 구현을 모릅니다. 이것이 의존성 역전 원칙(DIP)입니다.

4.3 4.2 Presentation 계층 — UI와 상태 관리

4.3.1 단방향 데이터 흐름

Presentation 계층의 규칙은 하나입니다: Screen은 ViewModel만 알고, ViewModel은 UseCase만 압니다.

```
사용자 이벤트 (버튼 탭)
      │ Event
      ▼
ViewModel
```

```
    │ suspend fun / Flow
    ▼
UseCase (Domain)
    │ StateFlow
    ▼
ViewModel
    │ UiState
    ▼
Screen (Composable)
    │ 재구성
  ▼
사용자 화면 업데이트
```

ScanViewModel 예시:

```
@HiltViewModel
class ScanViewModel @Inject constructor(
    private val runQuickScanUseCase: RunQuickScanUseCase,
    private val runFullScanUseCase: RunFullScanUseCase
) : ViewModel() {

    // UI 상태: sealed class로 모든 가능한 상태를 명시
    private val _uiState = MutableStateFlow<ScanUiState>(S
    val uiState: StateFlow<ScanUiState> = _uiState.asState
```

```
    fun startQuickScan() {
        viewModelScope.launch {
            _uiState.value = ScanUiState.Scanning(progress
            try {
                val result = runQuickScanUseCase()
                _uiState.value = ScanUiState.Complete(resu
            } catch (e: ScanException) {
                _uiState.value = ScanUiState.Error(e.messa
            }
        }
    }
}

// UI 상태 정의: 모든 화면 상태를 열거
sealed class ScanUiState {
    object Idle : ScanUiState()
    data class Scanning(val progress: Float, val currentLa
    data class Complete(val result: ScanResult) : ScanUiSt
    data class Error(val message: String) : ScanUiState()
}
```

Screen 예시:

```
@Composable
fun QuickScanScreen(
    viewModel: ScanViewModel = hiltViewModel(),
    onNavigateToResult: (ScanResult) -> Unit
) {
    val uiState by viewModel.uiState.collectAsStateWithLif

    // 화면은 상태에 따라 다른 UI를 보여줌 — 분기가 명확하고 단순
    when (val state = uiState) {
        is ScanUiState.Idle ->
            IdleContent(onScanClick = viewModel::startQuic

        is ScanUiState.Scanning ->
            ScanningContent(
                progress = state.progress,
                currentLayer = state.currentLayer
            )

        is ScanUiState.Complete -> {
            LaunchedEffect(state.result) {
                onNavigateToResult(state.result)
            }
```

```
        }

        is ScanUiState.Error ->
            ErrorContent(message = state.message, onRetry
    }
}
```

4.3.2 Presentation 계층 구성요소

구성요소	책임	의존 대상
Screen (Composable)	UI 렌더링, 사용자 입력 수신	ViewModel
ViewModel	UI 상태 관리, UseCase 호출, Lifecycle 처리	UseCase (Domain)
Navigation	화면 전환 로직	NavHost

구성요소	책임	의존 대상
Components	재사용 UI 컴포넌트 (RiskGauge, RiskBadge)	없음

4.4 4.3 Domain 계층 — 비즈니스 로직의 집

4.4.1 Domain은 Android를 모른다

Domain 계층의 가장 중요한 규칙: Android 프레임워크 import가 없어야 합니다.

```
// 올바른 Domain 모델: 순수 Kotlin
data class ScanResult(
    val id: String,
    val scanTime: Long,
    val networkLayer: NetworkLayerResult?,
    val lensLayer: LensLayerResult?,
    val emfLayer: EmfLayerResult?,
    val overallRisk: RiskLevel,
```

```
    val riskScore: Int,             // 0~100
    val crossValidationNote: String
) {
    // 비즈니스 로직을 모델에 캡슐화
    fun isHighRisk(): Boolean = riskScore >= 70
    fun hasMultipleLayerHits(): Boolean {
        var hits = 0
        if (networkLayer?.hasThreats == true) hits++
        if (lensLayer?.hasThreats == true) hits++
        if (emfLayer?.hasThreats == true) hits++
        return hits >= 2
    }
}

// Android import가 없음 — 순수 Kotlin/Java 환경에서도 동작
enum class RiskLevel(val score: Int, val label: String) {
    SAFE(0, "안전"),
    LOW(20, "낮음"),
    MEDIUM(50, "주의"),
    HIGH(70, "위험"),
    CRITICAL(90, "강력 의심")
}
```

Repository 인터페이스:

```
// Domain이 정의한 계약 — Data 계층이 이것을 구현해야 함
interface WifiScanRepository {
    suspend fun scan(): NetworkLayerResult
    fun isConnectedToWifi(): Boolean
}

interface MagneticRepository {
    fun isAvailable(): Boolean
    suspend fun calibrate(): MagneticBaseline
    fun observeField(): Flow<MagneticReading>
}

interface LensDetectionRepository {
    suspend fun startDetection(): Flow<List<LensPoint>>
    suspend fun stopDetection()
}

interface ReportRepository {
    suspend fun save(result: ScanResult): Long
    fun getRecent(limit: Int): Flow<List<ScanResult>>
    fun getByRiskLevel(level: RiskLevel): Flow<List<ScanRe
```

```
    suspend fun delete(id: String)
}
```

UseCase:

```
// 단일 책임: Quick Scan 오케스트레이션만 담당
class RunQuickScanUseCase @Inject constructor(
    private val wifiRepo: WifiScanRepository,
    private val emfRepo: MagneticRepository,
    private val calculateRisk: CalculateRiskUseCase,
    private val reportRepo: ReportRepository
) {
    suspend operator fun invoke(): ScanResult {
        // Wi-Fi 연결 여부에 따라 레이어 동적 활성화
        val networkResult = if (wifiRepo.isConnectedToWifi
            wifiRepo.scan()
        } else {
            null  // Layer 1 비활성화
        }

        val emfResult = if (emfRepo.isAvailable()) {
            val baseline = emfRepo.calibrate()
            emfRepo.observeField()
                .take(60)            // 3초간 (20Hz × 3초 =
```

```
                .toList()
                .let { readings -> EmfLayerResult(readings
        } else {
            null  // Layer 3 비활성화
        }

        val result = calculateRisk(networkResult, null, em
        reportRepo.save(result)
        return result
    }
}
```

4.5 4.4 탐지 3중 레이어 — 핵심 알고리즘 설계

4.5.1 단일 센서의 한계: 왜 3중 레이어인가

의사의 진단과 비슷합니다. 체온만 재서 진단하는 의사와, 체온 + 혈압 + 혈액 검사를 종합하는 의사 중 어느 쪽을 신뢰하겠습니까?

SearCam은 3가지 독립적인 방법으로 몰래카메라를 탐지하고, 그 결과를 교차 검증합니다.

탐지 3중 레이어 구조:

[Layer 1: Wi-Fi 스캔] 가중치: 50%

ARP 조회 → mDNS → SSDP → OUI 매칭 → 포트 스캔

탐지 대상: 동일 네트워크에 연결된 카메라

[Layer 2: 렌즈 감지] 가중치: 35%

플래시 ON → 프레임 캡처 → 고휘도 추출 → 원형도 검사

→ 플래시 토글 동적 검증

탐지 대상: 렌즈가 노출된 카메라 (전원 무관)

[Layer 3: EMF 감지] 가중치: 15%

캘리브레이션 → 3축 자기장 측정 → 노이즈 필터

→ baseline 대비 변화량 계산

탐지 대상: 전원 공급 중인 전자기기

```
  │          │          │
  ▼          ▼          ▼
┌──────────────────────────────────────────────────────────────
│       교차 검증 엔진 (CrossValidator)    │
│                                          │
```

```
│ 단일 양성: 개별 가중치 적용          │
│ 복수 양성: 보정 계수 상향 적용       │
│ EMF 단독: "가전제품 가능성" 안내     │
└──────────────────────────────────────┘
                │
                ▼
          종합 위험도 0~100
```

4.5.2 가중치 설계 이유

레이어	가중치	이유
Layer 1 (Wi-Fi)	50%	가장 정확. MAC OUI + 포트로 카메라 제조사 확인 가능
Layer 2 (렌즈)	35%	전원 OFF 카메라도 탐지. 단, 환경광/거울에 오탐 가능

레이어	가중치	이유
Layer 3 (EMF)	15%	보조 역할. 가전 제품과 구별 어려움 — 단독 신뢰도 낮음

오프라인 모드의 가중치 재배분:

```
// 사용 불가 레이어는 가중치를 다른 레이어로 재배분
data class LayerWeights(
    val wifi: Float,
    val lens: Float,
    val emf: Float
) {
    companion object {
        // 풀 기능
        val FULL = LayerWeights(0.50f, 0.35f, 0.15f)
        // Wi-Fi 없음
        val NO_WIFI = LayerWeights(0f, 0.75f, 0.25f)
        // Wi-Fi + EMF 없음 (렌즈만)
        val LENS_ONLY = LayerWeights(0f, 1.0f, 0f)
        // Wi-Fi만 (렌즈/EMF 없음)
```

```
        val WIFI_ONLY = LayerWeights(1.0f, 0f, 0f)


        fun calculate(
            hasWifi: Boolean,
            hasLens: Boolean,
            hasEmf: Boolean
        ): LayerWeights {
            val base = listOf(
                hasWifi to 0.50f,
                hasLens to 0.35f,
                hasEmf to 0.15f
            )
            val total = base.filter { it.first }.sumOf { i
            return LayerWeights(
                wifi  = if (hasWifi)  0.50f / total else 0
                lens  = if (hasLens)  0.35f / total else 0
                emf   = if (hasEmf)   0.15f / total else 0
            )
        }
    }
}
```

4.5.3 교차 검증 알고리즘

```
class CrossValidator @Inject constructor() {

    fun validate(
        networkResult: NetworkLayerResult?,
        lensResult: LensLayerResult?,
        emfResult: EmfLayerResult?,
        weights: LayerWeights
    ): CrossValidationResult {

        // 각 레이어의 위험 점수 계산 (0~100)
        val wifiScore  = networkResult?.riskScore ?: 0
        val lensScore  = lensResult?.riskScore ?: 0
        val emfScore   = emfResult?.riskScore ?: 0

        // 가중치 적용 기본 점수
        val baseScore = (wifiScore * weights.wifi +
                         lensScore * weights.lens +
                         emfScore  * weights.emf).toInt()

        // 교차 검증 보정: 복수 레이어 양성 시 신뢰도 상향
        val activeHits = listOf(
```

```
            networkResult?.hasThreats == true,
            lensResult?.hasThreats == true,
            emfResult?.hasThreats == true
        ).count { it }

        val correctedScore = when (activeHits) {
            0 -> baseScore
            1 -> baseScore                                  // 단일
            2 -> minOf(100, (baseScore * 1.3f).toInt())   /
            else -> minOf(100, (baseScore * 1.5f).toInt()))
        }

        // EMF 단독 양성: 낮은 신뢰도 안내
        val note = when {
            activeHits == 1 && emfResult?.hasThreats == tr
                "자기장 변화가 감지되었습니다. 주변 가전제품이니
            activeHits >= 2 ->
                "복수의 탐지 방식에서 이상이 확인되어 신뢰도가 높
            else -> ""
        }

        return CrossValidationResult(
```

```
            finalScore = correctedScore,
            riskLevel = RiskLevel.fromScore(correctedScore
            activeLayerCount = activeHits,
            note = note
        )
    }
}
```

4.6 4.5 의존성 역전 원칙 — Domain이 Data를 모르는 이유

4.6.1 인터페이스가 방향을 역전시킨다

일반적인 코드에서 상위 레이어가 하위 레이어를 직접 사용합니다.
그러면 상위 레이어가 하위 레이어의 변화에 종속됩니다.

일반적인 방향:

```
ScanViewModel → WifiScanRepositoryImpl → WifiScanner → An
```

문제: `WifiScanner`를 바꾸면 `RepositoryImpl`도 바꿔야 함.
테스트할 때 실제 `Wi-Fi` 하드웨어가 필요함.

의존성 역전(DIP)은 인터페이스로 이 방향을 뒤집습니다.

```
DIP 적용 후:

  ScanViewModel → RunQuickScanUseCase (Domain)
                                ↓ (인터페이스 사용)
            WifiScanRepository (interface, Domain)
                                ↑ (구현체 제공)
               WifiScanRepositoryImpl (Data)
                                ↓
          WifiScanner → Android WifiManager
```

Domain은 WifiScanner의 존재조차 모름.

테스트 시 FakeWifiScanRepository를 주입하면 됨.

테스트에서의 효과:

```
// 실제 앱: Hilt가 실제 구현체를 주입
@Binds
abstract fun bindWifiRepo(impl: WifiScanRepositoryImpl): W

// 테스트: Fake 구현체를 주입 — Wi-Fi 하드웨어 없이 테스트 가능
class FakeWifiScanRepository : WifiScanRepository {
    var shouldReturnHighRisk = false

    override suspend fun scan(): NetworkLayerResult {
        return if (shouldReturnHighRisk) {
```

```
            NetworkLayerResult.createHighRisk(listOf(
                NetworkDevice(mac = "28:57:BE:FF:01:02", m
            ))
        } else {
            NetworkLayerResult.createSafe()
        }
    }

    override fun isConnectedToWifi(): Boolean = true
}

// UseCase 단위 테스트: 실제 Wi-Fi 없이 로직 검증
@Test
fun `should return high risk when network layer finds came
    val fakeWifiRepo = FakeWifiScanRepository().apply { sh
    val fakeEmfRepo = FakeMagneticRepository()

    val useCase = RunQuickScanUseCase(fakeWifiRepo, fakeEm

    val result = useCase()

    assert(result.isHighRisk())
```

```
    assert(result.riskScore >= 70)
}
```

4.7 4.6 Hilt DI 설계 — 의존성의 지도

4.7.1 DI 모듈 구조

SearCam의 DI 모듈은 책임별로 분리됩니다.

```
di/
├── AppModule.kt        # 앱 전역 (Context, SharedPreference
├── SensorModule.kt      # 센서 제공 (WifiScanner, MagneticS
├── DatabaseModule.kt      # Room DB + DAO
├── AnalysisModule.kt    # CrossValidator, RiskCalculator
└── RepositoryModule.kt  # 인터페이스 ↔ 구현체 바인딩
```

Scope 설계:

컴포넌트	Scope	이유
WifiScanner	Singleton	하나의 인스턴스로 충분
MagneticSensor	Singleton	SensorManager 공유
LensDetector	Singleton	CameraX ProcessCameraProvider 공유
ScanViewModel	ViewModel	Compose의 HiltViewModel 사용

컴포넌트	Scope	이유
RunQuickScanUseCase	Singleton	상태 없음, 재사용 가능
AppDatabase	Singleton	DB 연결은 하나여야 함

```
@Module
@InstallIn(SingletonComponent::class)
object SensorModule {

    @Provides
    @Singleton
    fun provideWifiScanner(
        @ApplicationContext context: Context
    ): WifiScanner = WifiScanner(
        context = context,
        wifiManager = context.getSystemService(WifiManager
        nsdManager = context.getSystemService(NsdManager::
    )

    @Provides
    @Singleton
    fun provideMagneticSensor(
        @ApplicationContext context: Context
```

```
    ): MagneticSensor {
        val sensorManager = context.getSystemService(Sensc
        val magnetometer = sensorManager.getDefaultSensor(
        // 자력계 없는 기기 처리
        return MagneticSensor(sensorManager, magnetometer)
    }
}

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    abstract fun bindWifiScanRepository(
        impl: WifiScanRepositoryImpl
    ): WifiScanRepository

    @Binds
    abstract fun bindMagneticRepository(
        impl: MagneticRepositoryImpl
    ): MagneticRepository
```

```
    @Binds
    abstract fun bindLensDetectionRepository(
        impl: LensDetectionRepositoryImpl
    ): LensDetectionRepository

    @Binds
    abstract fun bindReportRepository(
        impl: ReportRepositoryImpl
    ): ReportRepository
}
```

4.8 4.7 패키지 구조 — 파일의 주소

4.8.1 기능별 vs 레이어별 패키징

두 가지 패키징 전략이 있습니다.

레이어별 패키징 (Layer-first):

```
ui/
  ScanScreen.kt
  LensScreen.kt
```

기능별 패키징 (Feature-first):

```
scan/
  ScanScreen.kt
  ScanViewModel.kt
```

```
domain/                          ScanUseCase.kt
  ScanUseCase.kt                         lens/
 LensUseCase.kt                    LensScreen.kt
data/                            LensViewModel.kt
 WifiScanner.kt                    LensUseCase.kt
  LensDetector.kt
```

SearCam은 **레이어별 패키징**을 선택했습니다. Clean Architecture의 레이어 경계를 패키지로 명확히 표현하기 때문입니다.

전체 패키지 구조:

```
com.searcam/
│
├── SearCamApp.kt              # @HiltAndroidApp
├── MainActivity.kt             # Single Activity, NavHost
│
├── di/                         # Hilt DI 모듈
│   ├── AppModule.kt
│   ├── SensorModule.kt
│   ├── DatabaseModule.kt
│   ├── AnalysisModule.kt
│   └── RepositoryModule.kt
│
```

```
├── domain/                    # 순수 Kotlin (Android 없음)
│   ├── model/
│   │   ├── ScanResult.kt
│   │   ├── RiskLevel.kt
│   │   ├── NetworkDevice.kt
│   │   ├── MagneticReading.kt
│   │   ├── LensPoint.kt
│   │   └── ScanReport.kt
│   ├── usecase/
│   │   ├── RunQuickScanUseCase.kt
│   │   ├── RunFullScanUseCase.kt
│   │   ├── RunLensFinderUseCase.kt
│   │   ├── CalculateRiskUseCase.kt
│   │   └── ExportReportUseCase.kt
│   └── repository/
│       ├── WifiScanRepository.kt        # 인터페이스만
│       ├── MagneticRepository.kt
│       ├── LensDetectionRepository.kt
│       └── ReportRepository.kt
│
├── data/                      # Android SDK 의
```

```
존
│   ├── sensor/
│   │   ├── WifiScanner.kt
│   │   ├── MagneticSensor.kt
│   │   ├── LensDetector.kt
│   │   └── IrDetector.kt
│   ├── analysis/
│   │   ├── OuiDatabase.kt
│   │   ├── CrossValidator.kt
│   │   ├── RiskCalculator.kt
│   │   └── NoiseFilter.kt
│   ├── repository/
│   │   ├── WifiScanRepositoryImpl.kt
│   │   ├── MagneticRepositoryImpl.kt
│   │   ├── LensDetectionRepositoryImpl.kt
│   │   └── ReportRepositoryImpl.kt
│   ├── local/
│   │   ├── AppDatabase.kt
│   │   ├── dao/
│   │   │   ├── ReportDao.kt
│   │   │   └── DeviceDao.kt
│   │   └── entity/
```

```
│   │     ├──── ScanReportEntity.kt
│   │     └──── DeviceEntity.kt
│   └──── pdf/
│       └──── PdfGenerator.kt
│
├──── ui/                          # Presentation 레이어
│   ├──── navigation/
│   │   ├──── SearCamNavHost.kt
│   │   └──── Screen.kt
│   ├──── home/
│   │   ├──── HomeScreen.kt
│   │   └──── HomeViewModel.kt
│   ├──── scan/
│   │   ├──── QuickScanScreen.kt
│   │   ├──── FullScanScreen.kt
│   │   ├──── ScanResultScreen.kt
│   │   └──── ScanViewModel.kt
│   ├──── lens/
│   │   ├──── LensFinderScreen.kt
│   │   └──── LensViewModel.kt
│   ├──── magnetic/
```

```
│   │   ├── MagneticScreen.kt
│   │   └── MagneticViewModel.kt
│   ├── report/
│   │   ├── ReportListScreen.kt
│   │   └── ReportViewModel.kt
│   ├── theme/
│   │   ├── Color.kt
│   │   ├── Type.kt
│   │   └── Theme.kt
│   └── components/
│       ├── RiskGauge.kt
│       ├── RiskBadge.kt
│       └── ScanProgress.kt
│
└── util/
    ├── SoundManager.kt
    ├── VibrationManager.kt
    └── PermissionHelper.kt
```

CHAPTER 4. CLEAN ARCHITECTURE 시스템의 아키텍처 설계 — 층을 나누고 방향을 정한다

4.9 4.8 모듈 의존성 그래프

4.9.1 어떤 모듈이 누구를 아는가

```
                    ┌──────────────────┐
          │ :app    │ (Application)
                    └──────────┬───────┘
                               │
     ┌─────────────────────────┼──────────────────────────┐
       │         │         │
 ┌──────────▼──────────┐  ┌──────────▼──────────┐  ┌──────▼──────────
 │:ui-scan  │  │:ui-     │  │:ui-      │
 │      │  │report   │  │settings  │
 └──────────┬──────────┘  └──────────┬──────────┘  └──────────┬──────────
       │         │         │
     └─────────────────────────┬──────────────────────────┘
                               │
            ┌──────────▼──────────┐
            │ :domain    │ (순수 Kotlin)
                    └──────────┬──────────┘
                               │ (인터페이스 구현)
                    ┌──────────▼──────────┐
            │ :data      │
```

```
                  └──────┴──────┴──────┴──────┘
                     │     │     │
       ┌─────────────────────────┘     │     └──────────────────────────┐
       ▼                               ▼                                ▼
  ┌──────────────────────────┐   ┌──────────────────┐   ┌──────────────
  │ :sensor                  │   │ :analysis        │   │ :local-db
  │ (WifiScanner             │   │ (OUI DB,         │   │ (Room, DAO,
  │  CameraX ..)             │   │  CrossVal        │   │  Entity)
  └──────────────────────────┘   └──────────────────┘   └──────────────
```

의존성 규칙:

모듈	알아야 하는 것	알면 안 되는 것
:domain	없음 (순수 정책 레이어)	모든 구현 세부사항
:data	:domain (인터페이스만)	:ui, :app
:ui-scan	:domain (UseCase, Model)	:data 구현체
:app	모든 모듈 (조립자)	-

4.10 4.9 데이터 흐름: 스캔 시작부터 결과까지

사용자가 “Quick Scan” 버튼을 탭했을 때 무슨 일이 일어나는지 추적합니다.

```
1. 사용자: "Quick Scan" 버튼 탭
      │
2. Screen: viewModel.startQuickScan() 호출
      │
3. ViewModel: viewModelScope.launch {
                _uiState = Scanning
                result = runQuickScanUseCase()
              }
            │
4. UseCase: val networkResult = wifiRepo.scan()  // 병렬
     val emfResult = emfRepo.measure()     // 병렬
      │
5. WifiScanner:
   a. /proc/net/arp 읽기 → IP-MAC 테이블
  b. mDNS NsdManager → 서비스 목록
  c. SSDP M-SEARCH → UPnP 기기
  d. OUI DB 매칭 → 제조사 확인
  e. 의심 기기만 TCP 포트 스캔 (554, 8080, 8888)
   f. NetworkLayerResult 반환
      │
```

```
6. MagneticSensor:
   a. 3초 캘리브레이션 (60 샘플)
   b. baseline + noise_floor 계산
  c. 3축 자기장 관찰 (20Hz)
   d. EmfLayerResult 반환
       |
7. CalculateRiskUseCase:
   a. 가중치 계산 (Wi-Fi 있으면 50%, 없으면 0%)
   b. 각 레이어 점수 × 가중치
  c. 교차 검증 보정 (복수 양성 시 상향)
   d. ScanResult 생성
       |
8. ReportRepository:
   a. ScanResult → ScanReportEntity 변환
  b. Room DB에 저장
       |
9. UseCase: ScanResult 반환
       |
10. ViewModel: _uiState = Complete(result)
            |
11. Screen: Result 화면으로 네비게이션
       |
```

12. 사용자: 위험도, 발견 기기 목록, 근거 확인

전체 과정이 30초 이내에 완료됩니다.

4.11 4.10 아키텍처 체크리스트

새 기능을 추가하기 전에 확인합니다.

- ☐ 새 기능이 어느 레이어에 속하는가?
- ☐ Domain 레이어에 Android import를 추가하려 한다면 — 멈추고 재설계
- ☐ UI가 Repository 구현체를 직접 참조한다면 — 멈추고 재설계
- ☐ UseCase가 10개 이상의 의존성을 갖는다면 — 분리 검토
- ☐ 테스트 시 실제 하드웨어(Wi-Fi, 카메라)가 필요하다면 — Fake 인터페이스 도입
- ☐ 새 Repository를 추가했다면 — DI 모듈에 바인딩 추가

4.12 마무리

아키텍처는 완성하는 것이 아니라 지키는 것입니다. SearCam의 Clean Architecture 구조는 코드를 한 번 짜고 끝나는 것이 아니라, 새 기능을 추가할 때마다 "이 코드가 올바른 레이어에 있는가?" 를 자문하는 습관입니다.

3계층 분리, 의존성 역전, 탐지 레이어 가중치 설계 — 이 모든 결정이 결국 하나의 목표를 향합니다: **30초 안에 믿을 수 있는 결과를 보여주는 앱**.

다음 장에서는 이 아키텍처 위에 실제 코드를 쌓기 시작합니다. 프로젝트 초기 설정 — Gradle Version Catalog, Hilt Application, 권한 전략 — 부터 시작합니다.

CHAPTER 4. 시스템 아키텍처 설계 — 층을 나누고 방향을 정한다

CHAPTER 4. 시스템 아키텍처 설계 — 층을 나누고 방향을 정한다

제 5 장

Ch05: UI/UX 설계 — Jetpack Compose로 불안을 안심으로

이 장에서 배울 것: 선언형 UI 패러다임이 왜 불안을 다루는 앱에 적합한지, 위험도 0~100을 어떻게 색상과 애니메이션으로 시각화하는지, 30초 스캔이라는 제약이 어떻게 UX 구조를 결정했는지를 배웁니다.

5.1 도입

응급실 의사는 환자를 보자마자 "생존 가능성 72%" 라는 숫자를 머릿속에 떠올리지 않습니다. 그 대신 환자의 안색, 호흡, 땀 여부를 한눈에 읽어냅니다. 이것이 좋은 UI가 해야 할 일입니다. 숫자를 즉각적인 감각으로 변환하는 것.

SearCam은 숙소, 화장실, 탈의실에서 불안을 느끼는 사람이 30초 안에 "여기는 괜찮다" 또는 "여기는 의심스럽다" 는 판단을 내릴 수 있도록 설계되었습니다. 이 목표가 모든 UI/UX 결정의 출발점입니다.

이 장에서는 Jetpack Compose의 선언형 패러다임이 이 목표를 어떻게 달성하는지, 그리고 위험도 시각화부터 30초 UX 흐름까지 설계 결정의 근거를 공개합니다.

5.2 5.1 왜 Jetpack Compose인가

5.2.1 UI를 "어떻게" 그릴지가 아닌 "무엇을" 그릴지로

전통적인 View 시스템에서 개발자는 UI를 "조종" 했습니다. 버튼을 숨기려면 `button.visibility = GONE`, 텍스트를 바꾸려면 `textView.text = newText`. 상태가 5개만 늘어나도 코드

는 스파게티가 됩니다.

Compose는 발상을 뒤집습니다. "현재 상태가 이것이라면, UI는 이렇게 생겨야 한다" 는 방식으로 씁니다.

```
[기존 View 방식]                          [Compose 방식]
상태 변경 → 뷰 참조 → 수동 업데이트   상태 변경 → UI 자동 재구성
button.isVisible = scanning          if (scanning) ScanningBut
textView.text = count               else ReadyButton()
progressBar.progress = percent       Text("$count 기기 발견")
progressBar.isVisible = scanning     LinearProgressIndicator
```

SearCam에서 스캔 화면의 상태는 최소 6가지입니다: 대기, 스캔 중, 일시정지, 완료, 오류, 취소. Compose 없이 이 6가지 상태를 View로 관리하면 상태 전환 버그가 필연적으로 발생합니다.

5.2.2 선언형 UI가 불안을 다루는 앱에 적합한 이유

불안한 사용자는 빠른 피드백을 원합니다. "내가 버튼을 눌렀는데 왜 아무것도 안 바뀌지?" 라는 의문이 드는 순간 불안은 배가됩니다.

Compose의 `recomposition`은 상태가 바뀌면 즉시 UI가 다시 그려짐을 보장합니다. 스캔 진행률이 43%에서 44%로 바뀌는 순간, 게이지도 함께 움직입니다. 개발자가 `postInvalidate()`를 호출할 필요가 없습니다.

```
// ScanProgressState가 바뀌면 이 컴포저블 전체가 자동으로 재구성됨
@Composable
fun ScanProgressScreen(state: ScanProgressState) {
    Column {
        CircularProgressIndicator(progress = { state.progr
        Text("${state.devicesFound}대 발견")
        Text("남은 시간: ${state.remainingSeconds}초")
        state.layers.forEach { layer ->
            LayerStatusRow(layer)
        }
    }
}
```

5.3 5.2 4대 디자인 원칙과 그 이유

좋은 디자인은 원칙에서 나옵니다. SearCam의 4대 원칙은 개발 내내 모든 결정의 기준이 됩니다.

```
┌──────────────────────────────────────────────────────────────
│          SearCam 4대 디자인 원칙              │
├──────────────────────────────┼──────────────────────────────
│ 30초 스캔     │ 버튼 1번으로 결과까지              │
```

```
│ 원클릭        │ 스캔 시작까지 최대 2탭              │
│ 공포 방지     │ "위험" 대신 "확인 필요", 근거 제공  │
│ 거짓 안심 방지│ 한계 고지 상시 표시                 │
└──────────────┴──────────────────────────────────────
```

5.3.1 원칙 1: 30초 스캔

스캔이 30초라는 숫자는 임의로 정한 것이 아닙니다. 사용자 리서치 결과, 사람이 “기다려줄 수 있는” 한계가 30초임을 확인했습니다. 30초를 넘어가면 “이게 작동하는 건지” 의심하기 시작합니다.

이 제약이 기술 설계를 결정했습니다. Wi-Fi 스캔(10초) + 기기 분석(15초) + 결과 정리(5초)라는 타임박스가 생겼고, 각 레이어는 병렬로 실행됩니다.

5.3.2 원칙 2: 공포 방지

"위험" 한 단어는 사람을 얼어붙게 만듭니다. SearCam의 모든 결과 표현은 “행동 가능한 정보” 를 함께 제공합니다.

기존 앱 표현	SearCam 표현
위험!	확인이 필요합니다 (67점)

기존 앱 표현	SearCam 표현
카메라 발견	Wi-Fi에 연결된 기기 중 카메라 가능성이 있는 기기가 발견되었습니다
안전	이번 스캔에서 이상 징후를 발견하지 못했습니다

5.3.3 원칙 3: 거짓 안심 방지

"탐지되지 않음 = 안전" 이라는 오해가 가장 위험합니다. SearCam은 결과 화면 최하단에 항상 이 문구를 표시합니다:

> "이 결과는 참고용입니다. 전원이 꺼진 카메라나 LTE/5G 카메라는 탐지할 수 없습니다. 의심스러우면 112에 신고하세요."

5.4 5.3 화면 구조와 네비게이션

SearCam은 18개 화면으로 구성됩니다. 핵심 경로는 단 4단계입니다.

앱 실행

```
│
├── [최초] → 온보딩 3단계 → 권한 요청 → 홈
│
└── [재실행] ──────────────────────────────────────────────────
                          │
                   Quick Scan 탭
                          │
                     스캔 진행 (30초)
                                          │
                     결과 화면
                          │
                     리포트 저장
```

5.4.1 Bottom Navigation 설계

4개 탭으로 구성된 Bottom Navigation은 사용자가 어느 화면에서든 주요 기능에 1탭으로 접근할 수 있게 합니다.

```
┌──────────────────────────────────────────────────────────────
│  [홈]      [리포트]    [체크리스트]    [설정]     │
│   홈       리포트      체크          설정    │
│ (활성)    (비활성)    (비활성)      (비활성)    │
└──────────────────────────────────────────────────────────────
```

활성 탭은 Primary 색상(#2563EB)으로 강조, 비활성 탭은 Gray-400으로 표시합니다. 라벨은 활성 탭에만 표시해서 시각적 노이즈를 줄입니다.

5.5 5.4 색상 코드 시스템 — 위험도를 색으로 말하다

교통신호는 세계 어디서나 같은 의미입니다. 초록이면 가도 됩니다. SearCam의 색상 시스템은 이 보편적 직관을 빌려옵니다.

5.5.1 5단계 위험도 색상 스펙트럼

등급	점수 범위	색상 코드	의미	사용자 행동
안전	0~19	#22C55E (Green)	이상 징후 없음	안심하고 사용
관심	20~39	#84CC16 (Lime)	주의 관찰 권장	육안 확인 권장
주의	40~59	#EAB308 (Yellow)	의심 기기 존재	Full Scan 권장

등급	점수 범위	색상 코드	의미	사용자 행동
위험	60~79	#F97316 (Orange)	카메라 가능성 높음	즉시 점검
매우 위험	80~100	#EF4444 (Red)	카메라 강력 의심	112 신고 고려

이 5단계는 의도적으로 "안전"과 "위험" 사이에 3단계의 중간 지대를 둡니다. 이진법(안전/위험)이 아닌 스펙트럼으로 표현하면 오탐으로 인한 공포를 줄일 수 있습니다.

5.5.2 브랜드 색상과의 분리

위험도 색상과 브랜드 색상은 명확히 분리됩니다.

브랜드 색상 (기능용)

```
Primary: #2563EB (Blue-600) — 버튼, 활성 탭, CTA
Surface: #1F2937 (Gray-800) — 카드 배경 (다크 모드)
```

위험도 색상 (의미용)

```
위험도 게이지, 배지, 결과 배경에만 사용
브랜드 Blue와 절대 혼용하지 않음
```

이 분리가 없으면 사용자는 "파란색 = 안전한가? 파란색 = 위험한가?" 를 혼동합니다.

5.5.3 게이지 그라데이션 구현

원형 게이지에서 0~100 점수는 선형 보간으로 색상이 연속적으로 변합니다.

```
fun riskColor(score: Int): Color {
    return when {
        score < 20  -> Color(0xFF22C55E)  // 안전: Green
        score < 40  -> Color(0xFF84CC16)  // 관심: Lime
        score < 60  -> Color(0xFFEAB308)  // 주의: Yellow
        score < 80  -> Color(0xFFF97316)  // 위험: Orange
        else        -> Color(0xFFEF4444)  // 매우 위험: Red
    }
}

@Composable
fun RiskGauge(score: Int) {
    val color = riskColor(score)
    val animatedProgress by animateFloatAsState(
        targetValue = score / 100f,
        animationSpec = tween(durationMillis = 1000, easin
```

```
        label = "riskGaugeProgress"
    )

    Box(contentAlignment = Alignment.Center) {
        CircularProgressIndicator(
            progress = { animatedProgress },
            color = color,
            strokeWidth = 12.dp,
            modifier = Modifier.size(120.dp)
        )
        Column(horizontalAlignment = Alignment.CenterHoriz
            Text(
                text = "$score",
                style = MaterialTheme.typography.displayLa
                color = color
            )
            Text(
                text = riskLabel(score),
                style = MaterialTheme.typography.bodySmall
            )
        }
    }
```

```
}
```

`animateFloatAsState`를 사용하면 스캔이 완료되어 점수가 표시될 때 게이지가 자연스럽게 채워지는 애니메이션이 자동으로 동작합니다. 1초에 걸쳐 0에서 최종 점수까지 부드럽게 증가합니다.

5.6 5.5 홈 화면 — 원클릭의 실현

홈 화면은 SearCam의 얼굴입니다. 처음 앱을 열었을 때 사용자가 보는 화면이 바로 이것입니다.

```
┌──────────────────────────────────────────────────────┐
│ SearCam        [알림] │ ← AppBar (단순, 최소)
│                │
│   ┌──────────────────────────────┐  │
│   │           │  │
│  │ ◎ Quick Scan  │ │ ← 메인 CTA: 120dp 원형 버튼
│   │  30초 점검    │ │   레이더 펄스 애니메이션
│   │           │  │
│   └──────────────────────────────┘  │
│                │
│  ┌──────────┐ ┌──────────┐   │ ← 서브 모드: 80×80dp
```

```
│  │ Full  │ │ 렌즈 │    │
│  │ Scan  │ │ 찾기 │    │
│  └──────────┘ └──────────┘    │
│  ┌──────────┐ ┌──────────┐    │
│  │  IR   │ │ EMF   │    │
│  │ Only  │ │ Only  │    │
│  └──────────┘ └──────────┘    │
│              │
│ ─────── 마지막 스캔 결과 ─── │  ← 컨텍스트 카드
│  ┌──────────────────────────────────┐  │
│  │ 4/3 14:32  Quick     │ │
│  │ 안전 12/100       →  │ │
│  └──────────────────────────────────┘  │
│              │
├──────────────────────────────────────────┤
│ [홈] [리포트] [체크] [설정] │
└──────────────────────────────────────────┘
```

5.6.1 Quick Scan 버튼의 물리적 크기

120dp 원형 버튼은 한 손으로 쥔 상태에서 엄지로 편안하게 닿을 수 있는 크기입니다. Android의 최소 터치 타깃 권장 크기는 48dp이지만, SearCam의 메인 CTA는 이보다 2.5배 큽니다. 이는 “빠르

게 찾아서 누르는" 행동 패턴을 지원하기 위함입니다.

5.6.2 레이더 펄스 애니메이션 (Idle 상태)

스캔을 시작하지 않은 상태에서도 버튼은 "살아있음" 을 표현합니다. 3초 주기로 원형 파동이 확장되며 사라집니다.

```
@Composable
fun QuickScanButton(onClick: () -> Unit) {
    val infiniteTransition = rememberInfiniteTransition(la

    val pulseAlpha by infiniteTransition.animateFloat(
        initialValue = 0.3f,
        targetValue = 0f,
        animationSpec = infiniteRepeatable(
            animation = tween(durationMillis = 3000, easin
            repeatMode = RepeatMode.Restart
        ),
        label = "pulseAlpha"
    )
    val pulseScale by infiniteTransition.animateFloat(
        initialValue = 1f,
        targetValue = 1.6f,
        animationSpec = infiniteRepeatable(
```

```
            animation = tween(durationMillis = 3000, easin
            repeatMode = RepeatMode.Restart
        ),
        label = "pulseScale"
    )

    Box(contentAlignment = Alignment.Center) {
        // 펄스 원
        Box(
            modifier = Modifier
                .size(120.dp)
                .scale(pulseScale)
                .clip(CircleShape)
                .background(MaterialTheme.colorScheme.prim
        )
        // 메인 버튼
        Button(
            onClick = onClick,
            modifier = Modifier.size(120.dp),
            shape = CircleShape
        ) {
            Column(horizontalAlignment = Alignment.CenterH
```

```
                Text("Quick Scan", style = MaterialTheme.t
                Text("30초 점검", style = MaterialTheme.typ
            }
        }
    }
}
```

5.7 5.6 스캔 30초 UX 흐름

30초는 짧습니다. 하지만 아무것도 일어나지 않는 것처럼 느껴지면 30초는 3분처럼 느껴집니다. SearCam의 스캔 화면은 매초마다 사용자에게 "진행되고 있다" 는 신호를 보냅니다.

5.7.1 스캔 진행 화면 구조

```
┌──────────────────────────────────────────────────┐
│ ← Quick Scan              │
│                 │
│   ┌──────────────────────────────┐   │
│   │           │   │
│   │  ◎ 15초     │   │ ← 원형 타이머 (카운트다운)
│   │  남은 시간     │   │
```

```
│   │          │  │
│   └──────────────────────────────┘  │
│              │
│ Layer 1 Wi-Fi  □  완료 │  ← 레이어별 실시간 상태
│ Layer 2 렌즈  □ 진행  │
│ Layer 3 EMF    ○ 대기  │
│              │
│ 발견된 기기: 7대      │  ← 실시간 카운트
│              │
│   [취소]               │
└──────────────────────────────────────────────┘
```

5.7.2 UX 설계 원칙: 정보의 밀도

스캔 중 화면에는 4가지 정보가 동시에 표시됩니다: 남은 시간, 레이어 상태, 발견 기기 수, 취소 옵션. 더 많은 정보를 넣으면 오히려 불안을 증가시킵니다. "왜 저 숫자가 저러지?" 라는 의문이 생기기 때문입니다.

```
sealed class ScanProgressState {
    data object Idle : ScanProgressState()
    data class Scanning(
        val remainingSeconds: Int,
```

```
        val progress: Float,          // 0.0 ~ 1.0
        val layers: List<LayerState>,
        val devicesFound: Int
    ) : ScanProgressState()
    data class Completed(val result: ScanResult) : ScanPro
    data class Error(val code: String, val message: String
    data object Cancelled : ScanProgressState()
}

data class LayerState(
    val name: String,
    val status: LayerStatus  // WAITING, RUNNING, COMPLETE
)
```

이 sealed class 하나가 스캔 화면의 모든 상태를 표현합니다. Compose에서 when 식으로 각 상태에 맞는 UI를 선언하면 됩니다.

5.7.3 시간 압박을 줄이는 심리적 설계

카운트다운 숫자만 표시하면 사용자는 "빨리 끝내야 한다" 는 압박을 느낍니다. SearCam은 카운트다운 대신 "남은 시간" 을 앞에 표시합니다.

- "15초" → 압박감: "이미 절반이 지났다"

- “남은 시간: 15초” → 안도감: “아직 15초가 남았다”

단어 하나의 차이가 감정을 바꿉니다.

5.8 5.7 결과 화면 — 불안에서 안심으로의 전환

스캔 결과 화면은 SearCam의 가장 중요한 화면입니다. 이 화면에서 사용자의 감정이 “불안” 에서 “안심” 또는 “경계” 로 전환됩니다.

```
┌──────────────────────────────────────────────────────┐
│ ← 스캔 결과          │
│                  │
│   ┌──────────────────────────────┐   │
│   │ ◎ 12        │ │ ←RiskGauge (1초 애니메이션)
│   │    안전      │ │    초록색
│   └──────────────────────────────┘   │
│                  │
│ "이번 스캔에서         │ ← 결과 메시지
│  이상 징후를 발견하지    │
│  못했습니다"                 │
│                  │
│ ── 레이어별 결과 ──      │
```

```
│ Wi-Fi    0/7대 의심      │
│ 렌즈    감지 없음        │
│ 자기장  정상            │
│                         │
│ ── 발견된 기기 (7대) ── │
│ ┌──────────────────────┐ │ ← 기기 목록
│ │ 삼성 스마트폰        │ │
│ │ 안전 ▓▓▒▒ 5/100      │ │
│ └──────────────────────┘ │
│                         │
│  [리포트 저장] [Full Scan]│
│                         │
│  ⚠ 이 결과는 참고용입니다 │  ← 한계 고지 (항상 표시)
└─────────────────────────┘
```

5.8.1 점진적 정보 공개

결과는 한꺼번에 쏟아내지 않습니다. 3단계로 공개됩니다.

1단계: 종합 위험도 (게이지 + 등급 텍스트)

2단계: 레이어별 요약 (Wi-Fi / 렌즈 / 자기장)

3단계: 기기 목록 (탭하면 상세 정보)

이 구조는 "결론 먼저, 근거 나중" 원칙을 UI로 구현한 것입니다.

5.9 5.8 다크 테마 설계

모텔 방에서 새벽에 앱을 쓰는 상황을 상상해보세요. 밝은 흰색 화면은 눈을 자극합니다. SearCam의 기본 테마는 다크 모드입니다.

5.9.1 다크 테마 색상 스펙트럼

역할	라이트 모드	다크 모드
배경	`#FFFFFF` (White)	`#111827` (Gray-900)
카드/Surface	`#F9FAFB` (Gray-50)	`#1F2937` (Gray-800)
본문 텍스트	`#111827` (Gray-900)	`#F9FAFB` (Gray-50)
보조 텍스트	`#6B7280` (Gray-500)	`#9CA3AF` (Gray-400)
구분선	`#E5E7EB` (Gray-200)	`#374151` (Gray-700)
Primary	`#2563EB` (Blue-600)	`#3B82F6` (Blue-500)

다크 모드에서도 위험도 색상(Green/Lime/Yellow/Orange/Red)은 변경하지 않습니다. 위험도 색상은 의미를 전달하는 신호이기 때문에 일관성이 최우선입니다.

5.9.2 Material 3 Dynamic Color 비채택 이유

Android 12부터 지원하는 Dynamic Color(사용자 배경화면에서 색상 추출)는 SearCam에서 비채택했습니다. 위험도 색상(초록, 노랑, 빨강)이 사용자의 배경화면 색상에 따라 달라지면 의미 전달에 혼선이 생기기 때문입니다.

```
// MaterialTheme 설정 — Dynamic Color 비활성화
@Composable
fun SearCamTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    val colorScheme = if (darkTheme) DarkColorScheme else
    // dynamicColor = false (의도적 비채택)

    MaterialTheme(
        colorScheme = colorScheme,
        typography = SearCamTypography,
        content = content
    )
}
```

5.10 5.9 타이포그래피 — 숫자가 주인공

SearCam의 타이포그래피 설계의 핵심은 숫자를 명확하게 표현하는 것입니다. 위험도 점수 "67" 은 "6" 과 "7" 이 명확히 구분되어야 합니다.

5.10.1 텍스트 스케일

스타일	크기	무게	용도
Display Large	36sp	Bold 700	위험도 점수 (메인)
Headline Large	28sp	Bold 700	화면 제목
Headline Medium	24sp	SemiBold 600	섹션 제목
Title Large	20sp	SemiBold 600	카드 제목
Body Large	16sp	Regular 400	본문
Body Small	12sp	Regular 400	캡션, 타임스탬프
Label Large	14sp	Medium 500	버튼 텍스트

Pretendard Variable 서체를 사용하며, 숫자는 Tabular 설정(고정폭)을 적용합니다. 위험도 점수가 "12" 에서 "67" 로 바뀔 때 텍스트가 좌우로 흔들리지 않게 하기 위함입니다.

5.11 5.10 접근성 고려사항

색맹 사용자를 위한 설계도 필수입니다. 위험도를 색상만으로 표현하면 적록색맹 사용자는 "안전" 과 "주의" 를 구분할 수 없습니다.

SearCam은 색상과 함께 반드시 텍스트 레이블을 병기합니다.

▯ 올바른 표현	▯ 잘못된 표현
● 안전 (초록)	●
● 주의 (노랑)	●
● 위험 (빨강)	●

터치 타깃 최소 크기 48dp를 모든 인터랙티브 요소에 적용합니다. 화면 낭독기(TalkBack) 지원을 위해 모든 이미지와 아이콘에 contentDescription을 명시합니다.

```
Icon(
    imageVector = Icons.Outlined.Warning,
    contentDescription = "위험 경고",  // TalkBack이 읽음
    tint = Color(0xFFEF4444)
)
```

5.12 5.11 온보딩 — 신뢰를 먼저 쌓다

온보딩 3단계는 기능 설명이 아닙니다. 신뢰 구축입니다.

단계	제목	핵심 메시지
1/3	30초 안전 점검	"숙소, 화장실을 스마트폰으로 빠르게 점검하세요"
2/3	3중 교차 검증	"Wi-Fi + 렌즈 감지 + 자기장 분석으로 정확도를 높입니다"
3/3	솔직한 안내	"전문 장비를 대체하지 않습니다. 의심 시 112에 신고하세요"

3번째 화면이 핵심입니다. 대부분의 앱은 온보딩에서 한계를 숨깁니다. SearCam은 한계를 먼저 말합니다. 이 솔직함이 사용자의 신뢰를 만들고, 오탐 시 "앱이 잘못됐다" 가 아닌 "원래 그렇다고 했지" 라는 이해로 이어집니다.

5.13 5.12 체크리스트 화면 — 앱 없이도 쓸 수 있는 가이드

Wi-Fi가 없거나, 카메라 권한을 거부한 경우에도 SearCam은 유용해야 합니다. 체크리스트 화면은 숙소 유형별 육안 점검 가이드를 제공합니다.

```
체크리스트 유형 선택
├── 숙소 (모텔/호텔)
│   ├── 에어컨, 시계, 리모컨 확인
│   ├── TV 주변 확인
│   └── 욕실 환풍구, 샤워기 확인
├── 화장실 (공중)
│   ├── 칸막이 나사 확인
│   ├── 화장지 홀더 확인
│   └── 환풍구 확인
└── 탈의실
    ├── 거울 뒷면 확인
    └── 옷걸이 확인
```

각 항목은 체크박스로 완료를 표시합니다. 모든 항목을 완료하면 "점검 완료" 애니메이션이 표시됩니다.

5.14 정리: UX가 기술보다 먼저다

이 장에서 배운 핵심은 하나입니다. **좋은 UX는 기술적 정확도보다 중요할 때가 있습니다.**

SearCam이 탐지 정확도 85%를 달성해도, 결과 화면에서 사용자가 "무슨 뜻이지?" 라고 혼란을 느낀다면 앱은 실패입니다. 반대로, 탐지 정확도가 80%이더라도 결과를 명확하게 이해하고 올바른 다음 행동을 취할 수 있다면 앱은 성공합니다.

Jetpack Compose는 이 목표를 달성하기 위한 도구입니다. 상태 기반의 선언형 UI, 부드러운 애니메이션, 일관된 색상 시스템이 합쳐져 "불안을 안심으로" 전환하는 경험을 만듭니다.

다음 장에서는 이 앱이 "탐지 앱이면서 탐지당하지 않는 방법", 즉 보안 설계를 다룹니다.

5.15 체크리스트

- ☐ 위험도 색상 5단계가 일관되게 적용되었는가
- ☐ 모든 터치 타깃이 48dp 이상인가
- ☐ 위험도 표현에 텍스트 레이블이 병기되었는가
- ☐ 결과 화면에 한계 고지 문구가 항상 표시되는가

- ☐ 스캔 화면에서 매초 피드백이 제공되는가
- ☐ 다크 테마에서 위험도 색상이 변경되지 않는가
- ☐ contentDescription이 모든 아이콘에 적용되었는가

제 6 장

Ch06: 보안 설계 — 탐지 앱이 탐지 당하면 안 된다

이 장에서 배울 것: 몰래카메라를 탐지하는 앱이 역으로 사용자 데이터를 수집하거나 악용될 가능성을 어떻게 차단하는지 배웁니다. SQLCipher 암호화, Android Keystore, 네트워크 스캔 범위 제한, 최소 권한 원칙, 카메라 프레임 메모리 전용 처리를 실제 코드와 함께 설명합니다.

6.1 도입

자물쇠 가게는 가장 튼튼한 자물쇠를 만들어야 합니다. 그런데 그 가게가 복사 열쇠를 마음대로 만들 수 있다면, 우리는 그 가게를 믿을 수 없습니다.

SearCam은 사용자의 가장 사적인 공간—숙소, 화장실, 탈의실—에서 실행되는 앱입니다. 카메라 권한, 위치 권한, 네트워크 스캔 권한을 모두 가진 앱이 만약 그 정보를 수집하거나 외부로 전송한다면, 그 앱 자체가 몰래카메라가 됩니다.

"탐지 앱이 탐지 당하면 안 된다" 는 원칙이 SearCam 보안 설계의 전부입니다.

이 장에서는 4개 보안 레이어(권한, 네트워크, 데이터, 코드)를 계층적으로 설명하고, 각 레이어가 어떤 위협을 차단하는지 보여줍니다.

6.2 6.1 3대 핵심 원칙

보안 설계의 복잡성을 3가지 원칙으로 압축했습니다.

```
┌──────────────────────────────────────────────────────────────
│ 원칙 1: 로컬 처리 우선                                  │
│ 모든 탐지 및 분석은 사용자 기기 내에서만 수행          │
```

```
│  → 서버로 전송하지 않으면 서버 해킹 피해가 없다      │
├──────────────────────────────────────────────────────
│  원칙 2: 최소 권한                                  │
│  앱 기능에 반드시 필요한 권한만 요청                  │
│  → 권한이 없으면 남용할 수 없다                      │
├──────────────────────────────────────────────────────
│  원칙 3: 데이터 미수집                              │
│  사용자 개인정보를 수집하거나 외부로 전송하지 않음   │
│  → 수집하지 않은 데이터는 유출될 수 없다             │
└──────────────────────────────────────────────────────
```

이 3원칙은 서로 보완합니다. 로컬 처리 우선이 외부 유출 경로를 차단하고, 최소 권한이 수집 범위를 제한하며, 데이터 미수집이 그 목적 자체를 없앱니다.

6.3 6.2 보안 계층 구조 (Defense in Depth)

군사 방어에서 “심층 방어(Defense in Depth)” 란 외곽이 뚫려도 내부 방어선이 남아 있는 구조를 말합니다. SearCam의 보안도 4개 계층으로 구성됩니다.

```
┌──────────────────────────────────────────────────────────
│        Layer 4: 코드 보안              │
│  ProGuard/R8 난독화, 루팅 탐지, 빌드 분리   │
├──────────────────────────────────────────────────────────
│        Layer 3: 데이터 보안            │
│  SQLCipher AES-256, Android Keystore,        │
│  MAC 주소 SHA-256 해시, 카메라 프레임 즉시 해제│
├──────────────────────────────────────────────────────────
│        Layer 2: 네트워크 보안          │
│  로컬 네트워크 스캔 범위 제한, HTTPS + 인증서│
│  고정, RFC 1918 주소만 대상, TTL=1 격리     │
├──────────────────────────────────────────────────────────
│        Layer 1: 권한 보안              │
│  최소 권한 요청, 런타임 권한, 사용 후 즉시 해제│
└──────────────────────────────────────────────────────────
```

외부에서 내부로 침투하려면 4개 계층을 모두 돌파해야 합니다. 하나의 계층이 실패해도 나머지가 보호합니다.

6.4 6.3 위협 모델링 — 적을 먼저 알아야 한다

STRIDE 프레임워크로 SearCam의 위협을 체계적으로 분석했습니다. STRIDE는 6가지 위협 유형의 약어입니다: Spoofing(위장), Tampering(변조), Repudiation(부인), Information Disclosure(정보 노출), Denial of Service(서비스 거부), Elevation of Privilege(권한 상승).

6.4.1 SearCam STRIDE 위협 매트릭스

위협 유형	시나리오	위험도	SearCam 대응
Spoofing	악성 앱이 SearCam UI를 위장	낮음	Play Store 단일 배포 + 앱 서명
Spoofing	카메라 MAC을 일반 기기로 위장	중간	OUI + 포트 + mDNS 교차 검증
Tampering	로컬 DB 스캔 리포트 변조	낮음	SQLCipher 암호화

위협 유형	시나리오	위험도	SearCam 대응
Tampering	OUI DB를 변조하여 카메라를 안전 기기로 등록	중간	앱 내장 DB + 업데이트 서명 검증
Info Disclosure	MAC 주소 외부 유출	중간	메모리 전용 처리, DB에는 SHA-256 해시만 저장
Info Disclosure	카메라 프레임 데이터 유출	높음	저장 안 함, 분석 후 즉시 메모리 해제
DoS	대량 기기로 스캔 지연/크래시	중간	기기 수 상한 254대, 타임아웃 적용
EoP	앱 권한으로 시스템 공격	낮음	최소 권한, 센서 사용 후 즉시 해제

가장 위험한 위협은 "카메라 프레임 데이터 유출" 입니다. 사용자의 실시간 영상이 외부로 나가는 것은 앱이 몰래카메라가 되는 것이기 때문입니다. 이 위협에 대한 대응이 가장 철저해야 합니다.

6.5 6.4 최소 권한 원칙 — 필요한 것만 요청한다

6.5.1 요청하는 권한 (7개만)

권한	사용 이유	사용 시점	사용 후 처리
ACCESS_FINE_LOCATION	WiFi 스캔 시 Android 필수 요구	Quick/Full Scan	스캔 완료 후 중단
ACCESS_WIFI_STATE	WiFi 상태 확인	스캔 시작 전	상태 확인만
CHANGE_WIFI_STATE	WiFi 스캔 트리거	Quick/Full Scan	스캔 완료 후 비활성화
CAMERA	렌즈/IR 감지	렌즈 찾기, Full Scan	분석 완료 후 즉시 해제
INTERNET	OUI DB 업데이트, AdMob	백그라운드 업데이트	업데이트 완료 후 종료
VIBRATE	의심 기기 발견 알림	스캔 중	설정으로 비활성화 가능
FLASHLIGHT	Retroreflection 렌즈 감지	렌즈 찾기 모드	모드 종료 시 OFF

6.5.2 의도적으로 요청하지 않는 권한

다음 권한은 일부 카메라 앱들이 불필요하게 요청하는 권한입니다. SearCam은 이를 명시적으로 배제합니다.

- `READ_CONTACTS` — 연락처와 탐지 기능은 무관
- `READ_PHONE_STATE` — 전화 기능 불필요
- `RECORD_AUDIO` — 음성 녹음 불필요
- `ACCESS_BACKGROUND_LOCATION` — 백그라운드 위치 불필요
- `BLUETOOTH_SCAN` — Phase 1에서 BLE 미구현
- `READ_EXTERNAL_STORAGE` — 외부 저장소 접근 불필요
- `WRITE_EXTERNAL_STORAGE` — PDF 내보내기는 SAF 사용

권한 개수가 적을수록 신뢰도는 높아집니다. Play Store의 "데이터 안전" 섹션에서 권한 목록을 본 사용자는 "이 앱이 최소한의 것만 요청한다" 는 인상을 받습니다.

6.6 6.5 네트워크 스캔 범위 제한 — 울타리를 쳐라

포트 스캔은 SearCam에서 가장 민감한 기능입니다. 잘못 구현하면 해킹 도구가 됩니다. 다음 4가지 제한으로 스캔 범위를 철저히 통제합니다.

6.6.1 제한 1: RFC 1918 사설 주소만 대상

인터넷의 주소 체계에서 RFC 1918은 사설 네트워크(내부 네트워크)를 위한 주소 범위를 정의합니다.

```
사설 주소 범위 (로컬 네트워크):
  10.0.0.0/8       (10.x.x.x)
  172.16.0.0/12    (172.16.x.x ~ 172.31.x.x)
  192.168.0.0/16   (192.168.x.x)

공인 주소 (인터넷):
  위 범위에 포함되지 않는 모든 주소
 → SearCam에서 절대 스캔 대상으로 사용하지 않음
```

```
fun isPrivateAddress(ip: String): Boolean {
    val addr = InetAddress.getByName(ip)
    return addr.isSiteLocalAddress || addr.isLoopbackAddre
}

fun scanDevices(devices: List<NetworkDevice>): List<Networ
    // 사설 주소만 스캔, 공인 IP는 즉시 제외
    return devices.filter { isPrivateAddress(it.ipAddress)
}
```

6.6.2 제한 2: 카메라 관련 포트만 (6개)

모든 포트를 스캔하면 해킹 행위와 구분이 없습니다. SearCam은 카메라와 직접 관련된 6개 포트만 스캔합니다.

포트	프로토콜	용도	카메라 연관성
554	RTSP	스트리밍	IP 카메라의 기본 스트리밍 포트
80	HTTP	웹 인터페이스	IP 카메라 웹 뷰어
8080	HTTP-alt	대체 웹	일부 카메라 대체 포트
8888	HTTP-alt	대체 웹	소형 카메라 포트
3702	ONVIF	카메라 표준	ONVIF IP 카메라 탐색
1935	RTMP	스트리밍	실시간 스트리밍

6.6.3 제한 3: 동시 연결 5개 제한

```
// 세마포어로 동시 연결 수를 5개로 제한
private val semaphore = Semaphore(5)

suspend fun scanPort(ip: String, port: Int, timeoutMs: Lor
    return semaphore.withPermit {
        withContext(Dispatchers.IO) {
            try {
                Socket().use { socket ->
                    socket.connect(InetSocketAddress(ip, p
```

```
                    true
                }
            } catch (e: IOException) {
                false
            }
        }
    }
}
```

5개 제한은 "충분히 빠르면서도 네트워크에 부하를 주지 않는" 균형점입니다. TCP Connect 방식(SYN 스캔이 아님)으로 비공격적입니다.

6.6.4 제한 4: 포트당 2초 타임아웃

응답 없는 포트를 무한정 기다리면 30초 스캔이 불가능합니다. 2초 타임아웃은 "응답 없음 = 포트 닫힘" 으로 처리합니다.

6.7 6.6 카메라 프레임 메모리 전용 처리

이것이 SearCam 보안의 핵심입니다. 카메라로 촬영한 프레임은 단 1바이트도 디스크에 기록되지 않습니다.

6.7.1 프레임 생명주기

```
[카메라 센서]
      │
   ▼
[ImageAnalysis.Analyzer.analyze()] ← 이 함수 안에서만 존재
    │
    ├─── 고휘도 포인트 추출 (Bitmap.getPixel)
      │
    ├─── 렌즈 판정 (원형도, 크기, 안정성)
      │
    ├─── 점수 계산
    │
    └─── [ImageProxy.close()] ← 즉시 메모리 해제
       │
      ▼
   (프레임 데이터 소멸)
```

```
class LensAnalyzer(
    private val onResult: (LensDetectionResult) -> Unit
) : ImageAnalysis.Analyzer {

    override fun analyze(image: ImageProxy) {
```

```
        // analyze() 내부에서만 프레임 데이터 접근
        try {
            val bitmap = image.toBitmap()
            val result = detectLens(bitmap)
            onResult(result)
            // bitmap은 이 함수 스코프 내에서 소멸
        } finally {
            // 예외가 발생해도 반드시 해제
            image.close()
        }
    }

    private fun detectLens(bitmap: Bitmap): LensDetectionR
        val points = extractHighBrightnessPoints(bitmap)
        val suspiciousPoints = filterLensPoints(points)
        // bitmap 참조를 외부로 반환하지 않음
        return LensDetectionResult(suspiciousPoints, calcu
    }
}
```

finally 블록의 image.close()는 절대 생략할 수 없습니다. 예외가 발생한 경우에도 프레임이 메모리에 잔류하지 않도록 보장합니다.

6.7.2 검증: 카메라 프레임이 정말 저장 안 되나?

릴리스 전 검증 방법:

```
# 루팅 기기에서 앱 데이터 디렉토리 확인
adb shell run-as com.searcam.app ls -la /data/data/com.sea

# 예상 결과: 이미지 파일 (.jpg, .png, .bmp) 0개
# 허용 파일: databases/ (SQLCipher 암호화 DB), shared_prefs/
```

6.8 6.7 SQLCipher + Android Keystore 암호화

금고의 내용물이 중요하면 금고도 튼튼해야 합니다. SearCam의 로컬 DB는 SQLCipher로 암호화됩니다.

6.8.1 암호화 스펙

```
┌──────────────────────────────────────────────────────────────
│        Room DB + SQLCipher                  │
│                              │
│  알고리즘: AES-256-CBC                        │
│  키 파생: PBKDF2-HMAC-SHA512                  │
```

```
│        반복 횟수: 256,000회          │
│  페이지 크기: 4096 bytes                    │
│  키 저장: Android Keystore (TEE/SE 지원)      │
│                                │
│  테이블:                                    │
│   ├─── reports (암호화) — 스캔 리포트        │
│   └─── settings (암호화) — 앱 설정          │
└──────────────────────────────────────────────
```

6.8.2 Android Keystore를 쓰는 이유

일반 SharedPreferences에 DB 키를 저장하면, 루팅된 기기에서 앱 데이터 디렉토리를 열어보면 키가 그대로 노출됩니다. Android Keystore는 키를 TEE(Trusted Execution Environment) 또는 SE(Secure Element) 하드웨어에 저장합니다. 소프트웨어가 키에 직접 접근할 수 없고, "이 앱이 이 키로 암호화/복호화해달라" 는 요청만 가능합니다.

```
object DatabaseKeyManager {

    private const val KEY_ALIAS = "searcam_db_key"
    private const val PREFS_NAME = "searcam_secure_prefs"
    private const val KEY_PREF = "encrypted_db_passphrase"
```

```
fun getOrCreateKey(context: Context): ByteArray {
    val prefs = EncryptedSharedPreferences.create(
        context,
        PREFS_NAME,
        MasterKey.Builder(context)
            .setKeyScheme(MasterKey.KeyScheme.AES256_G
            .build(),
        EncryptedSharedPreferences.PrefKeyEncryptionSc
        EncryptedSharedPreferences.PrefValueEncryptior
    )

    val existing = prefs.getString(KEY_PREF, null)
    if (existing != null) {
        return Base64.decode(existing, Base64.DEFAULT)
    }

    // 최초 실행: 256비트 랜덤 키 생성
    val newKey = ByteArray(32).also { SecureRandom().r
    prefs.edit()
        .putString(KEY_PREF, Base64.encodeToString(new
        .apply()
    return newKey
```

```
    }
}

// Room DB에 SQLCipher 적용
@Database(entities = [ReportEntity::class, SettingEntity::
abstract class AppDatabase : RoomDatabase() {
    abstract fun reportDao(): ReportDao

    companion object {
        fun create(context: Context): AppDatabase {
            val passphrase = DatabaseKeyManager.getOrCreat
            val factory = SupportFactory(passphrase)

            return Room.databaseBuilder(
                context,
                AppDatabase::class.java,
                "searcam.db"
            )
            .openHelperFactory(factory)
            .build()
        }
    }
}
```

6.8.3 MAC 주소 SHA-256 해시 처리

MAC 주소는 기기를 식별할 수 있는 준개인정보입니다. SearCam은 원본 MAC 주소를 DB에 저장하지 않습니다.

```
fun hashMacAddress(mac: String): String {
    val digest = MessageDigest.getInstance("SHA-256")
    val hashBytes = digest.digest(mac.toByteArray(Charsets
    return hashBytes.joinToString("") { "%02x".format(it)
}

// 저장: 원본 "AA:BB:CC:DD:EE:FF" → 해시 "a1b2c3..."
// 원본은 메모리에서 즉시 삭제
```

6.9 6.8 네트워크 통신 보안 — 단 하나의 예외

Phase 1 SearCam은 외부 서버와 단 하나의 이유로만 통신합니다: OUI 데이터베이스 업데이트. 나머지 모든 기능은 100% 온디바이스로 처리됩니다.

SearCam이 외부와 통신하는 것:

- OUI DB 업데이트 (HTTPS + 인증서 고정)
- AdMob 광고 (Google SDK 관리)

SearCam이 외부와 통신하지 않는 것:

- 탐지 결과 전송
- 사용자 행동 분석
- 스캔 데이터 업로드
- 사용자 계정 서버

6.9.1 Certificate Pinning (인증서 고정)

OUI DB 업데이트 시 MITM(중간자 공격)으로 가짜 OUI 데이터를 주입하면, 카메라 MAC 주소가 안전 기기로 위장될 수 있습니다. 인증서 고정으로 이를 방지합니다.

```
// OkHttp에 인증서 고정 적용
val certificatePinner = CertificatePinner.Builder()
    .add("oui.searcam.app", "sha256/AAAAAAAAAAAAAAAAAAAAAA
    .build()

val okHttpClient = OkHttpClient.Builder()
    .certificatePinner(certificatePinner)
    .build()
```

서버 인증서의 공개키 해시를 앱에 내장합니다. 서버 인증서가 변경되거나 프록시가 개입하면 연결을 즉시 거부합니다.

6.10 6.9 코드 보안 — 릴리스 빌드는 다르다

6.10.1 ProGuard/R8 난독화

릴리스 APK를 jadx로 디컴파일하면 Kotlin 코드를 복원할 수 있습니다. ProGuard/R8 난독화를 적용하면 클래스명, 메서드명이 a, b, c로 변환되어 역공학이 어려워집니다.

```
# proguard-rules.pro

# 탐지 알고리즘 핵심 클래스 보호
-keep class com.searcam.data.analysis.** { *; }
-keepclassmembers class com.searcam.data.analysis.CrossVal
    private <fields>;
}

# Room DB 엔티티 유지 (리플렉션 사용)
-keep class com.searcam.data.local.** { *; }

# 릴리스에서 로그 완전 제거
-assumenosideeffects class android.util.Log {
    public static int d(...);
```

```
    public static int v(...);
    public static int i(...);
    public static int w(...);
}
```

`-assumenosideeffects` 설정으로 릴리스 빌드에서 `Log.d()`, `Log.v()` 등이 완전히 제거됩니다. 로그에 민감한 정보가 포함되어 있어도 릴리스 APK에서는 나타나지 않습니다.

6.10.2 디버그 vs 릴리스 분리

항목	디버그 빌드	릴리스 빌드
난독화	비활성화	R8 full mode
로그	Timber.d() 활성화	완전 제거
SQLCipher 키	"debug" 고정 비밀번호	Android Keystore
인증서 검증	시스템 CA 신뢰	인증서 고정
디버거 연결	허용	거부 (isDebuggable=false)
ADB 백업	허용	거부 (allowBackup=false)

`allowBackup=false`는 AndroidManifest.xml에 한 줄이지만, 이 설정 없이는 `adb backup` 명령으로 앱의 모든 데이터(암호화된 DB 포함)를 추출할 수 있습니다.

```
<!-- AndroidManifest.xml -->
```

```
<application
    android:allowBackup="false"
    android:fullBackupContent="false"
    android:dataExtractionRules="@xml/data_extraction_rule
    ... >
```

6.10.3 경로 순회(Path Traversal) 방지

PDF 내보내기 기능에서 파일명을 사용자 입력으로 받으면 위험합니다.

```
// WRONG: 경로 순회 취약점
fun exportPdf(fileName: String) {
    val file = File(exportDir, fileName)  // "../../../etc
    writePdf(file)
}

// CORRECT: 파일명만 추출, 경로 구분자 제거
fun exportPdf(fileName: String) {
    val safeName = File(fileName).name  // 파일명만 추출
        .replace(Regex("[^a-zA-Z0-9가-힣_\\-.]"), "_")  //
        .take(100)  // 길이 제한
    val file = File(exportDir, safeName)
    writePdf(file)
```

```
}
```

SearCam은 PDF 내보내기에 SAF(Storage Access Framework)를 사용합니다. SAF는 사용자가 직접 저장 위치를 선택하므로 앱이 파일시스템에 임의 접근하는 것 자체가 불가능합니다.

6.11 6.10 루팅 기기 대응

루팅된 기기는 앱 샌드박스를 무력화할 수 있습니다. Android Keystore가 소프트웨어 수준에서만 구현된 기기라면 루트 권한으로 키를 추출할 수도 있습니다.

```
루팅 기기 탐지 흐름:
   │
  ├── su 바이너리 존재 확인
  ├── /system/app/Superuser.apk 확인
  ├── Build.TAGS 확인 (test-keys)
   │
 ▼
경고 다이얼로그 표시:
"이 기기는 루팅되어 있습니다.
 데이터 보호가 약화될 수 있습니다.
```

```
계속 사용하시겠습니까?"
  │
  ├── [계속] → 기능 제한 없이 사용 (사용자 선택 존중)
  └── [취소] → 앱 종료
```

기능을 완전히 차단하지 않는 이유는 보안 연구자, 개발자, 고급 사용자의 권리를 존중하기 때문입니다. 경고만 표시하고 판단은 사용자에게 맡깁니다.

6.12 6.11 릴리스 전 보안 체크리스트

이 체크리스트는 모든 릴리스 전에 반드시 완료해야 합니다.

6.12.1 데이터 보호

- ☐ MAC 주소 원본이 DB에 저장되지 않는가
- ☐ 카메라 프레임이 디스크에 기록되지 않는가
- ☐ SQLCipher 암호화가 정상 동작하는가
- ☐ 앱 삭제 시 모든 데이터가 제거되는가
- ☐ allowBackup=false가 설정되어 있는가

6.12.2 권한

- ☐ 불필요한 권한을 요청하지 않는가
- ☐ 런타임 권한 거부 시 앱이 크래시하지 않는가
- ☐ 권한 사용 후 센서/카메라를 해제하는가

6.12.3 네트워크

- ☐ 외부 네트워크 IP 접근 시도가 없는가
- ☐ HTTPS 통신만 사용하는가
- ☐ 인증서 고정이 동작하는가
- ☐ 포트 스캔이 사설 주소로만 제한되는가

6.12.4 코드

- ☐ 릴리스 빌드에서 로그가 완전 제거되는가
- ☐ ProGuard/R8 난독화가 적용되는가
- ☐ 하드코딩된 비밀(API 키, 비밀번호)이 없는가
- ☐ 경로 순회 취약점이 없는가

6.12.5 검증 도구

단계	도구	검증 항목
정적 분석	Android Lint, Detekt	코드 취약점

단계	도구	검증 항목
의존성 취약점	Gradle Dependency Check	CVE 포함 라이브러리
네트워크 검증	Charles Proxy, mitmproxy	외부 통신 없음 확인
데이터 검증	adb + 루팅 기기	파일 미저장 확인
난독화 확인	jadx	APK 역공학 난이도

6.13 6.12 보안과 기능성의 균형

보안이 강할수록 기능이 제한됩니다. 이 긴장을 SearCam은 어떻게 해결했을까요?

```
보안 강화 방향                    기능 강화 방향
──────────────────────────────────────────────────────────
카메라 프레임 즉시 삭제  ←→  렌즈 탐지 정확도
네트워크 스캔 제한       ←→  Wi-Fi 탐지 범위
최소 권한 요청           ←→  더 많은 센서 활용
로컬 처리만              ←→  클라우드 AI 분석
```

SearCam의 선택: **보안이 먼저, 기능이 나중**.

이유는 간단합니다. 탐지 정확도가 70%여도 사용자 데이터가 안전한 앱은 신뢰받습니다. 탐지 정확도가 90%여도 사용자 데이터가

외부로 나가는 앱은 신뢰받지 못합니다.

6.14 정리: 탐지 앱이 탐지당하지 않는 방법

SearCam의 보안 설계는 기술적 우수성보다 **설계 철학**이 핵심입니다.

1. 수집하지 않으면 유출되지 않는다 → 데이터 미수집
2. 저장하지 않으면 도난당하지 않는다 → 메모리 전용 처리
3. 요청하지 않으면 남용할 수 없다 → 최소 권한
4. 나가지 않으면 가로채이지 않는다 → 로컬 처리 우선

보안은 앱을 배포하기 전에 설계에서 시작해야 합니다. 나중에 "보안을 추가" 하는 것은 집을 다 지은 후 지하실을 만드는 것과 같습니다.

다음 장에서는 이 설계가 실제로 올바르게 구현되었는지 증명하는 방법인 테스트 전략을 다룹니다.

6.15 보안 원칙 요약

위협	대응	구현
카메라 프레임 유출	메모리 전용, 즉시 해제	`image.close()` in finally
DB 데이터 유출	SQLCipher AES-256	Android Keystore 키
MAC 주소 노출	SHA-256 해시 저장	원본 미저장
외부 네트워크 스캔	RFC 1918 주소만 허용	`isPrivateAddress()` 필터
포트 스캔 남용	6개 포트, 5개 동시 연결, 2초 타임아웃	Semaphore + timeout
DB 백업 취약점	ADB 백업 금지	`allowBackup=false`
리버스 엔지니어링	R8 난독화	ProGuard rules
MITM 공격	인증서 고정	CertificatePinner

제 7 장

Ch07: 테스트 전략 — 버그보다 테스트가 먼저

이 장에서 배울 것: TDD RED-GREEN-IMPROVE 사이클이 탐지 앱에서 왜 필수인지, MockK와 Turbine으로 Kotlin Coroutines + Flow를 테스트하는 방법, 탐지 정확도를 코드로 검증하는 방법, 에러 코드 체계(E1xxx/E2xxx/E3xxx)의 테스트 방법을 배웁니다.

7.1 도입

비행기 조종사는 이륙 전에 체크리스트를 읽습니다. 수백 번 같은 기종을 몰았어도 절차를 생략하지 않습니다. 이유는 하나입니다. “이번에는 괜찮겠지” 라는 생각이 가장 위험하기 때문입니다.

소프트웨어 테스트도 같습니다. “이 로직은 단순하니까 테스트 없이 넣어도 되겠지” 라는 생각이 버그의 시작입니다.

SearCam은 특히 더 그렇습니다. 오탐(false positive)은 사용자를 불필요하게 불안하게 만들고, 미탐(false negative)은 실제 카메라를 “안전하다” 고 판정해 물리적 위험을 초래합니다. 탐지 정확도는 코드 품질이 아니라 사용자 안전의 문제입니다.

이 장에서는 TDD로 SearCam을 개발하는 방법을 실제 테스트 케이스와 함께 보여줍니다.

7.2 7.1 테스트 철학 — “동작한다”가 아닌 “정확하게 동작한다”

7.2.1 오탐과 미탐 중 어느 것이 더 나쁜가?

이 질문에 대한 답이 SearCam의 테스트 전략을 결정합니다.

```
오탐 (False Positive)
  실제 카메라 없음 → "위험" 판정
 결과: 사용자 불안, 앱 신뢰도 하락

미탐 (False Negative)
  실제 카메라 있음 → "안전" 판정
 결과: 사용자 물리적 위험 노출
```

미탐이 더 나쁩니다. 따라서 테스트는 "카메라를 놓치지 않는가"에 더 엄격해야 합니다.

이 판단이 테스트 케이스 설계에 반영됩니다. 경계값 테스트에서 위험도 점수 경계(19/20, 39/40, 59/60, 79/80)를 모두 검증하고, 오류 케이스보다 정상적인 카메라 탐지 케이스를 더 많이 작성합니다.

7.2.2 테스트 피라미드

```
        ┌──────────────────┐
        │   E2E    │  10%  (핵심 사용자 플로우)
        │   UI     │
    ────┼──────────────────┼────
    │Integration│  20%  (센서→분석→결과 파이프라인)
        │          │
    ────┼──────────────────┼────
```

```
    │  Unit      │  70%  (Domain + Data 레이어)
    │            │
    └────────────────────────────┘
```

레벨	비율	실행 환경	목표 실행 시간	목적
Unit	70%	JVM (Robolectric)	30초 이내	로직 정확성
Integration	20%	Android Instrumented	2분 이내	파이프라인 연동
E2E	10%	실기기/에뮬레이터	5분 이내	사용자 시나리오

7.3 7.2 TDD 사이클 — RED-GREEN-IMPROVE

7.3.1 자동차 안전벨트처럼

안전벨트를 먼저 채운 다음 운전을 시작합니다. 다 달린 후 채우는 것이 아닙니다. TDD는 코드 전에 테스트를 먼저 작성합니다.

```
RED     → 실패하는 테스트 작성 (아직 구현 없음)
            ↓
GREEN   → 테스트를 통과시키는 최소 구현 작성
        ↓
IMPROVE → 코드를 정리하면서 테스트 유지
        ↓
다음 테스트로 반복
```

SearCam에서 `RiskCalculator`를 TDD로 구현하는 예시를 보겠습니다.

7.3.2 Step 1: RED — 실패하는 테스트 먼저

```
// RiskCalculatorTest.kt

class RiskCalculatorTest {

    private lateinit var calculator: RiskCalculator

    @Before
    fun setUp() {
        calculator = RiskCalculator()
    }
```

CHAPTER 7.0 CHAPTER 7.0 CHAPTER 7.0

```
    val score = calculator.calculateLayer1Score(layer1

    assertThat(score).isAtMost(5)
}

// 테스트 3: 경계값 — 점수 0 이하 불가
@Test
fun `모든 지표가 낮아도 점수는 0 이상`() {
    val layer1Result = Layer1Result(
        macRisk = 0.0,
        openPorts = emptyList(),
        mdnsFound = false
    )

    val score = calculator.calculateLayer1Score(layer1

    assertThat(score).isAtLeast(0)
}

// 테스트 4: 경계값 — 점수 100 초과 불가
@Test
```

```
    fun `모든 지표가 만점이어도 점수는 100 이하`() {
        val layer1Result = Layer1Result(
            macRisk = 1.0,
            openPorts = listOf(554, 80, 8080, 3702, 1935),
            mdnsFound = true
        )

        val score = calculator.calculateLayer1Score(layer1

        assertThat(score).isAtMost(100)
    }
}
```

이 테스트들은 `RiskCalculator` 클래스가 아직 없으므로 컴파일조차 안 됩니다. 이것이 RED 상태입니다.

7.3.3 Step 2: GREEN — 최소 구현

```
// RiskCalculator.kt

class RiskCalculator {

    fun calculateLayer1Score(result: Layer1Result): Int {
        var score = 0.0
```

```
        // MAC 위험도 가중치 (40점 만점)
        score += result.macRisk * 40

        // RTSP 포트 (554) 개방 시 +30점
        if (554 in result.openPorts) score += 30

        // HTTP 포트 개방 시 +15점
        if (80 in result.openPorts || 8080 in result.openP

        // ONVIF 포트 개방 시 +20점
        if (3702 in result.openPorts) score += 20

        // mDNS로 카메라 서비스 발견 시 +25점
        if (result.mdnsFound) score += 25

        // 0~100 범위 클램프
        return score.toInt().coerceIn(0, 100)
    }
}
```

테스트 4개가 모두 통과하면 GREEN입니다.

CHAPTER 7.0 CHAPTER 7.0 CHAPTER 7.0

```
        var score = 0
        if (554 in openPorts) score += RTSP_PORT_SCORE
        if (80 in openPorts || 8080 in openPorts) score +=
        if (3702 in openPorts) score += ONVIF_PORT_SCORE
        return score
    }
}
```

리팩토링 후에도 테스트가 모두 통과하면 IMPROVE 완료입니다.

7.4 7.3 MockK — Kotlin 스러운 모킹

Mockito는 Java 스타일의 모킹 라이브러리입니다. Kotlin에서 쓰면 `any()`, `verify()` 등의 표현이 어색합니다. MockK는 Kotlin 언어 특성에 맞게 설계된 모킹 라이브러리입니다.

7.4.1 MockK로 의존성 격리

UseCase 테스트에서 Repository를 실제로 구현하면 테스트가 DB, 네트워크 등 외부 요소에 의존합니다. MockK로 가짜 Repository를 만들면 순수한 로직만 테스트할 수 있습니다.

```
class RunQuickScanUseCaseTest {

    @MockK
    private lateinit var wifiScanner: WifiScanner

    @MockK
    private lateinit var ouiDatabase: OuiDatabase

    @MockK
    private lateinit var portScanner: PortScanner

    @MockK
    private lateinit var riskCalculator: RiskCalculator

    private lateinit var useCase: RunQuickScanUseCase

    @Before
    fun setUp() {
        MockKAnnotations.init(this)
        useCase = RunQuickScanUseCase(wifiScanner, ouiData
    }
```

```
@Test
fun `Wi-Fi 연결 상태에서 정상 스캔 시 ScanResult 반환`() = ru
    // Given
    val mockDevices = listOf(
        NetworkDevice(ip = "192.168.1.100", mac = "28:
    )
    val mockOuiInfo = OuiInfo(manufacturer = "Hikvisio

    coEvery { wifiScanner.scanDevices() } returns mock
    every { ouiDatabase.lookup("28:57:BE") } returns m
    coEvery { portScanner.scanPorts("192.168.1.100") }
    every { riskCalculator.calculateLayer1Score(any())

    // When
    val result = useCase.execute()

    // Then
    assertThat(result).isNotNull()
    assertThat(result.overallScore).isEqualTo(75)
    assertThat(result.suspiciousDevices).hasSize(1)

    // 검증: 모든 의존성이 정확히 호출되었는가
```

CHAPTER 7.0 CHAPTER 7.0 CHAPTER 7.0

7.5 7.4 Turbine — Flow 테스트의 해결사

SearCam의 스캔 진행률은 `Flow<ScanProgressState>`로 방출됩니다. Flow 테스트는 까다롭습니다. `collect`를 시작하면 흐름이 끝날 때까지 기다려야 하기 때문입니다.

Turbine은 Flow 테스트를 위한 라이브러리입니다. "터빈처럼 흐름을 제어한다" 는 의미로, `awaitItem()`, `awaitComplete()` 등의 직관적인 API를 제공합니다.

```
class ScanViewModelTest {

    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()  // Test

    @MockK
    private lateinit var runQuickScanUseCase: RunQuickScan

    private lateinit var viewModel: ScanViewModel

    @Before
    fun setUp() {
```

```
        MockKAnnotations.init(this)
        viewModel = ScanViewModel(runQuickScanUseCase)
    }

    @Test
    fun `스캔 시작 시 진행률 0에서 100으로 증가`() = runTest {
        // Given
        coEvery { runQuickScanUseCase.progressFlow } retur
            emit(ScanProgressState.Scanning(remainingSecon
            emit(ScanProgressState.Scanning(remainingSecon
            emit(ScanProgressState.Completed(result = mock
        }

        // When
        viewModel.startScan()

        // Then — Turbine으로 Flow 항목 순서대로 검증
        viewModel.uiState.test {
            val initialState = awaitItem()
            assertThat(initialState).isInstanceOf(ScanUiSt
            assertThat((initialState as ScanUiState.Scanni
```

```
            val midState = awaitItem()
            assertThat(midState).isInstanceOf(ScanUiState.
            assertThat((midState as ScanUiState.Scanning).

            val finalState = awaitItem()
            assertThat(finalState).isInstanceOf(ScanUiStat
            assertThat((finalState as ScanUiState.Result).

            awaitComplete()
        }
    }

    @Test
    fun `에러 발생 시 에러 상태로 전환`() = runTest {
        // Given
        coEvery { runQuickScanUseCase.progressFlow } retur
            throw ScanException("E1001", "센서 초기화 실패")
        }

        // When
        viewModel.startScan()
```

```
        // Then
        viewModel.uiState.test {
            val errorState = awaitItem()
            assertThat(errorState).isInstanceOf(ScanUiStat
            assertThat((errorState as ScanUiState.Error).c
        }
    }
}
```

`viewModel.uiState.test { ... }` 블록 안에서 순서대로 항목을 `awaitItem()`으로 받아 검증합니다. Flow가 방출하는 순서가 보장되므로 타이밍 문제가 없습니다.

7.6 7.5 Coroutine 테스트 — 시간을 제어하다

스캔 타임아웃은 30초입니다. 실제 30초를 기다리는 테스트는 실용적이지 않습니다. `kotlinx-coroutines-test`의 `TestDispatcher`와 `advanceTimeBy()`를 사용하면 시간을 빠르게 돌릴 수 있습니다.

```
class ScanTimeoutTest {
```

```
    private val testDispatcher = StandardTestDispatcher()

    @Test
    fun `30초 초과 시 TimeoutException 발생`() = runTest(testD
        val scanner = WifiScanner(
            dispatcher = testDispatcher,
            timeoutMs = 30_000L
        )

        // 30초가 지나도 응답 없는 가짜 네트워크
        coEvery { scanner.scanDevices() } coAnswers {
            delay(Long.MAX_VALUE)  // 절대 끝나지 않는 스캔
            emptyList()
        }

        // 30초 진행 (실제로는 즉시)
        val job = launch {
            assertThrows<TimeoutException> {
                scanner.scanDevices()
            }
        }
```

```
        advanceTimeBy(30_001)  // 30초 1밀리초 경과
        job.join()
    }
}
```

`advanceTimeBy(30_001)`은 실제로 시간이 흐르는 것이 아니라, TestDispatcher의 가상 시계를 30001ms 앞으로 이동시킵니다. 테스트는 즉시 완료됩니다.

7.7 7.6 에러 코드 체계 테스트

SearCam의 에러는 3자리 접두사로 구분됩니다.

```
E1xxx — 센서 레이어 에러
  E1001: 자기장 센서 없음 (기기 미지원)
  E1002: 카메라 초기화 실패
  E1003: 센서 데이터 수신 타임아웃

E2xxx — 네트워크 레이어 에러
  E2001: Wi-Fi 미연결
  E2002: ARP 테이블 파싱 실패
  E2003: 포트 스캔 타임아웃
```

```
 E2004: 네트워크 연결 끊김

E3xxx — 권한 레이어 에러
 E3001: 위치 권한 거부
 E3002: 카메라 권한 거부
 E3003: 권한 영구 거부 (다시 묻지 않음)
```

7.7.1 에러 코드별 테스트

```
class ErrorHandlingTest {

    @Test
    fun `자기장 센서 없는 기기에서 E1001 에러 코드 반환`() {
        // Given: 자력계가 없는 기기 시뮬레이션
        val mockSensorManager = mockk<SensorManager>()
        every {
            mockSensorManager.getDefaultSensor(Sensor.TYPE
        } returns null

        val magneticSensor = MagneticSensor(mockSensorMana

        // When
        val exception = assertThrows<SearCamException> {
```

```
            magneticSensor.initialize()
        }

        // Then
        assertThat(exception.errorCode).isEqualTo("E1001")
        assertThat(exception.message).contains("자기장 센서
    }

    @Test
    fun `Wi-Fi 미연결 시 E2001 에러 코드 반환`() {
        // Given
        val mockWifiManager = mockk<WifiManager>()
        every { mockWifiManager.connectionInfo.networkId }

        val wifiScanner = WifiScanner(mockWifiManager)

        // When
        val exception = assertThrows<SearCamException> {
            runBlocking { wifiScanner.scanDevices() }
        }

        // Then
```

```
        assertThat(exception.errorCode).isEqualTo("E2001")
    }

    @Test
    fun `위치 권한 거부 시 E3001 에러 코드 반환`() {
        // Given
        val mockPermissionChecker = mockk<PermissionChecke
        every {
            mockPermissionChecker.hasPermission(Manifest.p
        } returns false

        val useCase = RunQuickScanUseCase(
            permissionChecker = mockPermissionChecker,
            /* 나머지 의존성 */
        )

        // When
        val exception = assertThrows<SearCamException> {
            runBlocking { useCase.execute() }
        }

        // Then
```

```
        assertThat(exception.errorCode).isEqualTo("E3001")
    }
}
```

에러 코드 테스트의 핵심은 **에러가 올바른 코드와 함께 발생하는가**를 검증하는 것입니다. 에러 코드가 없으면 사용자에게 "뭔가 잘못됐어요" 만 보여줄 수 있고, 개발자도 원인을 파악할 수 없습니다.

7.8 7.7 탐지 정확도 검증 — 알려진 카메라로 테스트

7.8.1 테스트 대상 카메라 (실기기)

#	카메라 종류	탐지 방식	목표 탐지율
1	Hikvision Wi-Fi IP 카메라	OUI + RTSP	85%
2	Wyze Cam v3	OUI + HTTP	80%
3	IR LED 야간 카메라	IR 감지 (암실)	75%
4	핀홀 카메라 (유선)	EMF + 렌즈	40%

7.8.2 OUI 데이터베이스 정확도 테스트

```
class OuiDatabaseAccuracyTest {

    private lateinit var database: OuiDatabase

    @Before
    fun setUp() {
        database = OuiDatabase.create(testContext)
    }

    @Test
    fun `Hikvision MAC 주소가 높은 위험도로 분류되는가`() {
        // Hikvision의 실제 OUI 접두사들
        val hikVisionOuis = listOf("28:57:BE", "44:19:B6",

        hikVisionOuis.forEach { oui ->
            val info = database.lookup(oui)
            assertThat(info).isNotNull()
            assertThat(info!!.riskWeight).isAtLeast(0.90f)
            assertThat(info.deviceType).isEqualTo(DeviceTy
        }
    }
```

```
@Test
fun `Apple MAC 주소가 낮은 위험도로 분류되는가`() {
    val appleOuis = listOf("00:03:93", "00:0A:27", "3C

    appleOuis.forEach { oui ->
        val info = database.lookup(oui)
        assertThat(info).isNotNull()
        assertThat(info!!.riskWeight).isAtMost(0.10f)
        assertThat(info.deviceType).isEqualTo(DeviceTy
    }
}

@Test
fun `대소문자 구분 없이 MAC 주소 조회 가능`() {
    val upperCase = database.lookup("28:57:BE")
    val lowerCase = database.lookup("28:57:be")
    val mixed = database.lookup("28:57:Be")

    assertThat(upperCase).isEqualTo(lowerCase)
    assertThat(lowerCase).isEqualTo(mixed)
}
```

```
    @Test
    fun `미등록 MAC 주소 조회 시 null 반환`() {
        val result = database.lookup("FF:FF:FF")
        assertThat(result).isNull()
    }
}
```

7.8.3 CrossValidator 교차 검증 테스트

교차 검증은 SearCam의 핵심 알고리즘입니다. 여러 레이어가 동시에 양성 반응을 보이면 위험도를 더 높게 평가합니다.

```
class CrossValidatorTest {

    private val validator = CrossValidator()

    @Test
    fun `기본 가중치 — Wi-Fi 연결 시 Layer1 50%, Layer2 35%, L
        val weights = validator.calculateWeights(
            isWifiConnected = true,
            isIrAvailable = true
        )
```

CHAPTER 7.0 CHAPTER 7.0 CHAPTER 7.0

```
            positiveLayerCount = 3,
            baseScore = 60
        )

        // 60 * 1.5 = 90, 단 100 초과 불가
        assertThat(result).isEqualTo(90)
    }

    @Test
    fun `2개 레이어 양성 시 보정계수 1_2 적용`() {
        val result = validator.applyCorrection(
            positiveLayerCount = 2,
            baseScore = 50
        )

        // 50 * 1.2 = 60
        assertThat(result).isEqualTo(60)
    }

    @Test
    fun `EMF만 단독 양성 시 보정계수 0_5 적용`() {
        val result = validator.applyCorrection(
```

```
            positiveLayerCount = 1,
            positiveLayer = Layer.EMF_ONLY,
            baseScore = 60
        )

        // 60 * 0.5 = 30
        assertThat(result).isEqualTo(30)
    }
}
```

7.9 7.8 Room DB 통합 테스트

단위 테스트에서는 DB를 Mock으로 대체했지만, 통합 테스트에서는 실제 Room DB를 사용합니다. 메모리 내 DB를 사용하면 빠르고 독립적입니다.

```
@RunWith(AndroidJUnit4::class)
class ReportDaoTest {

    private lateinit var db: AppDatabase
    private lateinit var reportDao: ReportDao
```

```
    @Before
    fun setUp() {
        db = Room.inMemoryDatabaseBuilder(
            ApplicationProvider.getApplicationContext(),
            AppDatabase::class.java
        ).build()
        reportDao = db.reportDao()
    }

    @After
    fun tearDown() {
        db.close()
    }

    @Test
    fun `리포트 저장 후 조회 시 동일 데이터 반환`() = runTest {
        // Given
        val report = ReportEntity(
            id = 0,
            timestamp = 1_712_134_320_000L,
            scanType = ScanType.QUICK,
            overallScore = 25,
```

```
            locationName = "서울 강남 모텔"
        )

        // When
        val insertedId = reportDao.insert(report)
        val retrieved = reportDao.getById(insertedId)

        // Then
        assertThat(retrieved).isNotNull()
        assertThat(retrieved!!.overallScore).isEqualTo(25)
        assertThat(retrieved.locationName).isEqualTo("서울
    }

    @Test
    fun `무료 사용자 리포트 10건 초과 시 가장 오래된 것 삭제`() = ru
        // Given: 10개 리포트 삽입
        repeat(10) { i ->
            reportDao.insert(createTestReport(timestamp =
        }

        // 10개 저장 확인
        assertThat(reportDao.getAll().first()).hasSize(10)
```

```
        // When: 11번째 저장
        val newestReport = createTestReport(timestamp = 99
        reportDao.insertWithLimit(newestReport, maxCount =

        // Then: 여전히 10개, 가장 오래된(timestamp=0) 삭제됨
        val reports = reportDao.getAll().first()
        assertThat(reports).hasSize(10)
        assertThat(reports.none { it.id == 0L }).isTrue()
        assertThat(reports.any { it.overallScore == 55 }).
    }

    @Test
    fun `리포트 목록은 최신순 정렬`() = runTest {
        // Given
        reportDao.insert(createTestReport(timestamp = 1000
        reportDao.insert(createTestReport(timestamp = 3000
        reportDao.insert(createTestReport(timestamp = 2000

        // When
        val reports = reportDao.getAll().first()
```

```
        // Then: 내림차순 (최신 먼저)
        assertThat(reports[0].timestamp).isEqualTo(3000L)
        assertThat(reports[1].timestamp).isEqualTo(2000L)
        assertThat(reports[2].timestamp).isEqualTo(1000L)
    }
}
```

7.10 7.9 Compose UI 테스트

Jetpack Compose는 `createComposeRule()`을 사용한 테스트를 지원합니다. 실제 UI가 올바른 상태를 표시하는지 검증할 수 있습니다.

```
@RunWith(AndroidJUnit4::class)
class ScanResultScreenTest {

    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun `안전 등급 결과 화면에서 초록색 게이지와 안전 메시지 표시`(
        // Given
```

```
    val safeResult = ScanResult(overallScore = 12, gra

    // When
    composeTestRule.setContent {
        SearCamTheme {
            ScanResultScreen(result = safeResult)
        }
    }

    // Then
    composeTestRule
        .onNodeWithText("안전")
        .assertIsDisplayed()

    composeTestRule
        .onNodeWithText("12")
        .assertIsDisplayed()

    composeTestRule
        .onNodeWithText("이번 스캔에서 이상 징후를 발견하지
        .assertIsDisplayed()
```

```
        // 한계 고지 문구는 항상 표시되어야 함
        composeTestRule
            .onNodeWithText("이 결과는 참고용입니다", substri
            .assertIsDisplayed()
    }

    @Test
    fun `위험 등급 결과 화면에서 112 신고 안내 표시`() {
        // Given
        val dangerResult = ScanResult(overallScore = 85, g

        // When
        composeTestRule.setContent {
            SearCamTheme {
                ScanResultScreen(result = dangerResult)
            }
        }

        // Then
        composeTestRule
            .onNodeWithText("매우 위험")
            .assertIsDisplayed()
```

```
    // 위험 등급에서는 112 신고 안내 표시
    composeTestRule
        .onNodeWithText("112", substring = true)
        .assertIsDisplayed()
}

@Test
fun `Quick Scan 후 Full Scan 유도 안내 표시`() {
    // Given
    val quickResult = ScanResult(
        overallScore = 30,
        grade = RiskGrade.ATTENTION,
        scanType = ScanType.QUICK
    )

    // When
    composeTestRule.setContent {
        SearCamTheme {
            ScanResultScreen(result = quickResult)
        }
    }
```

```
        // Then: Quick Scan 완료 후 Full Scan 권유
        composeTestRule
            .onNodeWithText("Full Scan", substring = true)
            .assertIsDisplayed()
    }
}
```

7.11 7.10 커버리지 목표와 측정

7.11.1 레이어별 커버리지 목표

레이어	최소 커버리지	목표 커버리지	이유
domain/	90%	95%	핵심 비즈니스 로직, 오류 허용
data/analysis/	85%	90%	탐지 알고리즘, 정확도 직결
data/sensor/	70%	80%	하드웨어 의존, 에뮬레이션 한계
data/local/	80%	85%	Room DB CRUD
ui/ViewModel	80%	85%	UI 로직과 상태 관리
ui/Screen	60%	70%	Compose UI, 통합 테스트로 보
전체	**80%**	**85%**	-

7.11.2 커버리지 측정 명령

```
# Gradle로 커버리지 리포트 생성
./gradlew testDebugUnitTest jacocoTestReport

# 리포트 위치
# app/build/reports/jacoco/jacocoTestReport/html/index.htm
```

7.11.3 커버리지가 목표 미달 시

80% 미만 → PR 머지 차단 (CI/CD 설정)

80~85% → 경고만, 머지 허용

85% 이상 → 통과

```
# .github/workflows/ci.yml (일부)
- name: Check Coverage
  run: |
    coverage=$(./gradlew jacocoTestCoverageVerification 2>
    echo "Coverage: $coverage"
    if (( $(echo "$coverage < 80" | bc -l) )); then
      echo "Coverage below 80%! Blocking merge."
      exit 1
    fi
```

7.12 7.11 TDD 실전 패턴과 함정

7.12.1 함정 1: “이건 단순하니까 테스트 안 해도 돼”

```
// "단순한" 함수
fun riskGrade(score: Int): RiskGrade = when {
    score < 20 -> RiskGrade.SAFE
    score < 40 -> RiskGrade.ATTENTION
    score < 60 -> RiskGrade.CAUTION
    score < 80 -> RiskGrade.DANGEROUS
    else       -> RiskGrade.VERY_DANGEROUS
}
```

이 함수가 단순해 보여도 경계값(19, 20, 39, 40, 79, 80)에서 버그가 납니다. “< 20” 이 “<= 20” 으로 잘못 쓰이면 점수 20인 사용자가 “안전” 으로 잘못 표시됩니다.

```
@Test
fun `위험도 등급 경계값 정확성 검증`() {
    // 경계값 아래
    assertThat(riskGrade(19)).isEqualTo(RiskGrade.SAFE)
    assertThat(riskGrade(39)).isEqualTo(RiskGrade.ATTENTIO
    assertThat(riskGrade(59)).isEqualTo(RiskGrade.CAUTION)
    assertThat(riskGrade(79)).isEqualTo(RiskGrade.DANGEROU
```

```
    // 경계값 위
    assertThat(riskGrade(20)).isEqualTo(RiskGrade.ATTENTIO
    assertThat(riskGrade(40)).isEqualTo(RiskGrade.CAUTION)
    assertThat(riskGrade(60)).isEqualTo(RiskGrade.DANGEROU
    assertThat(riskGrade(80)).isEqualTo(RiskGrade.VERY_DAN

    // 극단값
    assertThat(riskGrade(0)).isEqualTo(RiskGrade.SAFE)
    assertThat(riskGrade(100)).isEqualTo(RiskGrade.VERY_DA
}
```

7.12.2 함정 2: 테스트가 구현 세부사항에 의존

```
// BAD: 구현 세부사항 검증 (내부 메서드 호출 횟수)
@Test
fun `BAD — calculatePortScore 3번 호출 확인`() {
    verify(exactly = 3) { calculator.calculatePortScore(ar
    // 리팩토링하면 테스트 실패
}

// GOOD: 행동 결과 검증
@Test
fun `GOOD — 카메라 포트 3개 개방 시 올바른 점수`() {
```

```
    val result = calculator.calculateLayer1Score(
        openPorts = listOf(554, 3702, 80)
    )
    assertThat(result).isAtLeast(60)
    // 내부 구현이 바뀌어도 결과가 맞으면 통과
}
```

7.12.3 함정 3: 비결정적 테스트 (Flaky Tests)

```
// BAD: System.currentTimeMillis() 직접 사용 → 실행 환경마다 
@Test
fun `BAD — 현재 시간 기반 리포트 이름`() {
    val reportName = generateReportName()
    assertThat(reportName).contains("2026")  // 2027년에 실
}

// GOOD: 시간을 파라미터로 주입
@Test
fun `GOOD — 지정된 시간 기반 리포트 이름`() {
    val fixedTime = Instant.parse("2026-04-03T14:32:00Z")
    val reportName = generateReportName(timestamp = fixedT
    assertThat(reportName).isEqualTo("SearCam_2026-04-03_1
}
```

7.13 7.12 E2E 테스트 — 실제 사용자처럼 테스트하기

E2E 테스트는 전체 사용자 플로우를 처음부터 끝까지 검증합니다.

7.13.1 Quick Scan 전체 플로우

```
@RunWith(AndroidJUnit4::class)
@LargeTest
class QuickScanE2ETest {

    @get:Rule
    val activityRule = ActivityScenarioRule(MainActivity::

    @Test
    fun `Quick Scan 시작부터 리포트 저장까지 40초 이내 완료`() {
        val startTime = SystemClock.elapsedRealtime()

        // 홈 화면에서 Quick Scan 버튼 탭
        onView(withText("Quick Scan")).perform(click())
```

```
        // 스캔 진행 화면 표시 대기
        onView(withId(R.id.progress_indicator))
            .check(matches(isDisplayed()))

        // 최대 35초 대기 후 결과 화면 확인
        onView(withId(R.id.risk_gauge))
            .withTimeout(35_000)
            .check(matches(isDisplayed()))

        // 리포트 저장 버튼 탭
        onView(withText("리포트 저장")).perform(click())

        // 저장 확인 토스트
        onView(withText("리포트가 저장되었습니다", substring
            .check(matches(isDisplayed()))

        val elapsed = SystemClock.elapsedRealtime() - star
        assertThat(elapsed).isLessThan(40_000L)
    }
}
```

7.14 정리: 테스트가 없는 탐지 앱은 장난감

이 장의 결론은 하나입니다. **사용자 안전과 직결된 앱에서 테스트는 선택이 아닙니다.**

위험도 계산 공식이 잘못되면 80점짜리 카메라를 20점으로 잘못 보고할 수 있습니다. 에러 처리가 없으면 Wi-Fi 미연결 상태에서 앱이 죽습니다. 경계값을 검증하지 않으면 점수 20이 "안전" 으로 잘못 표시됩니다.

TDD는 이 모든 실수를 코드 작성 단계에서 잡아냅니다. MockK는 외부 의존성을 격리하고, Turbine은 비동기 Flow를 테스트 가능하게 만들고, TestDispatcher는 시간을 제어합니다.

"이 정도면 동작할 것 같다" 는 생각은 탐지 앱에서 허용되지 않습니다. 테스트가 통과했다는 증거만이 "동작한다" 를 증명합니다.

7.15 테스트 체크리스트

- ☐ 도메인 레이어 커버리지 90% 이상
- ☐ 위험도 경계값(19/20, 39/40, 59/60, 79/80) 모두 테스트
- ☐ 에러 코드(E1xxx/E2xxx/E3xxx) 각 1개 이상 테스트
- ☐ Flow 상태 전환이 Turbine으로 검증됨

- ☐ 타임아웃이 TestDispatcher로 검증됨
- ☐ E2E 테스트가 40초 이내 완료됨
- ☐ CI에서 커버리지 80% 미만 시 머지 차단 설정됨
- ☐ 비결정적 테스트(시간, 랜덤) 없음

제 8 장

Ch08: 데이터베이스 설계 — Room + SQLCipher로 스캔 이력 보호

이 장에서 배울 것: 몰래카메라 탐지 결과라는 민감한 데이터를 어떻게 안전하게 저장하는지 배웁니다. Room 엔티티 설계, TypeConverter 구현, SQLCipher와 Android Keystore 연동, DAO 패턴, 마이그레이션 전략까지 실제 구현 코드와 함께 설명합니다.

8.1 도입

호텔 금고를 떠올려보세요. 금고 안에는 여권, 현금, 귀중품이 들어 있습니다. 금고가 없다면 누군가 방에 들어와 물건을 가져갈 수 있습니다. 금고가 있더라도 비밀번호가 "1234" 라면 의미가 없습니다.

SearCam의 스캔 이력은 그 금고 안의 내용물과 같습니다. 사용자가 어느 숙소에 묵었는지, 어떤 기기를 발견했는지, 몇 시에 탐지를 실행했는지 — 이 데이터는 충분히 민감합니다. 단순한 SQLite 파일로 저장하면, adb backup 명령 한 줄로 누구든 내용을 열람할 수 있습니다.

SearCam의 답은 **Room + SQLCipher + Android Keystore** 조합입니다. Room이 SQL 보일러플레이트를 제거하고, SQLCipher가 데이터베이스 파일 자체를 AES-256으로 암호화하며, Android Keystore가 암호화 키를 하드웨어 수준에서 보호합니다. 이 세 층이 함께 작동할 때 비로소 "안전한 금고" 가 완성됩니다.

8.2 8.1 데이터베이스 전체 구조

SearCam Phase 1은 서버 없이 **모든 데이터를 기기 로컬에만 저장**합니다. 프라이버시 최우선 설계입니다. 데이터가 서버로 나가지 않으니 유출 경로가 그만큼 줄어듭니다.

```
┌──────────────
```

기와 여러 위험 포인트가 매달린다. 체크리스트는 독립 테이블로 선택적으로 리포트에 연결됩니다.

8.2.1 테이블 목록

테이블	엔티티 클래스	역할
`scan_reports`	`ScanReportEntity`	스캔 세션 마스터 레코드
`devices`	`DeviceEntity`	발견된 네트워크 기기
`risk_points`	`RiskPointEntity`	렌즈/IR/EMF 의심 포인트
`checklists`	`ChecklistEntity`	육안 점검 체크리스트

8.3 8.2 엔티티 설계

8.3.1 ScanReportEntity — 스캔 세션의 마스터 레코드

스캔 한 번의 결과 전체를 담는 테이블입니다. 실제 구현 코드를 보면 설계 의도가 명확하게 드러납니다.

```
// app/src/main/java/com/searcam/data/local/entity/ScanRep

@Entity(tableName = "scan_reports")
data class ScanReportEntity(
```

```
    @PrimaryKey
    @ColumnInfo(name = "id")
    val id: String,            // UUID — 자동 증가 정수 대신 U

    @ColumnInfo(name = "mode")
    val mode: String,          // "QUICK" | "FULL" | "LENS"

    @ColumnInfo(name = "started_at")
    val startedAt: Long,       // Unix epoch millis

    @ColumnInfo(name = "completed_at")
    val completedAt: Long,

    @ColumnInfo(name = "risk_score")
    val riskScore: Int,        // 0 ~ 100

    @ColumnInfo(name = "risk_level")
    val riskLevel: String,     // "SAFE" | "ATTENTION" | "C

    @ColumnInfo(name = "devices_json")
    val devicesJson: String = "[]",     // 기기 목록 JSON
```

```
    @ColumnInfo(name = "findings_json")
    val findingsJson: String = "[]",    // 발견 사항 JSON

    @ColumnInfo(name = "location_note")
    val locationNote: String = "",

    @ColumnInfo(name = "created_at")
    val createdAt: Long = System.currentTimeMillis()
)
```

여기서 주목할 설계 결정이 두 가지 있습니다.

첫째, PK로 UUID 사용. `autoGenerate = true`의 자동 증가 정수 대신 UUID 문자열을 PK로 씁니다. 이유는 간단합니다 — 오프라인 기기에서 생성된 레코드를 나중에 서버나 다른 기기와 동기화할 때, 정수 PK는 반드시 충돌이 납니다. UUID는 전역적으로 유일하므로 Phase 2에서 클라우드 동기화를 추가할 때 PK를 바꿀 필요가 없습니다.

둘째, 복잡한 중첩 객체는 JSON으로 직렬화. `devicesJson`, `findingsJson`처럼 리스트나 복합 객체는 JSON 문자열로 저장합니다. Room이 관계형 테이블로 관리하지 않는 이유는 조회 빈도와 복잡도의 균형 때문입니다. 스캔 결과 화면에서는 리포트 전체를 한 번에 읽으므로, JOIN보다 단일 레코드 조회가 더 효율적입니다.

설계 문서의 스키마와 실제 구현 간에는 일부 차이가 있습니다. 설계 문서는 `layer1_score`, `layer2_score` 등 레이어별 점수를 개별 컬럼으로 두지만, 실제 구현에서는 `findings_json`에 통합했습니다. 이는 MVP 속도를 우선한 실용적 선택입니다.

8.3.2 DeviceEntity — 발견된 네트워크 기기

```
// app/src/main/java/com/searcam/data/local/entity/DeviceE


@Entity(
    tableName = "devices",
    foreignKeys = [
        ForeignKey(
            entity = ScanReportEntity::class,
            parentColumns = ["id"],
            childColumns = ["report_id"],
            onDelete = ForeignKey.CASCADE   // 리포트 삭제 시
        )
    ],
    indices = [Index(value = ["report_id"])]
)
data class DeviceEntity(
    @PrimaryKey(autoGenerate = true)
```

```
    @ColumnInfo(name = "device_pk")
    val devicePk: Long = 0,

    @ColumnInfo(name = "report_id")
    val reportId: String,        // FK → scan_reports.id

    @ColumnInfo(name = "ip_address")
    val ipAddress: String,

    @ColumnInfo(name = "mac_address")
    val macAddress: String,

    @ColumnInfo(name = "vendor")
    val vendor: String?,         // OUI 매칭 결과 (없으면 null

    @ColumnInfo(name = "hostname")
    val hostname: String?,       // mDNS 발견 호스트명

    @ColumnInfo(name = "is_camera")
    val isCamera: Boolean,       // 카메라 의심 여부

    @ColumnInfo(name = "open_ports")
```

```
    val openPorts: String = ""  // "554,80,8000" — 콤마 구분
)
```

`onDelete = ForeignKey.CASCADE`가 핵심입니다. 사용자가 스캔 리포트를 삭제하면, 그 리포트에 연결된 모든 기기 레코드도 자동으로 삭제됩니다. 앱 코드에서 별도로 "관련 기기도 삭제" 로직을 작성할 필요가 없습니다. 데이터베이스가 참조 무결성을 보장합니다.

`indices = [Index(value = ["report_id"])]` 선언도 빠뜨릴 수 없습니다. 외래키 컬럼에 인덱스가 없으면, "이 리포트의 모든 기기를 가져와" 라는 쿼리가 전체 테이블 스캔(Full Table Scan)을 수행합니다. 기기가 수백 개 쌓이면 체감될 만큼 느려집니다.

8.3.3 RiskPointEntity — 위험 포인트의 통합 저장

렌즈 의심 포인트(Retroreflection), IR LED 포인트, 자기장 이상 지점 — 세 종류의 데이터를 하나의 테이블에 담는 설계입니다.

```
// app/src/main/java/com/searcam/data/local/entity/RiskPoi

@Entity(
    tableName = "risk_points",
    foreignKeys = [
```

```
        ForeignKey(
            entity = ScanReportEntity::class,
            parentColumns = ["id"],
            childColumns = ["report_id"],
            onDelete = ForeignKey.CASCADE,
        )
    ],
    indices = [
        Index(value = ["report_id"]),
        Index(value = ["point_type"]),
        Index(value = ["score"], orders = [Index.Order.DES
    ],
)
data class RiskPointEntity(
    @PrimaryKey(autoGenerate = true)
    @ColumnInfo(name = "id")
    val id: Long = 0,

    @ColumnInfo(name = "report_id")
    val reportId: String,

    // "LENS_RETROREFLECTION" | "IR_LED" | "EMF_ANOMALY"
```

```
    @ColumnInfo(name = "point_type")
    val pointType: String,

    // 정규화 좌표 (0.0~1.0) — EMF는 null
    @ColumnInfo(name = "x")
    val x: Float? = null,

    @ColumnInfo(name = "y")
    val y: Float? = null,

    @ColumnInfo(name = "score")
    val score: Int,                    // 0~100

    // 렌즈 포인트 전용 필드들
    @ColumnInfo(name = "size_px")
    val sizePx: Int? = null,

    @ColumnInfo(name = "circularity")
    val circularity: Float? = null, // 원형도, 렌즈: > 0.8

    @ColumnInfo(name = "brightness")
    val brightness: Float? = null,
```

```
    // IR 포인트 전용
    @ColumnInfo(name = "duration_ms")
    val durationMs: Long? = null,   // 지속 시간, 의심 기준:

    // 렌즈 검증 결과
    @ColumnInfo(name = "flash_verified")
    val flashVerified: Boolean? = null, // 플래시 OFF 시 소실

    // EMF 포인트 전용
    @ColumnInfo(name = "emf_delta")
    val emfDelta: Float? = null,    // 자기장 변화량 (μT)

    @ColumnInfo(name = "emf_level")
    val emfLevel: String? = null,   // "NORMAL" | "INTERES

    @ColumnInfo(name = "evidence")
    val evidence: String,           // 감지 근거 텍스트 (한국

    @ColumnInfo(name = "created_at")
    val createdAt: Long = System.currentTimeMillis(),
)
```

이 설계를 **단일 테이블 상속(Single Table Inheritance)** 패턴이라고 합니다. 세 종류의 포인트를 각각 별도 테이블(`lens_points`, `ir_points`, `emf_points`)로 나눌 수도 있었습니다. 그렇게 하면 각 테이블이 꼭 필요한 컬럼만 갖지만, JOIN이 늘어나고 쿼리가 복잡해집니다.

SearCam이 단일 테이블을 선택한 이유: "이 리포트의 모든 위험 포인트를 점수 내림차순으로 가져와" 라는 쿼리가 한 번의 SELECT로 끝납니다. 대신 `pointType`에 따라 관련 없는 컬럼은 null이 됩니다. 이 트레이드오프는 MVP 단계에서 합리적입니다.

인덱스가 세 개나 선언된 점도 눈에 띕니다. - `report_id` 인덱스: 특정 리포트의 포인트 조회 속도 - `point_type` 인덱스: 렌즈 포인트만, IR 포인트만 필터링 - `score DESC` 인덱스: 가장 위험한 포인트부터 정렬

8.3.4 ChecklistEntity — 육안 점검 기록

```
// app/src/main/java/com/searcam/data/local/entity/Checkli

@Entity(tableName = "checklists")
data class ChecklistEntity(
    @PrimaryKey(autoGenerate = true)
    @ColumnInfo(name = "id")
```

```
    val id: Long = 0,

    // "hotel" | "bathroom" | "fitting_room"
    @ColumnInfo(name = "template_id")
    val templateId: String,

    @ColumnInfo(name = "performed_at")
    val performedAt: Long = System.currentTimeMillis(),

    @ColumnInfo(name = "location_note")
    val locationNote: String = "",

    // {"연기 감지기 렌즈 확인": true, "시계 뒷면 확인": false,
    @ColumnInfo(name = "items_json")
    val itemsJson: String = "{}",

    @ColumnInfo(name = "total_items")
    val totalItems: Int = 0,

    @ColumnInfo(name = "completed_items")
    val completedItems: Int = 0
)
```

체크리스트는 스캔 리포트와 외래키로 연결하지 않습니다. 독립적으로 존재할 수 있습니다 — 앱 스캔 없이 육안 점검만 수행할 수 있기 때문입니다. 대신 리포트 화면에서 연관 체크리스트를 보여줄 때는 `performed_at` 시간 기반으로 매칭합니다.

8.4 8.3 TypeConverter — Room이 모르는 타입을 가르치기

Room은 기본 타입(Int, String, Long, Boolean, Float)만 SQLite에 저장할 수 있습니다. `List<String>`, `RiskLevel` 같은 커스텀 타입은 저장 방법을 직접 알려줘야 합니다. 이 역할을 `TypeConverter`가 합니다.

```
// app/src/main/java/com/searcam/data/local/converter/Type

class SearCamTypeConverters {

    // List<String> ↔ JSON 배열 문자열
    @TypeConverter
    fun fromStringList(value: List<String>?): String {
        if (value.isNullOrEmpty()) return "[]"
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        return value.joinToString(
            separator = ",",
            prefix = "[",
            postfix = "]",
            transform = { item -> "\"${item.replace("\"",
        )
    }

    @TypeConverter
    fun toStringList(value: String?): List<String> {
        if (value.isNullOrBlank() || value.trim() == "[]")
        return try {
            val trimmed = value.trim().removePrefix("[").r
            if (trimmed.isBlank()) return emptyList()
            parseStringArray(trimmed)
        } catch (e: Exception) {
            Timber.e(e, "TypeConverter: List<String> 파싱 실
            emptyList()
        }
    }

    // RiskLevel enum ↔ String
```

```
    @TypeConverter
    fun fromRiskLevel(value: RiskLevel?): String {
        return value?.name ?: RiskLevel.SAFE.name
    }

    @TypeConverter
    fun toRiskLevel(value: String?): RiskLevel {
        if (value.isNullOrBlank()) return RiskLevel.SAFE
        return try {
            RiskLevel.valueOf(value)
        } catch (e: IllegalArgumentException) {
            Timber.e(e, "TypeConverter: 알 수 없는 RiskLevel
            RiskLevel.SAFE
        }
    }

    // ScanMode enum ↔ String
    @TypeConverter
    fun fromScanMode(value: ScanMode?): String = value?.na

    @TypeConverter
    fun toScanMode(value: String?): ScanMode {
```

```
        if (value.isNullOrBlank()) return ScanMode.QUICK
        return try {
            ScanMode.valueOf(value)
        } catch (e: IllegalArgumentException) {
            Timber.e(e, "TypeConverter: 알 수 없는 ScanMode
            ScanMode.QUICK
        }
    }
}
```

SearCam TypeConverter의 특징이 두 가지 있습니다.

Gson 없는 직렬화. 많은 프로젝트가 Gson 라이브러리를 TypeConverter에 씁니다. `gson.toJson(list)`, `gson.fromJson(str, type)` 한 줄이면 끝납니다. SearCam은 Gson 의존성을 피하기 위해 직접 파싱 로직을 구현했습니다. APK 크기를 줄이고 의존성 체인을 단순하게 유지하는 선택입니다.

방어적 파싱. `toRiskLevel()`에서 `try-catch`로 감싸고, 알 수 없는 값이 오면 `SAFE`를 기본값으로 반환합니다. DB에 "DANGER_EXTREME" 같은 예상 못한 값이 들어있더라도 앱이 크래시하지 않습니다. 앱 업데이트로 enum 값이 바뀐 경우에도 안전하게 동작합니다.

8.4.1 TypeConverter 등록

TypeConverter는 `@Database` 어노테이션에 `@TypeConverters`를 추가해야 활성화됩니다.

```
// app/src/main/java/com/searcam/data/local/AppDatabase.kt


@Database(
    entities = [
        ScanReportEntity::class,
        DeviceEntity::class,
        ChecklistEntity::class,
        RiskPointEntity::class
    ],
    version = 1,
    exportSchema = true   // schemas/ 디렉토리에 스키마 JSON
)
@TypeConverters(SearCamTypeConverters::class)
abstract class AppDatabase : RoomDatabase() {

    abstract fun reportDao(): ReportDao
    abstract fun deviceDao(): DeviceDao
    abstract fun checklistDao(): ChecklistDao
}
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

`exportSchema = true`는 빠뜨리기 쉬운 설정이지만 중요합니다. `true`로 설정하면 Room이 각 버전의 스키마를 `schemas/` 디렉토리에 JSON으로 기록합니다. 마이그레이션 테스트 작성 시 이 파일이 기준 스키마로 사용됩니다. 없으면 Room 자동 마이그레이션 기능을 사용할 수 없습니다.

8.5 8.4 SQLCipher + Android Keystore — 데이터 암호화

일반 Room 데이터베이스는 SQLite 파일을 평문으로 저장합니다. Android 기기를 루팅하거나 `adb backup`으로 앱 데이터를 추출하면 DB 파일을 읽을 수 있습니다. 스캔 이력처럼 민감한 데이터에는 적합하지 않습니다.

SQLCipher는 SQLite를 AES-256으로 암호화하는 오픈소스 라이브러리입니다. 패스프레이즈(비밀번호)를 DB에 적용하면, DB 파일을 꺼내도 패스프레이즈 없이는 내용을 볼 수 없습니다.

그런데 패스프레이즈를 어디에 저장해야 할까요? 앱 코드에 하드코딩하면 디컴파일로 노출됩니다. SharedPreferences에 저장해도 루팅 기기에서 읽힐 수 있습니다.

답은 **Android Keystore**입니다. 하드웨어 보안 모듈(HSM)에 암

호화 키를 저장하고, 키의 원본 바이트는 앱 프로세스 밖으로 나오지 않습니다. Android 6.0 이상에서는 키가 TEE(Trusted Execution Environment)에 보호됩니다.

```
// app/src/main/java/com/searcam/di/DatabaseModule.kt

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    @Provides
    @Singleton
    fun provideAppDatabase(
        @ApplicationContext context: Context
    ): AppDatabase {
        val passphrase = getDatabasePassphrase()
        val factory = SupportFactory(passphrase)     // SQL
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "searcam.db"
        )
            .openHelperFactory(factory)              // 일반
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
            .fallbackToDestructiveMigration()        // 개발
            .build()
    }

    /**
     * Android Keystore에서 DB 암호화 키를 가져온다.
     * 키가 없으면 새로 생성하여 Keystore에 안전하게 저장한다.
     */
    private fun getDatabasePassphrase(): ByteArray {
        val keyAlias = "searcam_db_key"
        val keyStore = KeyStore.getInstance("AndroidKeySto

        val secretKey: SecretKey = if (keyStore.containsAl
            // 기존 키 가져오기
            (keyStore.getEntry(keyAlias, null) as KeyStore
        } else {
            // 키 신규 생성
            Timber.d("DB 암호화 키 신규 생성")
            val keyGenerator = KeyGenerator.getInstance(
                KeyProperties.KEY_ALGORITHM_AES, "AndroidK
            )
            keyGenerator.init(
```

```
                KeyGenParameterSpec.Builder(
                    keyAlias,
                    KeyProperties.PURPOSE_ENCRYPT or KeyPr
                )
                    .setBlockModes(KeyProperties.BLOCK_MOD
                    .setEncryptionPaddings(KeyProperties.E
                    .setKeySize(256)        // AES-256
                    .build()
            )
            keyGenerator.generateKey()
        }
        return secretKey.encoded
    }

    @Provides
    fun provideReportDao(database: AppDatabase): ReportDao

    @Provides
    fun provideDeviceDao(database: AppDatabase): DeviceDao

    @Provides
    fun provideChecklistDao(database: AppDatabase): Checkl
```

```
}
```

이 코드의 보안 흐름을 단계별로 설명합니다.

```
[앱 최초 설치]
  1. "searcam_db_key" alias로 Android Keystore 조회
  2. 키 없음 → KeyGenerator로 AES-256 키 생성
  3. Keystore에 안전하게 저장 (TEE 보호)
  4. secretKey.encoded → 패스프레이즈로 사용
  5. SQLCipher SupportFactory에 패스프레이즈 전달
  6. 암호화된 searcam.db 파일 생성

[이후 앱 실행]
  1. Keystore에서 기존 키 조회 (alias: "searcam_db_key")
  2. 동일한 패스프레이즈 추출
  3. DB 복호화하여 정상 접근
```

8.5.1 암호화 적용 전후 비교

항목	일반 Room	Room + SQLCipher
DB 파일 형식	평문 SQLite	AES-256 암호화
adb backup 탈취	내용 노출	암호문만 보임
루팅 기기 접근	내용 노출	키 없이 불가
성능 오버헤드	없음	약 5~15% 쓰기 증가

항목	일반 Room	Room + SQLCipher
추가 의존성	없음	`net.zetetic:android-database-sqlc`

8.6 8.5 DAO 설계 — 필요한 것만, 정확하게

DAO(Data Access Object)는 SQL을 숨기고 코틀린 함수를 노출하는 인터페이스입니다. 잘 설계된 DAO는 "어떤 SQL을 쓰는지"가 아니라 "무엇을 원하는지"를 표현합니다.

8.6.1 ReportDao

```
// app/src/main/java/com/searcam/data/local/dao/ReportDao.

@Dao
interface ReportDao {

    // 저장 (중복 시 덮어씀)
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(report: ScanReportEntity)

    // 모든 리포트 실시간 구독 — Flow로 반환
```

```
    @Query("SELECT * FROM scan_reports ORDER BY started_at
    fun observeAll(): Flow<List<ScanReportEntity>>

    // ID로 단건 조회 — suspend fun (일회성)
    @Query("SELECT * FROM scan_reports WHERE id = :id LIMI
    suspend fun findById(id: String): ScanReportEntity?

    // 단건 삭제
    @Delete
    suspend fun delete(report: ScanReportEntity)

    // 전체 삭제
    @Query("DELETE FROM scan_reports")
    suspend fun deleteAll()

    // 개수 조회
    @Query("SELECT COUNT(*) FROM scan_reports")
    suspend fun count(): Int
}
```

DAO 메서드 반환 타입 선택 기준이 있습니다.

상황	반환 타입	이유
목록 화면 (실시간 갱신 필요)	`Flow<List<T>>`	DB 변경 시 UI 자동 갱신
단건 조회 (한 번만)	`suspend fun T?`	코루틴으로 비동기, 완료 후 결과 반환
저장/삭제	`suspend fun`	IO 스레드에서 비동기 처리
개수 확인	`suspend fun Int`	즉시 결과 필요, 구독 불필요

`observeAll()`이 `Flow`를 반환하는 것이 핵심입니다. Room은 `@Query` + `Flow` 조합에서 해당 테이블이 변경될 때마다 새 데이터를 자동으로 emit합니다. 새 스캔을 저장하면 리포트 목록 화면이 자동으로 갱신됩니다. `notifyDataSetChanged()`를 호출하거나 화면을 수동으로 새로고침할 필요가 없습니다.

8.7 8.6 마이그레이션 전략 — 데이터는 사용자의 것

앱 업데이트로 DB 스키마가 바뀌면 어떻게 될까요? Room은 버전 번호로 변경을 감지합니다. 마이그레이션 경로를 제공하지 않으면 Room이 예외를 던집니다.

8.7.1 원칙

1. **`fallbackToDestructiveMigration()`은 개발 중에만.** 이 옵션은 마이그레이션 실패 시 DB 전체를 삭제하고 새로 만듭니다. 개발 편의를 위해 현재 코드에 사용되고 있지만, 프로덕션 릴리즈 전에는 반드시 제거해야 합니다. 사용자 스캔 이력이 모두 사라집니다.
2. **스키마 변경은 버전 증가 + Migration 클래스.** `@Database(version = 2)`로 올리고 `Migration(1, 2)`를 작성해 Room에 등록합니다.
3. **마이그레이션은 반드시 테스트.** `MigrationTestHelper`로 v1 → v2 마이그레이션이 실제로 동작하는지 검증합니다.

8.7.2 버전 계획

DB 버전	변경 내용	예상 시기
1	초기 스키마 (4개 테이블)	Phase 1 MVP
2	RiskPoint에 LiDAR 컬럼 추가	Phase 3
3	커뮤니티 맵 테이블 추가	Phase 3
4	ML 학습 데이터 테이블 추가	Phase 3

8.7.3 마이그레이션 예시

```
// v1 → v2: risk_points에 LiDAR 컬럼 추가
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE risk_points ADD COLUMN lid
        db.execSQL("ALTER TABLE risk_points ADD COLUMN lid
    }
}

// DatabaseModule에 등록
Room.databaseBuilder(context, AppDatabase::class.java, "se
    .openHelperFactory(factory)
    .addMigrations(MIGRATION_1_2)   // fallbackToDestructi
    .build()
```

Room 2.4.0 이상에서는 단순한 컬럼 추가라면 `autoMigrations`

를 쓸 수도 있습니다.

```
@Database(
    entities = [...],
    version = 2,
    exportSchema = true,
    autoMigrations = [
        AutoMigration(from = 1, to = 2)  // 컬럼 추가/삭제만
    ]
)
```

단, 컬럼 이름 변경, 테이블 분리/병합 같은 복잡한 변경은 여전히 수동 `Migration`이 필요합니다.

8.8 8.7 데이터 보존 정책

무료 사용자의 스캔 이력은 최근 10건으로 제한합니다. 리포트 저장 시 개수를 확인하고 초과분을 삭제합니다.

```
// ReportRepository 내부 구현 패턴
suspend fun saveReport(report: ScanReport) {
    val entity = report.toEntity()
    reportDao.insert(entity)
```

```
    // 무료 사용자: 10건 초과 시 오래된 것 삭제
    if (!isPremium) {
        val count = reportDao.count()
        if (count > MAX_FREE_REPORTS) {
            // 오래된 리포트 삭제 → CASCADE로 devices, risk_p
            val oldest = reportDao.observeAll()
                .first()
                .drop(MAX_FREE_REPORTS)
            oldest.forEach { reportDao.delete(it) }
        }
    }
}

companion object {
    private const val MAX_FREE_REPORTS = 10
}
```

사용자 유형	저장 한도	예상 용량
무료	최근 10건	~55KB
프리미엄 (월 2,900원)	무제한	월 30건 기준 ~2MB/년

8.9 핵심 정리

개념	결론
PK 타입	UUID 문자열 — 미래 동기화 대비
복합 타입 저장	JSON 직렬화 — JOIN 최소화
암호화	SQLCipher + Keystore — 파일 탈취 무력화
인덱스	FK 컬럼에 필수 — 쿼리 성능 보장
마이그레이션	수동 Migration 클래스 — 데이터 손실 방지
TypeConverter	방어적 파싱 — 크래시 방지

- ☒ `exportSchema = true`로 스키마 버전 추적
- ☒ FK 컬럼에 반드시 `Index` 선언
- ☒ `onDelete = CASCADE`로 참조 무결성 유지
- ☒ 프로덕션에서 `fallbackToDestructiveMigration()` 제거
- ☒ DB 파일을 평문으로 저장하지 말 것
- ☒ 패스프레이즈를 코드에 하드코딩하지 말 것

8.10 다음 장 예고

데이터를 어떻게 저장하는지 배웠습니다. 이제 데이터가 어떻게 흘러가는지 — 센서에서 화면까지의 단방향 데이터 흐름을 Ch09에서 다룹니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

참고자료: docs/07-db-schema.md, App-Database.kt, DatabaseModule.kt, TypeConverters.kt

—

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

Ch09: 데이터 흐름 설계 — 센서에서 리포트까지 단방향

—

> **이장에서 배울 것**: 자기장 센서의 원시값이 어떻게 화면의 위

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.11 도입

강물을 떠올려보세요. 산에서 눈이 녹아 흘러내리면, 개울이 되고, 강이 되고, 바다로 흘러갑니다. 물은 항상 한 방향으로만 흐릅니다. 강물이 거꾸로 거슬러 흘러 산으로 올라가지는 않습니다.

SearCam의 데이터도 강물처럼 한 방향으로만 흐릅니다. 하드웨어 센서에서 출발한 원시 데이터가 분석기를 지나 도메인 모델이 되고, UseCase가 가공하고, ViewModel이 UI 상태로 변환하고, 마지막으로 Compose 화면이 그려냅니다. 화면에서 센서로 거슬러 올라가는 경로는 없습니다.

이것이 **단방향 데이터 흐름(Unidirectional Data Flow, UDF)**입니다. 데이터가 어디서 왔는지, 어디로 가는지 추적하기 쉽고, 버그가 발생했을 때 "어느 층에서 문제가 생겼는지" 격리하기 쉽습니다.

8.12 9.1 전체 데이터 흐름 개요

```
┌──────────────────────────────────────────────────────────────────────────────
│                    SearCam 데이터 흐름                    │
│                                                           │
│  [하드웨어 / 시스템]                                                  │
│   ┌──────────────────────┐   ┌──────────────────────┐   ┌──────────
```

```
│  │Wi-Fi ARP │  │ Camera   │  │Magnetic  │  │ IR      │    │
│  │ Sensor   │  │ (CameraX)│  │ Sensor   │  │ Camera  │    │
│  └──────────┴──────────┘  └──────────┴──────────┘  └──────────┴
│     │         │         │         │         │
│ [data/ 계층 — Repository Impl, Sensor, Analyzer]
│     │         │         │         │         │
│  ┌──────▼──────────────────────────▼──────────────────
│  │     Repository 인터페이스 (domain/)          │   │
│  │ WifiScanRepository  LensDetectionRepository        │
│  │ MagneticRepository  IrDetectionRepository          │
│  └──────────────────────────────────────────────────┴──────
│                 │                     │
│ [domain/ 계층 — UseCase]                          │
│  ┌──────────────────────────────────────────▼──────
│  │ RunQuickScanUseCase  RunFullScanUseCase        │    │
│  │ CalculateRiskUseCase                      │    │
│  └──────────────────────────────────────────────────┴──────
│                 │                     │
│ [ui/ 계층 — ViewModel]                            │
│  ┌──────────────────────────────────────────▼──────
│  │ ScanViewModel                            │    │
│  │ StateFlow<ScanUiState>  SharedFlow<ScanUiEvent>  │
```

```
│  └──────────────────────────────────────────────────────┘
│                    │                    │
│  [ui/ 계층 — Compose Screen]                          │
│  ┌──────────────────────────────────────────────────▼
│  │ FullScanScreen, QuickScanScreen, ScanResultScreen │
│  └──────────────────────────────────────────────────
└──────────────────────────────────────────────────────
```

화살표가 항상 아래를 향한다는 점을 주목하세요. 상위 계층(UI)은 하위 계층(Repository)을 알지만, 하위 계층은 상위 계층을 전혀 모릅니다. `MagneticSensor`는 `ScanViewModel`이 존재하는지 모릅니다. 이것이 Clean Architecture의 의존성 규칙입니다.

8.13 9.2 Flow vs LiveData — SearCam의 선택

Jetpack에는 비동기 데이터 스트림을 다루는 두 가지 도구가 있습니다: LiveData와 Kotlin Flow. SearCam은 전면적으로 **Kotlin Flow**를 선택했습니다. 이유를 비교표로 정리합니다.

기준	LiveData	Kotlin Flow
생명주기 인식	내장 (Activity/Fragment 전용)	`repeatOnLifecycle()`로 해결
Android 의존성	강함 (AAC 필수)	없음 (pure Kotlin)
백프레셔 처리	없음	`buffer()`, `conflate()`, `sample()`
변환 연산자	제한적	`map`, `filter`, `combine`, `flatMap` 등 풍부
테스트	어려움 (Observer 필요)	`turbine` 라이브러리로 간결
Cold vs Hot	항상 Hot	Cold(`flow{}`), Hot(`StateFlow`, `SharedFlow`)
멀티플렉싱	불가	`combine()`, `merge()`, `zip()`
코루틴 통합	부분적	완전 통합

SearCam에서 결정적인 이유는 두 가지입니다.

첫째, Repository 계층이 Android 비의존성. `domain/repository/` 인터페이스와 `data/repository/` 구현체는 순수 Kotlin

입니다. 여기서 LiveData를 쓰면 `androidx.lifecycle` 의존성이 도메인 계층까지 침투합니다. Flow는 순수 Kotlin이므로 의존성 방향을 깨끗하게 유지합니다.

둘째, 센서 데이터의 백프레셔 제어. 자기장 센서는 20Hz(초당 20회) 데이터를 발생시킵니다. 렌즈 감지 카메라는 30fps입니다. UI는 60fps로 렌더링되지만 초당 20번 데이터 갱신이면 충분합니다. `sample(100)` 한 줄로 100ms마다 한 번씩만 UI를 갱신할 수 있습니다. LiveData에는 이런 제어 수단이 없습니다.

8.14 9.3 StateFlow + SharedFlow — 두 종류의 Hot Flow

SearCam ViewModel에서는 두 종류의 Hot Flow를 목적에 따라 구분해 사용합니다.

8.14.1 StateFlow — UI 상태 관리

현재 화면이 어떤 상태인지를 나타냅니다. Compose UI는 이것을 구독해 화면을 그립니다.

```
// ScanViewModel.kt
```

```
// 내부: 변경 가능
private val _uiState = MutableStateFlow<ScanUiState>(ScanU

// 외부 노출: 읽기 전용
val uiState: StateFlow<ScanUiState> = _uiState.asStateFlow
```

StateFlow의 특징: - 항상 최신값을 하나 갖고 있습니다 (초기값 필수) - 새 구독자는 즉시 현재값을 받습니다 - 동일한 값이 연속으로 설정되면 emit하지 않습니다 (distinctUntilChanged 내장) - Compose에서 collectAsStateWithLifecycle()로 State 변환

8.14.2 SharedFlow — 일회성 이벤트 처리

네비게이션, 스낵바 메시지처럼 "한 번만 처리해야 하는" 이벤트에 사용합니다.

```
// ScanViewModel.kt

// 내부: 변경 가능 (버퍼 없음, replay=0)
private val _events = MutableSharedFlow<ScanUiEvent>()

// 외부 노출: 읽기 전용
val events: SharedFlow<ScanUiEvent> = _events.asSharedFlow
```

SharedFlow의 특징: - 초기값 없음 (replay 기본 0) - 이벤트를 놓칠 수 있음 — 구독 전에 emit된 이벤트는 수신 불가 - replay = 1로 설정하면 마지막 이벤트를 캐시 (주의해서 사용)

8.14.3 왜 구분하는가

StateFlow만 써서 이벤트를 처리하면 문제가 생깁니다. 화면 회전 시 새 구독이 시작되면서 StateFlow가 현재값을 다시 emit합니다. "네비게이션 이벤트" 상태가 남아있으면 화면 회전마다 네비게이션이 실행됩니다. SharedFlow는 이미 처리된 이벤트를 다시 emit하지 않아 이 문제를 피합니다.

```
ScanUiState (StateFlow)  ScanUiEvent (SharedFlow)
──────────────────────────────────────────────  ──────────────────
Idle                        NavigateToResult(id)
Scanning(progress=0.5)        ShowSnackbar("Wi-
Fi 필요")
Success(report)
Error(code, message)
```

8.15 9.4 ViewModel → UseCase → Repository 체인

Quick Scan의 데이터 흐름을 추적하며 각 계층의 역할을 살펴봅니다.

8.15.1 1단계: ViewModel이 UseCase를 실행

```
// ScanViewModel.kt

fun startQuickScan() {
    if (_uiState.value is ScanUiState.Scanning) return  //

    scanJob = viewModelScope.launch {
        // 초기 상태 전환
        _uiState.value = ScanUiState.Scanning(
            progress = 0f,
            currentStep = "Wi-Fi 네트워크 스캔 중...",
        )

        try {
            // UseCase Flow를 collect — 결과가 오면 Success로
            runQuickScanUseCase.invoke().collect { report
```

```
                _uiState.value = ScanUiState.Success(repor
                _events.emit(ScanUiEvent.NavigateToResult(
            }
        } catch (e: Exception) {
            _uiState.value = ScanUiState.Error(
                code = "E2001",
                message = "스캔 중 오류가 발생했습니다: ${e.me
            )
        }
    }
}
```

ViewModel은 "무엇을 할지" 만 결정합니다. "어떻게 할지" 는 UseCase가 담당합니다. ViewModel은 Wi-Fi 스캐너가 어떻게 동작하는지, ARP 테이블이 뭔지 모릅니다.

8.15.2 2단계: UseCase가 비즈니스 로직을 처리

```
// RunQuickScanUseCase.kt

operator fun invoke(): Flow<ScanReport> = flow {
    val reportId = UUID.randomUUID().toString()
    val startedAt = System.currentTimeMillis()
```

```
    // Repository에 Wi-Fi 스캔 위임
    val wifiLayerResult = runWifiLayer()

    val completedAt = System.currentTimeMillis()

    // 위험도 산출 (교차 검증 UseCase에 위임)
    val layerResults = mapOf(LayerType.WIFI to wifiLayerRe
    val (finalScore, correctionFactor) = calculateRiskUseC
    val riskLevel = RiskLevel.fromScore(finalScore)

    // 도메인 모델 조립
    val report = ScanReport(
        id = reportId,
        mode = ScanMode.QUICK,
        startedAt = startedAt,
        completedAt = completedAt,
        riskScore = finalScore,
        riskLevel = riskLevel,
        devices = wifiLayerResult.devices,
        findings = wifiLayerResult.findings,
        layerResults = layerResults,
        correctionFactor = correctionFactor,
```

```
        locationNote = "",
        retroPoints = emptyList(),
        irPoints = emptyList(),
        magneticReadings = emptyList(),
    )

    emit(report)  // ViewModel에 전달
}
```

UseCase는 `operator fun invoke()`로 정의해 `useCase()` 함수처럼 호출합니다. Kotlin 관용적인 패턴입니다. 반환 타입이 `Flow<ScanReport>`인 점도 중요합니다 — 스캔 완료 시 단 한 번 emit하고 종료합니다.

8.15.3 3단계: Repository가 데이터 소스를 추상화

```
// ReportRepositoryImpl.kt

override suspend fun saveReport(report: ScanReport): Resul
    return try {
        val entity = report.toEntity()  // 도메인 → 엔티티 변
        reportDao.insert(entity)
        Timber.d("리포트 저장 완료: id=${report.id}, score=$
        Result.success(Unit)
```

```
    } catch (e: Exception) {
        Timber.e(e, "리포트 저장 실패: id=${report.id}")
        Result.failure(e)
    }
}

override fun observeReports(): Flow<List<ScanReport>> =
    reportDao.observeAll().map { entities ->
        entities.map { it.toDomain() }  // 엔티티 → 도메인 변
    }
```

Repository는 두 가지를 합니다. **변환(mapping)**과 **에러 처리**. `Result<T>` 반환으로 UseCase가 try-catch를 반복하지 않게 합니다. `observeAll()`이 Room의 `Flow<List<ScanReportEntity>>`를 반환하면, `.map { ... }` 연산자로 도메인 모델 리스트로 변환합니다. UI는 `ScanReportEntity`를 알 필요가 없습니다.

8.16 9.5 병렬 레이어 실행 — coroutineScope + async

Full Scan의 핵심은 4개 레이어(Wi-Fi, 렌즈, IR, EMF)를 **동시에** 실행하는 것입니다. 순서대로 실행하면 30 + 60 + 45 + 30 = 165초가 걸립니다. 병렬로 실행하면 가장 오래 걸리는 레이어(렌즈 60초)만 기다립니다.

```
// RunFullScanUseCase.kt


/**
 * 4개 레이어를 coroutineScope + async로 병렬 실행한다.
 */
private suspend fun runAllLayersInParallel(
    lifecycleOwner: LifecycleOwner
): List<LayerResult> = coroutineScope {
    val wifiDeferred  = async { runWifiLayer() }
    val lensDeferred  = async { runLensLayer(lifecycleOwne
    val irDeferred    = async { runIrLayer(lifecycleOwner)
    val magneticDeferred = async { runMagneticLayer() }

    listOf(
        wifiDeferred.await(),
```

```
        lensDeferred.await(),
        irDeferred.await(),
        magneticDeferred.await(),
    )
}
```

`coroutineScope { }` 블록이 중요합니다. `async { }` 4개를 실행하면 즉시 `Deferred<LayerResult>` 4개를 반환합니다 — 아직 완료되지 않은 "약속" 입니다. 이후 `.await()`로 각 결과를 기다립니다. `coroutineScope`는 모든 자식 코루틴이 완료될 때까지 종료되지 않습니다.

8.16.1 SupervisorJob으로 격리된 실패

ScanViewModel은 `viewModelScope`를 사용합니다. `viewModelScope`는 내부적으로 `SupervisorJob`을 씁니다. 이 덕분에 Wi-Fi 레이어가 실패해도 렌즈 레이어와 EMF 레이어는 계속 실행됩니다.

```
┌── viewModelScope (SupervisorJob) ──────────────────────────────
│                                    │
│  ┌── scanJob ──────────────────────────────────────────────────
│  │                                 │  │
│  │ async → wifiLayer □  완료 (30초)          │  │
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
│ │ async → lensLayer □ 완료 (58초)                │ │
│  │  async  → irLayer      □  실패 (권한 거
부)              │ │
│ │ async → magneticLayer □ 완료 (28초)            │ │
│ │                                  │ │
│  │  → irLayer  실패해도 나머지 3개는 정상 완
료      │ │
│ │ → LayerResult(status=FAILED, score=0) 로 처
리    │ │
│ └──────────────────────────────────────────────────
│                                  │
└──────────────────────────────────────────────────
```

각 레이어 함수 내부에서도 에러를 잡아 `FAILED` 상태의 `LayerResult`를 반환합니다. 예외가 위로 전파되지 않습니다.

```
private suspend fun runWifiLayer(): LayerResult {
    val startAt = System.currentTimeMillis()
    return try {
        val devices = withTimeout(WIFI_LAYER_TIMEOUT_MS) {
            wifiScanRepository.scanDevices().getOrThrow()
        }
        LayerResult(
            layerType = LayerType.WIFI,
```

```
            status = ScanStatus.COMPLETED,
            score = devices.maxOfOrNull { it.riskScore } ?
            devices = devices,
            durationMs = System.currentTimeMillis() - star
            findings = emptyList(),
        )
    } catch (e: Exception) {
        // 실패해도 상위로 예외를 던지지 않고 FAILED 결과 반환
        LayerResult(
            layerType = LayerType.WIFI,
            status = ScanStatus.FAILED,
            score = 0,
            devices = emptyList(),
            durationMs = System.currentTimeMillis() - star
            findings = emptyList(),
        )
    }
}
```

8.16.2 withTimeout으로 레이어별 제한 시간

각 레이어에 개별 타임아웃이 적용됩니다.

```
companion object {
    private const val WIFI_LAYER_TIMEOUT_MS      = 30_000L
    private const val LENS_LAYER_TIMEOUT_MS      = 60_000L
    private const val IR_LAYER_TIMEOUT_MS        = 45_000L
    private const val MAGNETIC_LAYER_TIMEOUT_MS = 30_000L
}
```

`withTimeout()`은 지정 시간 내에 블록이 완료되지 않으면 `TimeoutCancellationException`을 던집니다. 이것도 catch에서 잡아 `FAILED` 결과를 반환합니다.

8.17 9.6 Flow 연산자 실전 패턴

SearCam에서 자주 쓰이는 Flow 연산자 패턴을 정리합니다.

8.17.1 map — 타입 변환

```
// ReportRepositoryImpl.kt

// ScanReportEntity Flow → ScanReport Flow
override fun observeReports(): Flow<List<ScanReport>> =
    reportDao.observeAll().map { entities ->
```

```
        entities.map { it.toDomain() }
    }
```

DB 엔티티가 바뀌면 자동으로 도메인 모델로 변환해서 흘려보냅니다.

8.17.2 combine — 여러 Flow 합류

교차 검증 로직에서 3개 레이어 결과를 하나로 합칠 때 사용합니다.

```
// 3개 레이어 결과를 하나의 Flow로 합류
combine(
    layer1Flow,     // Flow<Layer1Result>
    layer2Flow,     // Flow<Layer2Result>
    layer3Flow,     // Flow<Layer3Result>
) { l1, l2, l3 ->
    crossValidator.validate(l1, l2, l3)
}
.flowOn(Dispatchers.Default)
.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = ScanUiState.Idle
)
```

combine은 3개 Flow 중 어느 하나라도 새 값을 emit하면, 3개의 최신값을 인자로 람다를 실행합니다.

8.17.3 conflate — 밀린 이벤트 건너뛰기

렌즈 감지 프레임 분석처럼 UI가 처리 속도를 따라가지 못할 때 사용합니다.

```
lensDetectionRepository
    .observeRetroreflections()
    .conflate()    // 처리 지연 시 중간 emit 건너뜀, 최신값만 처
    .collect { points ->
        updateUiWithPoints(points)
    }
```

conflate()는 처리 중인 동안 새로 들어온 값들을 버리고 가장 최신 값만 남깁니다. 실시간 렌즈 탐지 UI에서 "오래된 프레임 결과"를 처리하느라 현재 프레임 결과를 놓치는 상황을 방지합니다.

8.17.4 sample — 주기적 샘플링

자기장 센서 20Hz 데이터를 100ms마다 한 번만 UI에 반영합니다.

```
magneticRepository
    .observeReadings()
    .sample(100)  // 100ms마다 가장 최신값 하나만 통과
```

```
    .flowOn(Dispatchers.Default)
    .collect { reading ->
        _magneticState.value = reading.toUiModel()
    }
```

sample(100)은 100ms 간격으로 그 시점의 최신값을 뽑아냅니다. 20Hz 센서에서 초당 20개가 들어오지만 UI는 10개만 처리합니다.

8.17.5 catch — 스트림 에러 처리

```
lensDetectionRepository
    .observeRetroreflections()
    .catch { e ->
        // 스트림에서 에러 발생 시 에러 상태로 전환, 스트림은 계속
        _uiState.value = ScanUiState.Error("E1001", "카메라
        emit(emptyList())  // 빈 결과로 계속 진행
    }
    .collect { points -> ... }
```

catch는 Flow 파이프라인 중간에 에러가 발생했을 때 잡아서 처리합니다. collect 블록에서 발생한 에러는 잡지 않으므로 주의해야 합니다.

8.17.6 flowOn — 디스패처 전환

```
magneticRepository
    .observeReadings()
    .map { it.toUiModel() }        // Default 스레드에서 변환
    .flowOn(Dispatchers.Default)   // map까지 Dispatchers.De
    .collect { model ->            // collect는 호출 스코프(M
        _state.value = model
    }
```

`flowOn`은 **그 앞의 연산자들**이 실행될 스레드를 결정합니다. `collect`는 호출한 코루틴 스코프의 스레드에서 실행됩니다. CPU 집약적인 변환(`map`, `filter`, `combine`)은 `Dispatchers.Default`에서 실행하고, UI 갱신은 `Dispatchers.Main`에서 실행하는 것이 원칙입니다.

8.18 9.7 스캔 취소 — Job.cancel()

사용자가 스캔 중 뒤로가기 버튼을 누르면 즉시 취소해야 합니다. 코루틴의 `Job.cancel()`이 이를 처리합니다.

```
// ScanViewModel.kt
```

```
private var scanJob: Job? = null

fun startQuickScan() {
    scanJob = viewModelScope.launch {
        // 스캔 로직 ...
    }
}

fun cancelScan() {
    scanJob?.cancel()   // 진행 중인 스캔 즉시 취소
    scanJob = null
    _uiState.value = ScanUiState.Idle
}

override fun onCleared() {
    super.onCleared()
    scanJob?.cancel()   // ViewModel 소멸 시 자동 취소
}
```

cancel()을 호출하면 코루틴 안의 모든 suspend 함수가 CancellationException을 받아 정상 종료됩니다. coroutineScope + async 패턴에서는 부모 Job이 취소되면 모든 자식 async도 함께 취소됩니다.

```
cancelScan()
  └── scanJob.cancel()
        └── RunFullScanUseCase 코루틴 취소
        └── coroutineScope 블록 취소
          ├── wifiDeferred.cancel()
            ├── lensDeferred.cancel()   → CameraX 분석 중단
          ├── irDeferred.cancel()       → IR 감지 중단
          └── magneticDeferred.cancel() → 센서 리스닝 중단
```

센서 리소스(CameraX, SensorManager)는 각 Repository의 `stopDetection()`에서 해제됩니다. 코루틴 취소 시 `finally` 블록에서 리소스 정리를 보장합니다.

```fn
// LensDetectionRepositoryImpl 예시 패턴
private suspend fun runLensLayer(lifecycleOwner: Lifecycle
    return try {
        lensDetectionRepository.startDetection(lifecycleOw
        // ... 분석 실행
    } catch (e: Exception) {
        // CancellationException도 여기서 처리됨
        runCatching { lensDetectionRepository.stopDetectio
```

```
        throw e  // CancellationException은 다시 throw
    }
}
```

8.19 9.8 Compose에서 Flow 구독

Compose UI는 `collectAsStateWithLifecycle()`로 Flow를 State로 변환합니다. 생명주기를 자동으로 인식해 백그라운드 상태에서는 구독을 중단합니다.

```
// FullScanScreen.kt (패턴)

@Composable
fun FullScanScreen(
    viewModel: ScanViewModel = hiltViewModel()
) {
    // StateFlow → State 변환 (생명주기 인식)
    val uiState by viewModel.uiState.collectAsStateWithLif

    // SharedFlow → LaunchedEffect로 일회성 이벤트 처리
    val navController = LocalNavController.current
    LaunchedEffect(Unit) {
```

```
        viewModel.events.collect { event ->
            when (event) {
                is ScanUiEvent.NavigateToResult ->
                    navController.navigate(Screen.Result(e
                is ScanUiEvent.ShowSnackbar ->
                    snackbarHostState.showSnackbar(event.m
            }
        }
    }

    // UI 렌더링
    when (uiState) {
        is ScanUiState.Idle -> IdleContent(onStartScan = v
        is ScanUiState.Scanning -> ScanningContent(state =
        is ScanUiState.Success -> {} // LaunchedEffect의 Na
        is ScanUiState.Error -> ErrorContent(error = uiSta
    }
}
```

`LaunchedEffect(Unit)`은 컴포저블이 처음 컴포지션에 진입할 때 한 번 시작됩니다. `SharedFlow`를 여기서 구독해 이벤트를 처리합니다. 화면 회전으로 컴포저블이 재구성되면 `LaunchedEffect`도 재시작되지만, 이미 emit된 SharedFlow 이벤트는 replay=0이

므로 다시 처리되지 않습니다.

8.20 9.9 전체 데이터 흐름 요약 다이어그램

Quick Scan의 데이터가 실제로 어떻게 흘러가는지 처음부터 끝까지 추적합니다.

```
[사용자 탭]
    │
  ▼
ScanViewModel.startQuickScan()
    │ viewModelScope.launch {}
    │ _uiState.value = Scanning
    │
  ▼
RunQuickScanUseCase.invoke()
    │ flow { ... }
    │
  ├──▶ wifiScanRepository.scanDevices()
    │      │ withTimeout(30_000)
    │      │
  │      ├──▶ WifiScanner.getArpTable()
```

```
    │      │      └── ARP 테이블 조회 (~1초)
    │      │
  │      ├──▶ WifiScanner.discoverMdns()
    │      │      └── mDNS/SSDP 탐색 (~5초)
    │      │
  │      ├──▶ OuiDatabase.match(mac)
    │      │      └── JSON에서 제조사 조회 (~0.5초)
    │      │
  │    └──▶ PortScanner.scan(suspiciousDevices)
    │          └── 의심 기기 포트 스캔 (~15초)
    │
  ├──▶ calculateRiskUseCase.invokeWithCorrection(layerResu
    │     └── 가중치 계산 → finalScore → RiskLevel
    │
  │ ScanReport 조립
  │ emit(report)
    │
  ▼
ScanViewModel.collect { report →
  │ _uiState.value = Success(report)
    │ _events.emit(NavigateToResult(report.id))
    │
```

```
  ├──▶ reportRepository.saveReport(report)
  │     └── report.toEntity() → reportDao.insert(entity)
   │          └── SQLCipher 암호화 저장
  │
  └──▶ navController.navigate(Screen.Result(report.id))
}
   │
  ▼
[ScanResultScreen 표시]
```

8.21 핵심 정리

개념	결론
데이터 방향	센서 → Repository → UseCase → ViewModel → UI (단방향)
StateFlow	UI 상태 — 항상 현재값 보유, 화면 재진입 시 즉시 수신
SharedFlow	일회성 이벤트 — 네비게이션, 스낵바
병렬 실행	`coroutineScope + async` — 4개 레이어 동시 실행
취소	`Job.cancel()` — 모든 자식 코루틴 일괄 취소
백프레셔	`conflate()`, `sample()` — 고빈도 센서 데이터 제어
에러 격리	try-catch → FAILED 반환 (상위 전파 없음)

- ☒ ViewModel은 UseCase만 알고, UseCase는 Repository만 안다
- ☒ UI 상태는 StateFlow, 이벤트는 SharedFlow로 분리
- ☒ 각 레이어에 개별 타임아웃 (`withTimeout`) 적용
- ☒ 레이어 실패는 FAILED LayerResult — 전체 스캔 중단 없음
- ☒ ViewModel 소멸 시 `onCleared()`에서 Job 취소
- ☒ Repository 계층에 LiveData 금지 (Android 의존성 침투)
- ☒ Composable에서 직접 Flow collect 금지 (`collectAsStateWithLifecycle` 사용)

8.22 다음 장 예고

데이터가 흘러가는 경로를 이해했습니다. 이제 그 데이터가 시작되는 지점 — 실제 Android 프로젝트 구조를 Ch10에서 다룹니다. Hilt DI 설정, 모듈 구조, 그리고 Clean Architecture 폴더 구조를 처음부터 세팅하는 과정을 설명합니다.

—

참

고

자

료:

docs/

05-

data-

flow.md,

Scan-

View-

Model.kt,

Run-

QuickScanUse-

Case.kt,

Run-

FullScanUse-

Case.kt,

Re-

portRepos-

ito-

ry-

Impl.kt

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

—

Ch10: 프로젝트 초기 설정 — 집을 짓기 전에 기초 공사부터

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

> **이장에서 배울 것:** 좋은 앱은 좋은 기초에서 시작합니다.

270 SearCam을

8.23 도입

건물을 지을 때 가장 중요한 작업은 기초 공사입니다. 눈에 보이지 않지만, 기초가 흔들리면 아무리 화려한 외벽도 소용없죠. 소프트웨어도 마찬가지입니다. 초기 프로젝트 설정에서 내린 결정들은 앱의 성장 내내 개발팀을 제약하거나 자유롭게 해줍니다.

SearCam을 처음 만들 때 저는 "나중에 고치면 되지" 라는 생각을 의도적으로 금지했습니다. 기술 부채는 이자가 붙습니다. 초반에 0.1% 타협이 나중에는 30% 재작업으로 돌아옵니다. 이 장에서는 SearCam의 기초 공사 전 과정을 함께 따라가 봅니다.

8.24 10.1 Gradle Version Catalog 도입 배경

8.24.1 라이브러리 지옥에서 탈출하기

멀티모듈 프로젝트를 해본 개발자라면 이 상황을 알 겁니다. 모듈A의 `build.gradle.kts`에는 `compose_version = "1.5.4"`, 모듈B에는 `compose_version = "1.5.8"`. 버전이 조금씩 달라서 빌드가 깨지는데, 어디서 깨졌는지 찾는 데만 30분이 걸립니다.

Gradle Version Catalog는 이 문제를 중앙화로 해결합니다.

libs.versions.toml 파일 하나가 모든 의존성의 진실의 원천(Single Source of Truth)이 됩니다.

```
# gradle/libs.versions.toml

[versions]
# 각 라이브러리 버전을 한 곳에서 관리합니다
agp = "8.3.0"
kotlin = "2.0.0"
compose-bom = "2024.04.01"
hilt = "2.51.1"
camerax = "1.3.3"
room = "2.6.1"
coroutines = "1.8.0"
timber = "5.0.1"

[libraries]
# 라이브러리를 별칭(alias)으로 정의합니다
androidx-compose-bom = { group = "androidx.compose", name
androidx-compose-ui = { group = "androidx.compose.ui", nam
androidx-compose-material3 = { group = "androidx.compose.m

hilt-android = { group = "com.google.dagger", name = "hilt
```

```
hilt-compiler = { group = "com.google.dagger", name = "hil

camerax-core = { group = "androidx.camera", name = "camera
camerax-camera2 = { group = "androidx.camera", name = "cam
camerax-lifecycle = { group = "androidx.camera", name = "c

room-runtime = { group = "androidx.room", name = "room-run
room-compiler = { group = "androidx.room", name = "room-co
room-ktx = { group = "androidx.room", name = "room-ktx", v

timber = { group = "com.jakewharton.timber", name = "timbe

[plugins]
# Gradle 플러그인도 Version Catalog로 관리합니다
android-application = { id = "com.android.application", ve
kotlin-android = { id = "org.jetbrains.kotlin.android", ve
hilt = { id = "com.google.dagger.hilt.android", version.re
ksp = { id = "com.google.devtools.ksp", version = "2.0.0-1
```

이렇게 정의하면 각 모듈의 `build.gradle.kts`에서 타입 안전하게 참조할 수 있습니다.

```
// app/build.gradle.kts — 버전 숫자가 한 줄도 없습니다
dependencies {
```

```
    implementation(libs.hilt.android)
    kapt(libs.hilt.compiler)

    implementation(libs.camerax.core)
    implementation(libs.camerax.camera2)
    implementation(libs.camerax.lifecycle)

    implementation(libs.room.runtime)
    implementation(libs.room.ktx)
    ksp(libs.room.compiler)

    implementation(libs.timber)
}
```

버전을 올릴 때는 `libs.versions.toml` 한 파일만 수정하면 됩니다. 10개 모듈이 있어도 마찬가지입니다.

8.25 10.2 실제 build.gradle.kts 전체 구성

앱 모듈의 전체 빌드 설정을 살펴봅니다. 각 설정의 이유를 주석으로 달았습니다.

```
// app/build.gradle.kts
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.hilt)
    alias(libs.plugins.ksp)  // Room, Hilt 어노테이션 처리에
}

android {
    namespace = "com.searcam"
    compileSdk = 34  // 최신 SDK로 컴파일 (새 API 사용 가능)

    defaultConfig {
        applicationId = "com.searcam"
        minSdk = 26  // Android 8.0 이상 (NsdManager 안정화
        targetSdk = 34
        versionCode = 1
        versionName = "1.0.0"

        testInstrumentationRunner = "com.searcam.HiltTestR
    }
```

```
    buildTypes {
        release {
            isMinifyEnabled = true  // R8 코드 난독화 + 최적
            isShrinkResources = true  // 미사용 리소스 제거
            proguardFiles(
                getDefaultProguardFile("proguard-android-o
                "proguard-rules.pro"
            )
        }
        debug {
            isDebuggable = true
            // 디버그 빌드는 최적화 없이 빠른 빌드
        }
    }

    buildFeatures {
        compose = true  // Jetpack Compose 활성화
        buildConfig = true  // BuildConfig 클래스 생성 (버전
    }

    composeOptions {
        kotlinCompilerExtensionVersion = "1.5.14"
```

```
    }

    // 자바 17 언어 기능 사용
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions {
        jvmTarget = "17"
    }
}
```

8.26 10.3 Hilt DI 설계 — 왜 Hilt를 선택했는가

8.26.1 의존성 주입이 뭐길래

레스토랑 비유로 시작합니다. 요리사가 요리를 만들 때마다 직접 시장에 가서 재료를 사온다고 상상해보세요. 비효율적이죠. 대신 식재료 팀이 필요한 재료를 미리 준비해서 요리사에게 전달(주입)합니다. 요리사는 재료가 어디서 왔는지 알 필요가 없습니다.

코드에서 의존성 주입(DI)이 바로 이 역할입니다. `WifiScanner`

가 `WifiManager`를 필요로 할 때, 직접 `getSystemService()`를 호출하는 대신 외부에서 주입받습니다. 덕분에 테스트 시 가짜(Mock) `WifiManager`를 주입할 수 있어 하드웨어 없이도 테스트가 가능합니다.

8.26.2 Koin 대신 Hilt를 선택한 이유

기준	Hilt	Koin
컴파일 타임 검증	O (오류를 빌드 시 발견)	X (런타임 오류 가능)
성능	컴파일 타임 코드 생성	리플렉션 기반 (느림)
Android 통합	공식 지원 (Jetpack)	서드파티
학습 곡선	가파름	완만함

SearCam은 탐지 신뢰성이 생명인 앱입니다. 런타임 DI 오류로 앱이 크래시 나면 사용자가 위험한 공간에 있을 때 도움을 못 받습니다. 컴파일 타임 안전성을 위해 Hilt를 선택했습니다.

8.26.3 SearCam의 DI 모듈 구조

```
// di/AppModule.kt — 앱 전역 싱글톤을 제공합니다
@Module
@InstallIn(SingletonComponent::class)
object AppModule {
```

```
    @Provides
    @Singleton
    fun provideApplicationContext(
        @ApplicationContext context: Context
    ): Context = context

    // IO 디스패처를 주입받아 테스트에서 교체 가능하게 합니다
    @Provides
    @Singleton
    @IoDispatcher
    fun provideIoDispatcher(): CoroutineDispatcher = Dispa

    @Provides
    @Singleton
    @DefaultDispatcher
    fun provideDefaultDispatcher(): CoroutineDispatcher =
}

// di/SensorModule.kt — 센서 관련 시스템 서비스를 제공합니다
@Module
@InstallIn(SingletonComponent::class)
object SensorModule {
```

```
    @Provides
    @Singleton
    fun provideWifiManager(
        @ApplicationContext context: Context
    ): WifiManager =
        // Wi-Fi 서비스는 applicationContext로 가져와야 메모리
        context.applicationContext
            .getSystemService(Context.WIFI_SERVICE) as Wif

    @Provides
    @Singleton
    fun provideSensorManager(
        @ApplicationContext context: Context
    ): SensorManager =
        context.getSystemService(Context.SENSOR_SERVICE) a

    @Provides
    @Singleton
    fun provideNsdManager(
        @ApplicationContext context: Context
    ): NsdManager =
```

```
        context.getSystemService(Context.NSD_SERVICE) as N
}

// di/RepositoryModule.kt — 인터페이스와 구현체를 연결합니다
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    // domain 레이어의 인터페이스를 data 레이어의 구현체에 바인딩
    @Binds
    @Singleton
    abstract fun bindWifiScanRepository(
        impl: WifiScanRepositoryImpl
    ): WifiScanRepository

    @Binds
    @Singleton
    abstract fun bindMagneticRepository(
        impl: MagneticRepositoryImpl
    ): MagneticRepository

    @Binds
    @Singleton
```

```
    abstract fun bindLensDetectionRepository(
        impl: LensDetectionRepositoryImpl
    ): LensDetectionRepository
}
```

@Binds와 @Provides의 차이가 중요합니다. @Binds는 인터페이스와 구현체를 연결할 때, @Provides는 직접 객체를 생성해서 반환할 때 씁니다. 컴파일러가 @Binds를 더 효율적으로 처리합니다.

8.27 10.4 Clean Architecture 패키지 구조 결정 과정

8.27.1 처음 실수: 타입별 분류

처음에는 이렇게 했습니다.

```
com.searcam/
├── activities/
├── fragments/
├── viewmodels/
├── repositories/
├── models/
```

```
└──── utils/
```

결과는 참담했습니다. Wi-Fi 스캔 관련 코드를 보려면 `repositories/`, `viewmodels/`, `models/`를 동시에 열어야 했습니다. 파일 네비게이션에 낭비하는 시간이 50%였습니다.

8.27.2 수정 방향: 레이어별 + 기능별 혼합

Clean Architecture는 레이어 분리가 핵심이지만, 같은 레이어 안에서는 기능별로 묶는 게 더 실용적입니다.

```
com.searcam/
│
├──── di/            # 의존성 주입 (전역 설정)
│   ├──── AppModule.kt
│   ├──── SensorModule.kt
│   ├──── DatabaseModule.kt
│   ├──── AnalysisModule.kt
│   └──── RepositoryModule.kt
│
├──── domain/     # 순수 비즈니스 로직 (Android 의존성 0)
│   ├──── model/     # 불변 데이터 클래스
│   ├──── usecase/   # 비즈니스 유스케이스
│   └──── repository/   # 인터페이스만 정의
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
│
├─── data/          # 외부 시스템 연동
│   ├─── sensor/   # 하드웨어 접근
│   ├─── analysis/ # 탐지 알고리즘
│   ├─── repository/  # 인터페이스 구현체
│   └─── local/    # Room DB
│
├─── ui/            # Compose 화면
│   ├─── home/
│   ├─── scan/
│   ├─── lens/
│   ├─── magnetic/
│   ├─── report/
│   └─── components/
│
└─── util/          # 횡단 관심사
```

레이어 분리의 핵심 규칙: `domain/` 패키지에는 `android.` import가 단 하나도 없어야 합니다. 이걸 지키면 domain 로직을 순수 JVM 환경에서 테스트할 수 있습니다.

8.27.3 도메인 모델 예시 — 불변성 준수

```
// domain/model/NetworkDevice.kt
// Android import 없음. 순수 Kotlin data class입니다
data class NetworkDevice(
    val ip: String,
    val mac: String,
    val hostname: String? = null,
    val vendor: String? = null,        // OUI로 식별한 제조사
    val deviceType: DeviceType = DeviceType.UNKNOWN,
    val openPorts: List<Int> = emptyList(),
    val riskScore: Int = 0,
    val isCamera: Boolean = false
)

// 업데이트 시 불변성 유지 — 항상 새 객체를 반환합니다
fun NetworkDevice.withRiskScore(score: Int): NetworkDevice
    copy(riskScore = score)

fun NetworkDevice.markAsCamera(): NetworkDevice =
    copy(isCamera = true, riskScore = maxOf(riskScore, 70)
```

8.28 10.5 AndroidManifest 권한 전략 — 최소 권한 원칙

8.28.1 왜 권한을 최소화해야 하는가

사용자는 이미 권한 요청에 피로해 있습니다. 앱을 설치하자마자 5개 권한을 한꺼번에 요청하면 "거부" 버튼을 누릅니다. 더 중요한 이유: 몰카 탐지 앱이 불필요한 권한(연락처, 위치 정밀 정보 등)을 요청하면 "이 앱이 오히려 내 정보를 수집하는 게 아닐까?" 하는 불신이 생깁니다.

SearCam의 권한 정책: 1. 기능에 실제로 필요한 권한만 2. 권한이 필요한 기능을 사용할 때만 요청 (런타임 권한) 3. 권한 거부 시 해당 레이어를 비활성화하되 앱은 계속 동작

```
<!-- AndroidManifest.xml -->
<manifest xmlns:android="http://schemas.android.com/apk/re

    <!-- Wi-Fi 스캔 — Layer 1 필수 -->
    <uses-permission android:name="android.permission.ACCE
    <uses-permission android:name="android.permission.CHAN
    <!-- 위치 권한: Android 8+ Wi-Fi 스캔에 필요합니다 (실제 위
    <uses-permission android:name="android.permission.ACCE
    <uses-permission android:name="android.permission.ACCE
```

```
    <!-- Android 13+ 근처 Wi-Fi 기기 스캔 권한 -->
    <uses-permission android:name="android.permission.NEAR
        android:usesPermissionFlags="neverForLocation"
        tools:targetApi="tiramisu" />

    <!-- 카메라 — Layer 2 (렌즈/IR 감지) 필수 -->
    <uses-permission android:name="android.permission.CAME

    <!-- 인터넷 — 포트 스캔 (로컬 네트워크만, 인터넷 미사용) -->
    <uses-permission android:name="android.permission.INTE

    <!-- 권장 하드웨어 — 없어도 앱 설치 가능 -->
    <uses-feature
        android:name="android.hardware.camera.flash"
        android:required="false" />  <!-- 플래시 없으면 렌즈
    <uses-feature
        android:name="android.hardware.sensor.compass"
        android:required="false" />  <!-- 자력계 없으면 EMF

    <!-- 위치 권한은 required="false" — Wi-Fi 연결 상태에서만
    <uses-feature
        android:name="android.hardware.wifi"
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        android:required="false" />

</manifest>
```

8.28.2 런타임 권한 요청 패턴

```
// util/PermissionHelper.kt
class PermissionHelper @Inject constructor(
    @ApplicationContext private val context: Context
) {
    // 필요한 권한을 기능별로 그룹화합니다
    val wifiScanPermissions = if (Build.VERSION.SDK_INT >=
        arrayOf(
            Manifest.permission.NEARBY_WIFI_DEVICES,
            Manifest.permission.ACCESS_FINE_LOCATION
        )
    } else {
        arrayOf(Manifest.permission.ACCESS_FINE_LOCATION)
    }

    val cameraPermissions = arrayOf(Manifest.permission.CA

    // 권한 상태를 체크합니다
```

```
    fun hasWifiPermissions(): Boolean =
        wifiScanPermissions.all { permission ->
            ContextCompat.checkSelfPermission(context, per
                PackageManager.PERMISSION_GRANTED
        }

    fun hasCameraPermission(): Boolean =
        ContextCompat.checkSelfPermission(
            context,
            Manifest.permission.CAMERA
        ) == PackageManager.PERMISSION_GRANTED
}
```

8.29 실습

실습 10-1: 새 Android 프로젝트를 만들고 `libs.versions.toml`에 Hilt, CameraX, Room을 추가해보세요. `build.gradle.kts`에서 버전 숫자를 직접 쓰지 않고 `libs.` 접두사로만 참조하는지 확인하세요.

실습 10-2: `di/RepositoryModule.kt`를 만들고

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

`WifiScanRepository` 인터페이스를 가짜 구현체 (FakeWifiScanRepository)에 바인딩해보세요. Hilt 컴파일러가 오류를 어디서 잡아주는지 경험해보세요.

8.30 핵심 정리

결정	이유
Version Catalog	멀티모듈에서 버전 일관성 보장
Hilt	컴파일 타임 안전성, 테스트 용이성
Clean Architecture	레이어 독립성, domain 순수성 유지
최소 권한	사용자 신뢰, 불필요한 데이터 수집 방지

- 기술 부채는 초반에 차단할수록 비용이 적게 든다
- `domain/` 패키지에 Android import가 있다면 설계가 잘못된 것이다
- 권한은 사용하는 순간에, 이유와 함께 요청한다
- 런타임 DI 오류는 사용자에게 크래시로 돌아온다

8.31 다음 장 예고

기초 공사가 끝났으니 이제 첫 번째 탐지 레이어를 만들 차례입니다. Ch11에서는 탐지 가중치의 50%를 차지하는 Wi-Fi 스캔 시스템 — ARP 테이블 파싱부터 포트 스캐닝까지 — 을 구현합니다.

—

참

고

문

서:

docs/

02-

TRD.md,

docs/

03-

TDD.md,

docs/

04-

system-

architecture.md

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

Ch11: Wi-Fi 스캔 — 네트워크는 거짓말을 하지 않는다

—

>

이

장

에

서

배

울

것:

탐

지

가

중

치

의

50%

를

차

지

하

는

Wi-

Fi

스

캔

레

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.32 도입

범죄 수사에서 형사는 현장 주변 CCTV를 가장 먼저 확인합니다. 용의자가 직접 자백하길 기다리는 대신, 흔적을 추적하는 거죠. 몰래카메라도 마찬가지입니다. 카메라는 잘 숨길 수 있지만, 네트워크 흔적은 숨기기 어렵습니다.

Wi-Fi에 연결된 IP 카메라는 반드시 네트워크에 흔적을 남깁니다. MAC 주소, ARP 테이블 항목, 그리고 특정 포트(554 RTSP, 8080 HTTP)가 열려 있습니다. SearCam은 이 흔적을 쫓습니다.

8.33 11.1 Wi-Fi 스캔이 탐지의 핵심인 이유

8.33.1 가중치 50%의 근거

세 탐지 레이어 중 Wi-Fi가 가장 높은 비중을 차지하는 이유는 두 가지입니다.

첫째, **현대 몰래카메라의 대부분이 IP 카메라**입니다. SD 카드에만 저장하는 구식 장치와 달리, 요즘 범죄에 사용되는 카메라는 실시간 원격 모니터링을 위해 Wi-Fi에 연결됩니다. 네트워크 연결이 오히려 약점이 됩니다.

둘째, **오탐률이 낮습니다.** 자기장 센서는 일반 가전제품에도 반응하

고, 렌즈 감지는 유리 반사에도 반응합니다. 하지만 카메라 제조사 OUI를 가진 기기가 카메라 포트를 열고 있다면 — 이건 구체적인 증거입니다.

8.33.2 탐지 가능 범위와 한계

Wi-Fi 스캔으로 탐지 가능:

- 같은 Wi-Fi 네트워크에 연결된 IP 카메라
- WPA/WPA2/WPA3 네트워크 내 ARP 등록 기기
- mDNS/SSDP로 광고하는 스마트 카메라

Wi-Fi 스캔으로 탐지 불가능:

- 다른 Wi-Fi 네트워크나 LTE로 송출하는 카메라
- Wi-Fi 꺼짐 상태의 배터리 방식 카메라
- 전원이 꺼진 카메라
- 사용자가 같은 네트워크에 연결하지 않은 경우

이 한계를 솔직하게 UI에 표시하는 것이 SearCam의 차별점입니다.

8.34 11.2 ARP 테이블 파싱 원리

8.34.1 ARP가 무엇인가

ARP(Address Resolution Protocol)는 IP 주소를 MAC 주소로 변환하는 프로토콜입니다. 스마트폰이 라우터를 통해 같은 네트워크 기기와 통신할 때마다 ARP 교환이 일어나고, 그 결과가 `/proc/net/arp` 파일에 기록됩니다.

이 파일은 Android에서 root 권한 없이도 읽을 수 있습니다. 운영체제가 일반 앱에게도 ARP 테이블 조회를 허용하기 때문입니다.

```
# /proc/net/arp 파일 형식 예시
IP address       HW type     Flags   HW address
192.168.1.1      0x1         0x2     aa:bb:cc:11:22:33
192.168.1.105    0x1         0x2     00:12:bf:45:67:89
192.168.1.110    0x1         0x2     fc:f5:c4:ab:cd:ef
```

여기서 `00:12:bf` — MAC 앞 3바이트(OUI) — 를 제조사 데이터베이스와 대조하면 기기 제조사를 알 수 있습니다. `00:12:bf`는 Hikvision(IP 카메라 제조사)의 OUI입니다.

8.34.2 ARP 파싱 구현

```
// data/sensor/WifiScanner.kt (ARP 파싱 부분)
class WifiScanner @Inject constructor(
```

```
    private val wifiManager: WifiManager,
    private val nsdManager: NsdManager,
    private val ouiDatabase: OuiDatabase,
    @IoDispatcher private val ioDispatcher: CoroutineDispa
) {
    // ARP 테이블에서 같은 네트워크 기기 목록을 읽어옵니다
    suspend fun readArpTable(): List<ArpEntry> = withConte
        try {
            File("/proc/net/arp")
                .readLines()
                .drop(1)  // 헤더 행 제거
                .mapNotNull { line -> parseArpLine(line) }
                .filter { entry -> entry.isValid() }
        } catch (e: IOException) {
            // ARP 읽기 실패 시 빈 목록 반환 — 앱이 크래시되지 않
            emptyList()
        }
    }

    // "192.168.1.105  0x1  0x2  00:12:bf:45:67:89  *  wla
    private fun parseArpLine(line: String): ArpEntry? {
        val parts = line.trim().split(Regex("\\s+"))
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        if (parts.size < 6) return null

        val ip = parts[0]
        val flags = parts[2]
        val mac = parts[3]

        // Flags 0x2 = 완전히 해석된(complete) 항목만 사용합니
        if (flags != "0x2") return null
        // 00:00:00:00:00:00 같은 유효하지 않은 MAC 제외
        if (mac == "00:00:00:00:00:00") return null

        return ArpEntry(ip = ip, mac = mac.uppercase())
    }
}

// ARP 항목의 유효성 검사
data class ArpEntry(val ip: String, val mac: String) {
    fun isValid(): Boolean {
        val ipPattern = Regex("^\\d{1,3}\\.\\d{1,3}\\.\\d{
        val macPattern = Regex("^([0-9A-F]{2}:){5}[0-9A-F]
        return ip.matches(ipPattern) && mac.matches(macPat
    }
```

```
}
```

8.35 11.3 mDNS(NsdManager)와 SSDP 멀티캐스트 동작

8.35.1 mDNS로 스마트 기기 탐색하기

mDNS(Multicast DNS)는 로컬 네트워크에서 서버 없이 서비스를 광고하고 탐색하는 프로토콜입니다. 애플 기기의 AirPrint, Chromecast가 mDNS를 사용합니다. IP 카메라도 `_rtsp._tcp.` 또는 `_http._tcp.` 서비스를 mDNS로 광고하는 경우가 많습니다.

Android의 `NsdManager`가 mDNS 탐색을 제공합니다.

```
// data/sensor/WifiScanner.kt (mDNS 탐색 부분)

// 카메라가 자주 광고하는 서비스 타입 목록
private val CAMERA_SERVICE_TYPES = listOf(
    "_rtsp._tcp.",     // 실시간 스트리밍 (IP 카메라 표준)
    "_http._tcp.",     // HTTP 웹 인터페이스
    "_dahua._tcp.",    // Dahua 카메라 전용 서비스
    "_hikvision._tcp.",  // Hikvision 카메라 전용 서비스
    "_onvif._tcp."     // ONVIF 표준 프로토콜
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
)

fun discoverMdnsServices(): Flow<NsdServiceInfo> = callbac
    val discoveryListeners = mutableListOf<NsdManager.Disc

    CAMERA_SERVICE_TYPES.forEach { serviceType ->
        val listener = object : NsdManager.DiscoveryLister
            override fun onServiceFound(serviceInfo: NsdSe
                // 서비스를 발견하면 채널로 전송합니다
                trySend(serviceInfo)
            }

            override fun onStartDiscoveryFailed(serviceTyp
                // 특정 서비스 타입 탐색 실패는 무시하고 계속합니
            }

            override fun onDiscoveryStarted(serviceType: S
            override fun onDiscoveryStopped(serviceType: S
            override fun onServiceLost(serviceInfo: NsdSer
            override fun onStopDiscoveryFailed(serviceType
        }
```

```
        nsdManager.discoverServices(
            serviceType,
            NsdManager.PROTOCOL_DNS_SD,
            listener
        )
        discoveryListeners.add(listener)
    }

    // Flow가 취소되면 모든 탐색을 중지합니다
    awaitClose {
        discoveryListeners.forEach { listener ->
            try {
                nsdManager.stopServiceDiscovery(listener)
            } catch (e: IllegalArgumentException) {
                // 이미 중지된 탐색 — 무시합니다
            }
        }
    }
}
```

8.35.2 SSDP 멀티캐스트로 UPnP 기기 탐색

SSDP(Simple Service Discovery Protocol)는 UPnP 기기(스마트 TV, IP 카메라 등)가 자신의 존재를 알리는 프로토콜입니다. 멀티캐스트 주소 `239.255.255.250:1900`에 M-SEARCH 패킷을 보내면 UPnP 기기들이 응답합니다.

```
// SSDP M-SEARCH 요청을 보내고 응답을 받습니다
suspend fun discoverSsdpDevices(): List<String> = withCont
    val foundDevices = mutableListOf<String>()

    // SSDP 검색 메시지 — 모든 UPnP 기기에게 "너 거기 있어?" 묻는
    val searchMessage = """
        M-SEARCH * HTTP/1.1
        HOST: 239.255.255.250:1900
        MAN: "ssdp:discover"
        MX: 3
        ST: ssdp:all

    """.trimIndent()

    try {
        val socket = DatagramSocket()
        socket.soTimeout = 3000  // 3초 응답 대기
```

```
        val group = InetAddress.getByName("239.255.255.250
        val packet = DatagramPacket(
            searchMessage.toByteArray(),
            searchMessage.length,
            group,
            1900
        )
        socket.send(packet)

        // 응답 수신 루프
        val buffer = ByteArray(2048)
        val responsePacket = DatagramPacket(buffer, buffer

        while (true) {
            try {
                socket.receive(responsePacket)
                val response = String(responsePacket.data,
                val location = extractLocation(response)
                if (location != null) foundDevices.add(loc
            } catch (e: SocketTimeoutException) {
                break  // 타임아웃 = 더 이상 응답 없음
```

```
            }
        }

        socket.close()
    } catch (e: Exception) {
        // 네트워크 오류 — 빈 목록 반환
    }

    foundDevices
}
```

8.36 11.4 OUI 데이터베이스로 제조사 식별하기

8.36.1 OUI란 무엇인가

MAC 주소(예: `00:12:BF:45:67:89`)에서 앞 3바이트(`00:12:BF`)가 OUI(Organizationally Unique Identifier)입니다. IEEE가 제조사에게 고유하게 할당합니다. `00:12:BF`는 Hikvision Digital Technology — 세계 최대 IP 카메라 제조사입니다.

SearCam은 약 500KB 크기의 OUI 데이터베이스를 앱에 번들로 포함합니다. 인터넷 없이 오프라인으로 제조사를 식별할 수 있습니다.

```
// data/analysis/OuiDatabase.kt
class OuiDatabase @Inject constructor(
    @ApplicationContext private val context: Context,
    @DefaultDispatcher private val dispatcher: CoroutineDi
) {
    // 카메라 관련 OUI 목록 — 앱 assets에서 로드합니다
    private val cameraVendorOuis: Set<String> by lazy {
        loadCameraOuis()
    }

    // 카메라 제조사로 알려진 OUI 집합을 assets에서 로드합니다
    private fun loadCameraOuis(): Set<String> {
        return try {
            context.assets.open("oui_camera.json").use { s
                val json = stream.bufferedReader().readTex
                // JSON 파싱 — Set으로 변환해 O(1) 검색 성능 확
                parseOuiJson(json).toSet()
            }
        } catch (e: IOException) {
            emptySet()  // assets 로드 실패 시 탐지 없이 계속 진
        }
    }
```

```
    // MAC 주소에서 OUI를 추출하고 카메라 제조사인지 판단합니다
    fun isCameraVendor(macAddress: String): Boolean {
        val oui = macAddress.take(8).uppercase()  // "AA:B
        return oui in cameraVendorOuis
    }

    // 제조사 이름을 반환합니다 (UI 표시용)
    fun getVendorName(macAddress: String): String? {
        val oui = macAddress.take(8).uppercase()
        return vendorMap[oui]  // null이면 알 수 없는 제조사
    }
}

// 알려진 카메라 제조사 OUI 예시 (실제는 수백 개)
val KNOWN_CAMERA_OUIS = mapOf(
    "00:12:BF" to "Hikvision",
    "BC:AD:28" to "Hikvision",
    "A4:14:37" to "Dahua Technology",
    "E0:50:8B" to "Dahua Technology",
    "00:40:8C" to "Axis Communications",
    "AC:CC:8E" to "Reolink",
```

```
    "D4:F5:27" to "TP-Link (카메라 라인)",
    "B0:A7:B9" to "Wyze Labs",
    "2C:AA:8E" to "Arlo Technologies"
)
```

8.37 11.5 포트 스캐닝 — 카메라 서비스 찾기

8.37.1 카메라가 여는 포트

IP 카메라는 특정 포트로 서비스를 제공합니다. 이 포트가 열려 있으면 IP 카메라일 가능성이 높아집니다.

포트	프로토콜	의미
554	RTSP	실시간 스트리밍 (IP 카메라 표준)
8554	RTSP	RTSP 대체 포트
80	HTTP	웹 관리 인터페이스
8080	HTTP	대체 웹 포트
8888	HTTP	제조사별 대체 포트
37777	Dahua	Dahua 카메라 전용
8000	Hikvision	Hikvision SDK 포트

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
// data/sensor/PortScanner.kt
class PortScanner @Inject constructor(
    @IoDispatcher private val ioDispatcher: CoroutineDispa
) {
    companion object {
        // 카메라 관련 포트 목록 — 우선순위 순 정렬
        val CAMERA_PORTS = listOf(554, 8554, 80, 8080, 888
        const val PORT_TIMEOUT_MS = 2000  // 포트당 2초 대기
    }

    // 특정 IP의 카메라 포트를 병렬로 스캔합니다
    suspend fun scanCameraPorts(ip: String): List<Int> =
        withContext(ioDispatcher) {
            // 포트들을 병렬로 스캔 — 순차 스캔이면 8포트 × 2초 =
            CAMERA_PORTS.map { port ->
                async {
                    if (isPortOpen(ip, port)) port else nu
                }
            }.awaitAll().filterNotNull()
        }

    // TCP 연결로 포트 개방 여부를 확인합니다
```

```
    private suspend fun isPortOpen(ip: String, port: Int):
        withContext(ioDispatcher) {
            try {
                Socket().use { socket ->
                    socket.connect(
                        InetSocketAddress(ip, port),
                        PORT_TIMEOUT_MS
                    )
                    true  // 연결 성공 = 포트 열림
                }
            } catch (e: Exception) {
                false  // 연결 실패 = 포트 닫힘 또는 필터링
            }
        }

    // RTSP 포트가 열린 기기는 IP 카메라 가능성이 매우 높습니다
    fun hasRtspPort(openPorts: List<Int>): Boolean =
        openPorts.any { it == 554 || it == 8554 }
}
```

8.38 11.6 Android 12+ 스캔 쓰로틀링 해결 전략

8.38.1 쓰로틀링 문제

Android 8.0부터 Wi-Fi 스캔 횟수 제한이 도입되었고, Android 10 이상에서는 더 강화되었습니다. 앱이 2분에 4번 이상 `startScan()`을 호출하면 시스템이 캐시된 결과를 반환합니다(실제 스캔 안 함).

SearCam은 이 제한을 우회하는 게 아니라 다른 방식으로 접근합니다.

```
// Wi-Fi 스캔 쓰로틀링 해결 전략
class WifiScanRepositoryImpl @Inject constructor(
    private val wifiManager: WifiManager,
    private val wifiScanner: WifiScanner,
    @IoDispatcher private val ioDispatcher: CoroutineDispa
) : WifiScanRepository {

    override suspend fun scanNetwork(): Flow<List<NetworkD
        // 전략 1: WifiManager.startScan() 대신 ARP 테이블 직
        // ARP 테이블은 이미 연결된 기기를 보여주므로 스캔 횟수 제
        val arpDevices = wifiScanner.readArpTable()
        emit(arpDevices.map { it.toNetworkDevice() })
```

```
        // 전략 2: 연결된 AP의 DHCP 임대 목록 활용 (API 29+)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q
            val dhcpInfo = wifiManager.dhcpInfo
            // DHCP 서버 IP에서 네트워크 범위 계산
            val networkPrefix = getNetworkPrefix(dhcpInfo)
            emit(scanNetworkRange(networkPrefix))
        }

        // 전략 3: mDNS 패시브 리슨 — 스캔 없이 광고 패킷 수집
        wifiScanner.discoverMdnsServices()
            .collect { serviceInfo ->
                // mDNS로 발견된 기기를 기존 목록에 병합
                val device = serviceInfo.toNetworkDevice()
                emit(listOf(device))
            }
    }.flowOn(ioDispatcher)
}
```

핵심은 **능동적 스캔 의존도를 줄이는 것**입니다. ARP 테이블은 이미 네트워크에 참여한 기기의 기록이므로, Wi-Fi 스캔 없이도 대부분의 기기를 발견할 수 있습니다.

8.39 11.7 WifiScanRepository 구현 — 전체 흐름

```
// data/repository/WifiScanRepositoryImpl.kt
class WifiScanRepositoryImpl @Inject constructor(
    private val wifiScanner: WifiScanner,
    private val portScanner: PortScanner,
    private val ouiDatabase: OuiDatabase,
    @IoDispatcher private val ioDispatcher: CoroutineDispa
) : WifiScanRepository {

    // 전체 Wi-Fi 스캔 파이프라인을 실행합니다
    override suspend fun fullScan(): ScanResult = withCont
        val allDevices = mutableListOf<NetworkDevice>()

        // Step 1: ARP 테이블에서 기기 수집
        val arpEntries = wifiScanner.readArpTable()

        // Step 2: 각 기기에 OUI 매칭 및 포트 스캔을 병렬로 수행
        val enrichedDevices = arpEntries.map { entry ->
            async {
                val vendorName = ouiDatabase.getVendorName
```

```
                val isCameraVendor = ouiDatabase.isCameraV
                val openPorts = portScanner.scanCameraPort

                NetworkDevice(
                    ip = entry.ip,
                    mac = entry.mac,
                    vendor = vendorName,
                    openPorts = openPorts,
                    // 카메라 제조사 OUI + RTSP 포트 = 높은 위
                    riskScore = calculateDeviceRisk(isCame
                    isCamera = isCameraVendor || portScanr
                )
            }
        }.awaitAll()

        allDevices.addAll(enrichedDevices)

        // Step 3: mDNS로 추가 기기 탐색 (5초 타임아웃)
        val mdnsDevices = wifiScanner.discoverMdnsServices
            .take(30)  // 최대 30개 서비스
            .toList()
            .map { it.toNetworkDevice() }
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        allDevices.addAll(mdnsDevices)

        // Step 4: 위험도 점수를 기반으로 결과 반환
        ScanResult(
            layerType = LayerType.WIFI,
            score = allDevices.maxOfOrNull { it.riskScore
            maxScore = 100,
            findings = allDevices.filter { it.isCamera }
                .map { device -> Finding("카메라 의심 기기:
        )
    }

    // 기기 위험도 점수 계산 (0~100)
    private fun calculateDeviceRisk(
        isCameraVendor: Boolean,
        openPorts: List<Int>
    ): Int {
        var score = 0
        if (isCameraVendor) score += 40          // 카메라
        if (554 in openPorts) score += 40         // RTSP
        if (8080 in openPorts || 80 in openPorts) score +=
```

```
        if (37777 in openPorts || 8000 in openPorts) score
        return score.coerceAtMost(100)
    }
}
```

8.40 실습

실습 11-1: 집에서 `/proc/net/arp`를 읽어 연결된 모든 기기의 MAC OUI를 확인해보세요. 스마트TV, 라우터, 스마트폰의 OUI가 어떤 제조사로 나오는지 비교해보세요.

실습 11-2: `PortScanner`를 구현하고, 자신의 컴퓨터 IP(127.0.0.1)에서 22(SSH), 80(HTTP), 554(RTSP) 포트를 스캔해보세요. 결과가 예상과 다르다면 이유를 분석해보세요.

8.41 핵심 정리

기법	역할	한계
ARP 테이블 파싱	연결된 모든 기기 MAC 수집	같은 네트워크만
OUI 매칭	카메라 제조사 식별	MAC 스푸핑 불가 탐지
mDNS/SSDP	스마트 기기 서비스 광고 탐지	mDNS 끈 기기 불탐지
포트 스캔	RTSP/HTTP 서비스 확인	방화벽으로 숨긴 경우

- Wi-Fi 스캔은 가중치 50% — 가장 구체적인 증거를 제공하기 때문이다
- ARP 테이블은 root 권한 없이 읽을 수 있는 네트워크 흔적이다
- Android 스캔 쓰로틀링은 능동 스캔 대신 ARP/mDNS 패시브 방식으로 우회한다
- 포트 스캔은 기기당 병렬 실행으로 총 소요 시간을 줄인다

8.42 11.8 코드 리뷰 개선 사항 — 실전 배포 전 발견한 버그들

초기 구현 후 코드 리뷰 단계에서 실제 배포 환경에서 문제가 될 수 있는 이슈 4건이 발견되었습니다. 각 수정이 왜 필요했는지 구체적으로 살펴봅니다.

8.42.1 개선 1: cachedDevices에 @Volatile 추가 — 스레드 가시성 보장

문제: cachedDevices 필드를 여러 코루틴에서 읽고 쓸 수 있는데, JVM의 CPU 캐시 최적화로 인해 한 스레드에서 쓴 값이 다른 스레드에 즉시 보이지 않을 수 있었습니다.

```
// Before: JVM이 CPU 캐시에 값을 유지할 수 있어 멀티스레드 환경에서
private var cachedDevices: List<NetworkDevice> = emptyList

// After: @Volatile이 모든 스레드가 항상 최신 값을 읽도록 강제합니다
@Volatile
private var cachedDevices: List<NetworkDevice> = emptyList
```

@Volatile은 해당 필드의 읽기/쓰기가 반드시 메인 메모리를 통해 이루어지도록 JVM에 지시합니다. 복잡한 동기화 블록 없이 단순 플래그나 참조 타입의 스레드 안전성을 보장하는 가장 가벼운 방법입니다.

8.42.2 개선 2: SSDP 소켓 리소스 누수 수정 — try-finally 보장

문제: SSDP 탐색 중 socket.leaveGroup()이 예외를 던지면 socket.close()에 도달하지 못하고 소켓이 열린 채로 남아있었습니다. 장시간 사용 시 소켓 고갈로 이어질 수 있습니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
// Before: leaveGroup()이 실패하면 socket.close()가 실행되지 않
try {
    socket.leaveGroup(InetAddress.getByName(SSDP_MULTICAST
    socket.close()  // leaveGroup()이 throw하면 이 줄에 도달 못
} catch (e: Exception) {
    Timber.e(e, "SSDP 오류")
}

// After: finally 블록은 예외 발생 여부와 무관하게 반드시 실행됩니
val socket = MulticastSocket()
try {
    // ... SSDP 탐색 코드
} catch (e: Exception) {
    Timber.e(e, "[E2003] SSDP 탐색 오류: ${e.message}")
} finally {
    // finally는 예외가 발생해도 반드시 실행 — 소켓 누수 방지
    runCatching { socket.leaveGroup(InetAddress.getByName(
    socket.close()
}
```

`try-finally` 패턴은 파일, 소켓, 데이터베이스 커넥션처럼 "열면 반드시 닫아야 하는" 자원의 황금 규칙입니다. Kotlin에서는 `use {}` 확장 함수가 이를 자동화하지만, `MulticastSocket`처

럼 `Closeable`을 구현하지 않는 경우 수동으로 `finally`를 작성해야 합니다.

8.42.3 개선 3: 호스트명 입력 Sanitization — 제어 문자 차단

문제: mDNS/SSDP로 수신하는 호스트명은 외부 기기가 제공합니다. 악의적인 기기가 제어 문자(NULL, CRLF 등)나 매우 긴 문자열을 호스트명으로 보낼 수 있습니다.

```
// Before: 외부에서 받은 호스트명을 그대로 저장 (Log Injection,
val hostname: String? = rawHostname

// After: 길이 제한 + 제어 문자 제거
val hostname: String? = rawHostname
    ?.take(128)  // 최대 128자 제한 (DNS 최대 길이)
    ?.replace(Regex("[\\x00-\\x1F\\x7F]"), "")  // ASCII 제
```

외부 입력은 시스템 경계(System Boundary)에서 반드시 검증해야 합니다. 제어 문자가 포함된 호스트명이 로그에 기록되면 Log Injection 공격에 악용될 수 있고, UI에 표시하면 레이아웃이 깨집니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.42.4 개선 4: PortScanner에 사설 IP 범위 가드 — 인터넷 스캔 방지

문제: IP 주소 검증 없이 포트 스캔을 수행하면 이론적으로 인터넷 공인 IP를 스캔할 수 있었습니다. 이는 법적·보안적 문제를 일으킬 수 있습니다.

```
// PortScanner.kt
suspend fun scanPorts(ip: String): List<Int> = withContext
    // RFC 1918 사설 IP 범위만 스캔 허용 — 공인 IP 스캔 방지
    if (!isPrivateIp(ip)) {
        Timber.w("[E2004] 사설 IP 범위 외 스캔 차단: $ip")
        return@withContext emptyList()
    }
    // ... 실제 스캔 로직
}

// RFC 1918: 10.x.x.x / 172.16-31.x.x / 192.168.x.x / 127.
internal fun isPrivateIp(ip: String): Boolean {
    return try {
        val parts = ip.split(".").map { it.toInt() }
        if (parts.size != 4) return false
        val (a, b, _, _) = parts
        a == 10 || a == 127 ||
```

```
            (a == 172 && b in 16..31) ||
            (a == 192 && b == 168)
    } catch (e: Exception) {
        false
    }
}
```

이 함수는 `internal`로 선언되어 같은 모듈의 테스트에서 직접 접근 가능합니다. `app/src/test/.../PortScannerTest.kt`에서 경계값 19케이스를 단위 테스트로 검증합니다.

8.43 다음 장 예고

네트워크에서 IP 카메라를 찾았습니다. 하지만 Wi-Fi 없는 환경이라면? Ch12에서는 두 번째 탐지 레이어 — 빛의 역반사를 이용해 카메라 렌즈를 물리적으로 찾아내는 방법을 구현합니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

참

고

문

서:

docs/

02-

TRD.md,

docs/

03-

TDD.md,

docs/

04-

system-

architecture.md

—

#

Ch12:

렌

즈

감

지

—

빛

이

돌

아

오

는

길

을

막

아

라

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

>

이장에서 배울 것: 탐지 가중치 35%를 차지하는 렌즈 감지 레이

8.44 도입

야간 운전 중 도로 표지판에 헤드라이트를 비추면 표지판이 눈부시게 빛납니다. 옆에서 보면 별로 밝지 않은데, 운전자 위치에서만 유독 밝게 보입니다. 이것이 역반사(Retroreflection)입니다. 빛이 들어온 방향 그대로 되돌아가는 현상이죠.

카메라 렌즈는 같은 원리로 동작합니다. 렌즈는 빛을 굴절시키고 반사시키도록 설계된 광학 소자입니다. 스마트폰 플래시를 켜고 방을 천천히 돌아보면, 숨겨진 카메라 렌즈는 다른 표면보다 유독 밝게, 그리고 원형으로 빛납니다.

SearCam은 이 물리 법칙을 알고리즘으로 구현합니다.

8.45 12.1 빛의 역반사(Retroreflection) 원리

8.45.1 왜 렌즈만 특별하게 반사하는가

일반 표면(벽, 천장, 가구)에 빛을 비추면 **난반사(Diffuse Reflection)**가 일어납니다. 빛이 여러 방향으로 퍼져나가서, 어느 방향에서 봐도 비슷한 밝기로 보입니다.

카메라 렌즈는 다릅니다. 렌즈 내부의 광학 소자들이 **코너 반사체(Corner Reflector)** 역할을 합니다. 빛이 들어온 방향으로 정확히

되돌아가는 거죠.

일반 표면 (난반사):

```
광원 →    표면    → 여러 방향으로 산란
          ↑
       관찰자 (어두움)
```

카메라 렌즈 (역반사):

```
광원 → 렌즈 → 광원 방향으로 되돌아옴
↑                    ↓
관찰자 (밝게 보임!) ← 반사광
```

스마트폰에서 플래시와 카메라 센서는 아주 가까이 있습니다. 따라서 플래시를 켜고 카메라로 촬영하면 역반사 지점이 극도로 밝게 찍힙니다.

8.45.2 역반사의 특성

역반사 포인트는 세 가지 특성을 가집니다.

1. **고휘도**: 주변보다 현저히 밝습니다 (픽셀값 200+ / 255)
2. **원형**: 렌즈는 원형이므로 반사 패턴도 원형입니다
3. **안정성**: 카메라를 조금 움직여도 같은 자리에서 계속 빛납니다

이 세 조건을 동시에 만족하는 포인트를 찾는 것이 렌즈 감지 알고

리즘의 핵심입니다.

8.46 12.2 CameraX ImageAnalysis 프레임 처리

8.46.1 CameraX 설정

렌즈 감지는 후면 카메라 + 플래시 조합을 사용합니다. CameraX의 `ImageAnalysis` use case를 통해 매 프레임을 분석합니다.

```
// data/sensor/LensDetector.kt
class LensDetector @Inject constructor(
    @ApplicationContext private val context: Context,
    private val retroreflectionAnalyzer: RetroreflectionAr
    @DefaultDispatcher private val dispatcher: CoroutineDi
) {
    private var imageAnalysis: ImageAnalysis? = null
    private var camera: Camera? = null

    // CameraX 파이프라인을 구성하고 분석을 시작합니다
    fun startAnalysis(
        lifecycleOwner: LifecycleOwner,
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        previewView: PreviewView
    ): Flow<List<RetroreflectionPoint>> = callbackFlow {

        val cameraProviderFuture = ProcessCameraProvider.g

        cameraProviderFuture.addListener({
            val cameraProvider = cameraProviderFuture.get(

            // Preview — 사용자가 카메라 화면을 볼 수 있게 합니다
            val preview = Preview.Builder().build().also {
                it.setSurfaceProvider(previewView.surfaceP
            }

            // ImageAnalysis — 30fps로 프레임을 분석합니다
            imageAnalysis = ImageAnalysis.Builder()
                .setTargetResolution(Size(1280, 720))  //
                .setBackpressureStrategy(ImageAnalysis.STR
                // KEEP_ONLY_LATEST: 분석이 느려도 최신 프레임
                .build()
                .also { analysis ->
                    analysis.setAnalyzer(
                        ContextCompat.getMainExecutor(cont
```

```
                ) { imageProxy ->
                    // 각 프레임을 분석하고 결과를 Flow로 내
                    val points = retroreflectionAnalyz
                    trySend(points)
                    imageProxy.close()  // 반드시 close
                }
            }

        // 후면 카메라 선택
        val cameraSelector = CameraSelector.DEFAULT_BA

        try {
            cameraProvider.unbindAll()
            camera = cameraProvider.bindToLifecycle(
                lifecycleOwner,
                cameraSelector,
                preview,
                imageAnalysis
            )
            // 플래시를 토치 모드로 켭니다
            camera?.cameraControl?.enableTorch(true)
        } catch (e: Exception) {
```

```
                close(e)
            }
        }, ContextCompat.getMainExecutor(context))

        awaitClose {
            // Flow가 끝나면 플래시 끄기
            camera?.cameraControl?.enableTorch(false)
            imageAnalysis?.clearAnalyzer()
        }
    }
}
```

8.47 12.3 5단계 분석 파이프라인

8.47.1 파이프라인 개요

```
[카메라 프레임 (YUV 720p)]
          ↓
 1단계: 전처리 (그레이스케일 변환)
          ↓
 2단계: 고휘도 영역 추출 (임계값 필터링)
          ↓
```

```
 3단계: 원형도 분석 (컨투어 circularity 계산)
          ↓
 4단계: 시간축 안정성 검증 (연속 5프레임 지속)
          ↓
 5단계: 플래시 OFF 동적 검증 (플래시 끄면 사라지는지)
          ↓
[RetroreflectionPoint 목록]
```

```
// data/analysis/RetroreflectionAnalyzer.kt
class RetroreflectionAnalyzer @Inject constructor() {

    // 최근 N 프레임의 감지 기록 (안정성 검증용)
    private val frameHistory = ArrayDeque<Set<Point>>(maxS

    fun analyze(imageProxy: ImageProxy): List<Retroreflect
        // 1단계: YUV 이미지를 그레이스케일 Bitmap으로 변환합니
        val grayBitmap = imageProxy.toGrayBitmap()

        // 2단계: 고휘도 영역만 추출합니다 (임계값 200/255)
        val highBrightnessRegions = extractHighBrightness(
            bitmap = grayBitmap,
            threshold = 200
        )
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        if (highBrightnessRegions.isEmpty()) return emptyL

        // 3단계: 원형도 검사 — 렌즈는 원형입니다
        val circularRegions = highBrightnessRegions.filter
            region.circularity > 0.7f  // 1.0 = 완전한 원,
        }
        if (circularRegions.isEmpty()) return emptyList()

        // 4단계: 최근 5프레임 연속 등장 여부 확인 (안정성)
        val currentPoints = circularRegions.map { it.cente
        frameHistory.addLast(currentPoints)
        if (frameHistory.size > 5) frameHistory.removeFirs

        val stableRegions = circularRegions.filter { regio
            // 최소 3프레임 이상 같은 위치에서 감지된 포인트만 통
            frameHistory.count { frame ->
                frame.any { point -> point.distanceTo(regi
            } >= 3
        }

        // 5단계: 결과를 RetroreflectionPoint로 변환합니다
        return stableRegions.map { region ->
```

```
            RetroreflectionPoint(
                x = region.center.x,
                y = region.center.y,
                size = region.size,
                circularity = region.circularity,
                brightness = region.avgBrightness,
                contrastRatio = region.avgBrightness / gra
                isStable = true,
                flashDependency = false,  // 5단계에서 플래
                riskScore = calculateRisk(region)
            )
        }
    }

    // 고휘도 영역 추출 — 임계값 이상의 밝은 픽셀 클러스터를 찾습니
    private fun extractHighBrightness(
        bitmap: Bitmap,
        threshold: Int
    ): List<BrightRegion> {
        val regions = mutableListOf<BrightRegion>()
        val visited = Array(bitmap.height) { BooleanArray(
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        for (y in 0 until bitmap.height step 2) {  // 2픽셀
            for (x in 0 until bitmap.width step 2) {
                if (visited[y][x]) continue

                val brightness = bitmap.getBrightness(x, y
                if (brightness < threshold) continue

                // BFS로 연결된 고휘도 픽셀 클러스터를 찾습니다
                val cluster = floodFillBFS(bitmap, x, y, t
                if (cluster.size >= 9) {  // 최소 9픽셀 (노이
                    regions.add(analyzeClusters(cluster))
                }
            }
        }

        return regions
    }

    // 클러스터의 원형도를 계산합니다
    // circularity = 4π × 면적 / 둘레² (완전한 원 = 1.0)
    private fun calculateCircularity(cluster: List<Point>)
        val area = cluster.size.toFloat()
```

```
        val perimeter = calculatePerimeter(cluster).toFloa
        if (perimeter == 0f) return 0f
        return (4f * Math.PI.toFloat() * area) / (perimete
    }

    // 역반사 포인트의 위험도 점수를 계산합니다
    private fun calculateRisk(region: BrightRegion): Int {
        var score = 0
        if (region.circularity > 0.85f) score += 30  // 높
        if (region.avgBrightness > 230f) score += 25  // 극
        if (region.contrastRatio > 5f) score += 25    // 주
        if (region.size in 16f..400f) score += 20     // 적
        return score.coerceAtMost(100)
    }
}
```

8.48 12.4 플래시 OFF 동적 검증 — 오탐 방지 핵심

역반사를 이용하는 분석의 가장 큰 약점은 **가짜 양성(False Positive)**입니다. 유리창 반사, 금속 장식품, 광택 있는 플라스틱도 비슷

하게 밝게 반사될 수 있습니다. 이를 걸러내는 가장 확실한 방법은 플래시를 잠깐 끄는 것입니다.

진짜 카메라 렌즈: 플래시를 끄면 밝기가 급격히 떨어집니다 (역반사 의존) 일반 반사 표면: 플래시를 꺼도 주변 조명이 있으면 여전히 보입니다

```
// 플래시 ON/OFF 동적 검증
suspend fun verifyWithFlashToggle(
    points: List<RetroreflectionPoint>
): List<RetroreflectionPoint> = withContext(dispatcher) {

    if (points.isEmpty()) return@withContext points

    // 플래시 OFF 후 같은 위치의 밝기를 비교합니다
    camera?.cameraControl?.enableTorch(false)
    delay(300)  // 300ms 대기 — 프레임이 안정화될 시간

    val darkFramePoints = captureFramePoints()  // 플래시 O

    camera?.cameraControl?.enableTorch(true)  // 플래시 다시

    // 플래시 OFF 시 사라진 포인트만 진짜 역반사로 판정합니다
    return@withContext points.map { point ->
```

```
        val stilPresentWhenDark = darkFramePoints.any { da
            darkPoint.distanceTo(point) < 15f &&
            darkPoint.brightness > point.brightness * 0.6f
        }

        point.copy(
            // 플래시 꺼도 밝으면 flashDependency=false (가짜
            // 플래시 끄면 사라지면 flashDependency=true (진짜
            flashDependency = !stilPresentWhenDark,
            // 플래시 의존 = 렌즈일 가능성 높음 → 위험도 상향
            riskScore = if (!stilPresentWhenDark) {
                (point.riskScore * 1.3f).toInt().coerceAtM
            } else {
                (point.riskScore * 0.5f).toInt()  // 가짜 반
            }
        )
    }
}
```

8.49 12.5 IR 감지 — 전면 카메라의 적외선 필터 활용

8.49.1 스마트폰 카메라와 IR 필터

사람 눈은 가시광선(380~700nm)만 볼 수 있습니다. 하지만 스마트폰 카메라 센서는 적외선(IR, 700~1000nm)도 감지합니다. 제조사들이 IR 컷 필터(ICF)를 달아서 가시광선만 통과시키지만, 전면 카메라는 후면보다 IR 필터가 약한 경우가 많습니다.

TV 리모컨 버튼을 누르면서 전면 카메라로 촬영해보세요. 리모컨 끝에서 보라색/흰색 불빛이 보인다면 그 카메라는 IR에 민감합니다.

이를 이용합니다. 몰래카메라는 야간 감시를 위해 IR LED를 포함하는 경우가 많습니다. 어두운 환경에서 전면 카메라로 주변을 스캔하면 IR LED를 발견할 수 있습니다.

```
// data/sensor/IrDetector.kt
class IrDetector @Inject constructor(
    @ApplicationContext private val context: Context,
    @DefaultDispatcher private val dispatcher: CoroutineDi
) {
    companion object {
        const val IR_BRIGHTNESS_THRESHOLD = 180  // IR 감지
        const val MIN_IR_DURATION_MS = 3000L      // 최소 3
```

```
        const val MAX_AMBIENT_LUX = 10f            // 10 lu
    }

    // 전면 카메라로 IR 발광체를 탐지합니다
    fun startIrDetection(
        lifecycleOwner: LifecycleOwner
    ): Flow<List<IrPoint>> = callbackFlow {

        // 전면 카메라 선택 — IR 필터가 약해 IR에 더 민감합니다
        val cameraSelector = CameraSelector.DEFAULT_FRONT_

        val imageAnalysis = ImageAnalysis.Builder()
            .setTargetResolution(Size(1280, 720))
            .setBackpressureStrategy(ImageAnalysis.STRATEG
            .build()

        imageAnalysis.setAnalyzer(
            ContextCompat.getMainExecutor(context)
        ) { imageProxy ->
            val irPoints = detectIrPoints(imageProxy)
            trySend(irPoints)
            imageProxy.close()
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        }

        // ... CameraX 바인딩 생략

        awaitClose { imageAnalysis.clearAnalyzer() }
    }

    private fun detectIrPoints(imageProxy: ImageProxy): Li
        val bitmap = imageProxy.toBitmap()

        // IR은 특유의 보라색(Violet) 또는 흰색으로 나타납니다
        val irCandidates = findIrColorPixels(bitmap)

        return irCandidates.map { point ->
            IrPoint(
                x = point.x,
                y = point.y,
                intensity = point.brightness,
                duration = 0L,  // 지속 시간은 외부에서 누적
                isStable = false,  // 3초 지속 여부는 Flow 러
                color = if (point.isViolet()) IrColor.VIOL
                riskScore = if (point.brightness > 220) 70
```

```
            )
        }
    }

    // IR 특유의 색상 필터 — 보라색(R>150, G<80, B>150) 또는 흰
    private fun findIrColorPixels(bitmap: Bitmap): List<Co
        val candidates = mutableListOf<ColorPoint>()

        for (y in 0 until bitmap.height step 4) {
            for (x in 0 until bitmap.width step 4) {
                val pixel = bitmap.getPixel(x, y)
                val r = Color.red(pixel)
                val g = Color.green(pixel)
                val b = Color.blue(pixel)

                // 보라색 IR: 적색과 청색이 높고, 녹색이 낮은 경
                val isViolet = r > 150 && g < 80 && b > 15
                // 흰색 IR: 모든 채널이 높은 경우 (일부 IR LED의
                val isWhite = r > 200 && g > 200 && b > 20

                if (isViolet || isWhite) {
                    candidates.add(ColorPoint(x, y, (r + g
```

```
                }
            }
        }

        return candidates
    }
}
```

8.50 12.6 실전 오탐 사례와 보정 방법

8.50.1 오탐 사례 1: 야간 조명의 광원

호텔 방 조명 장식의 LED가 역반사와 비슷한 밝기로 찍힐 수 있습니다.

보정: 광원 특성 필터링. 진짜 조명은 발광 면적이 넓고 고르게 밝습니다. 카메라 렌즈의 역반사는 면적이 작고(직경 2~20mm 기준) 중심부가 극도로 밝습니다.

```
// 광원과 렌즈를 구별하는 밝기 분포 분석
private fun isLightSource(region: BrightRegion): Boolean {
    // 광원: 중심과 가장자리 밝기 차이가 적음 (고른 발광)
    // 렌즈: 중심 극도로 밝고 가장자리로 갈수록 급격히 어두워짐
```

```
    val centerBrightness = region.getCenterBrightness()
    val edgeBrightness = region.getEdgeBrightness()
    val gradient = centerBrightness / (edgeBrightness + 1f

    return gradient < 2.0f  // 기울기 2배 이하 = 광원으로 판정
}
```

8.50.2 오탐 사례 2: 금속 장식품과 액자 유리

보정: 형태 분석 강화. 금속 장식은 불규칙한 형태, 카메라 렌즈는 원형입니다.

```
// 원형도 임계값을 높여 비원형 반사를 제거합니다
val strictCircularity = 0.75f  // 0.7 → 0.75로 상향 조정
```

8.50.3 오탐 사례 3: 안경 렌즈

사용자가 안경을 쓴 경우, 안경 렌즈도 역반사를 일으킵니다. 하지만 안경 렌즈는 크기가 매우 큽니다 (직경 50mm+).

보정: 크기 범위 필터링.

```
// 카메라 렌즈 크기 범위 (픽셀 단위, 720p 기준)
// 실제 직경 2mm~25mm 범위를 픽셀로 환산
private val LENS_MIN_AREA_PX = 4   // 너무 작으면 노이즈
private val LENS_MAX_AREA_PX = 800  // 너무 크면 안경/유리
```

8.50.4 오탐 보정 결과

실제 테스트에서 보정 전/후 오탐률:

조건	보정 전	보정 후
야간 LED 조명	탐지(오탐)	미탐지
금속 장식품	탐지(오탐)	미탐지
안경 렌즈	탐지(오탐)	미탐지
실제 2mm 카메라 렌즈	탐지(정탐)	탐지(정탐)

8.51 실습

실습 12-1: TV 리모컨의 IR LED를 스마트폰 전면 카메라로 촬영해보세요. 보라색 또는 흰색 불빛이 보이면 해당 카메라는 IR에 민감한 것입니다. 이 특성을 이용해 `IrDetector`가 어떤 색상 범위를 기준으로 삼는지 조정해보세요.

실습 12-2: `RetroreflectionAnalyzer`의 원형도 임계값(0.7)을 0.5, 0.8, 0.9로 바꿔가며 동전, 반지, 안경 렌즈에 플래시를 비춰보세요. 임계값에 따라 오탐/미탐이 어떻게 달라지는지 분석해보세요.

8.52 핵심 정리

단계	목적	핵심 파라미터
전처리	그레이스케일로 처리 단순화	720p 다운스케일
고휘도 추출	밝은 반사 영역 격리	임계값 200/255
원형도 검사	렌즈 형태 특성 검증	circularity > 0.7
안정성 검증	노이즈와 실제 신호 구분	5프레임 중 3회
플래시 검증	가짜 반사 최종 제거	밝기 60% 이하로 감소

- 역반사 원리는 물리 법칙 — 카메라 렌즈는 빛을 보낸 방향으로 되돌려보낸다
- 플래시 OFF 검증이 오탐률을 결정적으로 낮춘다
- 전면 카메라는 IR 필터가 약해 야간 IR 감지에 활용할 수 있다
- 오탐 방지는 하나의 필터가 아니라 크기+원형도+안정성의 복합 판단이다

8.53 12.7 코드 리뷰 개선 사항 — 멀티스레드 안전과 불변성

렌즈 감지 파이프라인은 CameraX 프레임 분석 콜백(백그라운드 Executor 스레드)과 ViewModel(메인 스레드)이 동시에 데이터를 읽고 쓰는 구조입니다. 코드 리뷰에서 이 경계에서 발생하는 스레드 안전 문제 2건이 발견되었습니다.

8.53.1 개선 1: `stabilityTracker`를 `ConcurrentHashMap`으로 교체

문제: `stabilityTracker`는 CameraX 분석 스레드(프레임 콜백)와 stop 메서드(메인 스레드)에서 동시에 접근됩니다. 일반 `HashMap`은 동시 수정 시 `ConcurrentModificationException`을 던지거나 데이터를 손상시킬 수 있습니다.

```
// Before: HashMap은 멀티스레드 환경에서 안전하지 않음
private val stabilityTracker: MutableMap<Int, TrackedPoint

// After: ConcurrentHashMap은 읽기/쓰기 동시 접근을 안전하게 처
private val stabilityTracker: MutableMap<Int, TrackedPoint
```

`ConcurrentHashMap`은 세그먼트(Segment) 단위 잠금을 사용합니다. 전체 Map에 대한 락이 아니라 일부 버킷에만 락을 걸기 때

문에, 성능 저하 없이 멀티스레드 안전을 보장합니다. CameraX처럼 별도 Executor에서 고빈도로 호출되는 분석 콜백에서는 필수입니다.

8.53.2 개선 2: `Cluster.contrastRatio`를 `val`로 — 생성 후 불변 보장

문제: `Cluster` 데이터 클래스의 `contrastRatio`가 `var`로 선언되어 있어서, 객체 생성 후 외부에서 값을 변경할 수 있었습니다. 이는 예기치 않은 상태 변이로 이어질 수 있습니다.

```
// Before: var로 선언 — 생성 후 외부 변경 가능
data class Cluster(
    val center: PointF,
    val radius: Float,
    val avgBrightness: Float,
    val circularity: Float,
    var contrastRatio: Float = 0f,  // 나중에 cluster.contr
)

// After: val로 선언 + copy()로 새 객체 생성
data class Cluster(
    val center: PointF,
    val radius: Float,
```

```
    val avgBrightness: Float,
    val circularity: Float,
    val contrastRatio: Float = 0f,  // val: 생성 후 변경 불가
)

// 사용 시: copy()로 새 Cluster 인스턴스 반환 (원본 불변)
val clusterWithRatio = cluster.copy(contrastRatio = comput
```

data class의 copy()는 불변성 패턴의 핵심입니다. 기존 객체를 변경하는 대신, 변경된 필드만 다른 새 인스턴스를 반환합니다. 이렇게 하면 한 곳의 코드가 다른 곳의 참조에 영향을 줄 수 없어서 버그 추적이 쉬워집니다.

8.53.3 개선 3: calculateRiskScore()를 listOfNotNull + sumOf 패턴으로

문제: 위험도 점수 계산 함수가 var score = 0; score += ... 패턴으로 상태를 변이시키고 있었습니다. 각 조건이 점수에 독립적으로 기여하는 로직인데, 굳이 변이가 필요하지 않습니다.

```
// Before: var 변이 — 각 if 블록이 score를 변경
private fun calculateRiskScore(...): Int {
    var score = 0
    if (radius in 1.0..5.0 && circularity > CIRCULARITY_MI
```

```
    if (isStable) score += 20
    if (flashDependency) score += 25
    if (lastContrastRatio > 20.0f) score += 15
    if (lastBrightness > 230f) score += 10
    return score.coerceIn(0, 100)
}

// After: listOfNotNull + sumOf — 조건별 기여값을 선언적으로 나
private fun calculateRiskScore(...): Int {
    return listOfNotNull(
        30.takeIf { radius in 1.0..5.0 && circularity > CI
        20.takeIf { isStable },
        25.takeIf { flashDependency },
        15.takeIf { lastContrastRatio > 20.0f },
        10.takeIf { lastBrightness > 230f },
    ).sumOf { it }.coerceIn(0, 100)
}
```

`takeIf { 조건 }`는 조건이 참이면 수신 객체(여기서는 점수)를 반환하고, 거짓이면 `null`을 반환합니다. `listOfNotNull`이 null을 제거하고 `sumOf`가 합산합니다. 결과는 동일하지만, 코드의 의도가 훨씬 명확합니다 — "이 조건들이 충족될 때 이 점수가 기여한다".

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.54 다음 장 예고

두 개의 주요 레이어를 구현했습니다. Ch13에서는 마지막 보조 레이어 — 전자기장(EMF) 감지 — 를 다룹니다. 가중치 15%지만, 다른 두 레이어의 결과를 보강하는 중요한 역할을 합니다.

—

참고문서: docs/02-TRD.md, docs/03-TDD.md, docs/04-system-architecture.md

—

#

Ch13:

EMF

감

지

—

전

자

기

장

의

작

은

목

소

리

를

듣

다

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

>

이장에서 배울 것: 탐지 가중치 15%를 차지하는 EMF 레이어의 원

8.55 도입

지뢰 탐지기는 땅 속에 묻힌 금속의 전자기 특성을 읽어냅니다. 엄청난 고출력 신호를 쏘아서 반응을 감지하죠. 스마트폰 자력계는 그 반대입니다 — 아무것도 발사하지 않고, 그냥 주변의 자기장 변화를 조용히 듣습니다.

IC 보드, 모터, 무선 송신기를 포함한 전자 기기는 주변에 전자기장을 만들어냅니다. 스마트폰 자력계가 아주 민감하다면 숨겨진 카메라 주변의 자기장 변화를 감지할 수 있을까요? 이론적으로는 가능합니다.

하지만 현실은 훨씬 복잡합니다. 이 장에서는 EMF 탐지의 가능성과 한계를 정직하게 살펴봅니다.

8.56 13.1 전자기장(EMF) 탐지의 원리와 한계

8.56.1 어떻게 탐지하는가

카메라를 비롯한 전자 기기는 동작 중 세 가지 방식으로 전자기장을 발생시킵니다.

1. **전원 회로**: 배터리 충전, 전압 변환기에서 자기장 발생
2. **무선 통신**: Wi-Fi, Bluetooth 모듈이 RF 신호 방출

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

3. **모터/구동계**: 팬이 있는 기기는 회전 모터의 자기장 방출

스마트폰의 3축 자력계(Magnetometer)는 TYPE_MAGNETIC_FIELD 센서로 x, y, z축의 자기장 강도를 마이크로테슬라(μT) 단위로 측정합니다.

8.56.2 현실적인 한계

EMF 탐지로 가능한 것:

- 전자 기기가 근처에 있을 때의 자기장 변화 감지 (10~30cm 이내)
- 배경 자기장 대비 이상 수치 탐지

EMF 탐지로 불가능한 것:

- 특정 기기 종류 식별 (카메라 vs 공유기 vs 충전기)
- 1m 이상 거리의 소형 기기 탐지
- 주변 전기 배선, 가전제품 노이즈와 구분
- 배터리만 사용하는 저전력 카메라 탐지

이것이 EMF 레이어에 15%라는 낮은 가중치를 할당한 이유입니다. 단독으로는 신뢰할 수 없지만, Wi-Fi 스캔이나 렌즈 감지와 조합하면 보강 신호가 됩니다.

8.57 13.2 SensorManager TYPE_MAGNETIC_FIELD 20Hz 수집

8.57.1 Android 자력계 API

Android는 `SensorManager`를 통해 자력계에 접근합니다. `TYPE_MAGNETIC_FIELD` 센서가 3축 자기장을 제공합니다.

```
// data/sensor/MagneticSensor.kt
class MagneticSensor @Inject constructor(
    private val sensorManager: SensorManager,
    private val noiseFilter: NoiseFilter,
    @DefaultDispatcher private val dispatcher: CoroutineDi
) {
    companion object {
        // 20Hz 샘플링 = SENSOR_DELAY_GAME (약 50ms 간격)
        // SENSOR_DELAY_FASTEST(~1ms)는 배터리 소모가 크고 노이
        const val SAMPLING_RATE = SensorManager.SENSOR_DEL

        // 탐지 임계값 — 배경 대비 이 값 이상 변화하면 이상 신호로
        const val ANOMALY_THRESHOLD_UT = 20f  // 마이크로테
    }

    // 센서 데이터를 Flow로 제공합니다
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    fun startMeasurement(): Flow<MagneticReading> = callba
        val magnetometer = sensorManager.getDefaultSensor(

        // 자력계가 없는 기기에서는 빈 Flow (EMF 레이어 비활성화
        if (magnetometer == null) {
            close()
            return@callbackFlow
        }

        val listener = object : SensorEventListener {
            override fun onSensorChanged(event: SensorEver
                if (event.sensor.type != Sensor.TYPE_MAGNE

                val rawX = event.values[0]  // x축 자기장 (μ
                val rawY = event.values[1]  // y축 자기장 (μ
                val rawZ = event.values[2]  // z축 자기장 (μ

                // 자기장 벡터의 크기 (magnitude) 계산
                val magnitude = sqrt(rawX * rawX + rawY *

                val reading = MagneticReading(
                    timestamp = event.timestamp,
```

```
                x = rawX,
                y = rawY,
                z = rawZ,
                magnitude = magnitude,
                delta = 0f  // 노이즈 필터 통과 후 계산
            )

            trySend(reading)
        }

        override fun onAccuracyChanged(sensor: Sensor,
            // 정확도 변화 — 필요 시 처리
        }
    }

    sensorManager.registerListener(
        listener,
        magnetometer,
        SAMPLING_RATE  // 20Hz
    )

    // Flow 취소 시 센서 등록 해제 (배터리 절약)
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        awaitClose {
            sensorManager.unregisterListener(listener)
        }
    }

    // 센서 가용성 확인
    fun isAvailable(): Boolean =
        sensorManager.getDefaultSensor(Sensor.TYPE_MAGNETI

    // 센서 정확도 확인 (ACCURACY_HIGH, MEDIUM, LOW, UNRELIAB
    fun checkAccuracy(): Int {
        val sensor = sensorManager.getDefaultSensor(Sensor
            ?: return SensorManager.SENSOR_STATUS_UNRELIAB
        // 실제 정확도는 onAccuracyChanged 콜백에서 업데이트됨
        return lastAccuracy
    }

    private var lastAccuracy = SensorManager.SENSOR_STATUS
}
```

8.58 13.3 이동 평균(Moving Average) 노이즈 필터 구현

8.58.1 왜 노이즈 필터가 필요한가

자력계 원시 데이터는 매우 불안정합니다. 스마트폰 자체의 전기 회로, 사용자의 움직임, 주변 금속의 영향으로 값이 튀는 **노이즈**가 많습니다. 이를 그대로 사용하면 아무것도 없는데도 경보가 울립니다.

이동 평균 필터는 최근 N개 값의 평균을 사용하는 가장 단순하면서도 효과적인 필터입니다. 잠깐 튀는 노이즈는 평균에 묻히고, 진짜 이상 신호는 여러 샘플에 걸쳐 지속됩니다.

```
// data/analysis/NoiseFilter.kt
class NoiseFilter @Inject constructor() {
    companion object {
        const val WINDOW_SIZE = 10  // 이동 평균 윈도우 크기 (
        const val SPIKE_THRESHOLD_UT = 50f  // 0.3초 내 50μ
        const val SPIKE_WINDOW_FRAMES = 6   // 0.3초에 해당
    }

    // 최근 WINDOW_SIZE 개의 magnitude 값을 저장하는 원형 버퍼
    private val magnitudeBuffer = ArrayDeque<Float>(WINDOW
    private var smoothedMagnitude = 0f
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    // 원시 자기장 측정값에 노이즈 필터를 적용합니다
    fun filter(reading: MagneticReading): MagneticReading
        val magnitude = reading.magnitude

        // 스파이크 감지: 최근 6프레임 내에서 50μT 이상 갑자기 변
        if (isSpike(magnitude)) {
            // 스파이크는 버퍼에 추가하지 않고 이전 평균값으로 대
            return reading.copy(
                magnitude = smoothedMagnitude,
                delta = 0f
            )
        }

        // 이동 평균 버퍼에 추가
        magnitudeBuffer.addLast(magnitude)
        if (magnitudeBuffer.size > WINDOW_SIZE) {
            magnitudeBuffer.removeFirst()
        }

        // 새 이동 평균 계산
        val newSmoothed = magnitudeBuffer.average().toFloa
```

```
        val delta = newSmoothed - smoothedMagnitude
        smoothedMagnitude = newSmoothed

        return reading.copy(
            magnitude = smoothedMagnitude,
            delta = delta
        )
    }

    // 스파이크 감지 — 최근 평균 대비 급격한 변화를 탐지합니다
    private fun isSpike(newMagnitude: Float): Boolean {
        if (magnitudeBuffer.size < SPIKE_WINDOW_FRAMES) re
        val recentAvg = magnitudeBuffer.takeLast(SPIKE_WIN
        return abs(newMagnitude - recentAvg) > SPIKE_THRES
    }

    // 필터 상태를 초기화합니다 (새 스캔 시작 시 호출)
    fun reset() {
        magnitudeBuffer.clear()
        smoothedMagnitude = 0f
    }
}
```

8.58.2 이동 평균의 트레이드오프

윈도우 크기가 클수록 노이즈가 줄지만 반응이 느려집니다. 작을수록 노이즈에 민감하지만 실제 신호에 빠르게 반응합니다.

윈도우 크기	대기 시간	노이즈 감쇠	적합한 용도
5 (0.25초)	빠름	낮음	빠른 움직임 탐지
10 (0.5초)	보통	보통	SearCam 선택값
20 (1.0초)	느림	높음	안정적 장소 측정

SearCam은 10을 선택했습니다. 30초 스캔 중에 스마트폰 움직임이 있을 수 있어 너무 느린 필터는 부적합하고, 너무 빠른 필터는 사용자의 움직임을 오탐할 수 있기 때문입니다.

8.59 13.4 캘리브레이션 — 배경 기준선 설정

8.59.1 왜 캘리브레이션이 필요한가

지구 자체가 자기장을 가지고 있습니다. 위치에 따라 30~60μT 사이입니다. 이 배경 자기장이 없다면 어떤 값이 "이상" 인지 판단할 수 없습니다.

캘리브레이션은 스캔 시작 전 3초 동안 "지금 이 환경의 정상 자기

장" 을 측정합니다. 이후 이 값보다 크게 벗어나는 측정값을 이상 신호로 판단합니다.

```
// domain/usecase/CalibrateEmfUseCase.kt
class CalibrateEmfUseCase @Inject constructor(
    private val magneticRepository: MagneticRepository
) {
    companion object {
        const val CALIBRATION_DURATION_MS = 3000L   // 3초
        const val CALIBRATION_SAMPLES = 60           // 20
    }

    // 배경 기준선(baseline)과 노이즈 바닥(noise_floor)을 계산합
    suspend operator fun invoke(): CalibrationResult {
        val samples = magneticRepository.collectSamples(CA

        if (samples.isEmpty()) {
            return CalibrationResult.Unavailable
        }

        val magnitudes = samples.map { it.magnitude }
        val baseline = magnitudes.average().toFloat()
        val stdDev = calculateStdDev(magnitudes, baseline)
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        // 노이즈 바닥 = 평균 ± 2 표준편차 (95% 신뢰 구간)
        val noiseFloor = stdDev * 2f

        return CalibrationResult.Success(
            baseline = baseline,
            noiseFloor = noiseFloor,
            // 탐지 임계값 = 기준선 + 노이즈 바닥 + 20μT 안전 마
            detectionThreshold = baseline + noiseFloor + 2
        )
    }

    private fun calculateStdDev(values: List<Float>, mean:
        val variance = values.map { (it - mean) * (it - me
        return sqrt(variance)
    }
}

// 캘리브레이션 결과 — 불변 sealed class
sealed class CalibrationResult {
    data class Success(
        val baseline: Float,       // 배경 자기장 기준선 (μT)
```

```
        val noiseFloor: Float,      // 노이즈 바닥 (μT)
        val detectionThreshold: Float  // 이상 신호 임계값 (μ
    ) : CalibrationResult()

    object Unavailable : CalibrationResult()  // 자력계 없는
}
```

8.59.2 캘리브레이션 결과 활용

```
// EMF 이상 탐지 로직
class EmfAnomalyDetector @Inject constructor(
    private val noiseFilter: NoiseFilter
) {
    private var calibration: CalibrationResult.Success? =

    fun setCalibration(result: CalibrationResult.Success)
        calibration = result
    }

    // 측정값이 이상 신호인지 판단합니다
    fun isAnomaly(reading: MagneticReading): EmfAnomaly? {
        val cal = calibration ?: return null  // 캘리브레이
```

```
        val filtered = noiseFilter.filter(reading)
        val deviation = filtered.magnitude - cal.baseline

        return when {
            deviation < cal.noiseFloor -> null  // 정상 범위
            deviation < cal.detectionThreshold -> EmfAnomaly.Weak(
                deviation = deviation,
                riskScore = 15  // 약한 이상 — 보조 신호
            )
            else -> EmfAnomaly.Strong(
                deviation = deviation,
                riskScore = 40  // 강한 이상 — 주목 필요
            )
        }
    }
}

// EMF 이상 신호 — 불변 sealed class
sealed class EmfAnomaly(
    open val deviation: Float,
    open val riskScore: Int
) {
```

```
    data class Weak(
        override val deviation: Float,
        override val riskScore: Int
    ) : EmfAnomaly(deviation, riskScore)

    data class Strong(
        override val deviation: Float,
        override val riskScore: Int
    ) : EmfAnomaly(deviation, riskScore)
}
```

8.60 13.5 EMF 단독으로 탐지 불가한 이유 — 보조 레이어 역할

8.60.1 일상적인 EMF 발생원

이것이 EMF 레이어의 근본적인 한계입니다. 호텔 방에는 자기장 발생원이 넘칩니다.

EMF 발생원	예상 자기장 강도
스마트폰 충전기	5~50μT (거리 10cm)

EMF 발생원	예상 자기장 강도
노트북 전원 어댑터	10~100μT
에어컨 실내기	2~20μT
TV	5~30μT
몰래카메라 (소형)	1~10μT

몰래카메라가 오히려 충전기보다 약한 신호를 냅니다. EMF만으로는 "방에 카메라가 있다" 고 말할 수 없습니다.

8.60.2 올바른 사용 방법 — 교차 검증 보강재

잘못된 사용:

```
EMF 이상 감지 → "카메라 발견!" (오탐률 90%+)
```

올바른 사용:

```
Wi-Fi 스캔: 카메라 의심 기기 발견 (70점)
+ 렌즈 감지: 역반사 포인트 1개 (80점)
+ EMF 이상: 해당 방향에서 자기장 상승 (+보정)
→ 교차 검증 엔진: 종합 위험도 85점
```

EMF 단독 감지에서는 "자기장 이상" 정도만 알려줍니다. 다른 레이어와 결합할 때 의미 있는 신호가 됩니다.

8.60.3 UI에서의 투명한 표시

```
// ui/magnetic/MagneticViewModel.kt
class MagneticViewModel @Inject constructor(
    private val magneticRepository: MagneticRepository,
    private val calibrateEmfUseCase: CalibrateEmfUseCase
) : ViewModel() {

    fun buildEmfMessage(anomaly: EmfAnomaly?): String = wh
        null -> "자기장 정상 — 배경 수준 내"
        is EmfAnomaly.Weak ->
            // 솔직하게 한계를 알려줍니다
            "자기장 약한 이상 (+${anomaly.deviation.toInt()}μ
            "주의: 충전기, 전자제품도 유사한 반응을 보일 수 있습니
        is EmfAnomaly.Strong ->
            "자기장 강한 이상 (+${anomaly.deviation.toInt()}μ
            "다른 탐지 레이어 결과와 함께 확인하세요"
    }
}
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.61 13.6 전체 EMF 레이어 통합

```
// domain/repository/MagneticRepository.kt (인터페이스)
interface MagneticRepository {
    fun startMeasurement(): Flow<MagneticReading>
    suspend fun collectSamples(count: Int): List<MagneticR
    fun isAvailable(): Boolean
}

// data/repository/MagneticRepositoryImpl.kt (구현체)
class MagneticRepositoryImpl @Inject constructor(
    private val magneticSensor: MagneticSensor,
    private val noiseFilter: NoiseFilter,
    private val emfAnomalyDetector: EmfAnomalyDetector,
    @DefaultDispatcher private val dispatcher: CoroutineDi
) : MagneticRepository {

    override fun startMeasurement(): Flow<MagneticReading>
        magneticSensor.startMeasurement()
            .map { reading -> noiseFilter.filter(reading)
            .flowOn(dispatcher)

    // 캘리브레이션용 샘플 수집
```

```
    override suspend fun collectSamples(count: Int): List<
        startMeasurement()
            .take(count)
            .toList()

    override fun isAvailable(): Boolean = magneticSensor.i
}
```

8.62 실습

실습 13-1: 스마트폰을 테이블 위에 평평하게 놓고, 스마트폰 자력계 앱(Physics Toolbox)으로 자기장 기준선을 측정해보세요. 그 다음 충전기를 가까이 가져가면서 자기장이 얼마나 변하는지 확인해보세요.

실습 13-2: `NoiseFilter`에서 윈도우 크기를 5와 20으로 바꿔가며 스마트폰을 흔들어보세요. 움직임 노이즈가 필터를 통과하는 차이를 로그로 확인해보세요.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.63 핵심 정리

구성 요소	역할
TYPE_MAGNETIC_FIELD	3축 자기장 20Hz 수집
NoiseFilter	이동 평균 + 스파이크 제거
CalibrateEmfUseCase	배경 기준선 + 임계값 산출
EmfAnomalyDetector	이상 신호 판단

- EMF 레이어 가중치 15%는 단독 탐지 불가 + 교차 보강 역할을 반영한다
- 캘리브레이션 없이는 어떤 값이 이상인지 판단할 수 없다
- 이동 평균은 가장 단순한 노이즈 필터 — 단순함이 곧 신뢰성이다
- 한계를 솔직하게 표시하는 것이 오탐보다 낫다

8.64 13.7 코드 리뷰 개선 사항 — 명확한 상수와 스레드 안전

8.64.1 개선 1: SENSOR_DELAY_FASTEST → 명시적 20Hz 상수

문제: 초기 구현에서 `SensorManager.SENSOR_DELAY_FASTEST`를 사용했습니다. 이름 그대로 가능한 최고 속도(기기마다 다르며 최대 ~1ms)입니다. 결과는 두 가지 문제였습니다. 첫째, 불필요하게 높은 CPU·배터리 소모. 둘째, 실제 Hz를 코드에서 알 수 없어 가독성 저하.

```
// Before: "가장 빠르게" — 실제 Hz를 알 수 없고 배터리 낭비
sensorManager.registerListener(listener, sensor, SensorMan

// After: μs 단위 직접 지정 — 20Hz = 50,000μs (50ms 간격)
companion object {
    /** 20Hz 샘플링 = 50ms 간격 = 50,000μs */
    private const val SENSOR_DELAY_20HZ_US = 50_000
}

sensorManager.registerListener(listener, sensor, SENSOR_DE
```

Android `SensorManager.registerListener()`의 세

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

번째 파라미터는 실제로 μs 단위 지연 값을 받습니다. SENSOR_DELAY_GAME(~20ms, ~50Hz), SENSOR_DELAY_UI(~60ms) 같은 상수도 있지만, 50_000을 직접 쓰면 코드를 읽는 사람이 "이 센서는 20Hz로 동작한다"는 사실을 즉시 알 수 있습니다.

8.64.2 개선 2: NoiseFilter 스파이크 상수를 companion object으로 이동

문제: SPIKE_THRESHOLD_UT = 50f와 SPIKE_TIME_WINDOW_MS = 300L이 함수 내부 지역 상수로 선언되어 있었습니다. 테스트 코드에서 이 임계값을 알려면 소스를 읽어야 했고, 두 곳 이상에서 같은 값이 사용되면 동기화가 어려웠습니다.

```
// Before: 함수 안에 숨어있는 매직 넘버
private fun isSpikeDetected(magnitude: Float, timestamp: L
    val timeDiff = timestamp - previousTimestamp
    val magnitudeDiff = abs(magnitude - previousMagnitude)
    return magnitudeDiff > 50f && timeDiff < 300L  // 50과
}

// After: companion object에 명명된 상수로 의도를 명확히 문서화
companion object {
    /** 급변 판정 임계값 (μT): 50μT 이상 급변 = 스마트폰 자체 간섭
    private const val SPIKE_THRESHOLD_UT = 50f
```

```
    /** 급변 판정 시간 창 (ms): 300ms 이내 급변만 감지 */
    private const val SPIKE_TIME_WINDOW_MS = 300L
}

private fun isSpikeDetected(magnitude: Float, timestamp: L
    val timeDiff = timestamp - previousTimestamp
    val magnitudeDiff = abs(magnitude - previousMagnitude)
    return magnitudeDiff > SPIKE_THRESHOLD_UT && timeDiff
}
```

이 상수들은 `NoiseFilterTest`에서도 참조 기준이 됩니다. 테스트 코드에서 "50μT 초과 + 300ms 이내" 케이스를 명시적으로 테스트합니다. 상수를 `companion object`에 두면 테스트의 가정(assumption)과 구현의 실제 값이 일치함을 보장할 수 있습니다.

8.65 다음 장 예고

세 개의 탐지 레이어를 모두 만들었습니다. 이제 가장 중요한 질문이 남았습니다 — 이 세 레이어의 결과를 어떻게 합쳐 "위험도 0~100"이라는 하나의 숫자로 만들까요? Ch14에서는 CrossValidator 설계와 위험도 산출 알고리즘을 다룹니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

참

고

문

서:

docs/

02-

TRD.md,

docs/

03-

TDD.md

—

Ch14: 교차 검증 — 세 개의 목격자가 일치할 때 믿어라

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—>

이장에서 배울 것: 세탐지레이어를 하나의 위험도 점수(0~100)로

8.66 도입

법원에서 한 명의 목격자 증언으로 유죄를 선고하는 건 위험합니다. 하지만 독립적인 세 명의 목격자가 같은 내용을 증언한다면 이야기가 달라집니다. 각자 다른 방향에서, 다른 방법으로, 같은 결론에 도달했다면 — 신뢰할 수 있습니다.

SearCam의 세 탐지 레이어가 바로 이 세 명의 목격자입니다. Wi-Fi 스캔, 렌즈 감지, EMF — 각각 완전히 다른 물리적 원리로 동작합니다. 이 세 레이어가 동시에 "의심스럽다" 고 말한다면, 그건 우연이 아닙니다.

CrossValidator는 이 세 목격자의 증언을 종합해서 판사 역할을 합니다.

8.67 14.1 세 레이어를 어떻게 합치는가 — CrossValidator 설계

8.67.1 기본 가중치 모델

각 레이어는 독립적으로 0~100 점수를 생성합니다. CrossValidator는 이 점수에 가중치를 곱해 최종 점수를 계산합니다.

기본 가중치:

```
  Layer 1 (Wi-Fi)    = 50%   → 0~50점 기여
 Layer 2 (렌즈)     = 35%   → 0~35점 기여
 Layer 3 (EMF)      = 15%   → 0~15점 기여
```

기본 계산:

```
   기반 점수 = (Wi-Fi 점수 × 0.5) + (렌즈 점수 × 0.35) + (EMF 점수 × 0.15)
```

하지만 이것만으로는 부족합니다. 레이어들이 서로를 확인해주는 **교차 검증 효과**를 반영해야 합니다.

8.67.2 CrossValidator 핵심 설계

```
// data/analysis/CrossValidator.kt
class CrossValidator @Inject constructor() {

    companion object {
        // 기본 가중치 (레이어 사용 가능 시)
        const val WEIGHT_WIFI = 0.50f
        const val WEIGHT_LENS = 0.35f
        const val WEIGHT_EMF = 0.15f

        // 교차 보정 계수 — 몇 개의 레이어가 양성인지에 따라 달라집
        const val CORRECTION_SINGLE = 0.7f    // 1개 양성:
```

```
        const val CORRECTION_DOUBLE = 1.2f    // 2개 양성:
        const val CORRECTION_TRIPLE = 1.5f    // 3개 양성:

        // 위험도 등급 임계값
        const val THRESHOLD_SAFE = 20
        const val THRESHOLD_INTEREST = 40
        const val THRESHOLD_CAUTION = 60
        const val THRESHOLD_DANGER = 80
    }

    // 세 레이어의 결과를 종합해 위험도를 산출합니다
    fun validate(
        wifiResult: ScanResult?,
        lensResult: ScanResult?,
        emfResult: ScanResult?
    ): ValidationResult {

        // 사용 가능한 레이어만 동적으로 가중치를 조정합니다
        val adjustedWeights = calculateAdjustedWeights(
            hasWifi = wifiResult != null,
            hasLens = lensResult != null,
            hasEmf = emfResult != null
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    )

    // 각 레이어의 가중 점수를 계산합니다
    val wifiScore = (wifiResult?.score ?: 0) * adjuste
    val lensScore = (lensResult?.score ?: 0) * adjuste
    val emfScore = (emfResult?.score ?: 0) * adjustedW

    // 기반 점수 합산
    val baseScore = wifiScore + lensScore + emfScore

    // 양성 레이어 수 카운트 (임계값 30 이상 = 양성으로 판정)
    val positiveCount = listOfNotNull(wifiResult, lens
        .count { it.score >= 30 }

    // 교차 보정 계수 적용
    val correctionFactor = when (positiveCount) {
        0 -> 1.0f
        1 -> CORRECTION_SINGLE   // 단일 탐지 — 신뢰도 낮
        2 -> CORRECTION_DOUBLE   // 이중 탐지 — 신뢰도 높
        else -> CORRECTION_TRIPLE  // 삼중 탐지 — 강력한
    }
```

```
        val finalScore = (baseScore * correctionFactor)
            .toInt()
            .coerceIn(0, 100)

        return ValidationResult(
            overallRisk = finalScore,
            riskLevel = scoreToRiskLevel(finalScore),
            positiveLayerCount = positiveCount,
            wifiContribution = wifiScore.toInt(),
            lensContribution = lensScore.toInt(),
            emfContribution = emfScore.toInt(),
            correctionFactor = correctionFactor,
            findings = buildFindings(wifiResult, lensResul
        )
    }

    // 위험도 점수를 등급으로 변환합니다
    fun scoreToRiskLevel(score: Int): RiskLevel = when {
        score < THRESHOLD_SAFE -> RiskLevel.SAFE
        score < THRESHOLD_INTEREST -> RiskLevel.INTEREST
        score < THRESHOLD_CAUTION -> RiskLevel.CAUTION
        score < THRESHOLD_DANGER -> RiskLevel.DANGER
```

CHAPTER 8 | 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        else -> RiskLevel.CRITICAL
    }
}

// 조정된 가중치 — 불변 데이터 클래스
data class AdjustedWeights(
    val wifi: Float,
    val lens: Float,
    val emf: Float
)

// 검증 결과 — 불변 데이터 클래스
data class ValidationResult(
    val overallRisk: Int,
    val riskLevel: RiskLevel,
    val positiveLayerCount: Int,
    val wifiContribution: Int,
    val lensContribution: Int,
    val emfContribution: Int,
    val correctionFactor: Float,
    val findings: List<Finding>
)
```

8.68 14.2 가중치 동적 조정 — Wi-Fi 없을 때

8.68.1 레이어가 비활성화되는 상황

세 레이어가 항상 모두 사용 가능하지는 않습니다.

상황	비활성화 레이어
Wi-Fi 꺼짐 또는 연결 없음	Layer 1 (Wi-Fi 스캔)
카메라 권한 거부	Layer 2 (렌즈 감지)
자력계 없는 기기	Layer 3 (EMF)
완전 오프라인 환경	Layer 1

이때 단순히 해당 레이어를 0점으로 처리하면 최대 가능 점수가 줄어들어 비현실적인 결과가 나옵니다. 대신 나머지 레이어들에 가중치를 재분배합니다.

```
// 사용 가능한 레이어에 가중치를 동적으로 재분배합니다
private fun calculateAdjustedWeights(
    hasWifi: Boolean,
    hasLens: Boolean,
    hasEmf: Boolean
): AdjustedWeights {
```

```
    // 사용 가능한 레이어의 기본 가중치 합계 계산
    val totalWeight = (if (hasWifi) WEIGHT_WIFI else 0f) +
                      (if (hasLens) WEIGHT_LENS else 0f) +
                      (if (hasEmf) WEIGHT_EMF else 0f)

    if (totalWeight == 0f) {
        // 모든 레이어 비활성화 — 탐지 불가 상태
        return AdjustedWeights(0f, 0f, 0f)
    }

    // 사용 가능한 레이어끼리 100%가 되도록 정규화합니다
    return AdjustedWeights(
        wifi = if (hasWifi) WEIGHT_WIFI / totalWeight else
        lens = if (hasLens) WEIGHT_LENS / totalWeight else
        emf = if (hasEmf) WEIGHT_EMF / totalWeight else 0f
    )
}
```

8.68.2 동적 조정 예시

```
Wi-Fi 없음 (Layer 1 비활성화):
  기본: Wi-Fi 50% + 렌즈 35% + EMF 15% = 100%
  조정: (렌즈 35% + EMF 15%) = 50% → 정규화
```

```
  결과: Wi-Fi 0% + 렌즈 70% + EMF 30% = 100%


렌즈+Wi-Fi만 (EMF 없음):
   기본: Wi-Fi 50% + 렌즈 35% = 85% → 정규화
  결과: Wi-Fi 58.8% + 렌즈 41.2% + EMF 0%
```

8.69 14.3 보정 계수 설계 근거

8.69.1 왜 ×0.7, ×1.2, ×1.5인가

이 수치는 경험적으로 도출한 값입니다. 실제 IP 카메라와 일반 가전제품을 대상으로 반복 테스트한 결과입니다.

단일 탐지 (×0.7): 하나의 레이어만 양성이면 다른 레이어가 침묵하는 겁니다. 즉, 두 개의 독립적인 검증이 "없음" 이라고 말하는 상황입니다. 신뢰도를 낮추는 게 맞습니다.

```
예시: Wi-Fi 스캔만 양성 (카메라 OUI 발견, 점수 75)
   기반 점수: 75 × 0.5 = 37.5
  보정 적용: 37.5 × 0.7 = 26.25 → 약 26점 (INTEREST 등급)
  해석: "네트워크에 의심 기기가 있지만, 렌즈/EMF 미확인"
```

이중 탐지 (×1.2): 두 개의 독립적인 물리 원리가 같은 방향을 가리킵니다. 우연의 일치가 아닐 가능성이 높습니다.

예시: Wi-Fi 양성 (75점) + 렌즈 양성 (60점), EMF 음성

기반 점수: (75 × 0.5) + (60 × 0.35) = 37.5 + 21 = 58.5

보정 적용: 58.5 × 1.2 = 70.2 → 약 70점 (DANGER 등급)

해석: "네트워크 기기 + 렌즈 역반사 이중 확인"

삼중 탐지 (×1.5): 세 레이어 모두 양성. 세 가지 독립적인 물리적 방법이 모두 같은 결론입니다. 강력한 증거입니다.

예시: 세 레이어 모두 양성 (Wi-Fi 80, 렌즈 70, EMF 50)

기반 점수: (80 × 0.5) + (70 × 0.35) + (50 × 0.15) = 40 + 24.5 +

보정 적용: 72 × 1.5 = 108 → 100점 (상한선, CRITICAL 등급)

해석: "세 레이어 모두 양성 — 즉시 확인 필요"

8.69.2 상한선 100점의 의미

점수가 100을 넘어도 100으로 제한합니다. `coerceIn(0, 100)` 사용. 이유는 두 가지입니다.

첫째, UI에서 게이지(0~100%)로 표시하기 위한 정규화입니다. 둘째, "100% 확실" 이라고 주장하지 않기 위해서입니다. 100점이라도 "높은 가능성" 이지 "확정" 이 아닙니다.

8.70 14.4 위험도 0~100 스코어 산출 알고리즘 — 전체 흐름

```
// domain/usecase/CalculateRiskUseCase.kt
class CalculateRiskUseCase @Inject constructor(
    private val crossValidator: CrossValidator
) {
    suspend operator fun invoke(
        wifiResult: ScanResult?,
        lensResult: ScanResult?,
        emfResult: ScanResult?
    ): RiskAssessment {

        // CrossValidator에 세 레이어 결과를 넘깁니다
        val validation = crossValidator.validate(wifiResul

        // 위험도에 따른 권고 사항을 생성합니다
        val recommendations = buildRecommendations(
            validation.riskLevel,
            validation.findings
        )
```

```
        return RiskAssessment(
            score = validation.overallRisk,
            level = validation.riskLevel,
            validation = validation,
            recommendations = recommendations,
            timestamp = System.currentTimeMillis()
        )
    }

    // 위험 등급별 사용자 권고 사항
    private fun buildRecommendations(
        level: RiskLevel,
        findings: List<Finding>
    ): List<String> = buildList {
        when (level) {
            RiskLevel.SAFE -> {
                add("현재 환경에서 의심 신호가 발견되지 않았습니
                add("정기적인 재스캔을 권장합니다 (30분마다).")
            }
            RiskLevel.INTEREST -> {
                add("약한 의심 신호가 있습니다. 추가 확인을 권장
                findings.forEach { finding -> add("• ${fin
```

```
            }
            RiskLevel.CAUTION -> {
                add("주의: 복수의 의심 신호가 감지되었습니다.")
                add("렌즈 찾기 모드로 육안 확인을 권장합니다.")
                findings.forEach { finding -> add("• ${fin
            }
            RiskLevel.DANGER, RiskLevel.CRITICAL -> {
                add("경고: 강한 의심 신호가 감지되었습니다.")
                add("즉시 육안으로 확인하거나 다른 장소로 이동하
                add("의심 기기 발견 시 덮개를 씌우거나 관할 경찰에
                findings.forEach { finding -> add("• ${fin
            }
        }
    }
}
```

8.71 14.5 실제 탐지 시나리오 3가지

8.71.1 시나리오 1: 숙박시설 (모텔/에어비앤비)

환경: 에어비앤비 원룸 체크인 직후 스캔

같은 Wi-Fi 연결, 플래시 켜고 렌즈 스캔, EMF 스캔

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

탐지 결과:

Layer 1 Wi-Fi:

- ARP 기기 3개: 라우터(ASUS), 스마트TV(Samsung), 의심 기기(Hikvision OUI)
- 의심 기기 포트: 554(RTSP) OPEN, 8080(HTTP) OPEN
- Wi-Fi 점수: 85

Layer 2 렌즈:

- 에어컨 리모컨 방향에서 역반사 포인트 1개 발견
- 원형도 0.82, 밝기 240, 플래시 OFF 시 소멸 확인
- 렌즈 점수: 78

Layer 3 EMF:

- 리모컨 방향 자기장 기준선 대비 +35μT 상승
- EMF 점수: 45

교차 검증:

기반 점수: (85 × 0.5) + (78 × 0.35) + (45 × 0.15)

= 42.5 + 27.3 + 6.75 = 76.55

양성 레이어: 3개 → 보정 계수 ×1.5

최종 점수: 76.55 × 1.5 = 114.8 → 100 (CRITICAL)

결과:

  CRITICAL — 위험도 100/100

"에어컨 리모컨 방향에서 강한 의심 신호:

카메라 제조사 기기(Hikvision) 네트워크 감지,

RTSP 포트 활성, 렌즈 역반사 확인, 자기장 이상"

8.71.2 시나리오 2: 공중화장실

환경: 대형 마트 화장실

Wi-Fi 연결 없음(공용 Wi-Fi 미연결), 플래시 렌즈 스캔

탐지 결과:

Layer 1 Wi-Fi: 비활성화 (네트워크 미연결)

→ 가중치 재분배: 렌즈 70%, EMF 30%

Layer 2 렌즈:

- 환기구 방향에서 역반사 포인트 1개
- 원형도 0.91, 밝기 245, 플래시 OFF 소멸 확인
- 렌즈 점수: 88

Layer 3 EMF:

- 기준선 정상 범위 내

- EMF 점수: 8

교차 검증:

기반 점수: (88 × 0.7) + (8 × 0.3)

= 61.6 + 2.4 = 64

양성 레이어: 1개(렌즈만) → 보정 계수 ×0.7

최종 점수: 64 × 0.7 = 44.8 → 44 (INTEREST)

결과:

☐ INTEREST — 위험도 44/100

"환기구 방향 렌즈 역반사 감지

(Wi-Fi 연결 없어 네트워크 확인 불가)

육안으로 환기구를 직접 확인하세요"

해석: 렌즈 점수가 높지만 Wi-Fi로 교차 확인이 안 됨.

단독 탐지이므로 보정으로 낮춤.

실제로는 화장실 조명 LED였을 가능성도 있음.

8.71.3 시나리오 3: 탈의실

환경: 헬스장 탈의실

Wi-Fi 연결됨, 렌즈 스캔 어려움 (조명 밝음)

탐지 결과:

Layer 1 Wi-Fi:

- ARP 기기: 라우터만 발견, 카메라 의심 기기 없음
- Wi-Fi 점수: 5

Layer 2 렌즈:

- 조명이 밝아 역반사 구별 어려움 — 오탐 포인트 3개
- 플래시 OFF 검증: 3개 모두 지속 (가짜 반사)
- 렌즈 점수: 10 (플래시 검증으로 대폭 하향)

Layer 3 EMF:

- 헤어드라이어 방향 자기장 강함 (+80μT) — 명확한 발생원
- 탐지 임계값 초과하지만 방향 명확
- EMF 점수: 20

교차 검증:

기반 점수: (5 × 0.5) + (10 × 0.35) + (20 × 0.15)
= 2.5 + 3.5 + 3 = 9

양성 레이어: 0개 → 보정 계수 ×1.0

최종 점수: 9 → 9 (SAFE)

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

결과:

☐ SAFE — 위험도 9/100

"현재 탐지된 의심 신호 없음.

Wi-Fi 네트워크 정상, 렌즈 역반사 미탐지"

8.72 14.6 교차 검증 결과 ViewModel 연동

```
// ui/scan/ScanViewModel.kt (관련 부분)
class ScanViewModel @Inject constructor(
    private val runFullScanUseCase: RunFullScanUseCase,
    private val calculateRiskUseCase: CalculateRiskUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow<ScanUiState>(S
    val uiState: StateFlow<ScanUiState> = _uiState.asState

    fun startFullScan() {
        viewModelScope.launch {
            _uiState.value = ScanUiState.Scanning(progress

            try {
```

```
            // 세 레이어를 병렬로 실행합니다
            val wifiResult = async { runFullScanUseCas
            val lensResult = async { runFullScanUseCas
            val emfResult = async { runFullScanUseCase

            // 모두 완료될 때까지 대기
            val results = awaitAll(wifiResult, lensRes

            // CrossValidator로 최종 위험도 산출
            val riskAssessment = calculateRiskUseCase(
                wifiResult = results[0],
                lensResult = results[1],
                emfResult = results[2]
            )

            _uiState.value = ScanUiState.Complete(
                riskScore = riskAssessment.score,
                riskLevel = riskAssessment.level,
                recommendations = riskAssessment.recom
                validation = riskAssessment.validation
            )
        } catch (e: Exception) {
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
                _uiState.value = ScanUiState.Error(
                    message = "스캔 중 오류가 발생했습니다: ${
                )
            }
        }
    }
}

// 스캔 UI 상태 — 불변 sealed class
sealed class ScanUiState {
    object Idle : ScanUiState()
    data class Scanning(val progress: Int) : ScanUiState()
    data class Complete(
        val riskScore: Int,
        val riskLevel: RiskLevel,
        val recommendations: List<String>,
        val validation: ValidationResult
    ) : ScanUiState()
    data class Error(val message: String) : ScanUiState()
}
```

8.73 실습

실습 14-1: 세 가지 시나리오의 숫자를 직접 계산해보세요. 보정 계수를 0.7/1.2/1.5 대신 0.6/1.3/1.8로 바꾸면 각 시나리오의 최종 점수가 어떻게 달라지는지 비교해보세요.

실습 14-2: `CrossValidator`의 양성 판정 임계값을 30에서 50으로 올려보세요. 어떤 시나리오에서 양성 레이어 수가 달라지는지, 결과적으로 최종 점수가 어떻게 변하는지 분석해보세요.

8.74 핵심 정리

개념	내용
기본 가중치	Wi-Fi 50%, 렌즈 35%, EMF 15%
동적 가중치	비활성 레이어 비중을 나머지가 나눠 가짐
단독 탐지 ×0.7	교차 미확인 → 신뢰도 하향
이중 탐지 ×1.2	독립 교차 확인 → 신뢰도 상향
삼중 탐지 ×1.5	강력한 증거 → 최고 신뢰도

- 교차 검증은 오탐률을 결정적으로 낮추는 핵심 설계다

- 단독 탐지 신호는 신뢰도를 낮춰 사용자에게 과도한 공포를 주지 않는다
- 삼중 탐지는 세 개의 독립적 물리 원리가 같은 결론 — 가장 신뢰할 수 있는 신호다
- 100점이라도 "확정" 이 아니라 "강한 가능성" — UI에서 솔직하게 전달해야 한다

8.75 14.7 코드 리뷰 개선 사항 — 단일 진실 출처와 중복 제거

8.75.1 개선 1: 가중치 상수를 LayerType 열거형으로 통합 — 단일 진실 출처

문제: 초기 구현에서 `CrossValidatorImpl`이 `WEIGHT_WIFI = 0.50f`, `WEIGHT_LENS = 0.35f` 같은 상수를 자체적으로 정의했습니다. 이미 `LayerType` 열거형에 동일한 가중치가 정의되어 있었는데 중복이었습니다.

```
// Before: CrossValidatorImpl 내부에 중복 상수
class CrossValidatorImpl : CrossValidator {
    companion object {
        private const val WEIGHT_WIFI = 0.50f    // LayerTy
```

```
        private const val WEIGHT_LENS = 0.35f
        private const val WEIGHT_EMF = 0.15f
    }
}

// After: LayerType.weight를 단일 진실 출처(Single Source of
class CrossValidatorImpl : CrossValidator {
    override fun calculateRisk(...): Int {
        val wifiWeight = LayerType.WIFI.weight
        val lensWeight = LayerType.LENS.weight + LayerType
        val emfWeight  = LayerType.MAGNETIC.weight
        // ...
    }
}

// LayerType 열거형 — 가중치의 유일한 정의 장소
enum class LayerType(
    val weight: Float,        // Wi-Fi 연결 시 기본 가중치
    val weightNoWifi: Float,  // Wi-Fi 미연결 시 가중치
    ...
) {
    WIFI(weight = 0.50f, weightNoWifi = 0.00f, ...),
```

```
    LENS(weight = 0.20f, weightNoWifi = 0.45f, ...),
    IR(weight = 0.15f,   weightNoWifi = 0.30f, ...),
    MAGNETIC(weight = 0.15f, weightNoWifi = 0.25f, ...),
}
```

단일 진실 출처(SSOT, Single Source of Truth)는 소프트웨어 설계의 핵심 원칙입니다. 가중치를 변경해야 할 때 `LayerType` 한 곳만 수정하면 모든 사용처에 자동으로 반영됩니다. 두 곳에 정의된 값은 언젠가 반드시 불일치가 발생합니다.

8.75.2 개선 2: `CalculateRiskUseCase.invoke()`가 `invokeWithCorrection()`에 위임 — 중복 로직 제거

문제: `invoke()`와 `invokeWithCorrection()` 두 함수가 거의 동일한 로직을 가지고 있었습니다. 34줄짜리 계산 로직이 복사·붙여넣기되어 있어서, 알고리즘을 수정하면 두 곳을 모두 수정해야 했습니다.

```
// Before: invoke()와 invokeWithCorrection()에 동일한 34줄 로
operator fun invoke(layerResults: Map<LayerType, LayerResu
    // 34줄의 가중치 계산 로직 (completedLayers, isWifiAvailab
    val finalScore = ...
    return finalScore.coerceIn(0, 100)
```

```
}

fun invokeWithCorrection(...): Pair<Int, Float> {
    // 동일한 34줄 로직 반복
    val finalScore = ...
    return Pair(finalScore, correctionFactor)
}

// After: invoke()가 invokeWithCorrection()에 완전 위임 — 1줄
operator fun invoke(layerResults: Map<LayerType, LayerResu
    invokeWithCorrection(layerResults).first
```

이 리팩토링으로 코드가 34줄에서 1줄로 줄었습니다. DRY(Don't Repeat Yourself) 원칙의 교과서적 적용입니다. 앞으로 알고리즘을 바꾸면 `invokeWithCorrection()` 한 곳만 수정하면 됩니다.

8.75.3 개선 3: `RunFullScanUseCase`에 `AllLayerResults` 네임드 data class 도입

문제: `runAllLayersInParallel()`이 `List<LayerResult>`를 반환하고, 호출부에서 인덱스 기반으로 구조 분해(destructuring)했습니다. 리스트 순서가 바뀌면 컴파일 오류 없이 wifi/lens/ir/magnetic이 뒤바뀌는 심각한 버그가 발생할 수 있었습니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
// Before: 위치 기반 destructuring — 순서 오류가 컴파일 타임에 잡
val (wifiResult, lensResult, irResult, magneticResult) = r
//  ↑ listOf() 안의 순서가 바뀌면 silently wrong data

// After: 이름 기반 data class — 잘못된 접근은 컴파일 오류
private data class AllLayerResults(
    val wifi: LayerResult,
    val lens: LayerResult,
    val ir: LayerResult,
    val magnetic: LayerResult,
)

private suspend fun runAllLayersInParallel(...): AllLayerR
    AllLayerResults(
        wifi     = wifiDeferred.await(),
        lens     = lensDeferred.await(),
        ir       = irDeferred.await(),
        magnetic = magneticDeferred.await(),
    )
}

// 사용부: 이름으로 접근 — 순서 변경에 영향받지 않음
```

```
val layersParallel = runAllLayersInParallel(lifecycleOwner
val wifiResult = layersParallel.wifi
val lensResult = layersParallel.lens
```

"무언의 실패(Silent failure)"는 가장 위험한 버그 패턴입니다. 위치 기반 destructuring은 컴파일러가 타입만 확인하므로, `List<LayerResult>`에서 순서 오류는 런타임에서야 발견됩니다. 이름 기반 접근은 이 위험을 컴파일 타임으로 앞당깁니다.

8.76 다음 장 예고

탐지 엔진의 핵심이 완성되었습니다. Ch15에서는 이 결과를 사용자에게 어떻게 전달할지 — ScanResult 화면, RiskGauge 컴포넌트, 리포트 저장 — Compose UI 레이어를 구현합니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

참

고

문

서:

docs/

02-

TRD.md,

docs/

03-

TDD.md,

docs/

04-

system-

architecture.md

—

Ch15: UI 구현 — Jetpack Compose로 스캔 화면 만들기

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

>

이장에서 배울 것: Jetpack Compose의 선언형 UI 철학이 SearCam의 스캔 화

8.77 도입

레고 블록을 쌓는다고 상상해보세요. 기존 XML 레이아웃 방식은 설계도(XML)와 조립(Java/Kotlin)이 분리되어 있었습니다. 설계도를 보면서 실제 조립 상태를 머릿속으로 그려야 했죠. Jetpack Compose는 이 둘을 하나로 합쳤습니다. 코드를 읽으면 화면이 어떻게 생겼는지 바로 보입니다. 조립과 설계가 같은 언어로 이루어지는 셈입니다.

SearCam의 스캔 화면은 이 철학의 좋은 예입니다. 위험도를 나타내는 반원 게이지, 레이더처럼 퍼지는 펄스 애니메이션, 스캔 상태에 따라 동적으로 바뀌는 UI — 이 모든 것을 Compose의 선언형 방식으로 구현했습니다. 이 장에서는 그 구현 과정을 처음부터 함께 따라갑니다.

8.78 15.1 선언형 UI의 핵심 개념

8.78.1 상태가 UI를 만든다

식당의 칠판 메뉴를 생각해보세요. 주방에서 재료가 떨어지면 웨이터가 칠판을 지우고 새로 씁니다. 칠판(UI)은 항상 현재 상태(재료 현황)를 반영합니다. Compose가 바로 이 방식입니다.

전통적인 View 시스템에서는 상태 변경 시 개발자가 직접 `textView.text = "새 값"` 처럼 UI를 명령해야 했습니다. Compose에서는 상태가 바뀌면 Compose 런타임이 자동으로 필요한 부분만 다시 그립니다(Recomposition). 개발자는 "이 상태일 때 화면은 이렇게 생겼다" 만 선언하면 됩니다.

```
// 명령형 방식 (View 시스템)
if (isScanning) {
    progressBar.visibility = View.VISIBLE
    scanButton.isEnabled = false
    statusText.text = "스캔 중..."
}

// 선언형 방식 (Compose)
// 상태가 바뀌면 Compose가 알아서 재구성합니다
when (uiState) {
    is ScanUiState.Scanning -> {
        LinearProgressIndicator()
        Text(text = "스캔 중...")
        // 버튼은 렌더링하지 않음 — 존재 자체가 없어짐
    }
    is ScanUiState.Idle -> {
        Button(onClick = { viewModel.startQuickScan() }) {
```

```
                Text("스캔 시작")
            }
        }
}
```

이 차이는 단순히 스타일의 문제가 아닙니다. 선언형 방식은 UI와 상태의 불일치(버튼은 비활성화됐는데 텍스트는 여전히 "시작" 인 상황)를 구조적으로 방지합니다.

8.78.2 Recomposition과 성능

Compose는 영리합니다. 상태가 바뀔 때 전체 화면을 다시 그리지 않습니다. 바뀐 상태를 읽는 Composable 함수만 선택적으로 재실행합니다. 이를 Recomposition이라 합니다.

성능을 위해 기억해야 할 원칙 두 가지: 1. Composable 함수는 순수 함수처럼 작성하세요. 같은 입력에 항상 같은 출력을 내야 합니다. 2. 비싼 계산은 `remember`로 캐시하세요. Recomposition 때마다 재실행되면 성능이 떨어집니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.79 15.2 ScanViewModel — StateFlow와 sealed class UiState

8.79.1 sealed class로 UI 상태 모델링

비행기 탑승 절차를 생각해보세요. "체크인 중", "탑승 대기", "탑승 완료", "지연" — 이 상태들은 동시에 존재할 수 없고, 각 상태마다 표시되는 정보가 다릅니다. SearCam의 스캔 화면도 마찬가지입니다. sealed class는 이런 "유한하고 배타적인 상태"를 표현하는 완벽한 도구입니다.

```
// ui/scan/ScanViewModel.kt


sealed class ScanUiState {
    /** 초기 대기 상태 */
    data object Idle : ScanUiState()


    /**
     * 스캔 진행 중
     *
     * @param elapsedSeconds 경과 시간 (초)
     * @param foundDevices 지금까지 발견된 기기 목록
     * @param progress 전체 진행률 (0.0 ~ 1.0)
     * @param currentStep 현재 단계 이름
```

```
     */
    data class Scanning(
        val elapsedSeconds: Int = 0,
        val foundDevices: List<NetworkDevice> = emptyList(
        val progress: Float = 0f,
        val currentStep: String = "스캔 준비 중...",
    ) : ScanUiState()

    /** 스캔 완료 */
    data class Success(val report: ScanReport) : ScanUiSta

    /**
     * 스캔 오류
     *
     * @param code 오류 코드 (E1xxx: 센서, E2xxx: 네트워크, E
     * @param message 사용자 표시 메시지
     */
    data class Error(val code: String, val message: String
}
```

data object와 data class의 차이에 주목하세요. Idle처럼 추가 데이터가 없는 상태는 data object로, Scanning처럼 상태와 함께 데이터를 전달해야 하는 경우는 data class로 선언합니

다. Kotlin 1.9부터 `data object`는 `equals()`와 `toString()`을 올바르게 구현해줍니다.

8.79.2 StateFlow와 SharedFlow의 역할 분리

ScanViewModel에는 두 종류의 Flow가 있습니다. StateFlow와 SharedFlow입니다. 이 둘의 역할은 명확하게 분리됩니다.

```
@HiltViewModel
class ScanViewModel @Inject constructor(
    private val runQuickScanUseCase: RunQuickScanUseCase,
    private val runFullScanUseCase: RunFullScanUseCase,
) : ViewModel() {

    // StateFlow: 현재 UI 상태. 항상 최신값을 가집니다.
    // 새로 구독해도 즉시 현재 상태를 받습니다 (replay = 1)
    private val _uiState = MutableStateFlow<ScanUiState>(S
    val uiState: StateFlow<ScanUiState> = _uiState.asState

    // SharedFlow: 일회성 이벤트. 화면 이동, 스낵바 등
    // replay = 0 — 구독 이전 이벤트를 받지 않습니다
    private val _events = MutableSharedFlow<ScanUiEvent>()
    val events: SharedFlow<ScanUiEvent> = _events.asShared
```

```
    // 취소 가능한 스캔 Job
    private var scanJob: Job? = null
```

StateFlow를 텔레비전 채널의 현재 방송으로, SharedFlow를 알림 벨소리로 비유할 수 있습니다. 채널을 켜면(구독하면) 지금 방송 중인 내용(현재 상태)을 바로 볼 수 있습니다. 반면 알림은 울리는 순간에 자리에 있어야만 들을 수 있습니다. 놓친 알림은 다시 받을 수 없습니다.

이 구분이 중요한 이유: 화면 이동 이벤트를 StateFlow에 넣으면 화면 회전 후 재구독할 때 같은 이동이 다시 발생합니다. SharedFlow는 이 문제를 방지합니다.

8.79.3 스캔 취소와 Job 관리

```
fun startQuickScan() {
    // 이미 스캔 중이면 중복 실행 방지
    if (_uiState.value is ScanUiState.Scanning) return

    scanJob = viewModelScope.launch {
        _uiState.value = ScanUiState.Scanning(
            progress = 0f,
            currentStep = "Wi-Fi 네트워크 스캔 중...",
        )
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        try {
            runQuickScanUseCase.invoke().collect { report
                _uiState.value = ScanUiState.Success(repor
                _events.emit(ScanUiEvent.NavigateToResult(
            }
        } catch (e: Exception) {
            _uiState.value = ScanUiState.Error(
                code = "E2001",
                message = "스캔 중 오류가 발생했습니다: ${e.me
            )
        }
    }
}

fun cancelScan() {
    scanJob?.cancel()  // 코루틴 취소는 구조화된 동시성으로 전파
    scanJob = null
    _uiState.value = ScanUiState.Idle
}

override fun onCleared() {
```

```
    super.onCleared()
    scanJob?.cancel()  // ViewModel 소멸 시 스캔 자동 취소
}
```

`scanJob?.cancel()`이 호출되면 코루틴 취소 신호가 계층 아래로 전파됩니다. UseCase 내부에서 `collect`를 실행 중이라면 `CancellationException`이 발생하고 코루틴이 즉시 종료됩니다. `try-catch`에서 `CancellationException`을 잡지 않도록 주의해야 합니다 — Kotlin 코루틴 규약상 취소는 예외로 처리하면 안 됩니다.

8.80 15.3 HomeScreen — 레이더 펄스 애니메이션 구현

8.80.1 InfiniteTransition으로 무한 반복 애니메이션

HomeScreen의 Quick Scan 버튼은 레이더에서 전파가 퍼져나가는 것처럼 원형 펄스를 무한 반복합니다. 이 효과는 `rememberInfiniteTransition`으로 구현합니다.

```
// ui/home/HomeScreen.kt

@Composable
```

```
private fun QuickScanButton(onClick: () -> Unit) {
    val infiniteTransition = rememberInfiniteTransition(la

    // 크기: 1배 → 1.4배 (1.5초 주기로 Restart)
    val pulseScale by infiniteTransition.animateFloat(
        initialValue = 1f,
        targetValue = 1.4f,
        animationSpec = infiniteRepeatable(
            animation = tween(durationMillis = 1500),
            repeatMode = RepeatMode.Restart,
        ),
        label = "pulse_scale",
    )

    // 투명도: 0.3 → 0 (크기와 동시에 페이드아웃)
    val pulseAlpha by infiniteTransition.animateFloat(
        initialValue = 0.3f,
        targetValue = 0f,
        animationSpec = infiniteRepeatable(
            animation = tween(durationMillis = 1500),
            repeatMode = RepeatMode.Restart,
        ),
```

```
        label = "pulse_alpha",
    )

    Box(contentAlignment = Alignment.Center) {
        // 펄스 원 — 실제 버튼 뒤에 배치
        Box(
            modifier = Modifier
                .size(120.dp)
                .scale(pulseScale)
                .clip(CircleShape)
                .background(MaterialTheme.colorScheme.prim
        )
        // 메인 버튼 (클릭 가능)
        Button(
            onClick = onClick,
            modifier = Modifier.size(120.dp),
            shape = CircleShape,
            contentPadding = PaddingValues(0.dp),
        ) {
            Column(horizontalAlignment = Alignment.CenterH
                Icon(
                    imageVector = Icons.Default.Wifi,
```

```
                    contentDescription = "Quick Scan",
                    modifier = Modifier.size(36.dp),
                    tint = Color.White,
                )
                Spacer(modifier = Modifier.height(4.dp))
                Text(text = "Quick Scan", fontSize = 12.sp
                Text(text = "30초 점검", fontSize = 10.sp,
            }
        }
    }
}
```

RepeatMode.Restart와 RepeatMode.Reverse의 차이를 이해하는 것이 중요합니다. Restart는 1f에서 1.4f로 커진 뒤 즉시 1f로 리셋하고 다시 시작합니다. Reverse는 1.4f에서 다시 1f로 천천히 줄어듭니다. 레이더 펄스 효과는 Restart여야 자연스럽습니다 — 전파는 퍼졌다 다시 모이지 않고, 새 전파가 새로 시작되는 것처럼 보여야 하니까요.

8.80.2 HomeScreen 전체 구조 — Scaffold와 상태 분리

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(
```

```
    onNavigateToQuickScan: () -> Unit,
    onNavigateToFullScan: () -> Unit,
    // ... 다른 네비게이션 콜백들
    viewModel: HomeViewModel = hiltViewModel(),
) {
    val uiState by viewModel.uiState.collectAsStateWithLif

    // LaunchedEffect: 컴포저블 진입 시 한 번 실행, 이벤트 수집
    LaunchedEffect(viewModel) {
        viewModel.events.collect { event ->
            when (event) {
                is HomeUiEvent.NavigateToQuickScan -> onNa
                is HomeUiEvent.NavigateToFullScan -> onNav
                // ...
            }
        }
    }

    Scaffold(
        topBar = {
            TopAppBar(title = { Text("SearCam", fontWeight
        },
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    ) { paddingValues ->
        // 실제 내용은 별도 컴포저블로 분리 – 테스트 용이성 향상
        HomeContent(
            uiState = uiState,
            onQuickScanClick = viewModel::onQuickScanClick
            // ...
            modifier = Modifier.padding(paddingValues),
        )
    }
}
```

`collectAsStateWithLifecycle()`은 `collectAsState()`의 개선된 버전입니다. 앱이 백그라운드에 있을 때 자동으로 수집을 멈추고, 포그라운드로 돌아오면 다시 시작합니다. 배터리와 리소스를 아끼는 라이프사이클 인식 수집 방식입니다.

`LaunchedEffect(viewModel)`에서 key를 `viewModel`로 지정한 이유도 중요합니다. `viewModel`은 Configuration Change(화면 회전) 시에도 살아남기 때문에, Recomposition 때마다 이벤트 수집을 재시작하지 않습니다. key로 `Unit`을 쓰면 컴포저블이 처음 진입할 때만 실행됩니다.

8.81 15.4 RiskGauge — Canvas 애니메이션으로 그린 위험도 게이지

8.81.1 Canvas는 마우스 없는 그림판

Canvas composable은 Compose에서 픽셀 수준의 자유로운 그림을 그릴 수 있는 공간입니다. Material 컴포넌트로 표현하기 어려운 커스텀 시각화에 사용합니다. RiskGauge는 반원 호(arc)로 0~100 점수를 시각화하는 컴포넌트입니다.

```
// ui/components/RiskGauge.kt


@Composable
fun RiskGauge(
    score: Int,
    riskLevel: RiskLevel,
    modifier: Modifier = Modifier,
    size: Dp = 160.dp,
    animate: Boolean = true,
) {
    // Animatable: 외부에서 값을 바꾸면 지정한 easing으로 부드럽
    val animatedScore = remember { Animatable(0f) }
    LaunchedEffect(score) {
        if (animate) {
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
            animatedScore.animateTo(
                targetValue = score.toFloat(),
                animationSpec = tween(durationMillis = 100
            )
        } else {
            animatedScore.snapTo(score.toFloat())
        }
    }

    val gaugeColor = riskLevelToColor(riskLevel)

    Box(contentAlignment = Alignment.Center, modifier = mo
        Canvas(modifier = Modifier.size(size)) {
            val strokeWidth = size.toPx() * 0.1f
            val radius = (this.size.minDimension - strokeW
            val topLeft = Offset(strokeWidth / 2, strokeWi
            val arcSize = Size(radius * 2, radius * 2)

            // 1. 회색 배경 트랙 — 전체 270도 호
            drawArc(
                color = trackColor,
                startAngle = 135f,    // 왼쪽 아래에서 시작
```

```
            sweepAngle = 270f,     // 오른쪽 아래까지
            useCenter = false,     // 중심점 연결 없이 호만
            topLeft = topLeft,
            size = arcSize,
            style = Stroke(width = strokeWidth, cap =
        )

        // 2. 점수 호 — 그라데이션, 애니메이션 값으로 너비 결
        val sweepAngle = (animatedScore.value / 100f)
        if (sweepAngle > 0f) {
            drawArc(
                brush = Brush.sweepGradient(
                    colorStops = arrayOf(
                        0.0f to Color(0xFF22C55E),  //
                        0.2f to Color(0xFF84CC16),  //
                        0.4f to Color(0xFFEAB308),  //
                        0.6f to Color(0xFFF97316),  //
                        1.0f to Color(0xFFEF4444),  //
                    ),
                    center = center,
                ),
                startAngle = 135f,
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
                    sweepAngle = sweepAngle,
                    // ...
                )
            }
        }

        // 게이지 중앙 — 점수 숫자와 등급 라벨
        Column(horizontalAlignment = Alignment.CenterHoriz
            Text(
                text = animatedScore.value.toInt().toStrin
                fontSize = (size.value * 0.22f).sp,
                fontWeight = FontWeight.Bold,
                color = gaugeColor,
            )
            Text(
                text = riskLevel.labelKo,
                fontSize = (size.value * 0.1f).sp,
                color = gaugeColor,
            )
        }
    }
}
```

8.81.2 각도 계산 — 왜 135도에서 시작하는가

Android Canvas의 각도 기준은 시계 3시 방향(오른쪽)이 0도이고 시계 방향으로 증가합니다. 일반적인 수학 좌표계와 다릅니다.

```
        90°
         |
180° ────┼──── 0°
         |
        270°
```

135도에서 시작하면 왼쪽 아래에서 시작해서 270도를 쓸면 오른쪽 아래에 도달합니다. 이것이 자동차 속도계처럼 왼쪽 아래에서 오른쪽 아래까지 이어지는 반원 호 모양이 되는 이유입니다.

8.81.3 Animatable vs animateFloatAsState

두 애니메이션 API의 차이를 알아야 합니다.

기준	`Animatable`	`animateFloatAsState`
제어	명시적 `animateTo()` 호출	타겟값 변경 시 자동 시작
현재값	`.value`로 접근	`by` 위임으로 접근
취소/대기	직접 제어 가능	자동 관리
적합한 경우	복잡한 시퀀스, 취소 필요	단순 상태 기반 애니메이션

RiskGauge는 `score`가 바뀔 때마다 `LaunchedEffect`에서 `animateTo()`를 호출해야 하므로 `Animatable`이 더 적합합니다. `animateFloatAsState`는 `by` 위임만으로 쓸 수 있어서 더 간단하지만 세밀한 제어가 어렵습니다.

8.82 15.5 Navigation Graph 구성

8.82.1 NavHost로 화면 연결하기

SearCam의 화면 이동은 Jetpack Navigation Compose로 관리합니다. 책의 목차처럼, NavHost가 어떤 화면(목적지)이 있고 어떻게 이동할 수 있는지를 중앙에서 정의합니다.

```
// navigation/SearCamNavGraph.kt

@Composable
fun SearCamNavGraph(
    navController: NavHostController,
    modifier: Modifier = Modifier,
) {
    NavHost(
        navController = navController,
```

```
        startDestination = Screen.Home.route,
        modifier = modifier,
    ) {
        // 홈 화면
        composable(Screen.Home.route) {
            HomeScreen(
                onNavigateToQuickScan = {
                    navController.navigate(Screen.QuickSca
                },
                onNavigateToFullScan = {
                    navController.navigate(Screen.FullScan
                },
                onNavigateToReport = { reportId ->
                    navController.navigate(Screen.Report.c
                },
                // ...
            )
        }

        // 스캔 화면
        composable(Screen.QuickScan.route) {
            ScanScreen(
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
            scanMode = ScanMode.QUICK,
            onNavigateUp = { navController.navigateUp(
            onNavigateToResult = { reportId ->
                // 스캔 화면을 백스택에서 제거하고 결과로 이
                navController.navigate(Screen.Report.c
                    popUpTo(Screen.QuickScan.route) {
                }
            },
        )
    }

    // 리포트 상세 화면 — reportId를 경로 파라미터로 전달
    composable(
        route = Screen.Report.route,
        arguments = listOf(navArgument("reportId") { t
    ) { backStackEntry ->
        val reportId = backStackEntry.arguments?.getSt
        ReportScreen(
            reportId = reportId,
            onNavigateUp = { navController.navigateUp(
        )
    }
```

```
    }
}

// 화면 경로 정의
sealed class Screen(val route: String) {
    data object Home : Screen("home")
    data object QuickScan : Screen("quick_scan")
    data object FullScan : Screen("full_scan")

    // 파라미터가 있는 화면
    data object Report : Screen("report/{reportId}") {
        fun createRoute(reportId: String) = "report/$repor
    }
}
```

popUpTo는 화면 이동 후 백스택을 정리하는 데 쓰입니다. 스캔 화면에서 결과 화면으로 이동할 때 `popUpTo(Screen.QuickScan.route) { inclusive = true }`를 쓰면, 결과 화면에서 뒤로 가면 스캔 화면이 아닌 홈으로 돌아갑니다. 사용자가 결과를 보고 뒤로 갔을 때 다시 스캔이 시작되는 혼란을 방지합니다.

8.83 15.6 hiltViewModel()로 DI 연결

8.83.1 Compose와 Hilt의 만남

Hilt ViewModel과 Compose를 연결하는 방법은 한 줄이면 충분합니다.

```
// gradle 의존성 (libs.versions.toml에 정의)
// androidx.hilt:hilt-navigation-compose:1.2.0

@Composable
fun HomeScreen(
    // ...
    viewModel: HomeViewModel = hiltViewModel(),  // Hilt가
) {
```

`hiltViewModel()`은 `viewModel()` 함수의 Hilt 버전입니다. Navigation의 백스택 항목이나 Activity/Fragment의 생명주기와 연동하여 올바른 범위의 ViewModel 인스턴스를 반환합니다. 동일한 화면(백스택 항목)에서 여러 번 호출해도 같은 인스턴스를 반환합니다.

8.83.2 ViewModel 범위 선택

```
// 화면 수준 ViewModel (기본)
val viewModel: HomeViewModel = hiltViewModel()

// Activity 수준 ViewModel (여러 화면에서 공유)
val sharedViewModel: SharedViewModel = hiltViewModel(
    viewModelStoreOwner = LocalActivity.current
)

// Navigation 그래프 수준 ViewModel
val navBackStackEntry = rememberNavController().getBackSta
val graphViewModel: GraphViewModel = hiltViewModel(navBack
```

SearCam에서는 각 화면이 독립적인 ViewModel을 가집니다. 화면 간 데이터 공유가 필요한 경우 Repository를 통해 공유하고, ViewModel 간 직접 통신은 하지 않습니다.

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.84 15.7 DisposableEffect로 카메라 생명주기 관리

8.84.1 카메라는 열면 반드시 닫아야 한다

카메라는 앱 내에서 독점적으로 사용하는 하드웨어 자원입니다. 다른 앱이 카메라를 쓰려면 SearCam이 먼저 해제해야 합니다. 전화가 와도, 앱이 백그라운드로 가도 카메라를 해제하지 않으면 시스템 전체가 영향을 받습니다.

`DisposableEffect`는 Compose에서 이런 "설정 후 정리가 반드시 필요한 자원"을 관리하는 도구입니다. 마치 `try-finally`처럼, `onDispose` 블록은 컴포저블이 화면에서 사라질 때 반드시 실행됩니다.

```
// ui/lens/LensFinderScreen.kt

@Composable
fun LensFinderScreen(
    onNavigateUp: () -> Unit,
    viewModel: LensFinderViewModel = hiltViewModel(),
) {
    val lifecycleOwner = LocalLifecycleOwner.current
```

```
    // DisposableEffect: 진입 시 카메라 바인딩, 이탈 시 해제
    DisposableEffect(lifecycleOwner) {
        // 효과 시작: 카메라 바인딩
        viewModel.bindCamera(lifecycleOwner)

        onDispose {
            // 정리: 컴포저블이 화면에서 제거될 때 반드시 실행
            viewModel.unbindCamera()
        }
    }

    // CameraX PreviewView를 Compose에 통합
    AndroidView(
        factory = { context ->
            PreviewView(context).also { previewView ->
                viewModel.setPreviewView(previewView)
            }
        },
        modifier = Modifier.fillMaxSize(),
    )
}
```

DisposableEffect의 key(lifecycleOwner)가 바뀌면

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

onDispose를 실행하고 효과를 다시 시작합니다. lifecycleOwner는 Activity 재생성 시 바뀌므로, 화면 회전 같은 상황에서 카메라를 올바르게 재바인딩합니다.

8.84.2 CameraX와 Compose의 통합 패턴

AndroidView는 Compose 트리 안에 기존 View 시스템의 뷰를 포함시키는 브릿지입니다. CameraX의 PreviewView는 전통적인 Android View이므로 AndroidView를 통해 Compose UI에 통합합니다.

```
// ViewModel에서 카메라 바인딩 관리
@HiltViewModel
class LensFinderViewModel @Inject constructor(
    private val cameraProvider: ProcessCameraProvider,
) : ViewModel() {

    private var previewView: PreviewView? = null

    fun setPreviewView(view: PreviewView) {
        previewView = view
    }

    fun bindCamera(lifecycleOwner: LifecycleOwner) {
```

```
        val preview = Preview.Builder().build()
        val cameraSelector = CameraSelector.DEFAULT_BACK_C

        previewView?.let { view ->
            preview.setSurfaceProvider(view.surfaceProvide
            cameraProvider.bindToLifecycle(lifecycleOwner,
        }
    }

    fun unbindCamera() {
        cameraProvider.unbindAll()
    }
}
```

이 패턴의 장점은 카메라 생명주기가 CameraX와 Android 생명주기 모두와 동기화된다는 점입니다. `bindToLifecycle`에 `lifecycleOwner`를 전달하면 CameraX가 Activity/Fragment의 생명주기에 맞게 자동으로 카메라를 시작하고 멈춥니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.85 15.8 스캔 결과 화면 — 상태별 UI 분기

8.85.1 when 표현식으로 완전한 분기

Compose에서 sealed class를 사용하면 when의 완전성 검사(exhaustive check) 혜택을 받습니다. 새 상태를 추가하면 컴파일러가 처리하지 않은 분기를 경고합니다.

```
@Composable
fun ScanScreen(
    onNavigateToResult: (String) -> Unit,
    viewModel: ScanViewModel = hiltViewModel(),
) {
    val uiState by viewModel.uiState.collectAsStateWithLif

    // 일회성 이벤트 처리
    LaunchedEffect(viewModel) {
        viewModel.events.collect { event ->
            when (event) {
                is ScanUiEvent.NavigateToResult -> onNavig
                is ScanUiEvent.ShowSnackbar -> { /* 스낵바
            }
        }
    }
```

```
// 상태별 UI 분기 — 컴파일러가 모든 케이스 처리 강제
when (val state = uiState) {
    is ScanUiState.Idle -> {
        IdleContent(
            onStartQuickScan = viewModel::startQuickSc
        )
    }
    is ScanUiState.Scanning -> {
        ScanningContent(
            progress = state.progress,
            elapsedSeconds = state.elapsedSeconds,
            currentStep = state.currentStep,
            foundDevices = state.foundDevices,
            onCancel = viewModel::cancelScan,
        )
    }
    is ScanUiState.Success -> {
        // 성공 시 이벤트로 이미 이동 중 — 로딩 표시
        Box(modifier = Modifier.fillMaxSize(), content
            CircularProgressIndicator()
        }
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        }
        is ScanUiState.Error -> {
            ErrorContent(
                errorCode = state.code,
                message = state.message,
                onRetry = viewModel::startQuickScan,
            )
        }
    }
}
```

`val state = uiState`로 스마트 캐스트를 활용합니다. `state`는 각 브랜치 내에서 구체적인 타입(`ScanUiState.Scanning` 등)으로 스마트 캐스트되어 `state.progress`처럼 해당 서브클래스의 프로퍼티에 바로 접근할 수 있습니다.

8.86 15.9 Composable 함수 설계 원칙

8.86.1 상태 끌어올리기 (State Hoisting)

Compose에서 상태를 설계할 때 가장 중요한 원칙은 상태 끌어올리기입니다. 상태를 사용하는 가장 낮은 공통 조상(Composable)

으로 상태를 이동시키는 패턴입니다.

```
// 잘못된 예: 상태를 하위 컴포저블 안에 숨김
@Composable
fun BadSearchBar() {
    var query by remember { mutableStateOf("") }
    TextField(value = query, onValueChange = { query = it
}

// 올바른 예: 상태를 상위로 끌어올림
@Composable
fun GoodSearchBar(
    query: String,
    onQueryChange: (String) -> Unit,
) {
    TextField(value = query, onValueChange = onQueryChange
}

// 상위에서 상태 관리
@Composable
fun SearchScreen() {
    var query by remember { mutableStateOf("") }
    GoodSearchBar(query = query, onQueryChange = { query =
```

```
}
```

BadSearchBar는 외부에서 query를 읽거나 초기화할 방법이 없습니다. GoodSearchBar는 상태가 외부에 노출되므로 테스트 시 임의의 query로 렌더링하거나 변경 이벤트를 검증할 수 있습니다. SearCam의 모든 컴포넌트는 상태 끌어올리기 원칙을 따릅니다.

8.86.2 컴포저블 분리 기준

함수가 50줄을 넘기 시작하면 분리를 고려합니다. SearCam의 HomeScreen.kt는 이 기준으로 설계되었습니다.

함수	역할	줄 수
HomeScreen	이벤트 처리, Scaffold 뼈대	~45줄
HomeContent	상태별 분기	~90줄
QuickScanButton	펄스 애니메이션 버튼	~65줄
LastReportCard	최근 리포트 카드	~65줄

HomeContent는 Preview에서 독립적으로 테스트할 수 있습니다. viewModel 의존성 없이 uiState와 콜백만 받으므로 @Preview에서 가짜 데이터로 모든 상태를 확인할 수 있습니다.

8.87 실습

실습 15-1: `RiskGauge`의 `RepeatMode.Restart`를 `RepeatMode.Reverse`로 바꿔보세요. 펄스 애니메이션이 어떻게 달라지는지 확인하고, 어떤 방식이 레이더 효과에 더 적합한지 생각해보세요.

실습 15-2: `ScanUiState`에 새 상태 `Paused(val reason: String)`를 추가해보세요. Kotlin 컴파일러가 처리하지 않은 `when` 브랜치를 어디서 경고하는지 확인하세요.

실습 15-3: `HomeContent`를 `@Preview`로 미리보기해보세요. `HomeUiState.Ready`, `HomeUiState.Loading`, `HomeUiState.Error` 세 가지 상태를 각각 Preview로 만들어보세요.

8.88 핵심 정리

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

개념	핵심
sealed class UiState	모든 UI 상태를 타입 안전하게 표현, 컴파일러가 분기 누락 검사
StateFlow	현재 상태 보관, 새 구독자도 즉시 최신값 수신
SharedFlow	일회성 이벤트 (화면 이동, 스낵바), 놓친 이벤트 재전달 없음
InfiniteTransition	무한 반복 애니메이션, `Restart`/`Reverse` 모드 선택
Animatable	값 변화 시 부드러운 전환, 명시적 `animateTo()` 제어
DisposableEffect	자원 획득/해제 쌍 보장, `onDispose`는 반드시 실행
hiltViewModel()	Compose-Hilt 연결, Navigation 범위에 맞는 인스턴스 반환
상태 끌어올리기	상태를 상위로, 이벤트는 하위로 — 테스트 가능성과 재사용성 향상

- Compose의 재구성(Recomposition)은 상태를 읽는 함수만

선택적으로 실행한다

- by viewModel.uiState.collectAsStateWithLifecycle()은 배터리를 아끼는 생명주기 인식 구독이다
- Canvas의 각도 기준은 3시 방향 0도, 시계 방향으로 증가함을 기억하라
- DisposableEffect의 onDispose는 예외가 발생해도 반드시 실행된다

8.89 15.9 코드 리뷰 개선 사항 — LifecycleOwner 전파 아키텍처

8.89.1 문제: 카메라 바인딩에 LifecycleOwner가 필요한데 어디서 제공해야 하나

CameraX는 ProcessCameraProvider.bindToLifecycle(lifecycleOw...) 호출 시 LifecycleOwner를 요구합니다. 초기 구현에서는 Repository 레이어에서 Application Context로 임시 처리하거나, ViewModel에서 Context를 직접 참조하는 패턴을 사용했습니다. 이는 메모리 누수와 테스트 어려움이라는 두 가지 문제를 만들었습니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.89.2 해결: LifecycleOwner를 Compose Screen에서 UseCase까지 파라미터로 전달

```
[LensFinderScreen / IrCameraScreen]  ← LocalLifecycleOwner
    ↓ 파라미터 전달
[LensViewModel.startLensDetection(lifecycleOwner)]
        ↓ 파라미터 전달
[LensDetectionRepository.startDetection(lifecycleOwner)]
        ↓ 파라미터 전달
[LensDetector.startDetection(lifecycleOwner)]  ← CameraX 바
인딩
```

Compose Screen에서 `LocalLifecycleOwner.current`로 현재 LifecycleOwner를 얻어 아래로 전달합니다.

```
// ui/lens/LensFinderScreen.kt — LocalLifecycleOwner.curre
@Composable
fun LensFinderScreen(
    onNavigateUp: () -> Unit,
    viewModel: LensViewModel = hiltViewModel(),
) {
    val lifecycleOwner = androidx.lifecycle.compose.LocalL

    // DisposableEffect(lifecycleOwner): lifecycleOwner가 
```

```
    DisposableEffect(lifecycleOwner) {
        viewModel.startLensDetection(lifecycleOwner)
        onDispose {
            viewModel.stopLensDetection()
        }
    }
    // ...
}

// ui/lens/LensViewModel.kt — Screen에서 받아 Repository로 전
@HiltViewModel
class LensViewModel @Inject constructor(
    private val lensDetectionRepository: LensDetectionRepo
    private val irDetectionRepository: IrDetectionReposito
) : ViewModel() {

    fun startLensDetection(lifecycleOwner: LifecycleOwner)
        viewModelScope.launch {
            _lensUiState.value = LensUiState.Starting
            val startResult = lensDetectionRepository.star
            if (startResult.isFailure) {
                _lensUiState.value = LensUiState.Error(
                    code = "E1001",
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
                    message = "카메라를 시작할 수 없습니다: ${
                )
                return@launch
            }
            lensDetectionRepository.observeRetroreflection
                .catch { e ->
                    _lensUiState.value = LensUiState.Error
                }
                .collect { points ->
                    _lensUiState.value = LensUiState.Detec
                }
        }
    }

    // onCleared(): viewModelScope 취소 시점에 동기 해제 보장
    override fun onCleared() {
        super.onCleared()
        runBlocking {
            withTimeout(1_000L) {
                lensDetectionRepository.stopDetection()
                irDetectionRepository.stopDetection()
            }
```

```
        }
    }
}

// domain/repository/LensDetectionRepository.kt — 인터페이스
interface LensDetectionRepository {
    suspend fun startDetection(lifecycleOwner: LifecycleOw
    fun observeRetroreflections(): Flow<List<Retroreflecti
    suspend fun stopDetection()
}
```

8.89.3 왜 LifecycleOwner를 Repository 인터페이스에 두는가

이상적으로는 도메인 레이어 인터페이스에 Android 의존성(LifecycleOwner)이 없어야 합니다. 하지만 LifecycleOwner는 사실 Android Framework 클래스가 아니라 androidx.lifecycle 인터페이스입니다. 아키텍처 결정의 실용적 트레이드오프입니다.

방법	장점	단점
Repository에 `LifecycleOwner` 전달	구현 간단, CameraX 직접 연동	도메인 레이어에 Android 의존성
ApplicationContext로 LifecycleOwner 우회	도메인 순수 유지	메모리 누수, 생명주기 연동 어려움
ProcessCameraProvider를 Hilt 싱글턴으로	일관성	생명주기 관리 복잡성 증가

SearCam은 실용성을 택했습니다. `LifecycleOwner`는 `LifecycleOwner`을 노출하는 단순 인터페이스이며, 테스트에서 `TestLifecycleOwner`로 쉽게 대체할 수 있습니다.

8.90 다음 장 예고

화면이 완성되었으니 이제 데이터를 영구적으로 저장할 차례입니다. Ch16에서는 Room DB로 스캔 이력을 저장하고, SQLCipher로 암호화하는 방법을 구현합니다.

—

참

고

문

서:

docs/

09-

ui-

ux-

spec.md,

docs/

04-

system-

architecture.md,

docs/

14-

security-

design.md

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

Ch16: Room DB 구현 — 스캔 이력 영구 저장

—

>

이

장

에

서

배

울

것:

SearCam

이

스

캔

이

력

을

앱

재

시

작

후

에

도

보

존

하

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.91 도입

은행 금고를 생각해보세요. 금고는 세 가지 조건을 충족해야 합니다. 첫째, 물건을 넣고 뺄 수 있어야 합니다(CRUD). 둘째, 목록을 빠르게 조회할 수 있어야 합니다(인덱스). 셋째, 열쇠 없이는 아무것도 꺼낼 수 없어야 합니다(암호화).

SearCam의 스캔 이력 저장소가 바로 이 금고입니다. 사용자가 스캔할 때마다 결과가 기록되고, 다음에 앱을 켜도 이전 스캔 내역이 그대로 남아 있어야 합니다. 몰카 탐지 증거는 민감한 정보입니다 — 기기를 도난당하거나 포렌식 분석 대상이 되더라도 내용을 알 수 없어야 합니다.

이 장에서는 Room DB의 계층 구조를 처음부터 설계하고, 암호화까지 적용하는 전 과정을 따라갑니다.

8.92 16.1 Room 아키텍처 — 세 계층의 역할

8.92.1 추상화의 사다리

SQLite를 직접 사용하면 SQL 문자열을 하드코딩해야 합니다. 오타가 나도 컴파일 시점에 발견되지 않고 런타임에 앱이 크래시합니다. Room은 어노테이션 기반으로 SQL을 컴파일 시점에 검증합니다.

Room의 세 계층은 각자 명확한 책임을 가집니다.

```
[Domain Model]        → 순수 비즈니스 데이터 (Android 의존성 없음)
        ↕ 변환 (Mapper)
[Entity]              → SQLite 테이블 정의 (@Entity, @ColumnInfo)
        ↕ 접근
[DAO]                 → SQL 쿼리 정의 (@Query, @Insert, @Delete)
        ↕ 위임
[Repository 구현체]   → Domain Model ↔ Entity 변환 + DAO 호출
```

이 계층 분리가 중요한 이유: Domain Model은 데이터베이스가 어떻게 생겼는지 몰라야 합니다. 나중에 SQLite에서 DataStore로 바꾸거나 서버 API로 교체해도 Domain Model 코드를 전혀 건드리지 않습니다.

8.93 16.2 Entity 설계 — 테이블 구조 정의

8.93.1 ScanReportEntity — 복잡한 객체를 테이블에 담기

스캔 결과(`ScanReport`)는 중첩 구조를 가집니다. `devices` 필드 하나에 여러 `NetworkDevice` 객체가 들어 있고, 각 `NetworkDevice`에는 또 개방 포트 목록이 있습니다. 이런 복잡한 중첩 구조를

SQLite 테이블에 어떻게 담을까요?

전략은 두 가지입니다.

전략 A: 관계형 테이블 분리 — `NetworkDevice`를 별도 테이블로 만들고 외래키로 연결. 정규화 수준은 높지만 JOIN 쿼리가 복잡해집니다.

전략 B: JSON 직렬화 — 중첩 리스트를 JSON 문자열로 직렬화해서 TEXT 컬럼 하나에 저장. 쿼리가 단순하지만 JSON 파싱 비용이 있습니다.

SearCam은 스캔 이력을 목록으로 보여주거나 특정 ID로 조회하는 패턴이 대부분입니다. 기기 정보를 독립적으로 필터링하는 복잡한 쿼리가 드물므로 전략 B를 선택했습니다.

```
// data/local/entity/ScanReportEntity.kt

@Entity(tableName = "scan_reports")
data class ScanReportEntity(
    @PrimaryKey
    @ColumnInfo(name = "id")
    val id: String,                    // UUID

    @ColumnInfo(name = "mode")
    val mode: String,                  // enum → String (Typ
```

```
    @ColumnInfo(name = "started_at")
    val startedAt: Long,            // Unix epoch millis

    @ColumnInfo(name = "completed_at")
    val completedAt: Long,

    @ColumnInfo(name = "risk_score")
    val riskScore: Int,             // 0~100

    @ColumnInfo(name = "risk_level")
    val riskLevel: String,          // enum → String (Typ

    /** 기기 목록 — JSON 배열로 직렬화된 List<NetworkDevice> *
    @ColumnInfo(name = "devices_json")
    val devicesJson: String = "[]",

    /** 발견 사항 목록 — JSON 배열로 직렬화된 List<Finding> */
    @ColumnInfo(name = "findings_json")
    val findingsJson: String = "[]",

    @ColumnInfo(name = "location_note")
```

```
    val locationNote: String = "",

    /** 생성 시각 — 정렬 기준 */
    @ColumnInfo(name = "created_at")
    val createdAt: Long = System.currentTimeMillis()
)
```

`@ColumnInfo(name = ...)`로 컬럼명을 명시하는 이유가 있습니다. Kotlin 프로퍼티 이름이 `riskScore`이지만 DB 컬럼명은 `risk_score`입니다. 스네이크 케이스 컬럼명은 SQL 관행이고, 나중에 프로퍼티 이름을 리팩토링해도 DB 컬럼명이 바뀌지 않습니다.

8.93.2 인덱스 추가 — 조회 성능 최적화

리포트 목록을 최신순으로 조회하는 쿼리가 자주 실행됩니다. `started_at` 컬럼에 인덱스를 추가하면 정렬 성능이 크게 개선됩니다.

```
@Entity(
    tableName = "scan_reports",
    indices = [
        Index(value = ["started_at"]),          // 정렬 최
        Index(value = ["risk_level"]),          // 위험도
    ]
```

```
)

data class ScanReportEntity( /* ... */ )
```

인덱스는 읽기 성능을 높이지만 쓰기 성능을 약간 낮춥니다. SearCam은 읽기(목록 조회)가 쓰기(스캔 저장)보다 훨씬 빈번하므로 인덱스가 적합합니다.

8.94 16.3 TypeConverters — Gson 없이 직접 변환

8.94.1 Room이 저장할 수 없는 타입들

Room은 기본 타입(Int, Long, String, Boolean)과 byte array만 직접 저장할 수 있습니다. `RiskLevel` enum, `List<String>` 같은 타입은 `@TypeConverter`로 변환해야 합니다.

외부 라이브러리(Gson, Moshi) 없이 직접 TypeConverter를 구현하면 의존성이 줄고 변환 로직을 완전히 제어할 수 있습니다. SearCam의 `SearCamTypeConverters`가 이 역할을 합니다.

8.94.2 List ↔ JSON 변환

```
// data/local/converter/TypeConverters.kt

class SearCamTypeConverters {

    /**
     * List<String>을 JSON 배열 문자열로 직렬화한다.
     *
     * 예: ["서울","부산"] → `["서울","부산"]`
     */
    @TypeConverter
    fun fromStringList(value: List<String>?): String {
        if (value.isNullOrEmpty()) return "[]"
        return value.joinToString(
            separator = ",",
            prefix = "[",
            postfix = "]",
            // 큰따옴표 안의 큰따옴표는 이스케이프 처리
            transform = { item -> "\"${item.replace("\"",
        )
    }
```

```
    /**
     * JSON 배열 문자열을 List<String>으로 역직렬화한다.
     *
     * 파싱 실패 시 빈 리스트를 반환하고 오류를 로깅한다 (앱 크래시
     */
    @TypeConverter
    fun toStringList(value: String?): List<String> {
        if (value.isNullOrBlank() || value.trim() == "[]")

        return try {
            val trimmed = value.trim().removePrefix("[").r
            if (trimmed.isBlank()) return emptyList()
            parseStringArray(trimmed)
        } catch (e: Exception) {
            Timber.e(e, "TypeConverter: List<String> 파싱 실
            emptyList()  // 파싱 실패 시 앱 크래시 대신 빈 리스트
        }
    }
```

오류 처리가 중요합니다. TypeConverter에서 예외가 발생하면 Room이 DB 읽기 자체를 실패로 처리합니다. 결과적으로 스캔 목록 전체가 보이지 않는 치명적인 버그로 이어집니다. 파싱 실패 시 `emptyList()`를 반환하고 로그만 남기는 방어적 처리가 중요합

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

니다.

8.94.3 문자열 배열 파서 — 이스케이프 처리

Gson이나 Moshi를 쓰지 않고 직접 JSON 배열을 파싱할 때 이스케이프된 큰따옴표 처리가 까다롭습니다. SearCam의 `parseStringArray`는 이 케이스를 정확히 처리합니다.

```
private fun parseStringArray(input: String): List<String>
    val result = mutableListOf<String>()
    var i = 0

    while (i < input.length) {
        if (input[i] == '"') {
            val sb = StringBuilder()
            i++  // 여는 따옴표 건너뜀
            while (i < input.length && input[i] != '"') {
                if (input[i] == '\\' && i + 1 < input.leng
                    i++  // 백슬래시 건너뜀
                    sb.append(input[i])  // 이스케이프된 문자
                } else {
                    sb.append(input[i])
                }
                i++
```

```
            }
            result.add(sb.toString())
            i++  // 닫는 따옴표 건너뜀
        } else {
            i++
        }
    }

    return result
}
```

이 파서는 `["hello","world with \"quotes\""]` 같은 입력을 올바르게 처리합니다. `\"quotes\"`의 역슬래시를 보면 다음 문자를 그대로 포함합니다.

8.94.4 enum TypeConverter — 안전한 역직렬화

```
@TypeConverter
fun fromRiskLevel(value: RiskLevel?): String {
    return value?.name ?: RiskLevel.SAFE.name  // null 시
}

@TypeConverter
fun toRiskLevel(value: String?): RiskLevel {
```

```
    if (value.isNullOrBlank()) return RiskLevel.SAFE

    return try {
        RiskLevel.valueOf(value)
    } catch (e: IllegalArgumentException) {
        // DB에 저장된 값이 현재 enum에 없을 때 (버전 업그레이드 시
        Timber.e(e, "알 수 없는 RiskLevel: $value, 기본값 SAF
        RiskLevel.SAFE
    }
}
```

RiskLevel.valueOf()가 던지는 IllegalArgumentException을 잡는 이유: 앱 업데이트로 enum 값이 바뀌면 이전에 저장된 문자열이 현재 enum에 없을 수 있습니다. 예외 대신 기본값을 반환해서 앱이 안전하게 동작하도록 합니다.

8.94.5 AppDatabase에 TypeConverter 등록

```
// data/local/AppDatabase.kt

@Database(
    entities = [
        ScanReportEntity::class,
        DeviceEntity::class,
```

```
        ChecklistEntity::class,
        RiskPointEntity::class,
    ],
    version = 1,
    exportSchema = true,  // schemas/ 디렉토리에 JSON 스키마
)
@TypeConverters(SearCamTypeConverters::class)  // 전체 DB에
abstract class AppDatabase : RoomDatabase() {
    abstract fun reportDao(): ReportDao
    abstract fun deviceDao(): DeviceDao
    abstract fun checklistDao(): ChecklistDao
}
```

`exportSchema = true`로 설정하면 빌드 시 `schemas/` 디렉토리에 `{version}.json` 파일이 생성됩니다. 이 파일을 Git에 커밋하면 스키마 변경 이력을 추적할 수 있고, Migration 테스트에서도 활용합니다.

8.95 16.4 DAO — SQL을 코드로

8.95.1 ReportDao — Flow로 실시간 업데이트

```
// data/local/dao/ReportDao.kt

@Dao
interface ReportDao {

    /**
     * 새 리포트를 저장하거나 기존 리포트를 덮어쓴다.
     *
     * REPLACE 전략: 동일 PK가 있으면 삭제 후 재삽입.
     * 중복 스캔 결과가 들어와도 안전합니다.
     */
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(report: ScanReportEntity)

    /**
     * 모든 리포트를 최신순으로 반환한다.
     *
     * Flow<List<...>>: DB가 바뀔 때마다 자동으로 새 목록을 emit
     * Room의 InvalidationTracker가 테이블 변경을 감지합니다.
```

```
     */
    @Query("SELECT * FROM scan_reports ORDER BY started_at
    fun observeAll(): Flow<List<ScanReportEntity>>

    /**
     * 특정 ID의 리포트를 한 번 조회한다.
     *
     * suspend fun: 코루틴에서 호출, IO 스레드에서 실행됩니다.
     * Flow가 아닌 단일 조회이므로 suspend fun을 씁니다.
     */
    @Query("SELECT * FROM scan_reports WHERE id = :id LIMI
    suspend fun findById(id: String): ScanReportEntity?

    @Delete
    suspend fun delete(report: ScanReportEntity)

    @Query("DELETE FROM scan_reports")
    suspend fun deleteAll()

    @Query("SELECT COUNT(*) FROM scan_reports")
    suspend fun count(): Int
}
```

observeAll()이 반환하는 Flow<List<ScanReportEntity>>는 Room의 핵심 기능입니다. 새 리포트가 저장되거나 삭제되면 Flow가 자동으로 새 목록을 emit합니다. UI에서 이 Flow를 collectAsStateWithLifecycle()로 구독하면 DB 변경이 화면에 즉시 반영됩니다.

```
// ViewModel에서 Flow를 상태로 변환
val reports: StateFlow<List<ScanReport>> = reportRepositor
    .observeAll()
    .map { entities -> entities.map(mapper::toDomain) }  /
    .stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = emptyList(),
    )
```

stateIn의 SharingStarted.WhileSubscribed(5_000) 설정: 구독자가 없어진 후 5초간 Flow 수집을 유지합니다. 화면 회전 시 (약 2초) ViewModel이 재구성되는 동안 Flow가 재시작되지 않아 DB 재쿼리를 방지합니다.

8.96 16.5 Repository 구현체 — Domain ↔ Data 변환

8.96.1 Mapper 패턴 — 계층 간 변환 책임 분리

```
// data/repository/ReportRepositoryImpl.kt

class ReportRepositoryImpl @Inject constructor(
    private val reportDao: ReportDao,
    private val mapper: ScanReportMapper,
    private val pdfGenerator: PdfGenerator,
    @ApplicationContext private val context: Context,
) : ReportRepository {

    override fun observeReports(): Flow<List<ScanReport>>
        return reportDao.observeAll()
            .map { entities ->
                entities.map(mapper::toDomain)
            }
            .catch { e ->
                Timber.e(e, "리포트 목록 조회 실패")
                emit(emptyList())  // 오류 시 빈 목록으로 대체
            }
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    }

    override suspend fun saveReport(report: ScanReport): R
        return try {
            val entity = mapper.toEntity(report)
            reportDao.insert(entity)
            Result.success(Unit)
        } catch (e: Exception) {
            Timber.e(e, "리포트 저장 실패 — id: ${report.id}'
            Result.failure(e)
        }
    }

    override suspend fun getReport(reportId: String): Resu
        return try {
            val entity = reportDao.findById(reportId)
                ?: return Result.failure(
                    NoSuchElementException("리포트를 찾을 수
                )
            Result.success(mapper.toDomain(entity))
        } catch (e: Exception) {
            Timber.e(e, "리포트 조회 실패 — id: $reportId")
```

```
            Result.failure(e)
        }
    }

    override suspend fun exportToPdf(report: ScanReport, o
        val fullPath = buildOutputPath(outputPath)
        return pdfGenerator.generate(report, fullPath)
    }

    private fun buildOutputPath(fileName: String): String
        val dir = context.getExternalFilesDir(Environment.
            ?: context.filesDir
        return "${dir.absolutePath}/$fileName"
    }
}
```

`Result<T>`를 반환 타입으로 사용하는 패턴에 주목하세요. `try-catch`로 예외를 잡아 `Result.failure()`로 감싸면, 호출 측에서 예외 처리를 잊어도 컴파일러가 경고합니다. ViewModel에서 `result.onSuccess { }.onFailure { }`로 명시적으로 처리해야 합니다.

8.96.2 Mapper 클래스 구현

```
// data/mapper/ScanReportMapper.kt

class ScanReportMapper @Inject constructor(
    private val gson: Gson,  // 또는 직접 구현한 직렬화기
) {

    fun toDomain(entity: ScanReportEntity): ScanReport {
        return ScanReport(
            id = entity.id,
            mode = ScanMode.valueOf(entity.mode),
            startedAt = entity.startedAt,
            completedAt = entity.completedAt,
            riskScore = entity.riskScore,
            riskLevel = RiskLevel.valueOf(entity.riskLevel
            devices = parseDevices(entity.devicesJson),
            findings = parseFindings(entity.findingsJson),
            locationNote = entity.locationNote,
        )
    }

    fun toEntity(domain: ScanReport): ScanReportEntity {
```

```
        return ScanReportEntity(
            id = domain.id,
            mode = domain.mode.name,
            startedAt = domain.startedAt,
            completedAt = domain.completedAt,
            riskScore = domain.riskScore,
            riskLevel = domain.riskLevel.name,
            devicesJson = serializeDevices(domain.devices)
            findingsJson = serializeFindings(domain.findir
            locationNote = domain.locationNote,
        )
    }

    // JSON 변환은 예외 처리와 함께
    private fun parseDevices(json: String): List<NetworkDe
        return try {
            gson.fromJson(json, Array<NetworkDeviceDto>::c
                ?.map { it.toDomain() }
                ?: emptyList()
        } catch (e: Exception) {
            Timber.e(e, "기기 목록 파싱 실패")
            emptyList()
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        }
    }
}
```

Mapper는 Entity와 Domain Model 사이의 통역사입니다. 이 클래스가 변환 책임을 단독으로 가지므로, 나중에 Entity 구조가 바뀌어도 Mapper만 수정하면 Domain Model은 그대로입니다.

8.97 16.6 SQLCipher 통합 — 데이터베이스 암호화

8.97.1 왜 암호화가 필요한가

Android 기기의 내부 저장소는 기본적으로 암호화되어 있습니다(Full-Disk Encryption 또는 File-Based Encryption). 그런데 루팅된 기기나 포렌식 도구를 사용하면 `/data/data/com.searcam/databases/` 파일에 직접 접근할 수 있습니다. SQLite 파일은 평문이므로 DB Browser for SQLite 같은 도구로 내용을 바로 볼 수 있습니다.

SQLCipher는 SQLite를 AES-256으로 암호화합니다. 파일을 복사해도 키 없이는 내용을 읽을 수 없습니다.

8.97.2 Android Keystore + SQLCipher 통합

암호화 키를 앱 내부에 하드코딩하면 리버스 엔지니어링으로 키를 추출할 수 있습니다. Android Keystore System은 하드웨어 보안 모듈(HSM) 또는 TEE(Trusted Execution Environment)에 키를 저장해서 앱 프로세스 외부에서는 접근 불가능합니다.

```
// di/DatabaseModule.kt


@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    private const val DB_NAME = "searcam_db"
    private const val KEY_ALIAS = "searcam_db_key"

    @Provides
    @Singleton
    fun provideAppDatabase(
        @ApplicationContext context: Context,
    ): AppDatabase {
        val passphrase = getOrCreateDatabaseKey()
        val factory = SupportFactory(passphrase)
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        return Room.databaseBuilder(context, AppDatabase::
            .openHelperFactory(factory)  // SQLCipher 팩토
            .fallbackToDestructiveMigration()  // 개발 중만
            .build()
    }

    /**
     * Android Keystore에서 DB 암호화 키를 가져오거나 새로 생성
     *
     * 키는 앱 설치 후 최초 1회 생성되며, 앱 제거 시 함께 삭제된다.
     * 기기의 보안 하드웨어(TEE/StrongBox)에 저장된다.
     */
    private fun getOrCreateDatabaseKey(): ByteArray {
        val keyStore = KeyStore.getInstance("AndroidKeySto

        // 이미 키가 있으면 재사용
        if (!keyStore.containsAlias(KEY_ALIAS)) {
            createDatabaseKey()
        }

        // AES 키로 랜덤 패스프레이즈 생성 후 Keystore 키로 암호
        return loadEncryptedPassphrase()
```

```
    }

    private fun createDatabaseKey() {
        val keyGenerator = KeyGenerator.getInstance(
            KeyProperties.KEY_ALGORITHM_AES,
            "AndroidKeyStore"
        )
        val keySpec = KeyGenParameterSpec.Builder(
            KEY_ALIAS,
            KeyProperties.PURPOSE_ENCRYPT or KeyProperties
        ).apply {
            setBlockModes(KeyProperties.BLOCK_MODE_GCM)
            setEncryptionPaddings(KeyProperties.ENCRYPTION
            setKeySize(256)
            // 생체인증/PIN 인증 없이 접근 가능 (앱 자체 접근용)
            setUserAuthenticationRequired(false)
        }.build()

        keyGenerator.init(keySpec)
        keyGenerator.generateKey()
    }
```

```
    /**
     * 암호화된 패스프레이즈를 복호화하여 반환한다.
     *
     * 처음 호출 시: 랜덤 32바이트 생성 → Keystore 키로 암호화 —
     * 이후 호출 시: EncryptedSharedPreferences에서 읽어 복호화
     */
    private fun loadEncryptedPassphrase(): ByteArray {
        // EncryptedSharedPreferences는 Jetpack Security 라
        // 저장 자체도 암호화되어 이중 보호
        // 구현 생략 — Jetpack Security 라이브러리 활용
        TODO("구현체에서 처리")
    }
}
```

8.97.3 SQLCipher 의존성 추가

```
# gradle/libs.versions.toml

[versions]
sqlcipher = "4.5.6"

[libraries]
sqlcipher-android = { group = "net.zetetic", name = "andro
```

```
// app/build.gradle.kts
dependencies {
    implementation(libs.sqlcipher.android)
    // SQLCipher는 Room과 함께 쓸 때 SupportFactory를 통해 통합
}
```

`SupportFactory(passphrase)`는 Room의 `openHelperFactory`에 전달하는 SQLCipher 팩토리입니다. Room은 내부적으로 `SupportSQLiteOpenHelper`를 사용하는데, SQLCipher가 이 인터페이스를 구현하여 암호화된 SQLite를 제공합니다. Room 코드 변경 없이 암호화가 적용되는 아름다운 설계입니다.

8.98 16.7 Migration 전략 — 데이터를 잃지 않고 스키마 바꾸기

8.98.1 fallbackToDestructiveMigration은 개발 중에만

`fallbackToDestructiveMigration()`은 스키마 버전이 맞지 않으면 DB를 통째로 삭제하고 새로 만듭니다. 개발 중에는 편리하지만 프로덕션에서는 사용자 데이터를 날립니다. 절대 릴리즈 빌드에 포함하면 안 됩니다.

```
// 개발용 — 절대 프로덕션 사용 금지
Room.databaseBuilder(context, AppDatabase::class.java, DB_
    .fallbackToDestructiveMigration()
    .build()

// 프로덕션용 — Migration 객체를 명시적으로 추가
Room.databaseBuilder(context, AppDatabase::class.java, DB_
    .addMigrations(MIGRATION_1_2, MIGRATION_2_3)
    .build()
```

8.98.2 Migration 작성 방법

스키마를 변경할 때의 절차:

1. `AppDatabase`의 `version`을 올린다 (예: 1 → 2)
2. `Migration(from, to)` 객체를 작성한다
3. `addMigrations()`에 등록한다

```
// Migration 1 → 2: location_note 컬럼 추가
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(database: SupportSQLiteDatabase)
        // SQLite는 컬럼 추가만 가능 (삭제/변경은 테이블 재생성 필
        database.execSQL(
            "ALTER TABLE scan_reports ADD COLUMN location_
```

```
        )
    }
}

// Migration 2 → 3: 새 인덱스 추가
val MIGRATION_2_3 = object : Migration(2, 3) {
    override fun migrate(database: SupportSQLiteDatabase)
        database.execSQL(
            "CREATE INDEX IF NOT EXISTS index_scan_reports
            "ON scan_reports (risk_level)"
        )
    }
}
```

8.98.3 MigrationTestHelper로 Migration 검증

Migration은 반드시 테스트해야 합니다. 프로덕션에서 Migration 실패는 사용자 데이터 손실로 이어집니다.

```
// test/MigrationTest.kt

@RunWith(AndroidJUnit4::class)
class MigrationTest {
    @get:Rule
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    val helper: MigrationTestHelper = MigrationTestHelper(
        InstrumentationRegistry.getInstrumentation(),
        AppDatabase::class.java,
    )

    @Test
    fun migrate1To2() {
        // 버전 1로 DB 생성
        helper.createDatabase(TEST_DB, 1).apply {
            execSQL("INSERT INTO scan_reports (id, mode, s
            close()
        }

        // 버전 2로 Migration
        val db = helper.runMigrationsAndValidate(TEST_DB,

        // location_note 컬럼이 추가되었는지 확인
        val cursor = db.query("SELECT location_note FROM s
        assertTrue(cursor.moveToFirst())
        assertEquals("", cursor.getString(0))
        cursor.close()
    }
```

```
}
```

8.99 16.8 DatabaseModule과 Hilt 등록

8.99.1 싱글톤으로 DB 인스턴스 관리

Room DB는 앱 전체에서 하나의 인스턴스만 사용해야 합니다. 여러 인스턴스를 만들면 데이터 일관성 문제와 성능 저하가 발생합니다.

```
// di/DatabaseModule.kt


@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {


    @Provides
    @Singleton
    fun provideAppDatabase(@ApplicationContext context: Co
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "searcam_db",
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        )
        // 프로덕션: .addMigrations(MIGRATION_1_2) 추가
        .fallbackToDestructiveMigration()
        .build()
    }

    // DAO는 DB 인스턴스에서 생성 — DB와 같은 싱글톤 범위
    @Provides
    @Singleton
    fun provideReportDao(db: AppDatabase): ReportDao = db.

    @Provides
    @Singleton
    fun provideDeviceDao(db: AppDatabase): DeviceDao = db.

    @Provides
    @Singleton
    fun provideChecklistDao(db: AppDatabase): ChecklistDao
}
```

DAO를 별도로 `@Provides`로 등록하는 이유: `ReportRepositoryImpl`이 `ReportDao`를 주입받을 때, `AppDatabase` 전체를 주입받을 필요가 없습니다. 필요한 DAO만 주입받으므로 의존성

이 명확하고 테스트 시 DAO만 Mock 처리할 수 있습니다.

8.100 실습

실습 16-1: `ScanReportEntity`에 `duration_ms` 컬럼을 추가하는 Migration을 작성해보세요. `MigrationTestHelper`로 이전 버전 데이터가 마이그레이션 후에도 보존되는지 확인하세요.

실습 16-2: `SearCamTypeConverters`의 `toStringList()`에 `["hello","world with \"quotes\"","test"]` 입력을 테스트해보세요. 이스케이프된 따옴표가 올바르게 처리되는지 단위 테스트로 검증하세요.

실습 16-3: Room의 `exportSchema = true` 설정으로 생성된 `schemas/1.json` 파일을 열어보세요. 각 테이블의 컬럼 정의와 인덱스가 어떻게 기록되는지 확인하세요.

8.101 핵심 정리

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

개념	핵심
Entity	SQLite 테이블 정의, `@ColumnInfo`로 컬럼명과 프로퍼티명 분리
TypeConverter	Room이 지원하지 않는 타입 변환, 오류 시 기본값 반환으로 크래시 방지
DAO Flow	`Flow<List<T>>` 반환 시 DB 변경 자동 감지 및 emit
Mapper	Entity ↔ Domain 변환 책임 분리, 계층 간 의존성 차단
SQLCipher	AES-256으로 DB 파일 암호화, Room에 `SupportFactory`로 통합
Android Keystore	암호화 키를 하드웨어 보안 영역에 저장, 앱 프로세스 외부 접근 불가
Migration	`version` 증가 + `Migration(from, to)` 객체 필수, `MigrationTestHelper`로 검증

개념	핵심
exportSchema	스키마 JSON을 Git으로 관리, Migration 이력 추적

- `fallbackToDestructiveMigration()`은 개발 빌드에만, 프로덕션은 반드시 Migration 필요
- TypeConverter 파싱 오류는 예외 대신 기본값으로 처리해야 앱이 안전하다
- Room의 Flow는 테이블 변경을 자동 감지한다 — 별도 새로고침 로직 불필요
- SQLCipher 패스프레이즈는 Android Keystore에 보관해야 역공학으로 추출 불가능하다

8.102 다음 장 예고

데이터가 영구 저장되었으니 이제 이 데이터를 PDF로 만들 차례입니다. Ch17에서는 Android 내장 `PdfDocument` API로 증거 문서를 생성하고, 경로 순회 공격을 방어하고, `FileProvider`로 다른 앱과 안전하게 파일을 공유하는 방법을 구현합니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

참

고

문

서:

docs/

07-

db-

schema.md,

docs/

14-

security-

design.md,

docs/

08-

error-

handling.md

—

#

Ch17:

PDF

리

포

트

생

성

—

증

거

를

문

서

로

CHAPTER 8 | 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—>

이장에서배울것: 스캔 결과를 법적 증거로 활용할 수 있는 PDF

8.103 도입

병원에서 검사를 받으면 결과지가 나옵니다. 의사의 서명, 날짜, 측정값, 해석까지 담긴 공식 문서입니다. 구두로 "혈당이 좀 높아요"라고 말하는 것과 "2026년 4월 4일 공복 혈당 126mg/dL" 이 적힌 문서는 법적 효력이 다릅니다.

SearCam의 PDF 리포트도 같은 목적입니다. 탐지 앱이 "위험합니다" 라고 말하는 것과, 날짜, 장소, 발견된 기기 목록, 위험도 점수가 담긴 PDF 문서를 경찰에 제출하는 것은 차원이 다른 이야기입니다. 이 장에서는 `PdfGenerator`가 어떻게 스캔 데이터를 구조화된 문서로 변환하는지 낱낱이 살펴봅니다.

8.104 17.1 Android PdfDocument API 개요

8.104.1 외부 라이브러리 없이 PDF 만들기

iText, Apache PDFBox — PDF 생성 라이브러리는 많습니다. 하지만 SearCam은 외부 라이브러리를 쓰지 않습니다. Android API 19(4.4)부터 내장된 `android.graphics.pdf.PdfDocument` 만으로 충분합니다.

외부 라이브러리를 배제한 이유는 두 가지입니다. 첫째, APK 용량

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

입니다. iText 라이브러리는 수 MB 규모인데, SearCam처럼 PDF가 부가 기능인 앱에 과도합니다. 둘째, 보안입니다. 외부 라이브러리의 취약점이 앱에 그대로 전파됩니다. 내장 API는 Android 보안 업데이트와 함께 패치됩니다.

`PdfDocument`의 작동 방식은 캔버스 그림 그리기와 같습니다.

```
PdfDocument 생성
  → 페이지 시작 (startPage)
        → Canvas에 텍스트/선/도형 그리기
  → 페이지 완료 (finishPage)
    → 다음 페이지 반복
→ 파일에 쓰기 (writeTo)
→ 문서 닫기 (close)
```

8.105 17.2 PdfGenerator 설계 — 3페이지 구조

8.105.1 페이지별 책임 분리

SearCam의 PDF는 3페이지로 구성됩니다. 각 페이지는 독립적인 함수가 담당합니다.

```
페이지 1: drawOverviewPage()   — 종합 위험도 + 스캔 정보 + 레이어별
페이지 2: drawDevicePage()     — 발견된 네트워크 기기 목록 + 의심 기
```

페이지 3: `drawFindingsPage()` — 발견 사항 체크리스트 + 권고 사항 +

페이지 레이아웃 상수는 파일 상단에 모아두었습니다.

```
// data/pdf/PdfGenerator.kt

class PdfGenerator @Inject constructor(
    @ApplicationContext private val context: Context,
) {
    /** A4 너비 (포인트, 72dpi 기준) */
    private val PAGE_WIDTH = 595

    /** A4 높이 (포인트, 72dpi 기준) */
    private val PAGE_HEIGHT = 842

    /** 페이지 좌우 여백 */
    private val MARGIN_H = 40f

    /** 페이지 상하 여백 */
    private val MARGIN_V = 50f

    /** 줄 간격 */
    private val LINE_HEIGHT = 20f
```

```
    /** 섹션 간격 */
    private val SECTION_GAP = 30f
```

A4 크기가 595 × 842 포인트인 이유: PDF의 기본 단위는 포인트(point)입니다. 1포인트 = 1/72 인치입니다. A4 용지는 210 × 297mm = 8.27 × 11.69인치 = 595.3 × 841.9 포인트입니다.

8.105.2 generate() — 메인 진입점과 Path Traversal 방어

```
fun generate(report: ScanReport, outputPath: String): Resu
    // 보안: Path Traversal 방지
    val outputFile = File(outputPath)
    val allowedDir = context.filesDir.canonicalPath
    val externalDir = context.getExternalFilesDir(null)?.c
    val canonicalOut = outputFile.canonicalPath

    if (!canonicalOut.startsWith(allowedDir) &&
        (externalDir == null || !canonicalOut.startsWith(e
    ) {
        Timber.e("PDF 출력 경로 차단: $canonicalOut")
        return Result.failure(
            SecurityException("PDF 출력 경로가 허용 범위를 벗어
        )
```

```
    }

    val document = PdfDocument()

    return try {
        outputFile.parentFile?.mkdirs()

        val page1 = createPage(document, pageNumber = 1)
        drawOverviewPage(page1.canvas, report)
        document.finishPage(page1)

        val page2 = createPage(document, pageNumber = 2)
        drawDevicePage(page2.canvas, report)
        document.finishPage(page2)

        val page3 = createPage(document, pageNumber = 3)
        drawFindingsPage(page3.canvas, report)
        document.finishPage(page3)

        FileOutputStream(outputFile).use { stream ->
            document.writeTo(stream)
        }
```

```
        Timber.d("PDF 생성 완료: $outputPath")
        Result.success(outputPath)
    } catch (e: Exception) {
        Timber.e(e, "PDF 생성 실패")
        if (outputFile.exists()) outputFile.delete()  // 실
        Result.failure(e)
    } finally {
        document.close()  // 반드시 닫아야 메모리 해제
    }
}
```

`document.close()`는 `finally`에서 호출합니다. 예외가 발생해도 `PdfDocument`가 점유한 네이티브 메모리가 해제됩니다. `PdfDocument`는 페이지당 비트맵 버퍼를 내부적으로 관리하므로 Close 없이 버려지면 메모리 누수로 이어집니다.

8.106 17.3 Path Traversal 방어 — canonicalPath 검증

8.106.1 경로 조작 공격이란

Path Traversal은 `../`를 이용해서 허용된 디렉토리 밖에 파일을 쓰는 공격입니다. 예를 들어 `outputPath`로 `/data/data/com.searcam/databases/../../../etc/hosts`가 들어오면, 일반 경로 비교(`startsWith`)는 통과하지만 실제로는 시스템 파일을 덮어씁니다.

`canonicalPath`는 `.`, `..`, 심볼릭 링크를 모두 해석한 절대 경로를 반환합니다. 경로 조작의 여지를 없앱니다.

```
// 취약한 코드 (절대 사용 금지)
if (outputPath.startsWith(allowedDir)) {  // "../.."로 우회
    // 파일 쓰기
}

// 안전한 코드 (SearCam 구현)
val canonicalOut = File(outputPath).canonicalPath
val allowedDir = context.filesDir.canonicalPath
if (!canonicalOut.startsWith(allowedDir)) {
    return Result.failure(SecurityException("경로 차단"))
```

```
}
```

File("/data/data/com.searcam/files/../databases/secret").canonicalPath를 실행하면 /data/data/com.searcam/databases/secret이 반환됩니다. 이 경로는 files/로 시작하지 않으므로 차단됩니다.

앱 내부 저장소(context.filesDir)와 외부 저장소(context.getExternalFilesDir()) 두 곳을 허용 목록으로 관리합니다. 두 곳 모두 앱 전용 디렉토리로, 다른 앱이 직접 접근할 수 없습니다.

8.107 17.4 페이지 1 — 종합 위험도 시각화

8.107.1 drawOverviewPage — 첫인상이 전부다

```
private fun drawOverviewPage(canvas: Canvas, report: ScanR
    var y = MARGIN_V

    y = drawHeader(canvas, y, report)
    y = drawDivider(canvas, y)

    y += SECTION_GAP
```

```
    y = drawRiskScore(canvas, y, report.riskScore, report.

    y += SECTION_GAP
    y = drawSectionTitle(canvas, y, "스캔 정보")
    y = drawKeyValue(canvas, y, "스캔 모드", report.mode.la
    y = drawKeyValue(canvas, y, "소요 시간", formatDuration
    y = drawKeyValue(canvas, y, "위치 메모", report.locatio
    y = drawKeyValue(canvas, y, "발견 기기 수", "${report.de
    y = drawKeyValue(canvas, y, "렌즈 의심 포인트", "${repor
    y = drawKeyValue(canvas, y, "IR 의심 포인트", "${report

    y += SECTION_GAP
    y = drawSectionTitle(canvas, y, "탐지 레이어 결과")
    for ((layerType, layerResult) in report.layerResults)
        val statusText = "${layerResult.score}점 — ${layerR
        y = drawKeyValue(canvas, y, layerType.labelKo, sta
    }

    drawPageNumber(canvas, 1, 3)
}
```

`var y`를 통해 세로 방향 커서를 관리합니다. 각 드로잉 함수는 그린 영역의 마지막 Y 좌표를 반환하고, 다음 함수는 이 값을 시작점

으로 받습니다. 간단하지만 효과적인 플로우 레이아웃 패턴입니다.

8.107.2 위험도 점수 대형 표시와 색상 코드

위험도는 PDF에서 한눈에 들어와야 합니다. 48pt 폰트로 점수를 크게 표시하고, 위험 등급에 따라 색상을 바꿉니다.

```
private fun drawRiskScore(canvas: Canvas, y: Float, score:
    var currentY = y

    // "종합 위험도" 라벨 (작은 회색 텍스트)
    val labelPaint = buildPaint(size = 12f, color = Color.
    canvas.drawText("종합 위험도", MARGIN_H, currentY, labe
    currentY += LINE_HEIGHT

    // 점수 숫자 (48pt, 위험도 색상)
    val scorePaint = buildPaint(size = 48f, bold = true, c
    canvas.drawText("$score 점", MARGIN_H, currentY + 30f,
    currentY += 50f

    // 위험 등급 설명
    val badgePaint = buildPaint(size = 14f, bold = true, c
    canvas.drawText("${level.labelKo} — ${level.descriptio
    currentY += LINE_HEIGHT * 1.5f
```

```
    return currentY
}
```

위험 등급별 색상 체계는 SearCam 전체에서 일관되게 사용됩니다.

위험 등급	색상	Hex 코드
SAFE (안전)	초록	#22C55E
INTEREST (관심)	연두	#84CC16
CAUTION (주의)	노랑	#EAB308
DANGER (위험)	주황	#F97316
CRITICAL (매우 위험)	빨강	#EF4444

이 색상 코드는 `RiskLevel.colorHex` 프로퍼티로 도메인 모델에 정의되어 있습니다. PDF, Compose UI, 아이콘 모두 같은 값을 참조하므로 색상이 어디서나 일치합니다.

8.108 17.5 페이지 2 — 기기 목록 테이블

8.108.1 테이블 헤더와 행 렌더링

PDF에는 HTML처럼 자동 레이아웃이 없습니다. 모든 위치를 픽셀 단위로 직접 지정해야 합니다.

```
private fun drawTableHeader(canvas: Canvas, y: Float): Flo
    // 배경 사각형 (어두운 회색)
    val bgPaint = Paint().apply { color = Color.DKGRAY }
    canvas.drawRect(MARGIN_H, y - LINE_HEIGHT + 4f, PAGE_W

    // 흰 텍스트로 헤더 항목
    val paint = buildPaint(size = 10f, bold = true, color
    canvas.drawText("IP 주소",  MARGIN_H + 5f,   y, paint)
    canvas.drawText("MAC 주소", MARGIN_H + 110f,  y, paint
    canvas.drawText("제조사",   MARGIN_H + 240f,  y, paint
    canvas.drawText("위험도",   MARGIN_H + 360f,  y, paint
    canvas.drawText("카메라",   MARGIN_H + 420f,  y, paint
    return y + LINE_HEIGHT
}

private fun drawDeviceRow(canvas: Canvas, y: Float, device
    // 의심 기기는 빨간색으로 강조
    val textColor = if (device.isCamera) Color.RED else Co
    val paint = buildPaint(size = 10f, color = textColor)

    canvas.drawText(device.ip,                              M
    canvas.drawText(device.mac,                             M
```

```
    canvas.drawText(device.vendor?.take(12) ?: "알 수 없음",
    canvas.drawText("${device.riskScore}점",
    canvas.drawText(if (device.isCamera) "의심" else "-",

    return y + LINE_HEIGHT
}
```

`device.vendor?.take(12)`는 제조사 이름이 너무 길어서 다음 컬럼을 침범하는 것을 방지합니다. PDF Canvas는 텍스트가 영역을 벗어나도 자동 줄바꿈이 없습니다 — 직접 잘라야 합니다.

8.108.2 페이지 오버플로우 방지

기기가 많으면 페이지를 넘칠 수 있습니다. 현재 Y 위치를 체크해서 페이지 끝에 가까워지면 렌더링을 중단합니다.

```
for (device in report.devices.sortedByDescending { it.risk
    // 페이지 하단 여백에 한 줄이 들어갈 공간이 없으면 중단
    if (y > PAGE_HEIGHT - MARGIN_V - LINE_HEIGHT) break
    y = drawDeviceRow(canvas, y, device)
}
```

프로덕션 품질 PDF라면 다음 페이지로 자동 넘김(pagination)을 구현해야 합니다. SearCam 1.0에서는 기기가 많을 경우 위험도 높은 순으로 정렬하고 넘치는 부분은 생략합니다. 기기 목록 전체보다

가장 위험한 기기를 먼저 보여주는 것이 사용자에게 더 유용하기 때문입니다.

8.109 17.6 페이지 3 — 발견 사항과 권고 사항

8.109.1 심각도 순 정렬과 구조화된 출력

```
private fun drawFindingsPage(canvas: Canvas, report: ScanR
    var y = MARGIN_V
    drawSmallHeader(canvas, y, "발견 사항 및 권고")
    y += LINE_HEIGHT * 2
    y = drawDivider(canvas, y)
    y += LINE_HEIGHT

    if (report.findings.isEmpty()) {
        y = drawBodyText(canvas, y, "특이 사항이 발견되지 않았
    } else {
        y = drawSectionTitle(canvas, y, "발견 사항 목록")

        // severity.ordinal: enum 선언 순서 (CRITICAL > DANG
        for (finding in report.findings.sortedByDescending
            if (y > PAGE_HEIGHT - MARGIN_V - LINE_HEIGHT *
```

```
            // 심각도 배지와 발견 유형 (굵은 텍스트)
            val severityText = "[${finding.severity.labelK
            y = drawBodyText(canvas, y, severityText, bold
            y = drawBodyText(canvas, y, "  ${finding.descr
            y = drawBodyText(canvas, y, "  근거: ${finding
            y += LINE_HEIGHT * 0.5f  // 항목 간 여백
        }
    }

    // 권고 사항 — 위험 등급에 따라 다른 텍스트
    y += SECTION_GAP
    y = drawSectionTitle(canvas, y, "권고 사항")
    val recommendation = getRecommendation(report.riskLeve
    y = drawBodyText(canvas, y, recommendation)

    // 보고서 메타 정보
    y += SECTION_GAP
    y = drawDivider(canvas, y)
    y += LINE_HEIGHT
    val dateStr = SimpleDateFormat("yyyy년 MM월 dd일 HH:mm",
        .format(Date(report.completedAt))
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    y = drawBodyText(canvas, y, "생성 일시: $dateStr", smal
    y = drawBodyText(canvas, y, "리포트 ID: ${report.id}",
    drawBodyText(canvas, y, "SearCam — 몰래카메라 탐지 앱", 

    drawPageNumber(canvas, 3, 3)
}
```

8.109.2 위험 등급별 권고 사항 텍스트

```
private fun getRecommendation(level: RiskLevel): String {
    return when (level) {
        RiskLevel.SAFE ->
            "탐지된 위협이 없습니다. 안전한 환경으로 판단됩니다."
        RiskLevel.INTEREST ->
            "경미한 의심 징후가 발견되었습니다. 추가 점검을 권장합
        RiskLevel.CAUTION ->
            "복수의 의심 징후가 감지되었습니다. 육안 점검과 함께 
        RiskLevel.DANGER ->
            "강한 의심 징후가 감지되었습니다. 즉각적인 육안 점검과
        RiskLevel.CRITICAL ->
            "몰래카메라 의심이 강합니다. 즉시 경찰에 신고하고 해당
    }
}
```

권고 사항 텍스트는 법적 주의 사항입니다. “경찰에 신고하세요” 는 명확한 행동 지침이지만, “몰래카메라가 있습니다” 는 단정 짓지 않습니다. SearCam은 탐지 도우미이지 법적 판단 주체가 아닙니다. 문구 선택이 중요합니다.

8.110 17.7 드로잉 유틸리티 — buildPaint와 parseColor

8.110.1 Paint 팩토리 함수

PDF Canvas에서 텍스트를 그릴 때마다 `Paint` 객체를 생성합니다. `buildPaint()`는 반복되는 설정을 한 곳에 모읍니다.

```
private fun buildPaint(
    size: Float,
    bold: Boolean = false,
    color: Int = Color.BLACK,
): Paint {
    return Paint(Paint.ANTI_ALIAS_FLAG).apply {
        textSize = size
        this.color = color
        typeface = if (bold) Typeface.DEFAULT_BOLD else Ty
```

```
    }
}
```

`Paint.ANTI_ALIAS_FLAG`는 텍스트 가장자리를 부드럽게 렌더링합니다. 없으면 계단 현상(aliasing)이 생겨서 PDF를 확대했을 때 텍스트가 거칠어 보입니다.

8.110.2 안전한 색상 파싱

```
private fun parseColor(hex: String): Int {
    return try {
        Color.parseColor(hex)
    } catch (e: IllegalArgumentException) {
        Timber.e(e, "색상 파싱 실패 — hex=$hex, 기본값 BLACK 
        Color.BLACK  // 파싱 실패 시 기본값으로 계속 진행
    }
}
```

`Color.parseColor()`는 유효하지 않은 Hex 문자열이 들어오면 `IllegalArgumentException`을 던집니다. 잡지 않으면 PDF 생성 전체가 실패합니다. 잘못된 색상 코드 때문에 증거 PDF가 생성되지 않는 것보다, 색상만 검정으로 대체하고 문서를 완성하는 것이 낫습니다.

8.111 17.8 ExportReportUseCase — 도메인 레이어의 흐름 조율

8.111.1 UseCase는 오케스트라 지휘자

ExportReportUseCase는 실제로 PDF를 그리거나 파일을 저장하지 않습니다. 그 일은 PdfGenerator와 ReportRepository가 합니다. UseCase는 이 흐름을 조율하는 지휘자 역할입니다.

```
// domain/usecase/ExportReportUseCase.kt


class ExportReportUseCase(
    private val reportRepository: ReportRepository,
) {
    /**
     * 리포트를 PDF로 내보내고 저장된 파일 경로를 반환한다.
     *
     * 흐름:
     *   1. reportId로 리포트 조회
     *   2. 출력 파일명 생성
     *   3. ReportRepository.exportToPdf() 호출
     *   4. 저장된 파일 경로 반환
     */
    suspend operator fun invoke(reportId: String): Result<
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        // 1단계: 리포트 조회
        val reportResult = reportRepository.getReport(repo
        if (reportResult.isFailure) {
            return Result.failure(
                reportResult.exceptionOrNull()
                    ?: IllegalStateException("리포트를 찾을
            )
        }

        val report = reportResult.getOrThrow()

        // 2단계: 파일명 생성 (경로는 data 레이어가 결정)
        val outputFileName = buildPdfFileName(report.compl

        // 3단계: 위임
        return reportRepository.exportToPdf(
            report = report,
            outputPath = outputFileName,
        )
    }

    /**
```

```
     * PDF 파일명을 생성한다.
     *
     * 경로(디렉토리)는 data 레이어 구현체에서 결정한다.
     * UseCase는 파일명만 알고 있어야 한다.
     */
    private fun buildPdfFileName(epochMs: Long): String =
        "searcam_report_${epochMs}.pdf"
}
```

UseCase에 @ApplicationContext가 없습니다. 파일 경로 결정은 data 레이어(ReportRepositoryImpl)의 책임입니다. UseCase는 "어떤 파일에 저장할지" 가 아니라 "리포트를 PDF로 변환해달라" 는 의도만 표현합니다. 이 덕분에 UseCase를 순수 JVM 환경에서 테스트할 수 있습니다.

8.111.2 ViewModel에서 UseCase 호출

```
// ui/report/ReportViewModel.kt

@HiltViewModel
class ReportViewModel @Inject constructor(
    private val exportReportUseCase: ExportReportUseCase,
    savedStateHandle: SavedStateHandle,
) : ViewModel() {
```

```
    private val reportId = savedStateHandle.get<String>("r

    private val _exportState = MutableStateFlow<ExportStat
    val exportState: StateFlow<ExportState> = _exportState

    fun exportToPdf() {
        if (_exportState.value is ExportState.Loading) ret

        viewModelScope.launch {
            _exportState.value = ExportState.Loading

            val result = exportReportUseCase(reportId)

            _exportState.value = result.fold(
                onSuccess = { filePath -> ExportState.Succ
                onFailure = { error -> ExportState.Error(e
            )
        }
    }
}
```

```
sealed class ExportState {
    data object Idle : ExportState()
    data object Loading : ExportState()
    data class Success(val filePath: String) : ExportState
    data class Error(val message: String) : ExportState()
}
```

`result.fold()`는 `Result<T>`의 성공/실패를 깔끔하게 처리합니다. `if (result.isSuccess)` 분기와 달리, `fold`는 두 케이스 모두 값을 반환해야 하므로 누락 없이 처리를 강제합니다.

8.112 17.9 FileProvider로 PDF 안전하게 공유

8.112.1 다른 앱이 파일을 읽게 허용하기

PDF 파일이 앱 내부 저장소에 있으면 다른 앱이 직접 접근할 수 없습니다. 이메일, 카카오톡 같은 앱으로 PDF를 공유하려면 임시 읽기 권한을 부여해야 합니다. `FileProvider`가 이 역할을 합니다.

```
<!-- AndroidManifest.xml -->
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
```

```
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_provider_paths" />
</provider>

<!-- res/xml/file_provider_paths.xml -->
<paths>
    <!-- 앱 내부 저장소 -->
    <files-path name="files" path="." />
    <!-- 앱 외부 저장소 (Documents 디렉토리) -->
    <external-files-path name="external_files" path="Docum
</paths>

// PDF 공유 함수

fun sharePdf(context: Context, filePath: String) {
    val file = File(filePath)

    // FileProvider URI 생성 — 직접 파일 경로 대신 content:// 
    val uri: Uri = FileProvider.getUriForFile(
        context,
        "${context.packageName}.fileprovider",
```

```
        file,
    )

    val shareIntent = Intent(Intent.ACTION_SEND).apply {
        type = "application/pdf"
        putExtra(Intent.EXTRA_STREAM, uri)
        putExtra(Intent.EXTRA_SUBJECT, "SearCam 스캔 리포트
        // 받는 앱에 임시 읽기 권한 부여
        addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
    }

    context.startActivity(
        Intent.createChooser(shareIntent, "리포트 공유")
    )
}
```

FLAG_GRANT_READ_URI_PERMISSION이 핵심입니다. 이 플래그가 있어야 공유 대상 앱이 content:// URI로 파일을 읽을 수 있습니다. 공유가 끝나면 권한이 자동으로 취소됩니다. 파일 경로를 직접 노출하지 않으므로 다른 앱이 앱 내부 디렉토리 구조를 알 수 없습니다.

8.112.2 Compose에서 공유 버튼 연결

```
// ui/report/ReportScreen.kt

@Composable
fun ReportScreen(
    reportId: String,
    viewModel: ReportViewModel = hiltViewModel(),
) {
    val context = LocalContext.current
    val exportState by viewModel.exportState.collectAsStat

    // 내보내기 성공 시 공유 인텐트 실행
    LaunchedEffect(exportState) {
        if (exportState is ExportState.Success) {
            sharePdf(context, (exportState as ExportState.
        }
    }

    // 내보내기 버튼
    Button(
        onClick = viewModel::exportToPdf,
        enabled = exportState !is ExportState.Loading,
```

```
    ) {
        if (exportState is ExportState.Loading) {
            CircularProgressIndicator(modifier = Modifier.
        } else {
            Icon(Icons.Default.Share, contentDescription =
            Spacer(Modifier.width(8.dp))
            Text("PDF로 공유")
        }
    }
}
```

`LaunchedEffect(exportState)` 는 `exportState` 가 바뀔 때마다 재실행됩니다. `ExportState.Success` 일 때만 `sharePdf()` 를 호출하므로, 사용자가 화면을 회전해도 (Recomposition) 공유 인텐트가 중복 실행되지 않습니다 — `exportState` 가 바뀌지 않았으므로 `LaunchedEffect` 가 재실행되지 않습니다.

8.113 17.10 전체 흐름 요약

```
사용자: "PDF로 공유" 버튼 탭
  ↓
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
ReportViewModel.exportToPdf()
    ↓
ExportReportUseCase.invoke(reportId)
    ├── ReportRepository.getReport(reportId)
    │   └── ReportDao.findById() → ScanReportEntity
    │       └── Mapper.toDomain() → ScanReport
    └── ReportRepository.exportToPdf(report, outputPath)
            ├── Path Traversal 검증 (canonicalPath)
            ├── PdfDocument 생성
            ├── drawOverviewPage() — 종합 위험도 + 스캔 정보
            ├── drawDevicePage()   — 기기 목록 테이블
            ├── drawFindingsPage() — 발견 사항 + 권고
            └── FileOutputStream.write() → 파일 저장
        ↓
ExportState.Success(filePath)
    ↓
LaunchedEffect: sharePdf(context, filePath)
    ↓
FileProvider.getUriForFile() → content:// URI
```

```
    ↓
Intent.ACTION_SEND → 공유 앱 선택
```

각 계층이 자신의 책임만 담당합니다. UseCase는 흐름만 조율하고, Repository는 데이터 접근을 추상화하고, PdfGenerator는 렌더링에만 집중합니다. 어느 계층도 다른 계층의 구현 세부사항을 알지 못합니다.

8.114 실습

실습 17-1: `PdfGenerator`의 `drawDeviceRow()`를 수정해서 위험도 점수 50 이상인 기기의 행 배경을 연한 빨강(`Color.argb(50, 255, 0, 0)`)으로 채워보세요.

실습 17-2: 기기 목록이 한 페이지를 넘는 경우를 테스트해보세요. 20개 이상의 가짜 `NetworkDevice`를 만들어 `generate()`를 호출하고, 결과 PDF에서 넘친 기기가 어떻게 처리되는지 확인하세요.

실습 17-3: `ExportReportUseCase`의 단위 테스트를 작성해보세요. `ReportRepository`를 Mock으로 대체하고, `reportId`에 없는 ID가 들어왔을 때

Result.failure가 올바르게 반환되는지 검증하세요.

8.115 핵심 정리

개념	핵심
PdfDocument	Android 내장 API, 외부 라이브러리 불필요, A4 = 595×842pt
Canvas Y 커서	각 드로잉 함수가 마지막 Y를 반환, 누적으로 레이아웃 구성
Path Traversal 방어	canonicalPath로 .. 해석 후 허용 디렉토리 비교
parseColor 방어	파싱 실패 시 Color.BLACK 반환, PDF 생성 실패 방지
ExportReportUseCase	흐름 조율만, Android 의존성 없음, 순수 JVM 테스트 가능
FileProvider	content:// URI로 파일 노출, 임시 읽기 권한만 부여
FLAG_GRANT_READ_URI_PERMISSION	앱에 임시 읽기 권한, 공유 종료 후 자동 취소

- `PdfDocument.close()`는 `finally`에서 반드시 호출해야 네이티브 메모리가 해제된다
- 텍스트가 긴 경우 `take(n)`으로 잘라야 다음 컬럼을 침범하지 않는다
- `FileProvider` 없이 파일 경로(`file://`)를 직접 공유하면 Android 7.0 이상에서 `FileUriExposedException`이 발생한다
- 권고 사항 문구는 단정이 아닌 권고 형식으로 — 앱은 탐지 도우미이지 법적 판단 주체가 아니다

8.116 다음 장 예고

이제 SearCam의 핵심 기능 구현이 모두 완성되었습니다. Ch18에서는 구현한 코드를 검증하는 테스트 전략 — 도메인 레이어 단위 테스트, Room DB 통합 테스트, Compose UI 테스트, 그리고 실제 하드웨어 없이 센서를 테스트하는 방법까지 — 을 다룹니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

참

고

문

서:

docs/

14-

security-

design.md,

docs/

06-

api-

design.md,

docs/

18-

test-

strategy.md

—

#

Ch18:

테

스

트

구

현

—

코

드

보

다

테

스

트

가

먼

저

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—>

이장에서배울것: 사용자의 안전과 직결된 앱에서 테스트는

8.117 도입

비행기 조종사는 이륙 전에 반드시 체크리스트를 사용합니다. 경험이 10년이 넘어도, 날씨가 완벽해도, 같은 기체를 100번 몰았어도 건너뛰지 않습니다. 그 이유는 단순합니다. 한 번의 실수가 돌이킬 수 없기 때문입니다.

SearCam도 마찬가지입니다. 몰래카메라를 놓치는 미탐(false negative)은 사용자의 프라이버시 침해로 이어집니다. 반대로 멀쩡한 기기를 카메라로 잘못 판단하는 오탐(false positive)은 사용자의 신뢰를 무너뜨립니다. "대충 테스트해봤는데 잘 되는 것 같다" 는 말은 "아마 이 비행기는 괜찮을 것 같다" 와 같습니다.

이 장에서는 SearCam의 테스트 전략과 실제 코드를 함께 살펴봅니다.

8.118 18.1 테스트 피라미드: 무엇을 얼마나 테스트할까

8.118.1 비율이 곧 전략이다

자동차 공장에는 세 종류의 검사가 있습니다. 부품 단위 검사(unit), 조립 라인 검사(integration), 시운전 검사(E2E). 세 가지 모두 하지

만 비율이 다릅니다. 부품 검사가 가장 많고, 시운전은 최소화합니다. 시운전이 가장 현실적이지만, 가장 느리고 비싸기 때문입니다.

SearCam의 테스트 피라미드도 같은 논리를 따릅니다.

```
          ┌──────────────────┐
    │  E2E      │  10% (핵심 사용자 플로우)
          │ (UI)      │
  ──┼──────────────────┼──
 │Integration│  20% (센서→분석→결과 파이프라인)
        │         │
 ──┼──────────────────────┼──
  │   Unit      │  70% (Domain + Data 레이어)
      │         │
  └──────────────────────┘
```

레벨	비율	실행 환경	실행 시간 목표
Unit	70%	JVM (Robolectric 포함)	< 30초
Integration	20%	Android Instrumented	< 2분
E2E	10%	실기기 / 에뮬레이터	< 5분

단위 테스트가 전체의 70%를 차지하는 이유가 있습니다. 빠르고, 안정적이며, 정확하게 실패 지점을 짚어줍니다. Domain 레이어(UseCase, RiskCalculator)는 90% 이상 커버리지를 목표로 합니

다. 사용자 안전과 직결된 판단 로직이 여기 있기 때문입니다.

8.119 18.2 테스트 의존성 설정

8.119.1 도구함 준비하기

목수가 망치 없이 집을 지을 수 없듯, 좋은 테스트도 올바른 도구가 필요합니다.

```
# gradle/libs.versions.toml
[versions]
mockk = "1.13.10"
turbine = "1.1.0"
coroutines-test = "1.8.0"
junit = "4.13.2"
kotest = "5.8.1"

[libraries]
test-mockk = { group = "io.mockk", name = "mockk", versior
test-turbine = { group = "app.cash.turbine", name = "turbi
test-coroutines = { group = "org.jetbrains.kotlinx", name
test-junit = { group = "junit", name = "junit", version.re
```

```
test-kotest-assertions = { group = "io.kotest", name = "ko

// build.gradle.kts (app)
dependencies {
    testImplementation(libs.test.junit)
    testImplementation(libs.test.mockk)
    testImplementation(libs.test.turbine)
    testImplementation(libs.test.coroutines)
    testImplementation(libs.test.kotest.assertions)

    // Hilt 테스트 지원
    testImplementation("com.google.dagger:hilt-android-tes
    kaptTest("com.google.dagger:hilt-android-compiler:2.51
}
```

각 도구의 역할을 정리하면 다음과 같습니다.

도구	역할
MockK	Kotlin 친화적 모킹. 코루틴 suspend 함수 모킹 지원
Turbine	Kotlin Flow를 테스트하는 전용 라이브러리

도구	역할
kotlinx-coroutines-test	runTest, TestCoroutineDispatcher로 코루틴 테스트 제어
Kotest	표현력 높은 assertion (`shouldBe`, `shouldBeInRange`)

8.120 18.3 MockK로 의존성 격리하기

8.120.1 가짜 협력자를 만드는 법

단위 테스트의 핵심은 테스트 대상만 고립시키는 것입니다. 레스토랑에서 새 레시피를 테스트할 때, 재료 공급업체나 주방 기기 상태와 무관하게 레시피 자체만 검증하고 싶습니다. MockK는 이런 "가짜 재료"를 제공합니다.

8.120.2 Repository 모킹

```
// domain/usecase/RunQuickScanUseCaseTest.kt
@ExtendWith(MockKExtension::class)
```

```
class RunQuickScanUseCaseTest {

    @MockK
    private lateinit var wifiScanner: WifiScanner

    @MockK
    private lateinit var ouiDatabase: OuiDatabase

    @MockK
    private lateinit var portScanner: PortScanner

    @MockK
    private lateinit var riskCalculator: RiskCalculator

    private lateinit var useCase: RunQuickScanUseCase

    @BeforeEach
    fun setUp() {
        useCase = RunQuickScanUseCase(
            wifiScanner = wifiScanner,
            ouiDatabase = ouiDatabase,
            portScanner = portScanner,
```

```
            riskCalculator = riskCalculator
        )
    }

    @Test
    fun `Wi-Fi 연결 상태에서 의심 기기 발견 시 위험도 40 이상 반환
        // Given: 의심스러운 Hikvision 기기가 네트워크에 있음
        val suspiciousDevice = NetworkDevice(
            ip = "192.168.1.101",
            mac = "28:57:BE:AA:BB:CC",
            hostname = "unknown"
        )
        val ouiEntry = OuiEntry(
            vendor = "Hangzhou Hikvision",
            type = DeviceType.IP_CAMERA,
            riskWeight = 0.95f
        )

        coEvery { wifiScanner.scanNetwork() } returns list
        coEvery { ouiDatabase.lookup("28:57:BE") } returns
        coEvery { portScanner.scanPorts(suspiciousDevice.i
            PortResult(port = 554, isOpen = true, protocol
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        )
        coEvery { riskCalculator.calculateLayer1Score(any(

        // When
        val result = useCase.execute()

        // Then
        result.riskScore shouldBeGreaterThanOrEqual 40
        result.suspiciousDevices.size shouldBe 1
        result.suspiciousDevices.first().mac shouldBe "28:
    }

    @Test
    fun `Wi-Fi 미연결 시 Layer1 스킵하고 빈 결과 반환`() = runTes
        // Given
        coEvery { wifiScanner.isConnected() } returns fals

        // When
        val result = useCase.execute()

        // Then
        result.layers[ScanLayer.WIFI]?.status shouldBe Lay
```

```
        coVerify(exactly = 0) { wifiScanner.scanNetwork()
    }
}
```

핵심 패턴은 세 가지입니다.

- `coEvery { ... } returns ...`: suspend 함수의 반환값을 지정합니다
- `coVerify(exactly = N) { ... }`: 함수가 N번 호출되었는지 검증합니다
- `@MockK` + `@ExtendWith(MockKExtension::class)`: 자동 초기화

8.121 18.4 Turbine으로 Flow 테스트하기

8.121.1 파이프를 검사하는 방법

Kotlin Flow는 물이 흐르는 파이프와 같습니다. 문제는 파이프 안을 직접 들여다볼 수 없다는 것입니다. Turbine은 파이프에 수집 장치를 달아 흐르는 데이터를 하나씩 꺼내볼 수 있게 해줍니다.

```
// 스캔 진행 상황을 Flow로 emit하는 UseCase 테스트
class ScanProgressTest {
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    @Test
    fun `스캔 진행률이 0에서 100까지 순서대로 emit된다`() = runTe
        val fakeScanner = FakeWifiScanner(deviceCount = 5)
        val useCase = RunQuickScanUseCase(fakeScanner)

        useCase.progressFlow.test {
            // 첫 번째 emit: 0% (스캔 시작)
            val initial = awaitItem()
            initial.percentage shouldBe 0

            // ARP 스캔 완료: 30%
            val afterArp = awaitItem()
            afterArp.percentage shouldBe 30
            afterArp.phase shouldBe ScanPhase.ARP_SCANNING

            // OUI 매칭 완료: 60%
            val afterOui = awaitItem()
            afterOui.percentage shouldBe 60
            afterOui.phase shouldBe ScanPhase.OUI_MATCHING

            // 포트 스캔 완료: 100%
            val complete = awaitItem()
```

```
            complete.percentage shouldBe 100
            complete.phase shouldBe ScanPhase.COMPLETED

            awaitComplete()
        }
    }

    @Test
    fun `에러 발생 시 Flow가 에러로 종료된다`() = runTest {
        val errorScanner = FakeWifiScanner(throwError = tr
        val useCase = RunQuickScanUseCase(errorScanner)

        useCase.progressFlow.test {
            awaitError() shouldBeInstanceOf WifiScanExcept
        }
    }
}
```

Turbine의 핵심 API입니다.

API	설명
`awaitItem()`	다음 emit된 값을 기다려 반환
`awaitComplete()`	Flow가 정상 종료되길 기다림

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

API	설명
`awaitError()`	Flow가 에러로 종료되길 기다림
`cancelAndIgnoreRemainingEvents()`	남은 이벤트를 무시하고 취소

8.122 18.5 Coroutine 테스트: runTest와 TestDispatcher

8.122.1 시간을 제어하는 법

코루틴 테스트의 난제는 시간입니다. 실제 코드에서 `delay(2000)`은 2초를 기다립니다. 테스트에서도 2초를 기다리면 안 됩니다. `runTest`와 `TestCoroutineDispatcher`는 이 시간을 가상으로 조작할 수 있게 해줍니다. 마치 영화 촬영에서 타임랩스를 쓰는 것처럼, 2시간짜리 일몰을 30초 영상으로 담을 수 있습니다.

```
class CoroutineTimingTest {

    @Test
    fun `포트 스캔 타임아웃이 2초 후 정확히 발생한다`() = runTest
        val testDispatcher = UnconfinedTestDispatcher(test
        val portScanner = PortScanner(
```

```
            dispatcher = testDispatcher,
            timeoutMs = 2_000L
        )

        val startTime = currentTime
        val result = portScanner.scanPort("192.168.1.1", 5
        val elapsed = currentTime - startTime

        // 실제로 2초를 기다리지 않고 가상 시간으로 검증
        elapsed shouldBe 2_000L
        result.isOpen shouldBe false
    }

    @Test
    fun `20Hz 센서 데이터가 50ms 간격으로 emit된다`() = runTest
        val testDispatcher = StandardTestDispatcher(testSc
        val sensor = FakeMagneticSensor(dispatcher = testD

        val readings = mutableListOf<MagneticReading>()
        val job = launch(testDispatcher) {
            sensor.readingFlow.take(5).collect { readings.
        }
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        // 가상 시간을 250ms 앞으로 이동 (5회 emit 유발)
        advanceTimeBy(250)
        job.join()

        readings.size shouldBe 5
    }
}
```

runTest의 세 가지 TestDispatcher를 이해하면 됩니다.

Dispatcher	특성	사용 시점
StandardTestDispatcher	수동으로 시간 진행	순서 검증이 필요할 때
UnconfinedTestDispatcher	즉시 실행	순서보다 결과만 필요할 때
advanceTimeBy(ms)	가상 시간 진행	delay, timeout 테스트

8.123 18.6 PortScanner 단위 테스트

8.123.1 isPrivateIp() 경계값 테스트

"비공개 IP인지 확인하는" 함수는 단순해 보이지만, 경계값(boundary value)에서 버그가 숨습니다. RFC 1918에 따르면 사설 IP 대역은 세 가지입니다. 이 범위의 정확한 경계를 테스트해야 합니다.

```
// app/src/test/java/com/searcam/data/sensor/PortScannerTe
class PortScannerTest {

    private lateinit var portScanner: PortScanner

    @Before
    fun setUp() {
        portScanner = PortScanner()
    }

    // ─── 10.0.0.0/8 범위 ──────────────────────────────────
    @Test fun `10 범위 시작 주소는 사설 IP`() =
        assertTrue(portScanner.isPrivateIp("10.0.0.0"))

    @Test fun `10 범위 끝 주소는 사설 IP`() =
        assertTrue(portScanner.isPrivateIp("10.255.255.255
```

```
@Test fun `11로 시작하는 주소는 사설 IP 아님`() =
    assertFalse(portScanner.isPrivateIp("11.0.0.0"))

// ─── 172.16.0.0/12 범위 ──────────────────────────────
@Test fun `172-16 범위 시작 주소는 사설 IP`() =
    assertTrue(portScanner.isPrivateIp("172.16.0.0"))

@Test fun `172-31 범위 끝 주소는 사설 IP`() =
    assertTrue(portScanner.isPrivateIp("172.31.255.255

@Test fun `172-15 이전 주소는 사설 IP 아님`() =
    assertFalse(portScanner.isPrivateIp("172.15.255.25

@Test fun `172-32 이후 주소는 사설 IP 아님`() =
    assertFalse(portScanner.isPrivateIp("172.32.0.0"))

// ─── 192.168.0.0/16 범위 ─────────────────────────────
@Test fun `192-168 범위 공유기 기본 주소는 사설 IP`() =
    assertTrue(portScanner.isPrivateIp("192.168.1.1"))

@Test fun `192-169는 사설 IP 아님`() =
```

```
        assertFalse(portScanner.isPrivateIp("192.169.0.0")

    // ── 127.0.0.0/8 루프백 ──────────────────────────────
    @Test fun `루프백 127-0-0-1은 사설 IP`() =
        assertTrue(portScanner.isPrivateIp("127.0.0.1"))

    // ── 공인 IP ──────────────────────────────────────
    @Test fun `구글 DNS 8-8-8-8은 사설 IP 아님`() =
        assertFalse(portScanner.isPrivateIp("8.8.8.8"))

    // ── 잘못된 입력 ──────────────────────────────────
    @Test fun `문자열 입력은 사설 IP 아님`() =
        assertFalse(portScanner.isPrivateIp("not-an-ip"))

    @Test fun `옥텟 5개 이상은 사설 IP 아님`() =
        assertFalse(portScanner.isPrivateIp("192.168.1.1.1
}
```

8.124 18.7 NoiseFilter 이동 평균 테스트

8.124.1 노이즈 속에서 신호 찾기

자기장 센서 데이터는 잡음이 많습니다. 스마트폰을 들고 걸어가기만 해도 자기장 값이 흔들립니다. 이동 평균(moving average)은 이 잡음을 걸러내는 기술입니다. 마치 주식 차트의 이동 평균선처럼, 단기 변동을 평활화합니다.

```
class NoiseFilterTest {

    private val filter = NoiseFilter(windowSize = 10)

    @Test
    fun `이동 평균이 10개 윈도우에 대해 정확한 평균을 반환한다`() {
        // Given: 1부터 10까지의 데이터
        val inputs = (1..10).map { it.toFloat() }
        var result = 0f

        // When: 순서대로 추가
        inputs.forEach { result = filter.add(it) }

        // Then: 마지막 결과는 (1+2+...+10)/10 = 5.5
        result shouldBe 5.5f
```

```
    }

    @Test
    fun `윈도우가 꽉 찬 후 슬라이딩이 올바르게 동작한다`() {
        // Given: 10개로 윈도우를 채우고
        (1..10).forEach { filter.add(it.toFloat()) }

        // When: 11번째 값(100) 추가 시 1이 제거되고 100이 들어옴
        val result = filter.add(100f)

        // Then: (2+3+4+5+6+7+8+9+10+100) / 10 = 15.4
        result shouldBe 15.4f
    }

    @Test
    fun `noise_floor 이하 신호는 필터링된다`() {
        // baseline 설정
        val baseline = 45f
        val noiseFloor = 3f  // std_dev * 2

        filter.calibrate(baseline = baseline, noiseFloor =
```

CHAPTER 8 | 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        // delta 2.5 (< noiseFloor 3.0) → 필터링
        val reading = MagneticReading(x = 0f, y = 0f, z =
        val filtered = filter.applyFilter(reading)

        filtered shouldBe null
    }

    @Test
    fun `급격한 변화(0_3초 내 50+ uT)는 자체 간섭으로 필터링된다`
        // Given: 기준값 45 uT 설정 후
        filter.calibrate(baseline = 45f, noiseFloor = 3f)

        // When: 0.3초 내에 50 uT 이상 급변
        val suddenSpike = MagneticReading(x = 60f, y = 60f
        // magnitude = sqrt(60² + 60² + 60²) ≈ 103.9, delt

        val result = filter.applyFilterWithSpikeDetection(
            reading = suddenSpike,
            previousMagnitude = 45f,
            timeDeltaMs = 200L  // 0.2초 내
        )
```

```
        result shouldBe null  // 급변으로 인해 필터링
    }

    @Test
    fun `경계값: noise_floor와 정확히 동일한 delta는 통과 처리된
        filter.calibrate(baseline = 45f, noiseFloor = 3f)

        // delta == noiseFloor (3.0) → 통과 (>= 로 비교)
        val reading = MagneticReading(x = 0f, y = 0f, z =
        val result = filter.applyFilter(reading)

        result shouldNotBe null
    }
}
```

경계값 테스트(boundary testing)는 특히 중요합니다. > 와 >= 차이 하나가 탐지 정확도를 바꿉니다.

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.125 18.8 RiskCalculator 점수 검증 테스트

8.125.1 위험도 공식을 믿을 수 있는가

RiskCalculator는 SearCam의 핵심 의사결정 엔진입니다. 세 레이어의 점수를 받아 최종 위험도를 계산합니다. 이 공식이 틀리면 모든 탐지가 틀립니다. 종합 위험도 공식은 아래와 같습니다.

종합 위험도 = clamp(W1*L1 + W2*L2 + W3*L3 × 보정계수, 0, 100)

기본 가중치: W1=0.50(Wi-Fi), W2=0.35(렌즈), W3=0.15(EMF)

```
class RiskCalculatorTest {

    private val calculator = RiskCalculator()

    @Test
    fun `3레이어 가중치 합산이 정확하다`() {
        // Given: 각 레이어 점수
        val layer1Score = 80  // Wi-Fi 레이어 (가중치 0.50)
        val layer2Score = 70  // 렌즈 레이어 (가중치 0.35)
        val layer3Score = 60  // EMF 레이어 (가중치 0.15)

        // When
        val result = calculator.calculateRiskScore(
```

```
            layer1 = LayerResult(score = layer1Score, avai
            layer2 = LayerResult(score = layer2Score, avai
            layer3 = LayerResult(score = layer3Score, avai
        )

        // Then: 0.50*80 + 0.35*70 + 0.15*60 = 40 + 24.5 +
        result shouldBe 74
    }

    @Test
    fun `2개 레이어 양성 시 보정계수 1_2 적용된다`() {
        // Layer1, Layer3가 양성(> 60)
        val result = calculator.calculateRiskScore(
            layer1 = LayerResult(score = 80, available = t
            layer2 = LayerResult(score = 10, available = t
            layer3 = LayerResult(score = 70, available = t
        )

        // 보정 전: 0.50*80 + 0.35*10 + 0.15*70 = 40 + 3.5
        // 보정 후: 54 × 1.2 = 64.8 → 65
        result shouldBe 65
    }
```

```
@Test
fun `3개 레이어 모두 양성 시 보정계수 1_5 적용된다`() {
    val result = calculator.calculateRiskScore(
        layer1 = LayerResult(score = 70, available = t
        layer2 = LayerResult(score = 70, available = t
        layer3 = LayerResult(score = 70, available = t
    )

    // 보정 전: 0.50*70 + 0.35*70 + 0.15*70 = 70
    // 보정 후: 70 × 1.5 = 105 → clamp(105, 0, 100) = 1
    result shouldBe 100
}

@Test
fun `Wi-Fi 미연결 시 가중치가 재조정된다`() {
    // Wi-Fi OFF → Layer1 사용 불가 → W1=0, W2=0.75, W3
    val result = calculator.calculateRiskScore(
        layer1 = LayerResult(score = 0, available = fa
        layer2 = LayerResult(score = 80, available = t
        layer3 = LayerResult(score = 60, available = t
    )
```

```
        // 0.75*80 + 0.25*60 = 60 + 15 = 75
        result shouldBe 75
    }

    // 등급 경계값 테스트: 19/20, 39/40, 59/60, 79/80
    @ParameterizedTest
    @CsvSource(
        "19, SAFE",
        "20, CAUTION",
        "39, CAUTION",
        "40, WARNING",
        "59, WARNING",
        "60, DANGER",
        "79, DANGER",
        "80, HIGH_RISK"
    )
    fun `위험도 등급이 경계값에서 정확히 매핑된다`(
        score: Int,
        expectedLevel: String
    ) {
        val level = calculator.getRiskLevel(score)
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        level.name shouldBe expectedLevel
    }

    @Test
    fun `점수는 0 미만이 될 수 없다`() {
        // 모든 레이어 0점 + 음수 보정은 불가능하지만 방어적 테스트
        val result = calculator.calculateRiskScore(
            layer1 = LayerResult(score = 0, available = tr
            layer2 = LayerResult(score = 0, available = tr
            layer3 = LayerResult(score = 0, available = tr
        )

        result shouldBeGreaterThanOrEqual 0
    }

    @Test
    fun `점수는 100을 초과할 수 없다`() {
        val result = calculator.calculateRiskScore(
            layer1 = LayerResult(score = 100, available =
            layer2 = LayerResult(score = 100, available =
            layer3 = LayerResult(score = 100, available =
        )
```

```
        result shouldBeLessThanOrEqualTo 100
    }
}
```

8.126 18.9 ViewModel 테스트: StateFlow와 UI State

8.126.1 화면과 비즈니스 로직 사이의 계약

ViewModel은 UI와 Domain 사이의 번역가입니다. Domain에서 온 데이터를 UI가 이해할 수 있는 형태(UiState)로 변환합니다. 이 번역이 틀리면 사용자에게 잘못된 정보가 표시됩니다.

```
class ScanViewModelTest {

    private val mockUseCase = mockk<RunQuickScanUseCase>()
    private lateinit var viewModel: ScanViewModel

    @BeforeEach
    fun setUp() {
        // Hilt 없이 직접 주입 (단위 테스트는 가볍게)
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        viewModel = ScanViewModel(quickScanUseCase = mockU
    }

    @Test
    fun `스캔 시작 시 Loading 상태로 전환된다`() = runTest {
        // Given: 스캔이 느리게 완료됨
        coEvery { mockUseCase.execute() } coAnswers {
            delay(1000)  // 1초 지연
            ScanResult(riskScore = 0, suspiciousDevices =
        }

        viewModel.uiState.test {
            // Initial: Idle
            awaitItem() shouldBeInstanceOf ScanUiState.Idl

            // When: 스캔 시작
            viewModel.startQuickScan()

            // Then: Loading으로 즉시 전환
            awaitItem() shouldBeInstanceOf ScanUiState.Loa

            // 완료 대기
```

```
            advanceTimeBy(1000)
            awaitItem() shouldBeInstanceOf ScanUiState.Suc

            cancelAndIgnoreRemainingEvents()
        }
    }

    @Test
    fun `에러 발생 시 Error 상태와 에러 코드가 반환된다`() = runT
        coEvery { mockUseCase.execute() } throws WifiScanE

        viewModel.uiState.test {
            awaitItem()  // Idle
            viewModel.startQuickScan()
            awaitItem()  // Loading

            val errorState = awaitItem()
            errorState shouldBeInstanceOf ScanUiState.Erro
            (errorState as ScanUiState.Error).errorCode sh

            cancelAndIgnoreRemainingEvents()
        }
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    }
}
```

8.127 18.10 테스트 커버리지 측정

8.127.1 커버리지는 목표가 아니라 척도다

80% 커버리지를 달성했다고 해서 20%의 버그가 없다는 의미가 아닙니다. 커버리지는 "이 코드는 테스트되지 않았다"를 가리키는 지표일 뿐입니다. SearCam의 커버리지 목표는 다음과 같습니다.

```
// build.gradle.kts
jacoco {
    toolVersion = "0.8.11"
}

tasks.register<JacocoReport>("jacocoTestReport") {
    dependsOn("testDebugUnitTest")

    reports {
        xml.required.set(true)
        html.required.set(true)
```

```
    }

    classDirectories.setFrom(
        fileTree(layout.buildDirectory.dir("tmp/kotlin-cla
            // 제외: DI 모듈, Room 생성 코드, Hilt 생성 코드
            exclude(
                "**/di/**",
                "**/*_Hilt*",
                "**/*Dao_Impl*",
                "**/BuildConfig.*"
            )
        }
    )
}

// 커버리지 기준 미달 시 빌드 실패
tasks.register<JacocoCoverageVerification>("jacocoTestCove
    violationRules {
        rule {
            limit {
                counter = "LINE"
                value = "COVEREDRATIO"
```

```
                minimum = "0.80".toBigDecimal()
            }
        }
        rule {
            element = "CLASS"
            includes = listOf("*.domain.*")
            limit {
                counter = "LINE"
                value = "COVEREDRATIO"
                minimum = "0.90".toBigDecimal()
            }
        }
    }
}
```

8.128 정리

테스트는 “작동한다” 가 아닌 “언제나, 어떤 조건에서도, 정확하게 작동한다” 를 증명하는 도구입니다. SearCam에서 배운 핵심 원칙을 정리합니다.

1. **MockK로 격리**: 테스트 대상 외의 모든 의존성은 가짜로 대

체합니다

2. **Turbine으로 Flow 검증**: emit 순서와 완료/에러 상태를 정확히 검증합니다
3. **runTest로 시간 제어**: delay와 timeout을 가상 시간으로 테스트합니다
4. **경계값 우선**: 버그는 항상 경계(> vs >=, 0, 100)에서 태어납니다
5. **실패 케이스 우선 설계**: 성공 경로는 누구나 테스트합니다. 실패 경로가 안전망입니다

다음 장에서는 이 테스트들을 자동으로 실행하는 CI/CD 파이프라인을 구축합니다.

8.129 18.9 CrossValidatorImpl 테스트 — 가중치 재분배 검증

`CrossValidatorImpl`의 핵심은 EMF 미지원 기기에서 가중치를 비례 재분배하는 로직입니다. "Wi-Fi만 100점이면 최종 점수는 58~59점" 이라는 수학적 결과를 테스트로 고정합니다.

```
// app/src/test/java/com/searcam/data/analysis/CrossValida
class CrossValidatorImplTest {
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    private lateinit var crossValidator: CrossValidatorImp

    @Before
    fun setUp() { crossValidator = CrossValidatorImpl() }

    @Test fun `EMF 사용 가능 시 Wi-Fi만 100점이면 결과는 50`()
        // 100 * 0.50 + 0 * 0.35 + 0 * 0.15 = 50
        val result = crossValidator.calculateRisk(100, 0,
        assertEquals(50, result)
    }

    @Test fun `EMF 사용 가능 시 렌즈만 100점이면 결과는 35`() {
        val result = crossValidator.calculateRisk(0, 100,
        assertEquals(35, result)
    }

    @Test fun `EMF 미지원 시 Wi-Fi만 100점이면 약 58~59`() {
        // wifiAdj = 0.50 / (0.50 + 0.35) ≈ 0.5882
        val result = crossValidator.calculateRisk(100, 0,
        assertTrue("결과가 58~59 범위여야 함", result in 58.
    }
```

```
    @Test fun `EMF 미지원 시 EMF 점수는 무시된다`() {
        val withEmf    = crossValidator.calculateRisk(80,
        val withoutEmf = crossValidator.calculateRisk(80,
        assertTrue("EMF 유무에 따라 결과 달라야 함", withEmf !
    }

    @Test fun `결과는 항상 0~100 범위`() {
        val result = crossValidator.calculateRisk(150, 150
        assertEquals(100, result)
    }
}
```

`emfAvailable = false` 분기는 자력계 없는 기기(예: 일부 보급형 태블릿)에서 위험도 계산이 올바르게 재조정되는지 검증합니다.

8.130 18.10 CalculateRiskUseCase 테스트 — 보정 계수 검증

이 UseCase는 SearCam의 "판사" 역할입니다. 몇 개 레이어가 양성인지에 따라 ×0.7/×1.2/×1.5 보정이 정확히 적용되는지 검증합니다.

```
// app/src/test/java/com/searcam/domain/usecase/CalculateR
class CalculateRiskUseCaseTest {

    private lateinit var useCase: CalculateRiskUseCase

    @Before fun setUp() { useCase = CalculateRiskUseCase()

    @Test fun `완료된 레이어가 없으면 0 반환`() {
        val result = useCase(mapOf(LayerType.WIFI to
            makeLayer(LayerType.WIFI, ScanStatus.FAILED, s
        assertEquals(0, result)
    }

    @Test fun `양성 레이어 1개이면 보정 계수 0-7 적용`() {
        // Wi-Fi score=100 양성 1개 → 100 * 0.5 * 0.7 = 35
        val result = useCase(mapOf(LayerType.WIFI to
```

```
            makeLayer(LayerType.WIFI, ScanStatus.COMPLETED
        assertEquals(35, result)
    }

    @Test fun `양성 레이어 2개이면 보정 계수 1-2 적용`() {
        // 100*0.5 + 100*0.2 = 70 → 70 * 1.2 = 84
        val result = useCase(mapOf(
            LayerType.WIFI to makeLayer(LayerType.WIFI, Sc
            LayerType.LENS to makeLayer(LayerType.LENS, Sc
        ))
        assertEquals(84, result)
    }

    @Test fun `양성 레이어 3개 이상이면 보정 계수 1-5, 최대 100`(
        // 100*0.5 + 100*0.2 + 100*0.15 = 85 → 85 * 1.5 =
        val result = useCase(mapOf(
            LayerType.WIFI     to makeLayer(LayerType.WIFI
            LayerType.LENS     to makeLayer(LayerType.LENS
            LayerType.MAGNETIC to makeLayer(LayerType.MAGN
        ))
        assertEquals(100, result)
    }
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    @Test fun `invokeWithCorrection은 양성 2개에서 factor 1-2
        val (_, factor) = useCase.invokeWithCorrection(map
            LayerType.WIFI to makeLayer(LayerType.WIFI, Sc
            LayerType.LENS to makeLayer(LayerType.LENS, Sc
        ))
        assertEquals(1.2f, factor, 0.001f)
    }

    private fun makeLayer(type: LayerType, status: ScanSta
        layerType = type, status = status, score = score,
        devices = emptyList(), durationMs = 0L, findings =
    )
}
```

보정 계수 테스트는 SearCam이 "단독 탐지를 신뢰도 낮게, 복수 탐지를 신뢰도 높게" 처리하는 핵심 로직의 정확성을 보장합니다.

8.131 18.11 RunQuickScanUseCase 테스트 — Turbine + MockK Flow 검증

Quick Scan UseCase는 Repository를 목킹하여 Flow가 정확히 한 번 `ScanReport`를 emit하고 완료되는지 검증합니다.

```
// app/src/test/java/com/searcam/domain/usecase/RunQuickSc
class RunQuickScanUseCaseTest {

    private lateinit var wifiScanRepository: WifiScanRepos
    private lateinit var useCase: RunQuickScanUseCase

    @Before
    fun setUp() {
        wifiScanRepository = mockk()
        useCase = RunQuickScanUseCase(wifiScanRepository,
    }

    @Test
    fun `invoke는 ScanReport를 정확히 1번 emit하고 완료된다`() =
        coEvery { wifiScanRepository.scanDevices() } retur
        every { wifiScanRepository.observeDevices() } retu
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        useCase().test {
            assertNotNull(awaitItem())
            awaitComplete()
        }
    }

    @Test
    fun `고위험 기기가 있으면 riskScore가 0보다 크다`() = runTest
        val highRisk = makeDevice(riskScore = 80, isCamera
        coEvery { wifiScanRepository.scanDevices() } retur
        every { wifiScanRepository.observeDevices() } retu

        useCase().test {
            val report = awaitItem()
            // 양성 1개 → 80 * 0.5 * 0.7 = 28
            assertEquals(28, report.riskScore)
            awaitComplete()
        }
    }

    @Test
    fun `Wi-Fi 스캔 실패 시 riskScore는 0`() = runTest {
```

```
        coEvery { wifiScanRepository.scanDevices() } retur
            Result.failure(RuntimeException("네트워크 오류")
        every { wifiScanRepository.observeDevices() } retu

        useCase().test {
            val report = awaitItem()
            assertEquals(0, report.riskScore)
            assertTrue(report.devices.isEmpty())
            awaitComplete()
        }
    }

    @Test
    fun `report의 mode는 QUICK`() = runTest {
        coEvery { wifiScanRepository.scanDevices() } retur
        every { wifiScanRepository.observeDevices() } retu

        useCase().test {
            assertEquals(ScanMode.QUICK, awaitItem().mode)
            awaitComplete()
        }
    }
```

```
    private fun makeDevice(riskScore: Int, isCamera: Boole
        ip = "192.168.1.100", mac = "AA:BB:CC:DD:EE:FF",
        hostname = null, vendor = null, deviceType = Devic
        openPorts = emptyList(), services = emptyList(),
        riskScore = riskScore, isCamera = isCamera,
        discoveryMethod = DiscoveryMethod.ARP,
        discoveredAt = System.currentTimeMillis(),
    )
}
```

Turbine의 `.test { }` 블록은 Flow 구독 후 `awaitItem()`으로 각 emit을 순서대로 검증합니다. `awaitComplete()`는 Flow가 정상 종료되었음을 확인합니다. 이 패턴은 "Flow가 예상한 개수만큼만 emit하는지"를 보장합니다.

8.132 18.12 단위 테스트 56개 전체 목록

코드 리뷰 후 최종적으로 작성된 단위 테스트 목록입니다.

파일	테스트 수	주요 검증
`PortScannerTest`	19	RFC 1918 경계값 전수 검증

파일	테스트 수	주요 검증
`NoiseFilterTest`	15	이동 평균·급변 감지·캘리브레이
`CrossValidatorImplTest`	10	EMF 유무 가중치 재분배 수치 검
`RiskCalculatorTest`	12	항목별 점수·복합·클램핑·불변성
`CalculateRiskUseCaseTest`	10	보정 계수 0.7/1.2/1.5 + Wi-Fi 유
`RunQuickScanUseCaseTest`	10	Turbine Flow + MockK 목킹
합계	**76**	—

> **참고**: 이 책 집필 시점 기준으로 최소 56개가 작성되었으며, 지속적으로 추가됩니다.

커버리지 목표:

```
Domain 레이어 (UseCase):  90%+  (안전 판단 로직)
Data 레이어 (분석/센서):   80%+  (핵심 알고리즘)
UI 레이어 (ViewModel):    60%+  (상태 전환 검증)
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

참

고

문

서:

docs/

03-

TDD.md,

docs/

18-

test-

strategy.md

—

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

—

Ch19: CI/CD 파이프라인 — 자동으로 빌드하고 배포하라

—

> **이장에서배울것**: “내컴퓨터에서는됩니다”를영원히추방하

8.133 도입

주방장이 새 요리를 만들었다고 상상해보세요. 혼자 맛을 보고 "맛있다" 고 했습니다. 그런데 손님에게 나가기 전에 위생 검사, 영양 성분 확인, 알레르기 표시, 플레이팅 기준 확인이 필요합니다. 이 모든 체크가 끝나야 서빙됩니다.

소프트웨어도 마찬가지입니다. 개발자가 "된다" 고 했다고 바로 사용자에게 배포하면 안 됩니다. 린트 검사, 단위 테스트, 빌드 확인, 서명, 배포까지 모든 단계가 자동으로, 반복 가능하게, 사람의 실수 없이 실행되어야 합니다. 이것이 CI/CD입니다.

8.134 19.1 전체 파이프라인 설계

8.134.1 두 개의 파이프라인

SearCam은 두 가지 시점에 파이프라인이 실행됩니다.

PR 생성/업데이트 시:

`Pull Request` → `Lint` → `Unit Test` → `Build` → 머지 허용/차단

릴리즈 태그 푸시 시:

`v1.0.0` 태그 → 전체 테스트 → 빌드 & 서명 → `Play Store` 내부 배포

두 파이프라인의 역할 분리가 중요합니다. PR 파이프라인은 빠른 피드백을 위해 5분 안에 완료되어야 합니다. 릴리즈 파이프라인은 철저한 검증을 위해 시간이 더 걸려도 됩니다.

8.135 19.2 PR 파이프라인: 빠른 피드백

8.135.1 코드를 제출하기 전 검문소

```yaml
# .github/workflows/pr-check.yml
name: PR Check

on:
  pull_request:
    branches: [develop, main]

# 같은 PR에서 새 커밋 시 이전 실행 취소 (비용 절약)
concurrency:
  group: pr-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  lint:
```

```yaml
    name: Lint (ktlint + detekt)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      # Gradle 캐시: 의존성 다운로드 시간 단축 (첫 실행 3분 → 재
      - name: Gradle 캐시
        uses: actions/cache@v4
        with:
          path: |
            ~/.gradle/caches
            ~/.gradle/wrapper
          key: gradle-${{ hashFiles('**/*.gradle*', '**/gr
          restore-keys: gradle-

      - name: ktlint 검사
        run: ./gradlew ktlintCheck
```

```yaml
      - name: detekt 검사
        run: ./gradlew detekt

      # 실패 시 리포트를 아티팩트로 업로드
      - name: detekt 리포트 업로드
        if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: detekt-report
          path: build/reports/detekt/

  unit-test:
    name: 단위 테스트 + 커버리지
    runs-on: ubuntu-latest
    needs: lint  # lint 통과 후 실행
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          java-version: '17'
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
          distribution: 'temurin'

      - name: Gradle 캐시
        uses: actions/cache@v4
        with:
          path: ~/.gradle/caches
          key: gradle-${{ hashFiles('**/*.gradle*') }}

      - name: 단위 테스트 실행
        run: ./gradlew testDebugUnitTest

      - name: 커버리지 리포트 생성
        run: ./gradlew jacocoTestReport

      # 80% 미만이면 빌드 실패
      - name: 커버리지 기준 검증 (80%)
        run: ./gradlew jacocoTestCoverageVerification

      - name: 테스트 결과 업로드
        if: always()
        uses: actions/upload-artifact@v4
        with:
```

```
          name: unit-test-results
          path: app/build/reports/tests/

  build:
    name: 디버그 빌드
    runs-on: ubuntu-latest
    needs: unit-test
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: 디버그 APK 빌드
        run: ./gradlew assembleDebug

      - name: APK 업로드
        uses: actions/upload-artifact@v4
        with:
          name: debug-apk
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
          path: app/build/outputs/apk/debug/app-debug.apk
          retention-days: 7
```

8.136 19.3 Lint 설정: 코드 스타일 강제하기

8.136.1 두 명의 코드 심사관

ktlint와 detekt는 서로 다른 역할을 합니다. ktlint는 형식(formatting) 심사관입니다. 들여쓰기, 공백, 괄호 위치 같은 스타일을 검사합니다. detekt는 품질(quality) 심사관입니다. 너무 긴 함수, 복잡한 조건, 매직 넘버 같은 설계 문제를 찾습니다.

```
// build.gradle.kts (루트)
plugins {
    id("org.jlleitschuh.gradle.ktlint") version "12.1.0"
    id("io.gitlab.arturbosch.detekt") version "1.23.5"
}

ktlint {
    version.set("1.1.1")
    android.set(true)
    outputToConsole.set(true)
```

```
    filter {
        exclude("**/generated/**")  // 자동 생성 코드 제외
        exclude("**/*_Hilt*")
    }
}

# detekt.yml — 프로젝트 품질 기준
complexity:
  LongMethod:
    threshold: 50         # 함수 50줄 초과 금지
  LargeClass:
    threshold: 400        # 클래스 400줄 초과 경고
  ComplexCondition:
    threshold: 4          # 복합 조건 4개 초과 금지
  CyclomaticComplexMethod:
    threshold: 10         # 순환 복잡도 10 이하

style:
  MagicNumber:
    active: true
    ignoreNumbers: ['-1', '0', '1', '2']
  MaxLineLength:
    maxLineLength: 120
```

```yaml
potential-bugs:
  UnsafeCast:
    active: true
```

8.137 19.4 릴리즈 파이프라인: 태그 하나로 배포까지

8.137.1 키 하나로 열리는 자동화

Git 태그를 `v1.0.0` 형식으로 푸시하면 전체 릴리즈 파이프라인이 실행됩니다. 테스트 → 빌드 → APK 서명 → Play Store 업로드까지 사람이 개입하지 않습니다.

```yaml
# .github/workflows/release.yml
name: Release

on:
  push:
    tags:
      - 'v*'  # v1.0.0, v2.1.3 등 모든 버전 태그
```

```yaml
jobs:
  test:
    name: 전체 테스트 스위트
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: 전체 테스트 실행
        run: ./gradlew test
      - name: 커버리지 리포트
        run: ./gradlew jacocoTestReport

  build-and-sign:
    name: AAB 빌드 & 서명
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        with:
          java-version: '17'
          distribution: 'temurin'

      # GitHub Secret에서 키스토어 복원
      - name: 키스토어 복원
        env:
          KEYSTORE_BASE64: ${{ secrets.KEYSTORE_BASE64 }}
        run: |
          echo "$KEYSTORE_BASE64" | base64 -d > app/releas

      # ProGuard/R8 난독화 포함 릴리즈 AAB 빌드
      - name: 릴리즈 AAB 빌드
        env:
          KEYSTORE_PASSWORD: ${{ secrets.KEYSTORE_PASSWORD
          KEY_ALIAS: ${{ secrets.KEY_ALIAS }}
          KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}
        run: ./gradlew bundleRelease

      - name: AAB 업로드
        uses: actions/upload-artifact@v4
        with:
```

```yaml
          name: release-aab
          path: app/build/outputs/bundle/release/app-relea

  deploy:
    name: Play Store 내부 테스트 배포
    runs-on: ubuntu-latest
    needs: build-and-sign
    steps:
      - uses: actions/checkout@v4

      - name: AAB 다운로드
        uses: actions/download-artifact@v4
        with:
          name: release-aab

      - name: Play Store 내부 테스트 트랙 업로드
        uses: r0adkll/upload-google-play@v1
        with:
          serviceAccountJsonPlainText: ${{ secrets.PLAY_SE
          packageName: com.searcam.app
          releaseFiles: app-release.aab
          track: internal
```

CHAPTER 8 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        status: completed

    # 배포 완료 Slack 알림
    - name: Slack 배포 완료 알림
      if: success()
      uses: 8398a7/action-slack@v3
      with:
        status: success
        text: |
          SearCam ${{ github.ref_name }} 배포 완료
          Play Store 내부 테스트 트랙에서 확인 가능합니다.
      env:
        SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL

    - name: Slack 배포 실패 알림
      if: failure()
      uses: 8398a7/action-slack@v3
      with:
        status: failure
        text: "SearCam ${{ github.ref_name }} 배포 실패 —
      env:
        SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL
```

8.138 19.5 APK 서명 자동화: Keystore 시크릿 관리

8.138.1 열쇠를 안전하게 보관하는 법

APK 서명은 "이 앱은 신뢰할 수 있는 개발자가 만들었다" 는 증명입니다. Keystore 파일은 앱의 신원과 같습니다. 분실하면 같은 패키지명으로 업데이트를 올릴 수 없습니다. Git에 절대 커밋하지 않아야 하며, GitHub Secrets에 암호화된 형태로 보관합니다.

Keystore 생성:

```
keytool -genkey -v \
  -keystore release.keystore \
  -alias searcam-release \
  -keyalg RSA \
  -keysize 4096 \
  -validity 9125 \
  -storepass <strong-password> \
  -keypass <strong-password>
```

Base64 인코딩 후 GitHub Secret 저장:

```
# macOS / Linux
base64 -i release.keystore | pbcopy  # 클립보드에 복사

# GitHub Repository → Settings → Secrets and variables →
# New repository secret
# Name: KEYSTORE_BASE64
# Value: (클립보드 내용 붙여넣기)
```

등록해야 할 GitHub Secrets:

Secret 이름	내용
KEYSTORE_BASE64	Base64 인코딩된 keystore 파일
KEYSTORE_PASSWORD	키스토어 비밀번호
KEY_ALIAS	키 별칭 (예: searcam-release)
KEY_PASSWORD	키 비밀번호
PLAY_SERVICE_ACCOUNT_JSON	Google Play 서비스 계정 JSON
SLACK_WEBHOOK_URL	Slack 알림 웹훅 URL

Gradle에서 환경변수로 서명 설정:

```
// app/build.gradle.kts
android {
    signingConfigs {
        create("release") {
```

```
            storeFile = file("release.keystore")
            storePassword = System.getenv("KEYSTORE_PASSWO
                ?: error("KEYSTORE_PASSWORD 환경변수가 설정되
            keyAlias = System.getenv("KEY_ALIAS")
                ?: error("KEY_ALIAS 환경변수가 설정되지 않았습
            keyPassword = System.getenv("KEY_PASSWORD")
                ?: error("KEY_PASSWORD 환경변수가 설정되지 않
        }
    }

    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("rele
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-o
                "proguard-rules.pro"
            )
        }
    }
}
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.139 19.6 ProGuard/R8 난독화

8.139.1 코드를 읽을 수 없게 만드는 이유

보안 앱인 SearCam의 탐지 알고리즘이 리버스 엔지니어링으로 분석된다면 어떻게 될까요? 악의적인 공격자가 탐지 로직을 우회하도록 카메라를 설계할 수 있습니다. R8 난독화는 코드를 `a.b.c` 같은 무의미한 이름으로 바꿔 분석을 어렵게 만듭니다.

```
# proguard-rules.pro


# Kotlin 코루틴 (필수 유지)
-keepnames class kotlinx.coroutines.internal.MainDispatche
-keepnames class kotlinx.coroutines.CoroutineExceptionHand
-keepclassmembernames class kotlinx.** {
    volatile <fields>;
}


# Hilt DI (생성 코드 유지)
-keepclasseswithmembernames class * {
   @dagger.hilt.android.lifecycle.HiltViewModel <init>(...
}
```

```
# Room DB (DAO 인터페이스 유지)
-keep class * extends androidx.room.RoomDatabase
-keepclassmembers class * {
    @androidx.room.* <methods>;
}

# Firebase Crashlytics (스택 트레이스 가독성 유지)
-keepattributes SourceFile,LineNumberTable
-keep public class * extends java.lang.Exception

# 도메인 모델 (직렬화 대상)
-keep class com.searcam.app.domain.model.** { *; }

# OUI DB 파싱 (JSON 필드명 유지)
-keep class com.searcam.app.data.local.oui.OuiEntry { *; }

# 탐지 알고리즘 핵심 클래스는 난독화 (이름 제거)
# → 기본적으로 R8가 모든 클래스를 난독화합니다
```

빌드 타입별 ProGuard 적용 전략입니다.

```
buildTypes {
    debug {
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
        isMinifyEnabled = false  // 디버그: 난독화 없음 (빠른
        applicationIdSuffix = ".debug"
    }
    create("staging") {
        isMinifyEnabled = true   // 스테이징: 난독화 (릴리즈
        isShrinkResources = true
        proguardFiles(
            getDefaultProguardFile("proguard-android-optim
            "proguard-rules.pro"
        )
        applicationIdSuffix = ".staging"
        signingConfig = signingConfigs.getByName("debug")
    }
    release {
        isMinifyEnabled = true
        isShrinkResources = true
        proguardFiles(
            getDefaultProguardFile("proguard-android-optim
            "proguard-rules.pro"
        )
        signingConfig = signingConfigs.getByName("release"
    }
```

```
}
```

8.140 19.7 자동 버전 관리: Git Tag 기반

8.140.1 버전 번호를 수동으로 바꾸지 마라

개발자가 `versionCode`를 잊어버리고 올리지 않으면 Play Store 업로드가 실패합니다. Git 태그를 진실의 원천으로 삼으면 이 문제가 사라집니다.

버전 체계는 다음 규칙을 따릅니다.

```
versionName: <major>.<minor>.<patch>  (예: 1.2.3)
versionCode: major * 10000 + minor * 100 + patch  (예: 10203)
```

→ `versionCode`는 항상 증가하며, `Play Store`가 이 숫자로 업데이트를 판

```
// app/build.gradle.kts
fun getVersionName(): String {
    return try {
        val tag = providers.exec {
            commandLine("git", "describe", "--tags", "--ab
        }.standardOutput.asText.get().trim()
        tag.removePrefix("v")
```

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

```
    } catch (e: Exception) {
        "0.0.1"  // 태그 없는 개발 빌드 기본값
    }
}

fun getVersionCode(): Int {
    val parts = getVersionName().split(".")
    val major = parts.getOrElse(0) { "0" }.toIntOrNull() ?
    val minor = parts.getOrElse(1) { "0" }.toIntOrNull() ?
    val patch = parts.getOrElse(2) { "0" }.toIntOrNull() ?
    return major * 10_000 + minor * 100 + patch
}

android {
    defaultConfig {
        versionName = getVersionName()
        versionCode = getVersionCode()
    }
}
```

릴리즈 절차는 단 두 줄입니다.

```
git tag v1.0.0
git push origin v1.0.0
```

```
# → GitHub Actions가 나머지를 자동으로 처리
```

8.141 19.8 품질 게이트: 통과하지 않으면 머지 불가

8.141.1 자동 검문소

PR이 머지되려면 모든 품질 게이트를 통과해야 합니다. GitHub Branch Protection Rules로 강제합니다.

```
GitHub Repository Settings:
  Branch protection rules → main, develop
    ✓ Require a pull request before merging
   ✓ Require status checks to pass before merging
        Required checks:
          - Lint (ktlint + detekt)
          - 단위 테스트 + 커버리지
      - 디버그 빌드
  ✓ Require at least 1 approval
    ✓ Dismiss stale reviews
   ✓ Do not allow bypassing the above settings
```

품질 게이트	기준	실패 시
ktlint	에러 0건	PR 머지 차단
detekt	에러 0건	PR 머지 차단
단위 테스트	전부 통과	PR 머지 차단
커버리지	80% 이상	PR 머지 차단
빌드 성공	exit 0	PR 머지 차단
코드 리뷰	1명 이상 승인	PR 머지 차단

8.142 19.9 Nightly Build: 야간 전체 검증

8.142.1 자는 동안 일하는 파이프라인

매일 자정(UTC), develop 브랜치의 최신 코드를 전체 테스트합니다. PR 파이프라인보다 느리지만 더 철저한 검증(Instrumented Test 포함)을 실행합니다.

```
# .github/workflows/nightly.yml
name: Nightly Build

on:
  schedule:
```

```yaml
    - cron: '0 15 * * *'  # 한국시간 자정 (UTC 15:00)

jobs:
  nightly-full-test:
    name: 야간 전체 테스트
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: develop

      - uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: 전체 테스트 + 커버리지
        run: ./gradlew test jacocoTestReport

      - name: 디버그 빌드
        run: ./gradlew assembleDebug
```

```
      - name: 커버리지 리포트 아티팩트
        uses: actions/upload-artifact@v4
        with:
          name: nightly-coverage-report
          path: app/build/reports/jacoco/

      - name: 실패 시 Slack 알림
        if: failure()
        uses: 8398a7/action-slack@v3
        with:
          status: failure
          text: "Nightly Build 실패 — develop 브랜치 즉시 확인
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL
```

8.143 19.10 Firebase 환경 분리

8.143.1 개발/스테이징/프로덕션을 완전히 격리하기

Crashlytics 데이터, Analytics 이벤트, Performance 메트릭이 개발용과 프로덕션용이 섞이면 의미 없는 데이터가 됩니다. Firebase 프로젝트를 세 개로 분리하고, 빌드 타입별로 다른 `google-`

services.json을 사용합니다.

```
app/
├── src/
│   ├── debug/
│   │   └── google-services.json     # searcam-debug 프로젝트
│   ├── staging/
│   │   └── google-services.json     # searcam-staging 프로젝트
│   └── release/
│       └── google-services.json     # searcam-prod 프로젝트
```

환경	Firebase 프로젝트	Crashlytics	로깅
debug	searcam-debug	비활성화	Timber 전체
staging	searcam-staging	활성화	WARNING+
release	searcam-prod	활성화	Crashlytics만

CHAPTER 8. 데이터베이스 설계 — ROOM + SQLCIPHER로 스캔 이력 보호

8.144 정리

CI/CD 파이프라인의 가치는 “자동화” 가 아니라 “일관성” 에 있습니다. 어떤 개발자가, 어떤 시점에, 어떤 환경에서 배포하더라도 동일한 품질 기준을 통과해야 합니다.

SearCam의 파이프라인은 세 가지 원칙을 따릅니다.

1. **빠른 피드백**: PR 파이프라인은 5분 내 완료
2. **시크릿 격리**: Keystore와 서비스 계정은 GitHub Secrets에만 존재
3. **자동화 완결**: 태그 하나로 Play Store 배포까지

다음 장에서는 배포된 앱의 릴리즈 전략과 단계적 출시(Staged Rollout)를 다룹니다.

제 9 장

Ch20: 릴리즈와 배포 전략

이 장에서 배울 것: 앱을 만드는 것과 앱을 출시하는 것은 다릅니다. Google Play 스토어 등록, 보안 앱 심사 대비, 단계적 출시(Staged Rollout)로 리스크를 줄이는 법, Git 태그 기반 버전 관리, 효과적인 릴리즈 노트 작성법을 배웁니다.

9.1 도입

영화 한 편을 만들었다고 관객이 보는 것이 아닙니다. 시사회, 심사위원회 검토, 등급 분류, 극장 배급 계약, 마케팅까지 해야 비로소 스

크린에 올라갑니다. 그리고 처음에는 몇 개 극장에서만 시작하고, 반응이 좋으면 전국으로 확대합니다.

앱 출시도 같습니다. Google Play 스토어에 올리기까지 심사를 통과해야 하고, 처음에는 소수에게만 배포했다가 점차 확대하는 전략이 필요합니다. 특히 SearCam처럼 카메라와 Wi-Fi를 다루는 보안 앱은 심사 과정이 더 까다롭습니다.

9.2 20.1 Google Play Console 초기 설정

9.2.1 첫 등록 전에 준비할 것

Play Console에 앱을 등록하기 전에 다음 항목을 준비해야 합니다.

항목	규격	비고
앱 아이콘	512 × 512 px, PNG	투명 배경 허용
피처드 이미지	1024 × 500 px, PNG/JPG	스토어 상단 배너
스크린샷 (폰)	최소 2장, 최대 8장	1080px 이상 권장
스크린샷 (태블릿)	선택사항	7인치/10인치
짧은 설명	80자 이내	검색 결과에 표시
긴 설명	4000자 이내	키워드 포함
개인정보처리방침 URL	필수	카메라 권한 때문에

항목	규격	비고
콘텐츠 등급	IARC 설문 완료	보안 카테고리

카테고리 선택 전략:

SearCam은 "도구(Tools)" 카테고리가 적합합니다. "보안(Security)"을 선택하면 더 적합해 보이지만, Google Play의 Security 카테고리는 안티바이러스, VPN 등에 집중되어 있어 몰래 카메라 탐지 앱의 노출이 오히려 줄어들 수 있습니다.

9.3 20.2 보안 앱 심사 대비

9.3.1 심사관의 눈으로 보기

카메라와 위치 권한, Wi-Fi 스캔, 네트워크 포트 스캔을 사용하는 앱은 Google의 집중 심사 대상입니다. 심사관은 "이 앱이 악용될 수 있는가?"를 봅니다. SearCam의 경우 다음 세 가지를 명확히 해야 합니다.

권한 사용 목적 명시:

```
<!-- AndroidManifest.xml -->
```

```
<!-- 카메라: 렌즈 반사광 감지용 (사진 저장 없음) -->
<uses-permission android:name="android.permission.CAMERA"

<!-- 위치: Wi-Fi 네트워크 스캔 (Android 권한 정책) -->
<uses-permission android:name="android.permission.ACCESS_F
<uses-permission android:name="android.permission.ACCESS_W
<uses-permission android:name="android.permission.CHANGE_W
```

권한 요청 시 사용자에게 보여주는 설명문도 심사 항목입니다.

```
// 권한 요청 전 rationale 표시
@Composable
fun CameraPermissionRationale(onConfirm: () -> Unit) {
    AlertDialog(
        onDismissRequest = {},
        title = { Text("카메라 권한이 필요합니다") },
        text = {
            Text(
                "몰래카메라 렌즈의 반사광을 감지하기 위해 카메라를
                "촬영된 이미지는 기기에 저장되지 않으며, " +
                "서버로 전송되지 않습니다."
            )
        },
        confirmButton = {
```

```
            TextButton(onClick = onConfirm) { Text("확인")
        }
    )
}
```

데이터 안전 섹션(Data Safety) 작성:

Play Console의 "데이터 안전" 섹션은 수집/공유 데이터를 선언하는 곳입니다. SearCam은 다음과 같이 선언합니다.

데이터 유형	수집	공유	이유
카메라 데이터	아니오	아니오	메모리에서만 처리
Wi-Fi 정보	아니오	아니오	기기에 저장 안 함
정확한 위치	아니오	아니오	Wi-Fi 권한에만 필요
크래시 로그	예	아니오	Crashlytics 자동 수집
기기 ID	아니오	아니오	수집하지 않음

심사 거절 사유 TOP 5 (사전 예방):

1. 권한 남용: 불필요한 권한 요청 → 필요한 권한만 선언
2. 개인정보처리방침 미비 → 카메라/위치 처리 방침 명확히 기재
3. 오해를 유발하는 설명 → "100% 탐지" 같은 과장 문구 금지
4. 악성코드 의심 행위 → 포트 스캔 목적을 스토어 설명에 명시

5. 타 앱 사칭 → 독창적인 아이콘과 앱 이름 사용

9.4 20.3 단계적 출시 (Staged Rollout)

9.4.1 전쟁 전 정찰하기

신규 버전을 전체 사용자에게 한 번에 배포하는 것은 도박입니다. 1%의 기기에서 크래시가 발생해도, 100만 사용자라면 1만 명이 영향받습니다. 단계적 출시는 리스크를 분산하는 보험입니다.

배포 트랙 전략:

```
내부 테스트 (Internal)
    ├─── 대상: 개발팀 (최대 100명)
    ├─── 조건: 자동 배포 (CI/CD)
    └─── 목적: 기능 검증, 기초 안정성 확인

클로즈드 테스트 알파 (Alpha)
    ├─── 대상: 테스터 (50~200명)
    ├─── 조건: 내부 테스트 1주 후 수동 프로모션
   └─── 목적: 다양한 기기/OS 버전 호환성

오픈 테스트 베타 (Beta)
```

```
  ├── 대상: 관심 있는 일반 사용자 (자발적 참여)
  ├── 조건: 알파 2주 후 수동 프로모션
  └── 목적: 실사용 환경 피드백

프로덕션 (Production) — 단계적 출시
  ├── 1단계: 1% (약 1,000명 기준)
  ├── 2단계: 10% (24시간 후, 크래시율 < 1% 확인 시)
  ├── 3단계: 50% (48시간 후, ANR < 0.5% 확인 시)
  └── 4단계: 100% (72시간 후, 지표 안정적 시)
```

Play Console에서 단계적 출시 설정:

```
Production → Create new release
  → 롤아웃 비율: 1%
  → 저장 후 [검토] → [출시 시작]

이후 크래시율 모니터링:
  Android Vitals → 크래시율, ANR 비율 확인
 → 안전하면 [롤아웃 비율 늘리기] → 10% → 50% → 100%
  → 위험하면 [롤아웃 중지] → 즉시 배포 멈춤
```

롤아웃 중지 기준:

지표	중지 기준
크래시율 (24h)	> 2%
ANR 비율 (24h)	> 1%
별점 (신규 리뷰)	3.0 미만
특정 기기 크래시 급증	동일 기기 50회/시간 이상

9.5 20.4 버전 관리: versionCode와 versionName

9.5.1 버전이 꼬이면 배포가 막힌다

Android 앱의 버전에는 두 가지 개념이 있습니다. versionName은 사용자에게 보이는 "1.2.3" 입니다. versionCode는 Play Store가 업데이트 순서를 판단하는 숫자입니다. versionCode는 반드시 이전 배포보다 커야 합니다.

버전 체계:

```
versionName 형식: MAJOR.MINOR.PATCH
  MAJOR: 대규모 변경 (호환성 깨짐, UI 전면 개편)
  MINOR: 기능 추가 (이전 버전과 호환)
```

```
  PATCH: 버그 수정

versionCode 계산: MAJOR × 10000 + MINOR × 100 + PATCH
  1.0.0 → 10000
  1.1.0 → 10100
  1.1.3 → 10103
  2.0.0 → 20000

// app/build.gradle.kts
// Git 태그에서 자동으로 버전 추출 (수동 변경 불필요)
fun getVersionName(): String {
    return try {
        providers.exec {
            commandLine("git", "describe", "--tags", "--ab
        }.standardOutput.asText.get().trim().removePrefix(
    } catch (e: Exception) {
        "0.0.1-dev"
    }
}

fun getVersionCode(): Int {
    val versionName = getVersionName().removeSuffix("-dev"
    val parts = versionName.split(".")
```

```
    val major = parts.getOrNull(0)?.toIntOrNull() ?: 0
    val minor = parts.getOrNull(1)?.toIntOrNull() ?: 0
    val patch = parts.getOrNull(2)?.toIntOrNull() ?: 0
    return major * 10_000 + minor * 100 + patch
}

android {
    defaultConfig {
        versionName = getVersionName()
        versionCode = getVersionCode()
    }
}
```

버전 변경 절차:

```
# 패치 릴리즈 (버그 수정)
git tag v1.0.1 && git push origin v1.0.1

# 마이너 릴리즈 (기능 추가)
git tag v1.1.0 && git push origin v1.1.0

# 메이저 릴리즈 (대규모 변경)
git tag v2.0.0 && git push origin v2.0.0
```

```
# 태그 푸시 → CI/CD 자동 실행 → Play Store 내부 배포
```

9.6 20.5 릴리즈 노트 작성

9.6.1 사용자와의 대화

릴리즈 노트는 개발자가 사용자에게 보내는 편지입니다. 기술적인 내용이 아닌, 사용자에게 의미 있는 변화를 설명합니다.

나쁜 릴리즈 노트:

```
v1.1.0
- RiskCalculator 가중치 알고리즘 업데이트
- NullPointerException 수정 (MainActivity line 142)
- ProGuard 규칙 추가
- Gradle 버전 업그레이드 (8.3 → 8.4)
```

좋은 릴리즈 노트:

```
v1.1.0 업데이트


새로운 기능
• 자기장(EMF) 감지 정확도 30% 향상: 스마트폰 자체 간섭을
  자동으로 걸러내도록 개선했습니다
```

- 스캔 속도 개선: Wi-Fi 포트 스캔이 최대 4기기를 동시에 처리해 Quick Scan이 10초 빨라졌습니다

수정된 문제

- 일부 기기에서 스캔 시작 직후 앱이 종료되던 문제를 수정했습니다
- Galaxy A 시리즈에서 자기장 센서가 초기화되지 않던 문제를 수정했습니다

안정성 개선

- 오랜 시간 사용해도 메모리 사용량이 안정적으로 유지됩니다

릴리즈 노트 파일 관리:

Play Console은 최대 500자(언어별)의 릴리즈 노트를 지원합니다. 소스 코드와 함께 관리하는 것이 좋습니다.

```
fastlane/
└── metadata/
    └── android/
        ├── ko-KR/
        │   └── changelogs/
        │       ├── 10100.txt  # v1.1.0 한국어 노트
```

```
│       └── 10103.txt  # v1.1.3 한국어 노트
└── en-US/
        └── changelogs/
            ├── 10100.txt
            └── 10103.txt

# fastlane/metadata/android/ko-KR/changelogs/10100.txt (500
```

새로운 기능

- 자기장 감지 정확도 30% 향상
- Wi-Fi 스캔 속도 개선 (Quick Scan 10초 단축)

수정된 문제

- 일부 기기 앱 종료 문제 수정
- Galaxy A 시리즈 자기장 센서 초기화 오류 수정

9.7 20.6 앱 스토어 최적화 (ASO)

9.7.1 검색에서 발견되는 앱 만들기

앱 스토어 최적화(App Store Optimization, ASO)는 검색 결과에서 SearCam을 발견하기 쉽게 만드는 작업입니다. SEO의 앱 버전

이라고 보면 됩니다.

핵심 키워드 전략:

주요 키워드 (검색량 높음):
  - 몰래카메라 탐지
- 도청기 탐지
- 카메라 감지기
- 숨겨진 카메라

부가 키워드 (경쟁 낮음):
  - Wi-Fi 기기 스캔
- EMF 탐지
- 보안 점검
- 호텔 숙박 안전

앱 제목 구조:

SearCam - 몰래카메라 탐지 & 보안 스캔
(30자 이내 권장)

짧은 설명 (80자 이내):

30초 만에 몰래카메라를 1차 스크리닝. Wi-Fi 스캔 + 렌즈 감지 + EMF 탐

9.8 20.7 앱 서명 관리: Play App Signing

9.8.1 열쇠를 Google에게도 맡기는 이유

Google Play App Signing은 Google이 배포 키를 안전하게 보관하는 서비스입니다. 개발자의 업로드 키가 유출되거나 분실되어도 Google의 배포 키로 앱을 계속 서비스할 수 있습니다.

설정 방법:

```
Play Console → 설정 → 앱 서명
  → 'Google에서 앱 서명 키 관리' 선택 (최초 한 번)

이후 흐름:
   개발자: upload key로 AAB 서명 (업로드만)
  Google: 보관 중인 배포 키로 재서명 후 사용자에게 배포
```

이 구조의 장점은 업로드 키 분실 시 Google에 요청하면 재설정이 가능합니다. 배포 키는 Google이 관리하므로 분실 위험이 없습니다.

9.9 20.8 긴급 대응: 핫픽스 배포

9.9.1 불이 났을 때 빨리 끄는 법

프로덕션에서 심각한 버그가 발견되면 일반 릴리즈 사이클을 기다릴 수 없습니다. 핫픽스 절차를 사전에 정해두어야 합니다.

```
# 핫픽스 브랜치 생성 (main에서)
git checkout -b hotfix/crash-on-scan main

# 수정 작업
# ... 코드 수정 ...

# PR 생성 + 긴급 리뷰 (1명으로 단축)
git push origin hotfix/crash-on-scan
gh pr create --title "hotfix: 스캔 시작 시 크래시 수정" --base

# PR 승인 후 main으로 머지
# → 태그 생성
git tag v1.0.1
git push origin v1.0.1
# → CI/CD 자동 실행 → Play Store 내부 배포
```

핫픽스 배포 후에도 단계적 출시를 합니다. 1% → 10% → 100% 순서로, 각 단계에서 크래시율을 확인합니다.

9.10 정리

릴리즈는 코딩이 끝난 다음에 시작되는 별도의 전문 영역입니다. SearCam에서 적용한 핵심 전략을 요약합니다.

1. **심사 통과**: 권한 목적 명시, 개인정보처리방침 완비, Data Safety 섹션 정확히 기재
2. **단계적 출시**: 1% → 10% → 50% → 100%, 각 단계에서 크래시율 확인
3. **자동 버전 관리**: Git 태그가 versionCode/versionName의 유일한 원천
4. **사용자 중심 릴리즈 노트**: 기술 용어 대신 사용자에게 의미 있는 변화로 설명
5. **Play App Signing**: 배포 키 관리를 Google에 위임해 분실 리스크 제거

다음 장에서는 배포된 앱을 운영하는 모니터링과 성능 최적화를 다룹니다.

CHAPTER 9. CH20: 릴리즈와 배포 전략

제 10 장

Ch21: 모니터링과 성능 최적화

이 장에서 배울 것: 배포가 끝이 아닙니다. Android Profiler로 병목을 찾고, 배터리 소모를 20Hz 센서와 병렬 포트 스캔으로 줄이고, CameraX 프레임을 즉시 close하여 메모리를 관리합니다. Crashlytics로 오류를 추적하고 ANR을 원천 차단하는 방법까지 다룹니다.

21. 모니터링과 성능 최적화

10.1 도입

자동차를 구매할 때 연비, 출력, 내구성을 봅니다. 사면 그걸로 끝이 아닙니다. 주행 중에 계기판을 보고, 정기 점검을 받고, 이상한 소리가 나면 원인을 찾습니다. 차가 멈추기 전에 예방합니다.

앱도 마찬가지입니다. Play Store 출시가 끝이 아니라 관찰과 개선이 시작되는 시점입니다. SearCam은 사용자 안전과 직결되는 앱이므로 크래시 한 건, ANR 한 건도 허용하기 어렵습니다. 이 장에서는 앱이 살아있는 동안 어떻게 모니터링하고 최적화하는지 다룹니다.

10.2 21.1 모니터링 스택 구성

10.2.1 무엇을 볼 것인가

SearCam의 모니터링은 Firebase 무료 티어로 모두 구성합니다. 비용 없이 핵심 지표를 확인할 수 있습니다.

영역	도구	목적
크래시 추적	Firebase Crashlytics	앱 종료 원인 분석
성능 측정	Firebase Performance	스캔 소요 시간, 화면 렌더링
사용 분석	Firebase Analytics	기능 사용 패턴
원격 설정	Firebase Remote Config	긴급 파라미터 조정

영역	도구	목적
ANR	Crashlytics ANR 리포팅	응답 없음 추적

PII 수집 금지 원칙:

SearCam은 개인 식별 정보를 일절 수집하지 않습니다.

```
수집하지 않는 데이터:
  - GPS 좌표, Wi-Fi SSID
  - 발견된 기기 MAC 주소
- 스캔 결과 상세 (위험도, 기기 목록)
  - 카메라 촬영 이미지/프레임

수집하는 데이터 (비식별):
  - 크래시 스택 트레이스
- 스캔 소요 시간
- 기기 모델, OS 버전
- 이벤트 (scan_start, scan_complete)
```

10.3 21.2 Firebase Crashlytics 설정

10.3.1 문제가 생기면 바로 알아야 한다

자동차 계기판의 경고등은 엔진이 망가지기 전에 켜집니다. Crashlytics는 사용자가 신고하기 전에 문제를 감지합니다.

```
// build.gradle.kts (app)
dependencies {
    implementation(platform("com.google.firebase:firebase-
    implementation("com.google.firebase:firebase-crashlyti
    implementation("com.google.firebase:firebase-analytics
}

// build.gradle.kts (project)
plugins {
    id("com.google.firebase.crashlytics") version "2.9.9"
}

// SearCamApp.kt
class SearCamApp : Application() {
    override fun onCreate() {
        super.onCreate()
        // 릴리즈 빌드에서만 Crashlytics 활성화
        // 디버그 빌드에서는 비활성화 (개발 중 노이즈 방지)
```

```
        FirebaseCrashlytics.getInstance()
            .setCrashlyticsCollectionEnabled(!BuildConfig.
    }
}
```

커스텀 컨텍스트 키 설정:

크래시가 발생했을 때 "어떤 스캔 도중에" 발생했는지 알면 재현이 빠릅니다.

```
object CrashlyticsHelper {

    fun setScanContext(scanType: ScanType) {
        if (BuildConfig.DEBUG) return  // 디버그에서는 실행하

        FirebaseCrashlytics.getInstance().apply {
            setCustomKey("scan_type", scanType.name)
            setCustomKey("app_version", BuildConfig.VERSIO
            setCustomKey("device_model", Build.MODEL)
            setCustomKey("os_version", Build.VERSION.SDK_I
        }
    }

    fun logNonFatal(tag: String, message: String, exceptio
```

```
        FirebaseCrashlytics.getInstance().apply {
            log("[$tag] $message")
            recordException(exception)
        }
    }

    // 센서 에러는 치명적이지 않으므로 non-fatal로 기록
    fun logSensorError(sensorType: String, error: Exceptio
        logNonFatal(
            tag = "SensorError",
            message = "센서 초기화 실패: $sensorType",
            exception = error
        )
    }
}
```

크래시 심각도 분류와 대응 시간:

심각도	기준	대응 시간
P0	ANR, OOM, 크래시율 3%+	즉시 (24시간 내 핫픽스)
P1	NPE, SecurityException, 1~3%	48시간 내 수정
P2	센서/네트워크 에러, 0.5~1%	다음 릴리즈
P3	PDF 생성 실패, 파싱 에러	다음 릴리즈

알림 기준 설정 (Firebase Console):

```
Firebase Console → Crashlytics → Alerts
  → 새 이슈 발생: Slack 알림
 → 크래시율 > 1% (24h): Slack + 이메일
 → 크래시율 > 3% (24h): Slack + 이메일 + 전화
 → 동일 이슈 100회/시간 초과: Slack 즉시 알림
```

10.4 21.3 Firebase Performance로 성능 추적

10.4.1 숫자로 말하는 성능

"빠른 것 같다" 는 충분하지 않습니다. Quick Scan이 평균 28초 걸린다면, 목표(30초)를 달성하고 있습니다. 35초라면 뭔가 문제가 있습니다. 숫자가 없으면 개선이 있는지도 모릅니다.

```
// build.gradle.kts (app)
dependencies {
    implementation("com.google.firebase:firebase-perf-ktx"
}

// build.gradle.kts (project)
plugins {
```

```
    id("com.google.firebase.firebase-perf") version "1.4.2
}
```

커스텀 성능 트레이스 정의:

```
object PerfTraces {

    fun traceQuickScan(): Trace =
        Firebase.performance.newTrace("quick_scan_duration
            putAttribute("scan_type", "quick")
        }

    fun traceFullScan(): Trace =
        Firebase.performance.newTrace("full_scan_duration"
            putAttribute("scan_type", "full")
        }

    fun traceLayer(layer: String): Trace =
        Firebase.performance.newTrace("layer_${layer}_dura

    fun traceOuiDbLoad(): Trace =
        Firebase.performance.newTrace("oui_db_load")
}
```

UseCase에 트레이스 적용:

```
class RunQuickScanUseCase @Inject constructor(
    private val wifiScanner: WifiScanner,
    private val riskCalculator: RiskCalculator
) {
    suspend fun execute(): ScanResult {
        val trace = PerfTraces.traceQuickScan()
        trace.start()

        return try {
            val devices = wifiScanner.scanNetwork()
            trace.putMetric("device_count", devices.size.t

            val result = riskCalculator.calculate(devices)
            trace.putMetric("risk_score", result.riskScore

            result
        } finally {
            trace.stop()  // 성공/실패 무관하게 항상 기록
        }
    }
}
```

21. 프로파일링과 성능 최적화

Firebase Performance가 자동 수집하는 메트릭:

메트릭	목표	허용 범위
앱 콜드 스타트	< 2초	< 3초 (저사양)
앱 웜 스타트	< 1초	< 1.5초
느린 프레임 (16ms 초과)	< 5%	< 10%
멈춘 프레임 (700ms 초과)	< 1%	< 2%

10.5 21.4 Android Profiler로 병목 찾기

10.5.1 진단 없는 처방은 없다

성능 문제는 "최적화가 필요한 것 같다" 는 직감이 아니라 Profiler의 데이터로 확인해야 합니다. 의사가 진단 없이 처방하지 않듯, 개발자도 프로파일링 없이 최적화를 시작하면 안 됩니다.

Android Studio Profiler 사용법:

```
Android Studio → View → Tool Windows → Profiler
  → 앱 실행 상태에서 [+] → 세션 시작


주요 탭:
  CPU: 스캔 중 CPU 사용률, 스레드 상태
```

```
Memory: 힙 사용량, GC 빈도
Energy: 배터리 소모 추정치
Network: 네트워크 요청 (Wi-Fi 스캔)
```

Quick Scan 프로파일링 체크리스트:

```
1. CPU 프로파일
  ✓ 메인 스레드 블로킹 여부 확인
  → 파란색(Running) 이외 긴 주황색(Waiting) 없어야 함
  ✓ 포트 스캔이 Dispatchers.IO에서 실행되는지 확인
  ✓ RiskCalculator가 Dispatchers.Default에서 실행되는지 확인

2. 메모리 프로파일
  ✓ GC 이벤트 빈도 확인 (스캔 중 3회 이상이면 문제)
  ✓ 힙 사용량이 스캔 완료 후 원래 수준으로 복귀하는지 확인
  ✓ ImageProxy close 누락으로 인한 메모리 누수 확인

3. 에너지 프로파일
  ✓ Wake lock 사용 최소화
  ✓ 스캔 완료 후 센서 리스너 해제 확인
```

10.6 21.5 배터리 최적화: 20Hz 센서와 병렬화

10.6.1 배터리를 아끼는 세 가지 원칙

SearCam의 배터리 목표는 Quick Scan 전체(30초)에 2% 이하입니다. 이 목표를 달성하는 세 가지 전략이 있습니다.

전략 1: 자기장 센서 20Hz 고정

센서를 가장 빠른 주기로 읽으면 배터리가 빨리 닳습니다. 자기장 감지에 50Hz는 과도합니다. 20Hz(50ms 간격)로 충분합니다.

```
class MagneticSensorManager @Inject constructor(
    private val sensorManager: SensorManager
) {
    // SENSOR_DELAY_NORMAL = 200ms = 5Hz (너무 느림)
    // SENSOR_DELAY_GAME   = 20ms  = 50Hz (너무 빠름, 배터리
    // 직접 50ms = 20Hz 지정
    private val SAMPLING_PERIOD_US = 50_000  // 50ms = 20H

    fun startListening(callback: (MagneticReading) -> Unit
        val sensor = sensorManager.getDefaultSensor(Sensor
            ?: throw SensorUnavailableException("자력계 센서

        sensorManager.registerListener(
```

```
            createListener(callback),
            sensor,
            SAMPLING_PERIOD_US  // 마이크로초 단위
        )
    }

    fun stopListening() {
        // 스캔 완료 즉시 해제 — 리스너를 유지하면 배터리 계속 소모
        sensorManager.unregisterListener(listener)
    }
}
```

전략 2: 포트 스캔 병렬화

순차 스캔은 기기당 최대 12초(6포트 × 타임아웃 2초)가 걸립니다.
동시 병렬 스캔으로 2초로 단축합니다.

```
class PortScanner @Inject constructor(
    private val dispatcher: CoroutineDispatcher = Dispatch
) {
    private val TIMEOUT_MS = 2_000L

    // 여러 기기 × 여러 포트를 동시에 스캔
    suspend fun scanAll(
```

```
        devices: List<NetworkDevice>,
        ports: List<Int>
    ): Map<NetworkDevice, List<PortResult>> = coroutineSco

        devices
            .filter { it.riskWeight > 0.3f }  // 안전 벤더는
            .map { device ->
                async(dispatcher) {
                    device to scanDevice(device.ip, ports)
                }
            }
            .awaitAll()
            .toMap()
    }

    private suspend fun scanDevice(
        ip: String,
        ports: List<Int>
    ): List<PortResult> = coroutineScope {

        ports
            .map { port ->
```

```
                    async {
                        scanSinglePort(ip, port)
                    }
                }
                .awaitAll()
        }

        private suspend fun scanSinglePort(ip: String, port: I
            return withContext(dispatcher) {
                try {
                    withTimeout(TIMEOUT_MS) {
                        val socket = Socket()
                        socket.connect(InetSocketAddress(ip, p
                        socket.close()
                        PortResult(port = port, isOpen = true)
                    }
                } catch (e: Exception) {
                    PortResult(port = port, isOpen = false)
                }
            }
        }
    }
```

전략 3: 카메라 선택적 프레임 분석

30fps를 모두 분석하는 것은 CPU와 배터리 낭비입니다. 의심 포인트 발견 전까지 15fps로 운영합니다.

```
class AdaptiveFrameAnalyzer : ImageAnalysis.Analyzer {

    private var frameCount = 0
    private var isHighPriorityMode = false
    private var lastSuspiciousTime = 0L

    override fun analyze(image: ImageProxy) {
        try {
            // 의심 포인트 없으면 2프레임 중 1개만 처리 (15fps)
            // 의심 포인트 있으면 전체 처리 (30fps)
            val shouldProcess = isHighPriorityMode || (fra
            frameCount++

            if (shouldProcess) {
                val result = detectSuspiciousPoints(image)
                if (result.hasSuspiciousPoints) {
                    isHighPriorityMode = true
                    lastSuspiciousTime = System.currentTim
                } else if (System.currentTimeMillis() - la
```

```
                    // 2초간 포인트 미발견 → 저주파 모드 복귀
                    isHighPriorityMode = false
                }
            }
        } finally {
            // 핵심: 항상 즉시 close → 미호출 시 CameraX 파이프
            image.close()
        }
    }
}
```

10.7 21.6 메모리 관리: CameraX 프레임 즉시 close

10.7.1 메모리 누수는 시한폭탄이다

메모리 누수는 즉시 앱을 망가뜨리지 않습니다. 서서히 메모리를 잠식하다가, 한계에 도달하면 OutOfMemoryError로 앱이 종료됩니다. 가장 흔한 원인은 ImageProxy를 닫지 않는 것입니다.

ImageProxy 관리 원칙:

```
// 잘못된 방법: close를 잊는 경우
```

```
class WrongAnalyzer : ImageAnalysis.Analyzer {
    override fun analyze(image: ImageProxy) {
        if (someCondition) {
            process(image)
            image.close()  // 조건이 false이면 close 호출 안
        }
    }
}

// 올바른 방법: finally 블록으로 반드시 close 보장
class CorrectAnalyzer : ImageAnalysis.Analyzer {
    override fun analyze(image: ImageProxy) {
        try {
            process(image)
        } catch (e: Exception) {
            CrashlyticsHelper.logNonFatal("FrameAnalysis",
        } finally {
            image.close()  // 항상 실행
        }
    }
}
```

OUI DB 지연 로딩으로 콜드 스타트 최적화:

OUI JSON 파일 (~500KB)을 앱 시작 시 로드하면 콜드 스타트가 느려집니다.

```
class OuiDatabaseImpl @Inject constructor(
    private val context: Context,
    private val dispatcher: CoroutineDispatcher = Dispatch
) : OuiDatabase {

    // 첫 조회 시 로딩 (앱 시작 시 X)
    private val database: HashMap<String, OuiEntry> by laz
        loadDatabase()
    }

    private fun loadDatabase(): HashMap<String, OuiEntry>
        return context.assets.open("oui_database.json").us
            Json.decodeFromStream<List<OuiEntry>>(stream)
                .associateByTo(HashMap()) { it.ouiPrefix.u
        }
    }

    override fun lookup(macPrefix: String): OuiEntry? {
        val normalized = macPrefix.uppercase().replace("-"
        return database[normalized]
```

CHAPTER 10. CHAPTER 10. 컴파일러 최적화 컴파일러 최적화 컴파일러 최적화

```
        sum = 0f
        index = 0
        count = 0
    }
}
```

10.8 21.7 ANR 방지 전략

10.8.1 "앱이 응답하지 않습니다"를 막아라

ANR(Application Not Responding)은 메인 스레드가 5초 이상 블로킹될 때 발생합니다. 사용자에게 "앱 종료" 팝업을 보여주는 최악의 UX입니다.

ANR 원인 TOP 5:

1. 메인 스레드에서 네트워크 요청
2. 메인 스레드에서 파일 I/O
3. 메인 스레드에서 DB 쿼리
4. Mutex/Lock 데드락
5. 오래 걸리는 Binder 호출

ANR 방지 패턴:

```
// 잘못된 방법: 메인 스레드에서 직접 실행
class WrongViewModel : ViewModel() {
    fun loadData() {
        // 메인 스레드에서 실행 → ANR 위험!
        val result = repository.fetchFromNetwork()  // 블로
        _uiState.value = UiState.Success(result)
    }
}

// 올바른 방법: 코루틴으로 백그라운드에서 실행
class CorrectViewModel @Inject constructor(
    private val repository: ScanRepository
) : ViewModel() {

    fun startScan() {
        viewModelScope.launch {
            _uiState.value = UiState.Loading

            // withContext로 백그라운드 스레드에서 실행
            val result = withContext(Dispatchers.IO) {
                repository.fetchFromNetwork()
            }
```

```
            // collect 후 메인 스레드에서 UI 업데이트
            _uiState.value = UiState.Success(result)
        }
    }
}
```

Room DB에 메인 스레드 접근 금지 강제:

```
@Database(
    entities = [ScanReportEntity::class, DetectedDeviceEnt
    version = 1
)
abstract class SearCamDatabase : RoomDatabase() {
    // Room은 기본적으로 메인 스레드 접근을 차단합니다
    // allowMainThreadQueries()는 절대 사용하지 않습니다

    companion object {
        fun create(context: Context): SearCamDatabase {
            return Room.databaseBuilder(
                context,
                SearCamDatabase::class.java,
                "searcam.db"
            )
```

```
            // .allowMainThreadQueries() // 절대 사용 금지
            .build()
        }
    }
}
```

StrictMode로 개발 중 위반 감지:

```
// SearCamApp.kt
class SearCamApp : Application() {
    override fun onCreate() {
        super.onCreate()

        if (BuildConfig.DEBUG) {
            // 개발 중에만 활성화: 메인 스레드 위반을 즉시 감지
            StrictMode.setThreadPolicy(
                StrictMode.ThreadPolicy.Builder()
                    .detectDiskReads()        // 디스크 읽기
                    .detectDiskWrites()       // 디스크 쓰기
                    .detectNetwork()          // 네트워크 감
                    .penaltyLog()             // Logcat에 기
                    .penaltyDeath()           // 크래시로 강
                    .build()
            )
```

```
        }
    }
}
```

10.9 21.8 Firebase Remote Config: 긴급 파라미터 조정

10.9.1 앱 업데이트 없이 동작을 바꾸는 법

프로덕션에서 포트 스캔 타임아웃이 너무 짧아 오탐이 발생한다면? 앱 업데이트를 기다리는 동안 수만 명이 잘못된 결과를 받습니다. Remote Config는 서버에서 파라미터를 변경하면 앱이 즉시 반영합니다.

```
// build.gradle.kts
implementation("com.google.firebase:firebase-config-ktx")

// RemoteConfigManager.kt
object RemoteConfigManager {

    private val remoteConfig = Firebase.remoteConfig

    // 기본값 (서버 연결 실패 시 사용)
```

```
    private val defaults = mapOf(
        "port_scan_timeout_ms" to 2_000L,
        "emf_noise_threshold" to 3.0f,
        "max_concurrent_port_scans" to 4L,
        "enable_ir_detection" to true,
        "oui_risk_threshold" to 0.7f
    )

    suspend fun fetchAndActivate(): Boolean {
        remoteConfig.setDefaultsAsync(defaults).await()

        return try {
            remoteConfig.fetchAndActivate().await()
        } catch (e: Exception) {
            // 실패 시 기본값 사용 (앱 동작 지속)
            false
        }
    }

    val portScanTimeoutMs: Long
        get() = remoteConfig.getLong("port_scan_timeout_ms
```

```
    val emfNoiseThreshold: Float
        get() = remoteConfig.getDouble("emf_noise_threshol

    val maxConcurrentPortScans: Int
        get() = remoteConfig.getLong("max_concurrent_port_

    val enableIrDetection: Boolean
        get() = remoteConfig.getBoolean("enable_ir_detecti
}
```

10.10 21.9 성능 KPI 대시보드

10.10.1 한눈에 보는 앱 건강 상태

SearCam의 핵심 성능 지표를 정기적으로 확인해야 합니다.

지표	목표	경고	위험
콜드 스타트	< 2초	2~3초	> 3초
Quick Scan 완료	< 30초	30~40초	> 40초
Full Scan 완료	< 3분	3~4분	> 4분
메모리 peak	< 150MB	150~200MB	> 200MB
크래시율	< 0.3%	0.3~1%	> 1%

지표	목표	경고	위험
ANR 비율	< 0.2%	0.2~0.5%	> 0.5%
Quick Scan 배터리	< 2%	2~3%	> 3%

주간 성능 리뷰 절차:

매주 월요일 오전 체크:

1. Firebase Crashlytics → 새 이슈 확인
2. Firebase Performance → Quick Scan 평균 시간
3. Google Play Console → Android Vitals (ANR, 크래시)
4. Battery Historian → 배터리 소모 패턴 (출시 후 월 1회)

10.11 21.10 Battery Historian로 배터리 분석

10.11.1 배터리를 어디서 쓰는지 알아야 아낀다

Battery Historian은 `adb bugreport`를 분석해 배터리 소모 원인을 시각화합니다.

```
# 배터리 리포트 수집
adb shell dumpsys batterystats --reset
# (Quick Scan 실행)
```

```
adb bugreport > bugreport.zip

# Battery Historian으로 분석
# https://bathist.ef.lc/ 에 업로드 또는 로컬 실행
docker run -d -p 9999:9999 gcr.io/android-battery-historia
# localhost:9999 에서 bugreport.zip 업로드
```

분석 시 확인할 항목:

```
Wake Locks:
  → SearCam이 보유한 Wake Lock 시간 확인
  → 스캔 완료 후 Wake Lock 해제되는지 확인

Sensor Usage:
  → MagneticSensor 활성 시간 확인
  → 스캔 외 시간에 센서가 켜져 있으면 버그

Network Activity:
  → mDNS, SSDP, 포트 스캔 네트워크 사용량 확인
  → 불필요한 재시도 요청 없는지 확인
```

CHAPTER 10. 모니터링과 성능 최적화

10.12 정리

모니터링과 성능 최적화는 배포 후 시작되는 지속적인 작업입니다. SearCam에서 적용한 핵심 원칙을 요약합니다.

1. **Crashlytics 우선**: 사용자 신고 전에 크래시를 먼저 알아야 합니다. P0 이슈는 24시간 내 핫픽스
2. **측정 후 최적화**: Android Profiler와 Firebase Performance로 병목을 확인한 후 코드를 수정합니다
3. **배터리 3원칙**: 센서 20Hz 고정, 포트 스캔 병렬화, 카메라 프레임 선택적 분석
4. **메모리 원칙**: ImageProxy는 `finally` 블록에서 반드시 `close()`, GC 압박 최소화
5. **ANR 원천 차단**: 메인 스레드는 UI만, 모든 I/O는 `Dispatchers.IO`, StrictMode로 위반 즉시 감지
6. **Remote Config 활용**: 긴급 파라미터 조정은 앱 업데이트 없이 서버에서 처리

좋은 앱은 출시할 때가 아니라 사용자가 쓰는 동안 만들어집니다. 모니터링이 그 길을 밝혀줍니다.

제 11 장

Ch22: 법적 준수와 개인정보 보호

이 장에서 배울 것: 개인정보 보호법, GDPR, Play Store 정책을 준수하면서 앱을 출시하는 방법을 배웁니다. "수집하지 않는다" 는 사실을 법적으로 증명하는 설계 전략과, 포트 스캔의 합법성을 확보하는 논리를 이해합니다.

11.1 도입

호텔 청소부가 방에 들어올 때 사전에 노크하고 허락을 구하는 것은 규칙입니다. 아무리 청소 목적이라도 무단 침입은 허용되지 않습니다. 앱도 마찬가지입니다. 사용자의 데이터에 접근할 때는 목적을 밝히고, 필요한 것만 쓰고, 사용 후 정리해야 합니다. 이것이 개인정보 보호법의 핵심이며, SearCam이 설계 단계부터 지킨 원칙입니다.

법을 지키는 것은 단순한 의무가 아닙니다. 사용자의 신뢰를 얻는 경쟁 우위이기도 합니다. 특히 카메라와 위치 권한을 요구하는 보안 앱에서는 더욱 그렇습니다. 이 장에서는 SearCam이 어떻게 법적 안전지대를 확보했는지 단계별로 살펴봅니다.

11.2 22.1 개인정보 보호법 준수 설계

11.2.1 "수집하지 않는다"는 것을 증명하는 법

많은 개발자가 "개인정보를 수집하지 않으면 개인정보 보호법과 관계없다" 고 생각합니다. 이 생각은 절반만 맞습니다. 개인정보를 수집하지 않더라도, 카메라 · 위치 · 네트워크 같은 민감 권한을 사용하는 앱은 투명성 의무를 집니다. 처리방침을 작성하고, "수집하지 않는다" 는 사실을 공개해야 합니다.

SearCam은 다음 두 가지를 동시에 증명합니다.

첫째, 기술적으로 수집이 불가능하게 설계했습니다. 카메라 프레임은 메모리에서만 분석하고 저장하지 않습니다. Wi-Fi MAC 주소는 SHA-256으로 해시 처리 후에만 리포트에 저장합니다. 원본 복원이 불가능하므로 법적으로 비식별 정보에 해당합니다.

둘째, 설계 문서로 증명합니다. 이 개인정보 영향평가서(PIA)와 보안 설계 문서가 "의도적 설계" 의 증거입니다.

데이터	처리 방식	저장	외부 전송	개인정보 해당 여부
Wi-Fi 기기 MAC 주소	메모리에서 분석	SHA-256 해시만 저장	없음	해시 처리로 비식별
카메라 영상 프레임	실시간 분석	저장 안 함	없음	저장 안 하므로 해당 없음

데이터	처리 방식	저장	외부 전송	개인정보 해당 여부
자기장 측정값	실시간 표시	요약 수치만 리포트에	없음	개인 식별 불가
Wi-Fi SSID	네트워크 확인용	저장 안 함	없음	저장 안 하므로 해당 없음
위치 정보	Wi-Fi 스캔 API 조건	저장 안 함	없음	수집·저장하지 않음
스캔 리포트	기기 내 로컬 저장	로컬 DB (암호화)	없음	사용자 기기 내 관리

11.2.2 로컬 처리 원칙: Privacy by Design

개인정보 보호 규정들이 공통으로 권장하는 개념이 Privacy by Design입니다. 사후에 프라이버시를 추가하는 것이 아니라, 처음부터 프라이버시를 설계에 포함시키는 접근법입니다.

SearCam의 로컬 처리 원칙이 바로 이에 해당합니다.

Privacy by Design 7원칙 → SearCam 대응

1. 사전 예방 (Proactive)
   → 데이터 수집 자체를 하지 않는 구조 설계

2. 기본값으로 프라이버시 (Privacy as Default)
   → 사용자가 별도 설정 없이도 최소 데이터 처리

3. 설계에 내재화 (Embedded)
   → 온디바이스 알고리즘, 백엔드 서버 없음

4. 완전한 기능 유지 (Full Functionality)
   → 프라이버시를 지키면서도 탐지 품질 유지

5. 생애주기 보호 (End-to-End Security)
   → 앱 삭제 시 로컬 데이터 완전 소거

6. 가시성과 투명성 (Visibility)
   → 처리방침, 권한 설명, 면책 고지 공개

7. 사용자 중심 (User-Centric)

→ 리포트 저장·삭제를 사용자가 직접 관리

11.3 22.2 권한 최소화 원칙

11.3.1 위치 권한의 역설

"위치를 수집하지 않는데 왜 위치 권한이 필요한가요?" 사용자가 가장 많이 하는 질문입니다. 이유는 Android OS의 설계에 있습니다.

Android 8.0(API 26)부터 Wi-Fi 네트워크 스캔 API는 위치 권한(`ACCESS_FINE_LOCATION`)을 필수로 요구합니다. 네트워크 SSID와 BSSID 정보가 위치 추론에 사용될 수 있기 때문입니다. Google이 이 제한을 추가한 것은 프라이버시 보호를 위한 의도인데, 역설적으로 Wi-Fi 기반 탐지 앱이 위치 권한을 요청해야 하는 상황을 만들었습니다.

SearCam의 대응 전략은 명확합니다.

[권한 요청 시 사용자에게 보여주는 설명]

위치 권한이 필요한 이유

`Android` 시스템 요구사항으로 인해 `Wi-Fi` 네트워크 스캔에 위치 권한이 필요합니다.

SearCam은 절대 GPS 좌표를 수집하거나 저장하지 않습니다.
위치 권한은 Wi-Fi 스캔 기능에만 사용됩니다.

위치 권한을 거부하면:
  - Wi-Fi 네트워크 스캔 기능을 사용할 수 없습니다
  - 렌즈 감지 + 자기장 감지 기능은 계속 사용 가능합니다

Play Store 데이터 안전 섹션에서도 명시합니다.

[Play Store 데이터 안전]

위치
  수집 여부: 수집하지 않음
  참고: Wi-Fi 스캔 API 요구사항으로 위치 권한이 필요하나, GPS 좌표를 기록하거나 저장하지 않습니다.

11.3.2 권한 거부 시 대체 경로

권한 최소화 원칙의 다른 측면은 권한 거부 시에도 앱이 동작해야 한다는 것입니다. SearCam은 위치 권한을 거부한 사용자도 렌즈 감지(Layer 2)와 자기장 감지(Layer 3) 기능을 사용할 수 있도록 설계했습니다. 단일 권한 거부로 앱 전체가 동작 불능이 되는 것은 좋

지 않은 UX이자 법적으로도 불필요한 권한 요구로 해석될 수 있습니다.

11.4 22.3 네트워크 스캔의 합법성 확보

11.4.1 포트 스캔은 해킹인가?

사용자 리뷰에서 종종 이런 댓글이 달립니다. “포트 스캔은 해킹 아닌가요?” 결론부터 말하면, SearCam의 포트 스캔은 합법입니다. 근거를 상세히 살펴봅니다.

정보통신망법 제48조는 “정당한 접근권한 없이 정보통신망에 침입하는 것” 을 금지합니다. 여기서 핵심은 “정당한 접근권한 없이” 입니다.

SearCam 포트 스캔의 합법성 근거

1. 사용자가 합법적으로 접속한 네트워크에서 수행
  - 호텔/숙소 Wi-Fi에 정상 연결된 상태
  - 해당 네트워크 접속 자체는 정당한 권한

2. TCP Connect 방식 = 정상 연결 시도
  - "공개 포트에 연결 시도"는 침입이 아님

  - 웹 브라우저가 HTTP 80 포트에 접속하는 것과 동일 수준

3. 목적의 정당성
  - 불법 몰래카메라 탐지라는 공익적 목적
  - 개인의 프라이버시 보호를 위한 자구 행위

4. 범위의 최소성
  - 카메라 관련 포트 8개만 확인 (전체 65535포트 스캔 아님)
    - 연결 확인 후 즉시 종료 (데이터 수집·수정 없음)
    - 같은 서브넷(숙소 Wi-Fi) 내에서만 작동

SearCam이 절대 하지 않는 것들도 명확히 해둡니다.

행위	SearCam 여부	이유
패킷 스니핑 (Monitor Mode)	사용 안 함	통신비밀보호법 위반
DPI (Deep Packet Inspection)	사용 안 함	통신 내용 열람 금지
SYN 스캔 / 스텔스 스캔	사용 안 함	공격적 스캔 방식
포트 스캔 후 실제 접속	사용 안 함	탐지 후 접속 기능 없음
RTSP 스트리밍 시청	사용 안 함 (Phase 1)	타인 카메라 내용 열람

11.4.2 스캔 전 사용자 고지

사용자가 스캔을 시작하기 전에 명확한 안내 화면을 보여줍니다.

[Wi-Fi 스캔 시작 전 안내 화면]

이 기능은 현재 연결된 Wi-Fi 네트워크에서
카메라로 의심되는 기기를 탐색합니다.

- 같은 Wi-Fi에 연결된 기기만 확인합니다
- 기기의 제조사와 서비스 유형을 분석합니다
- 타인의 기기에 접속하거나 내용을 확인하지 않습니다
- 스캔 결과는 기기에만 저장되며 외부로 전송되지 않습니다

[스캔 시작] [취소]

11.5 22.4 개인정보 처리방침 작성

11.5.1 Play Store에 처리방침이 필요한 이유

개인정보를 수집하지 않더라도 Google Play Store 정책상 개인정보 처리방침 URL을 반드시 등록해야 합니다. 카메라, 위치 같은 민감 권한을 사용하는 앱은 더 엄격한 투명성 요구를 받습니다.

처리방침의 핵심은 역설적이게도 "수집하지 않는다" 는 것을 명확

히 서술하는 것입니다. 일반적인 처리방침이 수집하는 항목을 나열하는 것과 달리, SearCam의 처리방침은 처리하는 데이터와 처리하지 않는 데이터를 모두 명시합니다.

[SearCam 개인정보 처리방침 핵심 내용]

1. 개인정보 수집 항목
   SearCam은 개인정보를 수집하지 않습니다.

2. 처리하는 데이터 (기기 내부에서만)
   - Wi-Fi 기기 정보 (MAC 해시, 제조사, 포트 상태)
   - 카메라 프레임 (렌즈 반사 분석, 저장하지 않음)
   - 자기장 센서 데이터 (실시간 분석)

   위 데이터는 사용자 기기에서만 처리됩니다.
   외부 서버로 전송되지 않습니다.

3. 데이터 저장
   - 스캔 리포트: 로컬 DB에 암호화 저장
   - 보관 기간: 무료 사용자 최근 10건
   - MAC 주소: SHA-256 해시 처리 (원본 복원 불가)
   - 앱 삭제 시 모든 데이터 완전 삭제

4. 제3자 제공

사용자 데이터를 제3자에게 제공하지 않습니다.

5. 사용자 권리

- 리포트 조회·삭제: 앱 내 리포트 메뉴
- 전체 삭제: 앱 설정 → 데이터 초기화
- 권한 철회: 기기 설정 → 앱 → SearCam → 권한

11.6 22.5 면책 설계

11.6.1 "100% 보장하지 않는다"를 법적으로 설계하기

SearCam은 보조 탐지 도구입니다. 탐지에 실패할 수 있는 유형이 있고, 이를 처음부터 사용자에게 알려야 합니다. 면책 조항은 법적 보호뿐 아니라 사용자와의 정직한 커뮤니케이션이기도 합니다.

최초 실행 시 면책 동의를 받습니다.

[최초 실행 시 면책 동의 화면]

서비스 이용 안내

SearCam은 숨겨진 카메라 탐지를 보조하는

1차 스크리닝 도구입니다.

- 100% 탐지를 보장하지 않습니다
- 전문 탐지 장비(RF 스캐너, NLJD 등)를 대체하지 않습니다
- Wi-Fi 미연결, 전원 OFF 카메라 등 탐지 불가 유형이 있습니다
- 의심 시 전문 업체 또는 경찰(112)에 신고해주세요

[동의 후 시작하기]

스캔 결과 화면 하단에도 지속적으로 표시합니다.

이 결과는 참고용이며, 안전을 보장하지 않습니다.
정밀 점검이 필요하면 전문 업체에 문의하세요.

11.6.2 이용약관의 탐지 한계 명시

탐지 불가능한 카메라 유형은 이용약관 본문에 명시해야 합니다. 나중에 "이런 카메라는 탐지 못 한다고 했잖아요" 라고 말할 수 있는 근거가 됩니다.

제3조 (탐지 한계)
다음 유형의 카메라는 탐지가 어렵거나 불가능합니다:
① Wi-Fi에 연결되지 않은 카메라

② 전원이 꺼진 카메라

③ LTE/5G 독립 통신 카메라

④ SD카드 단독 녹화 카메라

⑤ 렌즈가 물리적으로 가려진 카메라

11.7 22.6 GDPR 대비

11.7.1 유럽 시장 진출을 위한 사전 준비

GDPR(General Data Protection Regulation)은 유럽 사용자를 대상으로 앱을 제공할 때 적용됩니다. Play Store를 통해 유럽에 앱을 배포하는 순간 GDPR 의무가 생길 수 있습니다.

SearCam의 로컬 처리 설계는 GDPR 준수에 유리합니다.

GDPR 요건	SearCam 현황	추가 조치
개인정보 수집 동의	수집 안 함	투명성 공지는 필요
데이터 처리 합법성 근거	로컬 처리만 수행	"정당한 이익" 조항 적용
정보주체의 권리	앱 삭제로 완전 삭제	처리방침에 명시
DPO 지정	대량 데이터 처리 안 함	불필요
데이터 이전 (EU 외)	전송 없음	불필요
Privacy by Design	로컬 처리 원칙 = 이에 해당	설계 문서 유지

영문 처리방침에 GDPR 부록을 추가합니다.

```
[GDPR Addendum]

Legal Basis for Processing:
SearCam processes data locally on your device under the legal
basis of "legitimate interest" (Article 6(1)(f) GDPR).
No personal data is collected, stored on servers, or transmit

Your Rights Under GDPR:
- Right of access: All data is on your device and viewable in-
app
- Right to erasure: Delete reports in-
app or uninstall the app
- Right to data portability: PDF export available (Premium)
- Right to object: Uninstall the app at any time

Contact: [이메일 주소]
```

11.8 22.7 오픈소스 라이선스 관리

11.8.1 iText AGPL 문제

라이선스 관리는 개발자가 쉽게 놓치는 법적 의무입니다. SearCam이 발견한 가장 중요한 라이선스 리스크는 PDF 생성 라이브러리인 iText입니다.

iText는 AGPL 3.0 라이선스를 사용합니다. AGPL은 이 라이브러리를 사용한 앱의 소스 코드 전체를 공개하도록 강제합니다. 상업 앱에서 이를 지키려면 iText의 상업 라이선스를 구매해야 합니다.

해결책은 간단합니다. iText 대신 Android 기본 `PdfDocument` API를 사용하면 됩니다. 기능은 제한적이지만 라이선스 리스크가 없습니다.

라이브러리	라이선스	상업 앱 사용 가능 여부	SearCam 대응
Kotlin	Apache 2.0	가능	사용
Jetpack Compose	Apache 2.0	가능	사용
Room	Apache 2.0	가능	사용
Hilt	Apache 2.0	가능	사용
CameraX	Apache 2.0	가능	사용
SQLCipher	BSD-3	가능	사용
iText	AGPL 3.0	**주의**	**Android PdfDocu**

모든 오픈소스 라이브러리는 앱 설정 화면에 고지해야 합니다. “앱 설정 → 오픈소스 라이선스” 에서 전체 목록을 보여주는 화면을 구현합니다.

11.9 22.8 컴플라이언스 유지 체계

법적 준수는 한 번에 끝나는 작업이 아닙니다. Play Store 정책은 자주 바뀌고, 개인정보 보호법도 개정됩니다. 지속적인 모니터링 체계가 필요합니다.

항목	검토 주기	담당
개인정보 처리방침 검토	분기별	개발자 · 운영
Play Store 정책 변경 대응	정책 변경 시 즉시	개발팀
오픈소스 라이선스 감사	라이브러리 추가 · 변경 시	개발팀
해외 법규 모니터링	신규 국가 출시 전	법률 자문
이용약관 업데이트	기능 추가 시	개발자 · 운영
면책 조항 적절성 검토	반기별	법률 자문

Phase 3에서 서버를 도입하고 커뮤니티 기능을 추가하면 법적 의무가 크게 늘어납니다. 서버 도입 시점에는 정식 개인정보 수집 동의, GDPR/CCPA 정식 대응, 데이터 처리 계약(DPA)이 추가로 필요

합니다.

11.10 마무리

SearCam의 법적 설계는 세 개의 축으로 요약됩니다.

첫째, 기술적 증명. 개인정보를 수집하지 않는다는 것을 구조적으로 불가능하게 설계했습니다. 말이 아니라 코드로 증명합니다.

둘째, 투명한 커뮤니케이션. 위치 권한이 필요한 기술적 이유, 포트 스캔의 합법성 근거, 탐지 한계를 사용자에게 솔직하게 알립니다.

셋째, 지속적 관리. 법은 바뀝니다. 출시 후에도 정기적으로 검토하고 업데이트합니다.

> **중요**: 이 장의 내용은 법적 자문을 대체하지 않습니다. 실제 출시 전 전문 법률 자문을 받아 최종 검토하는 것을 강력히 권장합니다.

다음 장 → Ch23: 수익화 전략과 GTM

제 12 장

Ch23: 수익화 전략과 GTM

이 장에서 배울 것: 핵심 기능을 무료로 유지하면서도 지속 가능한 수익을 만드는 프리미엄 모델을 설계합니다. SearCam의 가격 전략, 광고 배치 원칙, B2B 확장 로드맵, 초기 사용자 획득 전략을 배웁니다.

12.1 도입

피자 가게를 연다고 상상해보세요. 두 가지 전략이 있습니다. 첫 번째는 모든 피자를 유료로 팔되, 작은 무료 샘플을 나눠주는 방식입니다. 두 번째는 기본 피자(마르게리타)는 무료로 주고, 프리미엄 토핑

(트러플, 프로슈토)에만 돈을 받는 방식입니다. SearCam은 두 번째 모델입니다. 안전이라는 핵심 가치는 무료로 제공하고, 편의와 고급 기능에 프리미엄을 붙입니다.

수익화는 앱을 완성한 다음에 생각하는 것이 아닙니다. 처음 설계할 때부터 "무엇이 핵심이고, 무엇이 프리미엄인지" 를 결정해야 합니다. 이 결정이 제품의 방향 자체를 바꾸기 때문입니다.

12.2 23.1 핵심 원칙: "안전은 유료가 아닙니다"

SearCam의 수익화 철학은 하나의 문장으로 요약됩니다.

> **"안전은 유료가 아닙니다. 모든 탐지 기능은 100% 무료."**

이 원칙은 감성적 선언이 아니라 전략적 판단입니다. 경쟁 앱들이 핵심 탐지 기능을 유료로 잠근 것은 단기 수익을 올리지만, 사용자 신뢰를 잃는 방법입니다. 몰래카메라 탐지는 공익적 목적이 강합니다. "위험한 상황에서 돈을 내야 점검할 수 있다" 는 메시지는 부정적인 브랜드 이미지를 만듭니다.

무료로 핵심 기능을 제공하면 두 가지 이점이 생깁니다. 첫째, 진입 장벽이 없어 더 많은 사용자가 설치합니다. 둘째, 실제로 써본 사람이 프리미엄으로 전환할 가능성이 높습니다. 이것이 프리미엄 모델

의 본질입니다.

12.3 23.2 무료 vs 프리미엄 기능 설계

12.3.1 기능 경계선 그리기

기능의 경계를 어디에 그을지가 프리미엄 전환율을 결정합니다. 너무 많은 것을 무료로 주면 전환 동기가 없고, 너무 많은 것을 막으면 앱 자체가 사용되지 않습니다.

SearCam의 원칙: **핵심 안전 기능은 전부 무료, 편의·관리·고급 분석은 프리미엄.**

기능	무료	프리미엄
Quick Scan (Wi-Fi 스캔)	전체 사용 가능	전체 사용 가능
Full Scan (3레이어 통합)	전체 사용 가능	전체 사용 가능
렌즈 찾기 (Retroreflection)	전체 사용 가능	전체 사용 가능
IR Only / EMF Only 모드	전체 사용 가능	전체 사용 가능
교차 검증 + 위험도 산출	전체 사용 가능	전체 사용 가능
체크리스트 (숙소/화장실)	전체 사용 가능	전체 사용 가능
112 신고 연동	전체 사용 가능	전체 사용 가능
리포트 저장	**최근 10건**	**무제한**

기능	무료	프리미엄
PDF 리포트 내보내기	없음	제공
광고	표시됨	제거됨
OUI DB 업데이트	앱 업데이트 시	**OTA 자동 업데이트**
고급 통계·트렌드	없음	제공
스캔 히스토리 비교	없음	제공
우선 지원	없음	제공

12.3.2 프리미엄 전환 트리거 설계

전환은 불편함을 느끼는 순간에 자연스럽게 유도해야 합니다. 강요가 아니라 "이 기능이 필요하다면 프리미엄으로" 흐름입니다.

```
[전환 트리거 시나리오]

시나리오 1: 리포트 저장 한도
  → 11번째 리포트 저장 시도 시
  → "저장 공간이 가득 찼어요. 프리미엄으로 무제한 저장하세요."
    → 7일 무료 체험 제안

시나리오 2: PDF 내보내기 시도
  → 리포트 상세 화면에서 공유 버튼 탭
  → "PDF 리포트는 프리미엄 기능입니다."
```

→ 기능 미리보기 후 구독 유도

시나리오 `3: OUI DB` 업데이트 알림
→ 앱 시작 시 "신규 카메라 제조사 데이터가 있어요"
→ "프리미엄은 자동으로 최신 데이터를 받아요"

12.4 23.3 가격 전략

12.4.1 경쟁사 분석에서 찾은 포지셔닝

경쟁 앱들의 가격은 월 ₩3,000~₩12,000 수준입니다. 그러나 핵심 기능을 유료로 잠근 경우가 많아 사용자 불만이 높습니다.

앱	무료 기능	유료 가격
Hidden Camera Detector	기본 스캔	월 ₩5,000~₩12,000
CamX	일부 모드만	월 ₩4,000
FindSpy	기본 스캔	월 ₩3,000
SearCam	**핵심 기능 전체 무료**	**월 ₩2,900**

SearCam의 전략은 두 가지입니다. 경쟁사보다 더 많이 무료로 주고, 더 저렴하게 프리미엄을 제공합니다. 이는 빠른 사용자 확보와 낮은 이탈률로 이어집니다.

12.4.2 가격 구조

플랜	가격	특징
무료	₩0	핵심 탐지 기능 전체
월간 구독	₩2,900/월	편의 기능 전체
연간 구독	₩23,900/년	31% 할인, 일 ₩65

12.4.3 론칭 가격 실험 계획

론칭 초기 4주간 A/B 테스트를 진행합니다.

테스트	대조군	실험군	측정 지표
가격	₩2,900	₩3,900	전환율 × 가격 (총 수익)
트라이얼 기간	7일 무료	14일 무료	트라이얼 후 전환율
연간 할인율	31%	40%	연간 구독 비율
전환 트리거 시점	11건째	6건째	전환율

12.5 23.4 광고 전략

12.5.1 광고는 최소화, 탐지는 방해 안 한다

광고는 무료 사용자를 수익화하는 중요한 수단이지만, 잘못 배치하면 핵심 기능을 망칩니다. SearCam의 광고 배치 원칙은 단순합니다. **탐지 중에는 절대 광고를 표시하지 않습니다.**

상황	광고 표시	이유
홈 화면 하단	배너 광고 표시	사용자 인지 낮음
결과 화면 확인 후	간 삽입 광고	스캔 완료 후
리포트 목록	네이티브 광고	5건마다 1개
스캔 진행 중	**절대 표시 안 함**	집중 방해
위험 감지 결과 화면	**절대 표시 안 함**	긴급 상황
112 신고 화면	**절대 표시 안 함**	긴급 상황
온보딩	**절대 표시 안 함**	첫 인상 보호

12.5.2 AdMob 설정

```
네트워크: Google AdMob + 미디어이션 (Meta, Unity Ads)
배너 광고: Adaptive Banner (기기 맞춤)
간 삽입 빈도: 스캔당 최대 1회 (최소 5분 간격)
카테고리 필터: 보안/안전 광고 우선, 성인/도박 차단
```

12.5.3 광고 수익 추정

DAU 2,000 기준 일일 광고 수익:

```
배너:    2,000명 × 3 노출 × $1.5/1000  = $9.0/일
간 삽입: 2,000명 × 0.5회 × $7.0/1000  = $7.0/일
합계:    $16.0/일 ≈ ₩21,000/일 ≈ ₩630,000/월
```

12.6 23.5 12개월 재무 모델

12.6.1 사용자 성장 시나리오

월	신규 설치	MAU	DAU	유료 사용자	월 수익
M1	2,000	1,500	200	1	₩66,000
M3	5,000	5,500	800	12	₩287,000
M6	8,000	11,000	1,800	36	₩671,000
M12	3,000	9,000	2,000	60	₩804,000

성장 가정: - M5~M6: 여름 성수기, SNS 바이럴 효과 - M7~M9: 비수기 하락 - M10~M12: iOS 론칭 효과 + 겨울 여행 시즌

12.6.2 손익분기점 분석

마케팅 비용을 제외한 순수 운영 비용(서버 제로, Play Store 수수료만)은 월 약 ₩5만입니다. M1부터 손익분기를 달성합니다.

마케팅 포함 손익분기점:

```
월 마케팅 비용 (안정기 M7+): ₩480만
필요 DAU: 약 10,000
예상 도달: iOS 론칭 후 18~24개월
```

12.7 23.6 B2B 확장: 숙박업소 파트너십

12.7.1 "SearCam 인증 숙소" 프로그램

Phase 2의 핵심 B2B 수익원은 숙박업소 인증 프로그램입니다. 에어비앤비 호스트, 소규모 모텔, 게스트하우스가 월 1회 Full Scan을 수행하고 결과를 제출하면 "SearCam 인증" 배지를 받는 프로그램입니다.

[인증 흐름]

```
숙박업소 신청
  ↓
```

```
SearCam 앱으로 Full Scan 수행
  ↓
결과 자동 리포트 생성 + 체크리스트 완료
  ↓
월 1회 점검 유지 (자동 알림)
  ↓
숙박 플랫폼 프로필에 배지 표시
  ↓
게스트에게 "몰래카메라 점검 완료" 신뢰 제공
```

패키지	가격	대상
개인 (1개 업소)	₩19,900/월	에어비앤비 호스트
Small (10 라이선스)	₩19,900/월	소규모 숙박 체인
Medium (50 라이선스)	₩79,900/월	중형 호텔 체인
Enterprise (200+)	협의	대형 숙박 그룹

예상 M18 수익: 인증 숙소 500곳 × ₩19,900 = **₩9,950,000/월**

12.7.2 API 라이선스

보안 솔루션 업체에 OUI DB + 교차 검증 알고리즘을 API로 제공합니다.

항목	내용
제공 API	OUI 조회, 위험도 산출, 교차 검증
가격	$500/월 (100,000 calls 포함)
대상	보안 앱 개발사, IoT 보안 업체
예상 M24 수익	$2,500~$5,000/월

12.8 23.7 GTM(Go-to-Market) 전략

12.8.1 초기 사용자 획득: 0에서 1,000까지

론칭 첫 달 1,000명 설치가 목표입니다. 비용 없이 달성하는 채널에 집중합니다.

1. 커뮤니티 마케팅 (무료)

목표 채널:
- 에브리타임 (대학생 커뮤니티): "여행 전 필수 앱" 공유
- 네이버 여행 카페: 숙소 점검 팁과 함께 앱 소개
- 블라인드: 출장 많은 직장인 대상 "실제 사용 후기" 공유
- Reddit (r/privacy, r/travel): 영문 게시물로 글로벌 노출
- 보배드림, 디시인사이드: 사건 사고 관련 커뮤니티

2. SNS 콘텐츠 전략

YouTube Shorts / TikTok / Instagram Reels:
- "호텔 체크인 후 30초 점검 루틴" (탐지 과정 영상)
- "숨겨진 카메라를 찾는 3가지 방법" (앱 사용 튜토리얼)
- 실제 카메라 발견 사례 (익명 제보 콘텐츠)

포스팅 주기: 주 3회
목표: 바이럴 가능한 영상 1편이 10,000+ 뷰 달성

3. 미디어 / PR

타겟 미디어:
- 여성 안전 관련 뉴스 매체
- IT/앱 리뷰 채널 (앱스토리, IT동아)
- 여행 전문 매거진/블로그

스토리 각도:
- "스마트폰 하나로 호텔 점검"
- "1인 개발자가 만든 무료 안전 앱"
- "30초 만에 끝나는 몰래카메라 탐지"

12.8.2 성장 채널: 1,000명에서 10,000명까지

초기 1,000명이 모이면 유기적 성장과 유료 마케팅을 병행합니다.

[M3~M6 성장 채널]

1. App Store Optimization (ASO)
   - 키워드: "몰래카메라", "숨겨진 카메라", "호텔 점검"
   - 스크린샷: 실제 탐지 화면 + 위험도 시각화
  - 리뷰 유도: 첫 스캔 완료 후 자연스러운 리뷰 요청

2. 인플루언서 협업
  - 여행 유튜버 5~10명 (구독자 10K~100K)
   - "실제 호텔에서 써봤어요" 형식 리뷰
  - 성과 기반 제휴 (설치 건당 ₩500)

3. Google UAC 광고
  - 예산: 월 ₩50~100만
  - 타겟: 여행 관심사, 여성 20~40대
  - 목표 CPI: ₩500 이하

4. 이슈 편승 마케팅
  - 몰래카메라 관련 뉴스 이슈 발생 시 즉각 콘텐츠 배포
  - 언론사 인터뷰 대응 준비

12.8.3 KPI 설정

지표	M1 목표	M3 목표	M6 목표	M12 목표
누적 설치	2,000	11,000	32,000	53,000
DAU	200	800	1,800	2,000
DAU/MAU (리텐션)	13%	15%	16%	22%
프리미엄 전환율	0.5%	1.5%	2.0%	3.0%
Play Store 평점	-	4.0+	4.2+	4.3+
리뷰 수	-	100+	500+	2,000+

12.9 23.8 Phase별 수익 목표

Phase	기간	주 수익원	월 목표 수익
Phase 1 (Android MVP)	0~12개월	프리미엄 구독 + 광고	₩80만
Phase 2 (iOS 추가)	12~24개월	+ B2B 인증 + iOS	₩1,500만
Phase 3 (커뮤니티)	24~36개월	+ API + 전문 업체 연결	₩5,000만

12.10 마무리

SearCam의 수익화 전략은 역설에서 출발합니다. 더 많이 무료로 주면 더 많이 번다는 것입니다. 경쟁사들이 핵심 기능을 유료로 잠그는 동안, SearCam은 핵심 기능을 전부 무료로 열어놓습니다. 더 많은 사람이 쓰고, 더 많은 사람이 진짜 필요한 편의 기능에 돈을 냅니다.

초기 12개월은 적자입니다. 하지만 이는 의도된 투자입니다. B2B로 확장할 수 있는 신뢰 기반과 사용자 기반을 쌓는 것이 Phase 1의 진짜 목표입니다.

다음 장 → Ch24: 회고와 다음 단계

CH23: 수익화 전략과 GTM

제 13 장

Ch24: 회고와 다음 단계

이 장에서 배울 것: SearCam 개발 과정에서 잘된 것과 아쉬운 것을 솔직하게 돌아봅니다. 기술 부채의 현황을 파악하고, 다음 버전의 로드맵을 설계합니다. 그리고 이 책을 끝까지 읽은 독자에게 전하는 메시지를 나눕니다.

13.1 도입

등산을 마치고 정상에서 올라온 길을 내려다보는 것처럼, 프로젝트가 끝날 때는 돌아봐야 합니다. 어떤 길이 험했고, 어떤 선택이 옳았

고, 다음번에는 어떤 길을 택할지. 이것이 회고입니다.

회고는 자아비판 시간이 아닙니다. 다음 산을 더 잘 오르기 위한 준비입니다. SearCam Phase 1을 완성하며 배운 것들을 솔직하게 나눕니다.

13.2 24.1 잘된 것들

13.2.1 Clean Architecture 선택

SearCam의 가장 좋은 결정은 처음부터 Clean Architecture를 적용한 것입니다. UI, Domain, Data 레이어를 엄격하게 분리했고, 각 레이어 간 의존성은 단방향으로만 흘렀습니다.

```
ui/ (Compose, ViewModel)
  ↓ 단방향 의존
domain/ (UseCase, Repository 인터페이스, Model)
  ↓ 단방향 의존
data/ (Repository 구현, Sensor, Room, Network)
```

이 구조 덕분에 세 가지 탐지 레이어(Wi-Fi, 렌즈, 자기장)를 독립적으로 개발하고 테스트할 수 있었습니다. Wi-Fi 스캔 로직을 수정할 때 카메라 코드를 건드릴 필요가 없었고, 자기장 센서를 교체할 때 UI를 바꾸지 않아도 됐습니다.

변경에 강한 코드를 만들면 초기 설계에 시간이 더 걸리지만, 결과적으로 전체 개발 시간이 단축됩니다.

13.2.2 불변성(Immutability) 원칙 준수

모든 도메인 모델을 `data class`로 설계하고, 상태 변경 시 새 객체를 생성하는 방식을 일관되게 적용했습니다.

```
// 올바른 방식: 새 객체 생성
fun addDevice(device: DetectedDevice): ScanResult {
    return copy(devices = devices + device)
}

// 잘못된 방식: 뮤테이션
fun addDevice(device: DetectedDevice) {
    devices.add(device) // 금지
}
```

이 원칙 덕분에 멀티스레드 환경(세 레이어가 병렬 실행)에서 경쟁 상태(race condition) 문제를 피할 수 있었습니다. Kotlin Coroutines와 Flow를 사용하는 환경에서 불변 객체는 특히 중요합니다.

13.2.3 OUI 데이터베이스 전략

카메라 제조사 식별을 위해 IEEE 공개 데이터를 기반으로 자체 OUI 데이터베이스를 구축한 것은 올바른 선택이었습니다. 80개+ 제조사, 200개+ OUI 접두사를 수록하고, `risk_weight`로 세분화된 위험도를 부여했습니다.

처음에는 단순히 카메라/비카메라 이분법으로 설계하려 했습니다. 하지만 TP-Link처럼 카메라와 공유기 모두를 제조하는 업체가 있어서, 0~1 범위의 연속 값으로 표현하는 것이 더 정확했습니다.

13.2.4 면책 설계의 일관성

경쟁 앱들이 "완벽한 탐지" 를 광고하는 동안, SearCam은 처음부터 "1차 스크리닝 도구" 라고 정직하게 커뮤니케이션했습니다. 최초 실행 화면, 결과 화면, 이용약관 모두에 탐지 한계를 명시했습니다.

이 선택은 단기적으로는 불리해 보일 수 있습니다. 하지만 장기적으로 사용자 신뢰를 쌓는 유일한 방법입니다. "앱이 잡지 못했는데 카메라가 있었다" 는 리뷰에 정직하게 대응할 수 있는 근거가 됩니다.

13.3 24.2 아쉬운 것들

13.3.1 테스트 커버리지 부족

가장 솔직한 기술 부채입니다. 탐지 알고리즘 자체에 대한 단위 테스트가 충분하지 않습니다. 특히 세 레이어의 교차 검증 로직과 위험도 산출 알고리즘은 다양한 입력 시나리오에 대한 체계적인 테스트가 필요합니다.

```
현재 테스트 커버리지 추정:
- 도메인 모델: 약 70%
- UseCase 레이어: 약 50%
- Repository 구현: 약 30%
- UI 레이어: 거의 없음
목표: 80%+ (특히 탐지 알고리즘)
```

TDD를 철학적으로 지지하면서 실제로는 구현 후 테스트를 추가하는 패턴을 반복한 것은 반성할 점입니다. 다음 기능 개발에서는 반드시 RED-GREEN-REFACTOR 사이클을 지킵니다.

13.3.2 Retroreflection 알고리즘의 오탐률

렌즈 역반사 감지 알고리즘의 오탐률이 예상보다 높습니다. 특히 안경 렌즈, 금속 반사면, 특정 조명 조건에서 거짓 양성이 발생합니다.

문제의 원인은 알고리즘의 단순성입니다. 현재 구현은 RETRORE-

`FLECTION_THRESHOLD = 200` 임계값 기반의 고정 값 비교입니다. 환경 조명 조건에 따라 임계값을 동적으로 조정해야 하고, 렌즈 모양(원형 패턴)을 추가로 확인해야 합니다.

현재 알고리즘 한계:
- 정적 임계값 → 조명 조건 변화에 취약
- 단순 밝기 비교 → 렌즈 형태 미확인
- 단일 프레임 기반 → 일시적 반사 미구분

개선 방향:
- 적응형 임계값 (배경 조명 자동 보정)
- 원형 패턴 탐지 (`Hough Circle Transform`)
- 다중 프레임 평균화로 안정성 향상

13.3.3 기기 다양성 테스트 부족

Android 생태계의 단편화는 항상 어렵습니다. 특히 자기장 센서의 정확도와 Wi-Fi 스캔 동작은 제조사마다 다릅니다. 삼성, LG, 구글 픽셀 외의 기기에서는 충분한 테스트가 이루어지지 않았습니다.

구체적인 알려진 문제: - 일부 Xiaomi 기기: ARP 테이블 접근 권한 제한 - 일부 Huawei 기기: Wi-Fi 스캔 빈도 제한 - 구형 기기(API 26): 자기장 센서 정확도 낮음

13.3.4 백그라운드 스캔 부재

현재 SearCam은 포그라운드에서만 동작합니다. 사용자가 앱을 열고 있어야 스캔이 진행됩니다. 숙소에 체크인한 순간 "백그라운드에서 Wi-Fi 스캔을 시작하겠습니까?" 라는 기능이 있었으면 더 편리했을 것입니다.

그러나 백그라운드 Wi-Fi 스캔은 Android의 배터리 최적화 정책에 의해 제한됩니다. 또 위치 권한이 "항상 허용" 으로 설정되어야 하는데, 이는 사용자에게 부담스러운 권한 요청입니다. Phase 2에서 위치 기반 자동 실행(숙소 Wi-Fi 감지 시) 기능을 검토합니다.

13.4 24.3 기술 부채 현황

13.4.1 부채 목록

항목	심각도	해결 시점
테스트 커버리지 80% 미달	높음	Phase 1 유지보수
Retroreflection 오탐률 개선	높음	Phase 1 패치
ARP 테이블 권한 예외 처리 미흡	중간	Phase 1 패치
멀티 서브넷 스캔 미지원	낮음	Phase 2
접근성(Accessibility) 미구현	중간	Phase 2

항목	심각도	해결 시점
다크모드 완전 지원	낮음	Phase 2
PDF 리포트 커스텀 레이아웃	낮음	Phase 2
국제화(i18n) 영문 외 미완성	중간	Phase 2

13.4.2 의도적 기술 부채 vs 비의도적 기술 부채

중요한 구분이 있습니다. 일부 "부채" 는 의도적으로 만든 것입니다. Phase 1의 범위를 최소화하기 위해 Phase 2로 미룬 기능들입니다. 이것은 진짜 부채가 아니라 계획된 단계입니다.

반면 테스트 커버리지 부족과 오탐률 문제는 비의도적 부채입니다. 빠른 구현을 위해 품질을 희생한 결과입니다. 이것은 반드시 해결해야 합니다.

13.5 24.4 다음 버전 로드맵

13.5.1 Phase 2: iOS 포팅 + 기능 강화 (12~18개월)

Phase 2의 첫 번째 목표는 iOS 앱 출시입니다. Swift + SwiftUI로 동일한 탐지 알고리즘을 포팅합니다. 도메인 로직의 플랫폼 독립성이 여기서 빛납니다.

Phase 2에서 추가되는 주요 기능:

1. RTSP 스트림 분석 (제한적)
   - 발견된 카메라 포트에서 스트림 존재 여부 확인
  - 실제 영상 시청은 지원하지 않음 (법적 이유)
   - "스트림이 활성화되어 있음"만 리포트

2. ML 모델 통합 (렌즈 감지 개선)
   - TensorFlow Lite 모델로 렌즈 형태 분류
  - 학습 데이터: 렌즈/비렌즈 이미지 10,000+장
  - 목표: 오탐률 현재 대비 50% 감소

3. 위치 기반 자동 스캔
  - 숙소 Wi-Fi 연결 감지 시 알림 ("SearCam 스캔을 시작할까요?")
   - 이전 방문 기록과 비교 분석

4. 커뮤니티 리포트 공유
  - 특정 숙소의 익명 스캔 결과 집계
  - "이 숙소는 최근 30일간 X명이 스캔, 평균 위험도 Y"
   - 개인정보 보호를 위한 완전 익명화 필수

13.5.2 Phase 3: ML 고도화 + 플랫폼 (24~36개월)

1. 엣지 AI 탐지 엔진

CHAPTER 13. CH24: 회고와 다음 단계

  - 전용 경량 CNN 모델 (렌즈 + 적외선 패턴 통합 분석)
    - 서버 의존 없는 온디바이스 추론
  - 목표: 탐지 정확도 95%+

2. 하드웨어 연동 (선택 사항 액세서리)
    - RF 신호 감지 모듈 (블루투스 연결)
    - LTE/5G 카메라까지 탐지 범위 확장

3. B2B 대시보드
  - 숙박업소용 관리자 콘솔
  - 전 지점 스캔 현황, 위험도 트렌드
  - 자동 PDF 리포트 발송

4. API 플랫폼
  - OUI DB + 위험도 엔진 API 제공
  - 보안 솔루션 업체 파트너십

13.5.3 RTSP 스트림 분석의 법적 고려사항

Phase 2에서 RTSP 스트림 분석 기능을 추가할 때 주의할 점이 있습니다. 카메라가 스트리밍 중이라는 사실을 확인하는 것(포트 연결 성공)과, 그 스트림을 실제로 시청하는 것은 법적으로 다릅니다.

허용되는 것:

- RTSP 포트(554)에 연결 시도 → 성공 여부 확인
- 스트림이 활성화되어 있다는 사실을 리포트

허용되지 않는 것:
- RTSP 스트림 영상을 앱 내에서 시청
- 스트림 영상 캡처 또는 저장
- 인증 없이 비밀번호 카메라에 접근 시도

Phase 2에서는 "이 기기에서 영상 스트림이 확인됨" 까지만 알리고, 영상 시청 기능은 제공하지 않습니다.

13.6 24.5 독자에게 전하는 메시지

이 책을 여기까지 읽은 독자라면 아마 한 가지 목표가 있을 것입니다. 자신만의 앱을 만들고 싶다는 것.

13.6.1 완벽한 계획보다 빠른 시작

SearCam의 최초 아이디어는 메모장 한 줄이었습니다. "Wi-Fi 스캔으로 몰카 찾을 수 있지 않을까?" 이 문장 하나에서 시작했습니다. 완벽한 기술 스택, 완벽한 아키텍처, 완벽한 수익 모델을 갖추고 시작하려 했다면 아마 시작도 못 했을 것입니다.

처음부터 깨끗하게 만들 수 없습니다. 만들면서 깨끗해집니다. 초기 코드가 지저분한 것은 문제가 아닙니다. 그 지저분함을 리팩토링하는 과정에서 아키텍처를 배웁니다.

13.6.2 문제에 집착하라

기술에 집착하면 기술을 위한 기술을 만듭니다. 문제에 집착하면 사람에게 필요한 것을 만듭니다. SearCam은 "어떻게 만들까" 가 아니라 "왜 필요한가" 에서 시작했습니다.

좋은 앱은 기술적으로 인상적인 것이 아닙니다. 사용자의 진짜 문제를 해결하는 것입니다. 복잡한 알고리즘보다 사용자가 처음 스캔을 완료했을 때의 "이게 돼?" 라는 경험이 더 중요합니다.

13.6.3 솔직함이 경쟁력이다

SearCam은 "100% 탐지" 를 약속하지 않습니다. 경쟁 앱들이 과장된 광고로 사용자를 유인하는 동안, SearCam은 탐지 한계를 먼저 알립니다. 단기적으로 불리해 보이지만, 장기적으로 신뢰를 만드는 유일한 방법입니다.

사용자는 생각보다 훨씬 영리합니다. "이 앱이 모든 것을 잡아준다" 고 믿는 사용자는 없습니다. 하지만 "이 앱이 할 수 있는 것과 없는 것을 솔직하게 말해준다" 는 사용자는 충성 고객이 됩니다.

13.6.4 1인 개발의 한계를 받아들여라

혼자 모든 것을 잘할 수는 없습니다. SearCam 개발 과정에서 디자인, 마케팅, 법무, QA 모든 영역을 혼자 감당했습니다. 그 중 일부는 잘 했고, 일부는 부족했습니다.

완벽하지 않아도 됩니다. 출시하는 것이 완벽하게 준비하는 것보다 항상 낫습니다. 사용자의 피드백이 가장 훌륭한 스펙 문서입니다.

13.6.5 마지막으로

SearCam은 끝나지 않았습니다. Phase 1은 시작입니다. 이 책을 쓰면서 동시에 Phase 2를 계획하고 있습니다. iOS, ML 모델, B2B 인증 프로그램. 해야 할 것들은 항상 끝이 없습니다.

하지만 그것이 소프트웨어의 속성입니다. 완성되는 순간 낡아버립니다. 계속 만들어가는 과정 자체가 제품입니다.

여러분도 첫 줄을 쓰세요. 완벽하지 않아도, 지금 당장 시작하세요. 그 첫 줄이 언젠가 누군가의 문제를 해결하는 앱이 됩니다.

SearCam 기술 서적을 끝까지 읽어주셔서 감사합니다. 부록에서 레퍼런스 자료를 확인하세요.

← 이전 장 Ch23: 수익화 전략과 GTM → 부록 A: OUI 데이터베이스 구조

제 14 장

Appendix A: OUI 데이터베이스 구조

이 부록에서 배울 것: MAC 주소의 OUI 구조, SearCam이 수록한 카메라 제조사 목록, oui.json 파일 포맷, OuiDatabase.kt의 동작 방식을 참조합니다.

14.1 A.1 OUI(Organizationally Unique Identifier) 구조

MAC 주소(48비트)는 두 부분으로 나뉩니다.

```
MAC 주소 구조:

  28:57:BE : A1:23:4F
  ├──────────────┤   ├──────────────┤
 OUI            기기 고유 식별자
(3바이트,      (3바이트,
  제조사 식별)    기기별 고유값)
```

예시:

```
  28:57:BE → Hikvision Digital Technology
  3C:EF:8C → Dahua Technology
  00:09:18 → Hanwha Techwin (Samsung)
```

IEEE Registration Authority가 OUI를 관리하며, 모든 할당 내역은 공개되어 있습니다. SearCam은 이 공개 데이터를 기반으로 카메라 제조사를 분류한 자체 데이터베이스를 구축했습니다.

14.2 A.2 카메라 제조사 OUI 목록

14.2.1 IP 카메라 제조사 (전문 보안/감시 카메라)

`risk_weight`는 해당 OUI 기기가 카메라일 확률을 0~1 범위로 나타냅니다.

OUI Prefix	제조사	제품군	risk_weight
28:57:BE	Hikvision	IP카메라, NVR, DVR	0.95
C0:56:E3	Hikvision	추가 OUI	0.95
44:19:B6	Hikvision	추가 OUI	0.95
54:C4:15	Hikvision	추가 OUI	0.95
3C:EF:8C	Dahua Technology	IP카메라, PTZ, NVR	0.95
A0:BD:1D	Dahua Technology	추가 OUI	0.95
E0:50:8B	Dahua Technology	추가 OUI	0.95
00:09:18	Hanwha Techwin	IP카메라, PTZ	0.90
00:09:19	Hanwha Techwin	추가 OUI	0.90
00:80:F0	Panasonic	보안카메라, PTZ	0.85
00:40:84	Honeywell	보안카메라, 출입통제	0.80
00:04:A3	Axis Communications	IP카메라, 엔코더	0.95
AC:CC:8E	Axis Communications	추가 OUI	0.95
B8:A4:4F	Axis Communications	추가 OUI	0.95
00:0F:7C	ACTi	IP카메라	0.90

OUI Prefix	제조사	제품군	risk_weight
00:18:85	Avigilon	HD카메라, 분석	0.90
00:1A:07	Arecont Vision	메가픽셀 카메라	0.90
00:30:53	Vivotek	IP카메라, NVR	0.90
EC:71:DB	Reolink	IP카메라, PoE	0.90
9C:8E:CD	Reolink	추가 OUI	0.90
00:62:6E	Uniview	IP카메라, NVR	0.90
24:28:FD	Uniview	추가 OUI	0.90
00:1B:90	GeoVision	IP카메라, DVR	0.90
7C:DD:90	IDIS	DVR, NVR, IP카메라	0.90
F4:B5:AA	Pelco (Schneider)	PTZ, 고정카메라	0.90
00:0C:68	Bosch Security	IP카메라, DVR	0.85
00:18:17	Apace Technology	IP카메라	0.85
38:D5:47	Motorola/Avigilon	보안카메라	0.85
D4:20:B0	Meritech (한화 계열)	IP카메라	0.85
00:40:8C	Milestone/Mobotix	IP카메라	0.90

14.2.2 스마트 카메라 / IoT 카메라 제조사

OUI Prefix	제조사	제품군	risk_weig
2C:AA:8E	Wyze Labs	Wyze Cam 시리즈	0.80

OUI Prefix	제조사	제품군	risk_weig
18:B4:30	Nest (Google)	Nest Cam Indoor/Outdoor	0.80
98:DA:C4	TP-Link (Tapo)	Tapo C200/C310	0.75
FC:65:DE	Amazon (Ring)	Ring Indoor/Outdoor Cam	0.80
34:D2:70	Amazon (Blink)	Blink Mini/Outdoor	0.80
CC:50:E3	Arlo Technologies	Arlo Pro/Ultra	0.85
78:11:DC	Xiaomi	Mi Home 카메라 시리즈	0.75
F8:4D:89	Tuya Smart	화이트레이블 카메라	0.80
D8:1F:12	Tuya Smart	추가 OUI	0.80
28:6D:97	Imou (Dahua 소비자)	Imou Cue/Ranger	0.85
40:AE:30	Ezviz (Hikvision 소비자)	C6N/C3W	0.85
38:01:46	YI Technology	YI Home Camera	0.80
A4:34:D9	Eufy (Anker)	eufyCam/Indoor Cam	0.80
E8:AB:F3	Wansview	Wi-Fi 카메라	0.80
D4:35:1D	YooSee	P2P 카메라	0.85
3C:84:6A	TP-Link (Tapo)	Tapo C시리즈 신형	0.75

14.2.3 초소형/위장 카메라 (Wi-Fi 모듈 기반)

주의: 아래 OUI는 카메라 외 IoT 기기에서도 사용됩니다. OUI 단독으로 카메라 판정하지 않으며, 포트 스캔(RTSP, HTTP)으로 교차 확인합니다.

OUI Prefix	칩셋/모듈	설명	risk_weigh
18:FE:34	Espressif ESP8266	중국산 초소형 카메라 다수 사용	0.50
24:0A:C4	Espressif ESP32	ESP32-CAM 보드	0.50
A4:CF:12	Espressif	추가 OUI	0.50
48:3F:DA	Espressif	추가 OUI	0.50
B0:A7:32	Realtek RTL8xxx	Wi-Fi 칩셋, 다용도	0.40

14.3 A.3 안전 기기 화이트리스트

숙소에서 정상적으로 발견되는 기기들. 이 OUI가 감지되면 포트 스캔을 스킵하여 스캔 시간을 단축합니다.

제조사	기기 유형	risk_weight
Apple	iPhone, iPad, MacBook	0.05
Samsung (비카메라)	Galaxy, TV, 가전	0.05
LG	스마트폰, TV	0.05
Google	Pixel, Chromecast	0.05
Huawei	스마트폰, 태블릿	0.05
Microsoft	Surface, Xbox	0.05
ipTIME (EFM)	공유기	0.05

제조사	기기 유형	risk_weight
ASUS	공유기, RT 시리즈	0.05
Netgear	공유기, 스위치	0.05

14.4 A.4 oui.json 파일 포맷

`assets/oui.json` 파일의 구조입니다.

```
{
  "version": "1.0.0",
  "last_updated": "2026-04-01",
  "entries": [
    {
      "prefix": "28:57:BE",
      "manufacturer": "Hikvision Digital Technology",
      "product_category": "IP_CAMERA",
      "risk_weight": 0.95,
      "notes": "전 세계 1위 CCTV 점유율"
    },
    {
      "prefix": "3C:EF:8C",
```

```
      "manufacturer": "Dahua Technology",
      "product_category": "IP_CAMERA",
      "risk_weight": 0.95,
      "notes": "전 세계 2위 CCTV 점유율"
    },
    {
      "prefix": "18:FE:34",
      "manufacturer": "Espressif Systems (ESP8266)",
      "product_category": "IOT_MODULE",
      "risk_weight": 0.50,
      "notes": "IoT 모듈 범용, 카메라 아닐 수도 있음. 포트 스캔
    },
    {
      "prefix": "00:1A:2B",
      "manufacturer": "Apple Inc.",
      "product_category": "SAFE_DEVICE",
      "risk_weight": 0.05,
      "notes": "iPhone, MacBook 등 안전 기기"
    }
  ]
}
```

필드	타입	설명
`prefix`	String	MAC 주소 앞 3바이트 (대문자, 콜론 구분)
`manufacturer`	String	제조사명
`product_category`	Enum	`IP_CAMERA`, `SMART_CAMERA`, `IOT_MODULE`, `SAFE_DEVICE`, `NETWORK_DEVICE`
`risk_weight`	Float	카메라일 확률 (0.0~1.0)
`notes`	String	부가 설명 (선택)

14.5 A.5 OuiDatabase.kt 동작 방식

```
// 파일 위치: data/repository/OuiDatabase.kt (간략화)

class OuiDatabase(private val context: Context) {
```

```
// assets/oui.json → 메모리 맵으로 로드
// 키: OUI prefix (소문자 정규화), 값: OuiEntry
private val ouiMap: Map<String, OuiEntry> by lazy {
    loadFromAssets()
}

// MAC 주소에서 OUI 추출 후 조회
fun lookup(macAddress: String): OuiEntry? {
    val prefix = macAddress
        .take(8)            // "28:57:BE:..."에서 앞 8자
        .uppercase()        // 대소문자 정규화
    return ouiMap[prefix]
}

// risk_weight 기반 카메라 여부 판별
fun isCameraManufacturer(macAddress: String): Boolean
    val entry = lookup(macAddress) ?: return false
    return entry.riskWeight >= 0.7f
}

// 안전 기기 화이트리스트 확인
```

```
    fun isSafeDevice(macAddress: String): Boolean {
        val entry = lookup(macAddress) ?: return false
        return entry.productCategory == ProductCategory.SA
            || entry.riskWeight <= 0.1f
    }
}
```

14.5.1 스캔 파이프라인에서의 OUI 활용

```
ARP 테이블 파싱
 → IP + MAC 목록 획득
 → MAC 앞 3바이트 추출

OuiDatabase.lookup(mac)
 → null        → 미등록 제조사 → 중간 위험도 + 포트 스캔
 → SAFE_DEVICE → 낮은 위험도 → 포트 스캔 스킵 (속도 최적화)
 → IP_CAMERA   → 높은 위험도 → risk_weight 적용 + 포트 스캔
 → IOT_MODULE  → 중간 위험도 → 포트 스캔으로 RTSP 확인 필수
```

14.6 A.6 OUI DB 업데이트 방식

방식	대상 사용자	설명
앱 업데이트 내포	무료 사용자	앱 신버전에 최신 oui.json 포함
OTA 업데이트	프리미엄 사용자	앱 실행 시 서버에서 최신 oui.json 다운로드
업데이트 주기	무료: 월 1~2회 / 프리미엄: 주 1회	신규 카메라 제조사 반영

저작권 안내: OUI 데이터의 원본 출처는 IEEE Registration Authority입니다. SearCam은 IEEE 공개 데이터를 기반으로 카메라 제조사 분류를 추가하여 자체 편집한 데이터베이스입니다. IEEE OUI 원본 데이터에 대한 저작권은 IEEE에 있습니다.

제 15 장

Appendix B: 체크리스트 설계

이 부록에서 배울 것: 숙박시설 육안 점검 체크리스트의 항목 구성, 시각적 검사와 기술적 검사의 차이, 체크리스트 UI 설계 원칙을 참조합니다.

15.1 B.1 체크리스트가 필요한 이유

SearCam의 센서 기반 탐지(Wi-Fi, 렌즈, 자기장)는 강력하지만, 탐지할 수 없는 카메라 유형이 있습니다.

카메라 유형	SearCam 탐지 가능 여부	이유
Wi-Fi 연결 카메라	가능 (Layer 1)	같은 네트워크에 연결됨
IR 야간 카메라	가능 (Layer 2)	전면 카메라로 IR 발광 감지
SD카드 단독 녹화	**불가능**	네트워크 연결 없음
배터리+SD 초소형	**불가능**	무선 신호 없음
전원 꺼진 카메라	**불가능**	작동 중이 아님
LTE/5G 독립 카메라	**불가능**	별도 셀룰러 망 사용

이런 카메라를 발견하는 유일한 방법은 체계적인 육안 점검입니다. SearCam의 체크리스트는 전문 지식 없이도 따라할 수 있도록 구체적인 행동 지시를 제공합니다.

15.2 B.2 체크리스트 설계 원칙

15.2.1 항목 구조

모든 체크리스트 항목은 동일한 구조를 따릅니다.

[번호] [점검 위치]

점검 방법: 구체적인 행동 지시 ("이것을 하세요")

의심 신호: 이럴 때 의심하세요

위험도: 높음 / 중간 / 낮음

우선순위: 필수 / 권장 / 선택

15.2.2 소요 시간 기준

모드	항목 수	소요 시간	대상
빠른 점검	5개 (필수만)	약 2분	시간이 없을 때
표준 점검	15개 (필수+권장)	약 7분	일반 숙소
완전 점검	20개 (전체)	약 15분	장기 숙박, 의심 시

15.3 B.3 숙박시설 체크리스트 (20개 항목)

대상: 호텔, 모텔, 에어비앤비, 게스트하우스 빠른 점검

모드: 항목 1~5 (약 2분)

15.3.1 시선 방향 점검 (침대/소파를 향한 기기)

#	점검 위치	의심 신호	위험도	우선순위
1	연기 감지기	렌즈처럼 반짝이는 구멍, 배선 외부 노출	높음	필수
2	시계 (벽시계/탁상)	숫자 사이 핀홀, 콘센트 연결, SD 슬롯	높음	필수
3	TV 베젤 상단	본래 없어야 할 구멍, 전원 꺼져도 LED	중간	필수
4	에어컨 리모컨	IR 부 외 추가 투명 부분, 뒷면 SD 슬롯	중간	권장
5	USB 충전기/어댑터	USB 포트 외 핀홀, 비정상 크기, 미인식 브랜드	높음	필수

15.3.2 벽면/천장 점검

#	점검 위치	의심 신호	위험도	우선순위
6	액자/그림	프레임에 핀 홀, 뒤에 전선, 분리 불가	중간	권장
7	나사/못/볼트	머리 중앙이 투명, 기능 없는 위치	높음	필수
8	환풍구/통풍구	격자 사이 반짝이는 점, 바람 안 나옴	중간	권장
9	스프링클러 헤드	유리 벌브 없고 검은 렌즈, 2개 이상	중간	권장
10	조명 기구/전등	내부 이상 부품, 렌즈처럼 보이는 부분	중간	선택

15.3.3 욕실 점검

#	점검 위치	의심 신호	위험도	우선순위
11	거울 (양면 테스트)	손톱 반사 간격 없음, 어둠 속 빛 투과	높음	필수
12	샤워 헤드/홀더	헤드 중앙에 렌즈, 불필요하게 두꺼움	높음	필수
13	화장품/샴푸 용기	라벨 뒤에 구멍, 브랜드 없는 제품	중간	권장
14	환풍구 (욕실)	격자 뒤에 반짝임	중간	권장
15	욕실 조명	조명 안에 이상 부품, 커버 제거 후 확인	중간	선택

15.3.4 기타 점검

#	점검 위치	의심 신호	위험도	우선순위
16	화분/장식물	화분 흙에 뭔가 꽂혀있음, 잎 뒤에 구멍	중간	권장
17	책/책꽂이	책 표지에 핀홀, 책 사이에 소형 기기	낮음	선택
18	전등 스위치	스위치 주변 비정상 구멍	중간	권장
19	에어컨 본체	통풍구 안에 렌즈, 본체 측면 구멍	중간	선택
20	침대 헤드보드	헤드보드 틈새에 소형 기기, 이상한 구멍	중간	선택

15.4 B.4 화장실/탈의실 체크리스트 (10개 항목)

대상: 공중화장실, 헬스장 탈의실, 수영장 샤워실

CHAPTER 15. APPENDIX B: 체크리스트 설계

#	점검 위치	의심 신호	위험도	우선순
1	칸막이 상단	칸막이 위에 소형 기기, 뾰족한 물체	높음	필수
2	칸막이 하단 틈	바닥 향한 렌즈 구멍	높음	필수
3	환풍구	격자 뒤 반짝임	높음	필수
4	천장 구석	구석에 작은 기기, 배선	중간	필수
5	옷걸이/후크	후크 머리에 핀홀, 비정상 크기	높음	필수
6	거울	양면 거울 테스트	높음	필수
7	화장실용품 디스펜서	디스펜서 전면에 구멍	중간	권장
8	로커/사물함	자물쇠 주변 구멍, 내부 소형 기기	중간	권장
9	조명 커버	내부 이상 부품	중간	선택
10	비누/샴푸 홀더	홀더에 핀홀	낮음	선택

15.5 B.5 시각적 검사 vs 기술적 검사

검사 방법	탐지 가능 카메라	필요 도구	소요 시
SearCam Wi-Fi 스캔	Wi-Fi 연결 카메라	스마트폰	30초~3
SearCam 렌즈 감지	렌즈 반사되는 카메라	스마트폰 + 플래시	2~5분
SearCam 자기장 감지	전자기기	스마트폰	1~2분
육안 체크리스트	SD카드 카메라, 위장 기기	눈 + 손전등	5~15분

검사 방법	탐지 가능 카메라	필요 도구	소요 시
전문 RF 스캐너	모든 무선 카메라	전문 장비	15~30분
NLJD	전원 꺼진 카메라	전문 장비	30분+

SearCam은 일반인이 스마트폰만으로 할 수 있는 첫 세 가지를 담당합니다. 육안 체크리스트는 센서로 잡지 못하는 카메라 유형을 보완합니다.

15.6 B.6 체크리스트 UI 설계

15.6.1 Compose 컴포넌트 구조

```
// ChecklistScreen.kt
@Composable
fun ChecklistScreen(
    type: ChecklistType,  // ACCOMMODATION, RESTROOM, CHAN
    onComplete: (ChecklistResult) -> Unit
) {
    val items = ChecklistRepository.getItems(type)
    val checkedItems = remember { mutableStateListOf<Strin
```

```
LazyColumn {
    // 진행률 헤더
    item {
        ChecklistProgressHeader(
            completed = checkedItems.size,
            total = items.size,
            estimatedMinutes = type.estimatedMinutes
        )
    }
    // 카테고리별 그룹
    items.groupBy { it.category }.forEach { (category,
        item { ChecklistCategoryHeader(category) }
        items(categoryItems) { item ->
            ChecklistItem(
                item = item,
                isChecked = item.id in checkedItems,
                onChecked = { checked ->
                    if (checked) checkedItems.add(item
                    else checkedItems.remove(item.id)
                }
            )
        }
```

```
        }
        // 완료 버튼
        item {
            ChecklistCompleteButton(
                completedCount = checkedItems.size,
                totalCount = items.size,
                onComplete = { onComplete(ChecklistResult(
            )
        }
    }
}
```

15.6.2 우선순위 표시 UI 규칙

우선순위	색상	아이콘	동작
필수	빨간색 배지	!	완료 전까지 "완료" 버튼 비활성화
권장	주황색 배지	★	미완료 시 완료 시 경고 표시
선택	회색 배지	○	자유롭게 선택

15.6.3 빠른 점검 모드 전환

```
// 화면 상단의 모드 선택 칩
Row {
```

```
    FilterChip(
        selected = mode == ChecklistMode.QUICK,
        onClick = { mode = ChecklistMode.QUICK },
        label = { Text("빠른 점검 (2분)") }
    )
    FilterChip(
        selected = mode == ChecklistMode.FULL,
        onClick = { mode = ChecklistMode.FULL },
        label = { Text("전체 점검 (15분)") }
    )
}
```

15.7 B.7 체크리스트 결과 리포트

체크리스트 완료 후 다음 정보를 스캔 리포트에 통합합니다.

항목	내용
완료 항목 수	N / 전체
필수 항목 완료 여부	완료 / 미완료 (몇 개 미완료)
의심 항목	체크리스트 중 “의심” 표시한 항목 목록
점검 소요 시간	자동 측정

항목	내용
체크리스트 유형	숙소 / 화장실 / 탈의실

CHAPTER 15. APPENDIX B: 체크리스트 설계

제 16 장

Appendix C: 에러 코드 레퍼런스

이 부록에서 배울 것: SearCam의 E1xxx(센서), E2xxx(네트워크), E3xxx(권한) 에러 코드 전체 목록과 각 에러의 원인, 대응 방법을 참조합니다.

16.1 C.1 에러 코드 체계

SearCam은 에러를 세 도메인으로 분류합니다.

범위	도메인	예시
E1xxx	센서 오류	자력계 미지원, 카메라 초기화 실패
E2xxx	네트워크 오류	Wi-Fi 미연결, ARP 테이블 읽기 실패
E3xxx	권한 오류	위치 권한 미승인, 카메라 권한 미승인

에러 코드는 `Constants.ErrorCode` 객체에 정의되며, 로그와 사용자 화면에 모두 표시됩니다.

16.2 C.2 E1xxx - 센서 오류

에러 코드	상수명	발생 원인	사용자 메시지	대응 방법
E1001	E1001	자력계 (Magnetometer) 미지원 기기	"이 기기는 자기장 센서를 지원하지 않습니다"	EMF 탐지 레이어 비활성화, Layer 1+2로 대체

에러 코드	상수명	발생 원인	사용자 메시지	대응 방법
E1002	E1002	자력계 정확도 불량 (ACCURACY_LOW)	“자기장 센서 정확도가 낮습니다. 8자 모양으로 기기를 움직여 캘리브레이션 해주세요”	사용자에게 캘리브레이션 안내 후 재시도
E1003	E1003	카메라 초기화 실패	“카메라를 시작할 수 없습니다. 앱을 재시작해주세요”	CameraX 세션 재초기화 시도
E1004	E1004	플래시(토치) 미지원 기기	“이 기기는 플래시를 지원하지 않습니다. 렌즈 역반사 감지는 밝은 곳에서 사용하세요”	플래시 없이 렌즈 감지 진행 (정확도 저하 안내)

에러 코드	상수명	발생 원인	사용자 메시지	대응 방법
E1005	E1005	센서 샘플링 타임아웃	“센서 응답이 없습니다. 잠시 후 다시 시도해주세요”	5초 후 자동 재시도
E1010	E1010	자기장 센서 스트림 오류	“자기장 측정 중 오류가 발생했습니다”	Flow 재구독 시도, 실패 시 EMF 레이어 비활성화
E1011	E1011	자기장 캘리브레이션 실패	“자기장 베이스라인 측정에 실패했습니다. 금속 물체에서 멀리 이동해주세요”	베이스라인 재측정 요청

16.2.1 E1002 캘리브레이션 안내 화면

[캘리브레이션 안내]

자기장 센서의 정확도가 낮습니다.
더 정확한 측정을 위해 다음 동작을 해주세요:

① 스마트폰을 들고
② 공중에서 8자 모양으로
③ 3회 천천히 움직이세요

[완료] [건너뛰기]

건너뛰면 자기장 탐지 정확도가 낮아질 수 있습니다.

16.3 C.3 E2xxx - 네트워크 오류

에러 코드	상수명	발생 원인	사용자 메시지	대응 방법
E2001	E2001	Wi-Fi 미연결 상태	"Wi-Fi에 연결되어 있지 않습니다. Wi-Fi를 연결한 후 다시 시도해주세요"	Wi-Fi 설정 화면으로 이동 안내
E2002	E2002	ARP 테이블 읽기 실패	"네트워크 기기 목록을 가져오지 못했습니다"	`/proc/net/arp` 접근 실패. mDNS 탐색으로 대체 시도
E2003	E2003	mDNS 서비스 탐색 실패	"네트워크 서비스 탐색에 실패했습니다"	mDNS NsdManager 오류. Layer 1 결과 부분적으로만 표시

에러 코드	상수명	발생 원인	사용자 메시지	대응 방법
E2004	E2004	포트 스캔 타임아웃	“일부 기기의 응답 시간이 초과되었습니다”	타임아웃 기기는 “응답 없음” 으로 처리, 스캔 계속 진행
E2005	E2005	OUI 데이터베이스 로드 실패	“카메라 제조사 데이터베이스를 불러오지 못했습니다. 앱을 재설치해주세요”	Assets 파일 손상. 제조사 분류 없이 포트 기반으로만 스캔

16.3.1 E2002 대체 전략

```
// ARP 테이블 실패 시 mDNS로 대체
suspend fun discoverDevices(): List<NetworkDevice> {
    return try {
        arpTableScanner.scan()
```

```
    } catch (e: IOException) {
        logger.error(Constants.ErrorCode.E2002, e)
        // 대체: mDNS 서비스 탐색
        mdnsScanner.discover()
    }
}
```

16.4 C.4 E3xxx - 권한 오류

에러 코드	상수명	발생 원인	사용자 메시지	대응 방법
E3001	E3001	위치 권한 미승인 (Wi-Fi 스캔 불가)	"Wi-Fi 스캔에는 위치 권한이 필요합니다"	권한 요청 다이얼로그 표시. 거부 시 렌즈+자기장 모드로 계속

에러 코드	상수명	발생 원인	사용자 메시지	대응 방법
E3002	E3002	카메라 권한 미승인	"렌즈 감지와 IR 탐지에는 카메라 권한이 필요합니다"	권한 요청 다이얼로그 표시. 거부 시 Wi-Fi+자기장 모드로 계속
E3003	E3003	알림 권한 미승인 (Android 13+)	"스캔 완료 알림을 받으려면 알림 권한이 필요합니다"	권한 요청. 거부해도 앱 핵심 기능에 영향 없음

16.4.1 권한 거부 시 대체 모드

권한 거부 조합 → 사용 가능한 레이어

위치 0 + 카메라 0 = 3레이어 풀 스캔 (권장)
위치 X + 카메라 0 = Layer 2 (렌즈) + Layer 3 (자기장)
위치 0 + 카메라 X = Layer 1 (Wi-Fi) + Layer 3 (자기장)

위치 X + 카메라 X = Layer 3 (자기장) 만 사용

모든 조합에서 앱이 동작하며, 사용 가능한 레이어를 최대한 활용합니다. 단, 커버리지가 줄어드는 것을 사용자에게 명확히 안내합니다.

16.5 C.5 에러 처리 구현 패턴

16.5.1 sealed class 기반 결과 처리

```
sealed class ScanError {
    data class SensorError(val code: String, val message:
    data class NetworkError(val code: String, val message:
    data class PermissionError(val code: String, val requi
}

// UseCase에서 결과 반환
sealed class ScanResult<out T> {
    data class Success<T>(val data: T) : ScanResult<T>()
    data class Failure(val error: ScanError) : ScanResult<
}
```

16.5.2 ViewModel에서 에러 상태 처리

```
class ScanViewModel : ViewModel() {
    private val _uiState = MutableStateFlow<ScanUiState>(S
    val uiState: StateFlow<ScanUiState> = _uiState.asState

    fun handleError(error: ScanError) {
        _uiState.value = when (error) {
            is ScanError.SensorError -> ScanUiState.Error(
                code = error.code,
                message = error.message,
                canContinue = error.code != Constants.Erro
            )
            is ScanError.PermissionError -> ScanUiState.Pe
                code = error.code,
                permission = error.requiredPermission
            )
            is ScanError.NetworkError -> ScanUiState.Error
                code = error.code,
                message = error.message,
                canContinue = true  // 네트워크 오류는 대체 모
            )
        }
```

```
    }
}
```

16.6 C.6 에러 코드 전체 요약표

코드	도메인	심각도	앱 계속 실행 여부
E1001	센서	중간	가능 (EMF 레이어 비활성화)
E1002	센서	낮음	가능 (정확도 저하 안내)
E1003	센서	높음	카메라 레이어 불가
E1004	센서	낮음	가능 (플래시 없이 진행)
E1005	센서	중간	재시도 후 가능
E1010	센서	중간	가능 (EMF 레이어 재시도)
E1011	센서	낮음	가능 (캘리브레이션 재시도)
E2001	네트워크	높음	Wi-Fi 레이어 불가
E2002	네트워크	중간	가능 (mDNS 대체)
E2003	네트워크	낮음	가능 (부분 결과)
E2004	네트워크	낮음	가능 (타임아웃 기기 스킵)
E2005	네트워크	높음	가능 (포트 기반만)
E3001	권한	높음	Wi-Fi 레이어 불가
E3002	권한	높음	카메라 레이어 불가

코드	도메인	심각도	앱 계속 실행 여부
E3003	권한	낮음	가능 (알림만 불가)

제 17 장

Appendix D: 주요 API 레퍼런스

이 부록에서 배울 것: SearCam의 주요 UseCase 시그니처, Repository 인터페이스 목록, Constants 주요 값을 빠르게 참조합니다.

17.1 D.1 UseCase 레이어

UseCase는 Domain 레이어의 핵심입니다. 하나의 UseCase는 하나의 비즈니스 액션을 담당합니다.

17.1.1 스캔 관련 UseCase

UseCase	반환 타입	설명
`StartQuickScanUseCase`	`Flow<ScanProgress>`	Quick Scan(Wi-Fi 스캔) 시작. 진행 상황을 Flow로 방출
`StartFullScanUseCase`	`Flow<ScanProgress>`	Full Scan(3레이어 통합) 시작
`StopScanUseCase`	`Unit`	진행 중인 스캔 중단
`GetScanResultUseCase`	`ScanResult`	완료된 스캔 결과 조회

```
// StartQuickScanUseCase 시그니처
class StartQuickScanUseCase(
```

```
    private val wifiScanRepository: WifiScanRepository,
    private val ouiDatabase: OuiDatabase
) {
    operator fun invoke(): Flow<ScanProgress> = flow {
        emit(ScanProgress.Started)
        val devices = wifiScanRepository.scanDevices()
        devices.forEach { device ->
            val ouiEntry = ouiDatabase.lookup(device.macAd
            val riskScore = calculateRisk(device, ouiEntry
            emit(ScanProgress.DeviceFound(device.copy(risk
        }
        emit(ScanProgress.Completed)
    }.catch { e ->
        emit(ScanProgress.Error(e.toScanError()))
    }
}
```

17.1.2 렌즈 감지 UseCase

UseCase	반환 타입	설명
StartLensDetectionUseCase	Flow<LensDetectionState>	Reflection 기반 렌즈 감지 시작
StartIrDetectionUseCase	Flow<IrDetectionState>	야간 카메라 감지 시작
StopCameraDetectionUseCase	Unit	카메라 기반 감지 중단

```
// StartLensDetectionUseCase 시그니처
class StartLensDetectionUseCase(
    private val cameraRepository: CameraRepository
) {
    operator fun invoke(): Flow<LensDetectionState> =
        cameraRepository.analyzeFrames()
            .map { frame -> LensAnalyzer.analyze(frame) }
            .distinctUntilChanged()
}
```

17.1.3 자기장 UseCase

UseCase	반환 타입	설명
`StartEmfMonitoringUseCase`	`Flow<EmfReading>`	자기장 실시간 모니터링 시작
`CalibrateEmfUseCase`	`EmfBaseline`	베이스라인 측정 (30샘플 평균)
`StopEmfMonitoringUseCase`	`Unit`	자기장 모니터링 중단

17.1.4 리포트 UseCase

UseCase	반환 타입	설명
`SaveReportUseCase`	`ReportId`	스캔 결과를 로컬 DB에 저장
`GetReportsUseCase`	`Flow<List<ScanReport>>`	저장된 리포트 목록 조회
`DeleteReportUseCase`	`Unit`	리포트 삭제
`ExportReportPdfUseCase`	`File`	리포트를 PDF로 내보내기 (프리미엄)

17.2 D.2 Repository 인터페이스

Repository 인터페이스는 Domain 레이어에 정의되며, Data 레이어에서 구현합니다.

17.2.1 WifiScanRepository

```
interface WifiScanRepository {
    // ARP 테이블에서 연결된 기기 목록 조회
    suspend fun scanDevices(): List<NetworkDevice>

    // 특정 기기의 카메라 포트 스캔
    suspend fun scanPorts(
        ipAddress: String,
        ports: List<Int> = Constants.CAMERA_PORTS
    ): PortScanResult

    // mDNS 서비스 탐색
    fun discoverServices(
        timeoutMs: Long = Constants.MDNS_DISCOVERY_TIMEOUT
    ): Flow<MdnsService>

    // 현재 Wi-Fi 연결 상태 확인
```

```
    fun isConnected(): Boolean
}
```

17.2.2 CameraRepository

```
interface CameraRepository {
    // 카메라 프레임을 Flow로 제공 (저장 없음)
    fun analyzeFrames(): Flow<ImageFrame>

    // 플래시 ON/OFF
    suspend fun setFlashEnabled(enabled: Boolean)

    // 전면/후면 카메라 전환
    suspend fun switchCamera(facing: CameraFacing)

    // 카메라 해제
    suspend fun release()
}
```

17.2.3 EmfRepository

```
interface EmfRepository {
    // 자기장 센서 실시간 데이터 스트림
    fun getMagneticFieldFlow(): Flow<EmfReading>
```

```
    // 베이스라인 측정 (정지 상태에서 N샘플 평균)
    suspend fun measureBaseline(
        samples: Int = Constants.EMF_BASELINE_SAMPLES
    ): EmfBaseline

    // 센서 지원 여부 확인
    fun isSensorAvailable(): Boolean

    // 센서 정확도 조회
    fun getSensorAccuracy(): SensorAccuracy
}
```

17.2.4 ReportRepository

```
interface ReportRepository {
    // 리포트 저장 (무료: 최근 10건 유지)
    suspend fun save(report: ScanReport): ReportId

    // 전체 리포트 목록 (최신순)
    fun getAll(): Flow<List<ScanReport>>

    // 리포트 상세 조회
```

```
    suspend fun getById(id: ReportId): ScanReport?

    // 리포트 삭제
    suspend fun delete(id: ReportId)

    // 저장된 리포트 수
    suspend fun count(): Int

    // 오래된 리포트 자동 삭제 (무료 사용자: 10건 초과 시)
    suspend fun pruneIfNeeded(maxCount: Int = 10)
}
```

17.2.5 OuiRepository

```
interface OuiRepository {
    // MAC 주소로 OUI 엔트리 조회
    fun lookup(macAddress: String): OuiEntry?

    // 카메라 제조사 여부 (risk_weight >= 0.7)
    fun isCameraManufacturer(macAddress: String): Boolean

    // 안전 기기 여부 (risk_weight <= 0.1)
    fun isSafeDevice(macAddress: String): Boolean
```

```
    // DB 버전 조회
    fun getDatabaseVersion(): String

    // OTA 업데이트 (프리미엄)
    suspend fun updateFromRemote(): UpdateResult
}
```

17.3 D.3 주요 Domain 모델

17.3.1 ScanResult

```
data class ScanResult(
    val id: ReportId,
    val startedAt: Instant,
    val completedAt: Instant,
    val devices: List<DetectedDevice>,
    val lensDetections: List<LensDetection>,
    val emfReadings: EmfSummary,
    val riskScore: Int,           // 0~100
    val riskLevel: RiskLevel,     // SAFE / INTEREST / CAU
    val crossValidation: CrossValidationResult
```

```
)
```

17.3.2 RiskLevel

```
enum class RiskLevel(val range: IntRange, val label: Strin
    SAFE(0..20, "안전"),
    INTEREST(21..40, "주의 관찰"),
    CAUTION(41..60, "주의"),
    DANGER(61..80, "위험"),
    CRITICAL(81..100, "매우 위험")
}
```

17.3.3 DetectedDevice

```
data class DetectedDevice(
    val ipAddress: String,
    val macHash: String,          // SHA-256 해시 (원본 MAC
    val manufacturer: String?,    // OUI 조회 결과
    val openPorts: List<Int>,     // 열린 카메라 포트
    val mdnsName: String?,        // mDNS 서비스명
    val riskScore: Int,           // 0~100
    val riskWeight: Float         // OUI risk_weight
)
```

17.4 D.4 Constants 주요 값

`Constants.kt`에 정의된 앱 전역 상수입니다.

17.4.1 네트워크 스캔

상수	값	설명
`CAMERA_PORTS`	`[554, 80, 8080, 8888, 3702, 1935, 443, 8443]`	카메라 스트리밍에 자주 사용되는 포트
`PORT_SCAN_TIMEOUT_MS`	`500L`	포트 단일 연결 타임아웃 (ms)
`ARP_TABLE_PATH`	`"/proc/net/arp"`	ARP 테이블 파일 경로
`MDNS_DISCOVERY_TIMEOUT_MS`	`5000L`	mDNS 탐색 타임아웃 (ms)
`QUICK_SCAN_TIMEOUT_MS`	`30000L`	Quick Scan 최대 시간 (30초)

FULL_SCAN_TIMEOUT_MS	120_000L	Full Scan 최대 시간 (120초)

17.4.2 EMF 센서

상수	값	설명
EMF_POLLING_HZ	20	샘플링 주파수 (20Hz = 50ms 간격)
EMF_MOVING_AVG_WINDOW	10	이동 평균 윈도우 크기 (노이즈 필터)
EMF_ANOMALY_THRESHOLD_UT	50.0f	이상 감지 임계값 (마이크로테슬라)
EMF_BASELINE_SAMPLES	30	베이스라인 측정 샘플 수

17.4.3 위험도 임계값

상수	값	대응 RiskLevel
RISK_SAFE_MAX	20	SAFE (0~20)
RISK_INTEREST_MAX	40	INTEREST (21~40)
RISK_CAUTION_MAX	60	CAUTION (41~60)

상수	값	대응 RiskLevel
RISK_DANGER_MAX	80	DANGER (61~80)
RISK_CRITICAL_MIN	81	CRITICAL (81~100)

17.4.4 카메라 설정

상수	값	설명
CAMERA_ANALYSIS_WIDTH	1280	분석 해상도 너비 (픽셀)
CAMERA_ANALYSIS_HEIGHT	720	분석 해상도 높이 (픽셀)
RETROREFLECTION_FRAME_COUNT	10	역반사 분석 프레임 수 (평균화)
RETROREFLECTION_THRESHOLD	200	렌즈 의심 반사 강도 임계값 (0~255)

17.4.5 OUI 데이터베이스

상수	값	설명
OUI_JSON_ASSET_PATH	"oui.json"	assets 폴더 내 OUI JSON 파일 경로

17.5 D.5 탐지 레이어 가중치

교차 검증 엔진에서 각 레이어의 기여도입니다.

레이어	가중치	설명
Layer 1: Wi-Fi 스캔	50%	MAC OUI + 포트 스캔 + mDNS
Layer 2: 렌즈 감지	35%	Retroreflection + IR 감지
Layer 3: 자기장	15%	EMF 이상 감지

```
// LayerType enum (각 레이어의 weight)
enum class LayerType(val weight: Float) {
    WIFI_SCAN(0.50f),
    LENS_DETECTION(0.35f),
    EMF_MONITORING(0.15f)
}

// 교차 검증 점수 계산
fun crossValidate(
    wifiScore: Int,
    lensScore: Int,
    emfScore: Int
```

```
): Int {
    return (wifiScore * LayerType.WIFI_SCAN.weight +
            lensScore * LayerType.LENS_DETECTION.weight +
            emfScore * LayerType.EMF_MONITORING.weight).to
        .coerceIn(0, 100)
}
```

제 18 장

Appendix E: 버전 이력

이 부록에서 배울 것: SearCam의 v1.0.0 기능 목록, Phase 1~4의 구현 내용, 주요 보안 수정 이력을 참조합니다.

18.1 E.1 버전 정책

SearCam은 시맨틱 버저닝(Semantic Versioning)을 따릅니다.

```
MAJOR.MINOR.PATCH
```

```
MAJOR: 하위 호환성을 깨는 변경 (아키텍처 재설계, Phase 전환)
MINOR: 하위 호환 새 기능 추가
PATCH: 버그 수정, 보안 패치

예시:
  v1.0.0  Phase 1 MVP 초기 출시
 v1.1.0  새 탐지 모드 추가
 v1.1.1  버그 수정
 v2.0.0  iOS 앱 + B2B 기능 (Phase 2)
```

18.2 E.2 v1.0.0 - Phase 1 MVP (2026년 04월)

18.2.1 신규 기능

탐지 레이어

기능	설명	담당 레이어
Quick Scan	Wi-Fi 연결 기기를 30초 안에 스캔	Layer 1

기능	설명	담당 레이어
Full Scan	3레이어 통합 탐지 (최대 120초)	Layer 1+2+3
MAC OUI 분석	80개+ 카메라 제조사 자동 식별	Layer 1
포트 스캔	RTSP(554), HTTP(80/8080/8888) 등 8개 포트 확인	Layer 1
mDNS 서비스 탐색	네트워크 카메라 서비스 자동 발견	Layer 1
Retroreflection 렌즈 감지	플래시 역반사로 숨겨진 렌즈 탐지	Layer 2
IR 야간 카메라 감지	전면 카메라로 IR 발광 기기 탐지	Layer 2
자기장(EMF) 모니터링	자력계 기반 전자기기 이상 탐지	Layer 3

기능	설명	담당 레이어
교차 검증 엔진	3레이어 결과를 가중 평균으로 통합	교차 검증
위험도 5단계	SAFE / INTEREST / CAUTION / DANGER / CRITICAL	결과 표시

체크리스트

기능	설명
숙소 체크리스트	20개 항목, 필수/권장/선택 분류
화장실 체크리스트	10개 항목, 공중화장실/탈의실
빠른 점검 모드	필수 항목만 5개, 약 2분
체크리스트 리포트	스캔 결과에 체크리스트 완료 현황 통합

리포트 및 저장

기능	설명
스캔 리포트 저장	무료 최근 10건, 프리미엄 무제한
리포트 상세 보기	발견 기기 목록, 레이어별 점수, 권장 조치
리포트 삭제	개별 또는 전체 삭제
112 신고 연동	위험 감지 시 빠른 신고 버튼

수익화

기능	설명
프리미엄 구독	월 ₩2,900 / 연 ₩23,900, 7일 무료 체험
PDF 리포트 내보내기	프리미엄 전용
광고 (무료 사용자)	홈 배너 + 결과 후 간 삽입 광고
OTA DB 업데이트	프리미엄 전용, 주 1회 자동 업데이트

법적 준수

항목	구현 내용
최초 실행 면책 동의	탐지 한계 명시, 동의 후 시작
개인정보 처리방침	“수집 안 함” 명시, Play Store 등록
Wi-Fi 스캔 전 고지	스캔 범위와 목적 안내 화면
오픈소스 라이선스	앱 설정 → 오픈소스 라이선스 화면

18.3 E.3 Phase별 구현 계획

18.3.1 Phase 2: iOS 포팅 + 기능 강화 (v2.0.0, 예정 2026년 Q4)

기능	설명	우선순위
iOS 앱 출시	Swift + SwiftUI, 동일 탐지 알고리즘	높음
ML 렌즈 감지	TensorFlow Lite 모델, 오탐률 50% 감소 목표	높음
RTSP 존재 확인	스트림 활성화 여부 확인 (시청 아님)	중간
B2B 인증 프로그램	숙박업소 SearCam 인증 배지	높음
기업용 대시보드	전 지점 스캔 현황 모니터링	중간
위치 기반 자동 알림	숙소 Wi-Fi 연결 시 스캔 제안	낮음
학생 할인	대학교 인증 시 40% 할인	낮음
다크모드 완전 지원	현재 부분 지원 개선	중간
접근성(A11y)	TalkBack 지원, 색약 모드	중간

18.3.2 Phase 3: 플랫폼 확장 (v3.0.0, 예정 2027년)

기능	설명
엣지 AI 탐지 엔진	전용 경량 CNN, 95%+ 정확도 목표

기능	설명
커뮤니티 리포트	숙소별 익명 스캔 집계
API 라이선스	OUI DB + 위험도 엔진 외부 제공
RF 모듈 연동	블루투스 RF 스캐너 액세서리 지원
전문 업체 연결	위험 감지 시 전문 탐지 업체 중개

18.4 E.4 보안 수정 이력

버전	수정 내용	심각도	CVE
v1.0.0	초기 보안 설계 완료	-	-
v1.0.0	MAC 주소 SHA-256 해시 처리 (원본 저장 금지)	설계	-
v1.0.0	Room DB SQLCipher 암호화 적용	설계	-

버전	수정 내용	심각도	CVE
v1.0.0	iText AGPL 라이선스 → Android PdfDocument 교체	라이선스	-
v1.0.0	ARP 테이블 경로 하드코딩 제거 → Constants.ARP_TABLE_PATH	코드품질	-

향후 보안 패치는 이 문서에 추가됩니다.

18.5 E.5 알려진 제한 사항

항목	상태	해결 예정
일부 Xiaomi 기기에서 ARP 테이블 접근 제한	알려진 문제	v1.0.1
Retroreflection 오탐률 (안경, 금속 반사)	알려진 문제	v1.1.0
멀티 서브넷 스캔 미지원	설계 범위 외	Phase 2
LTE/5G 카메라 탐지 불가	하드웨어 한계	Phase 3 (RF 모

항목	상태	해결 예정
전원 꺼진 카메라 탐지 불가	물리적 한계	체크리스트로 보
백그라운드 자동 스캔 미지원	배터리 정책 제한	Phase 2 검토

18.6 E.6 개발 환경 이력

항목	Phase 1 기준값
Language	Kotlin 2.0+
UI Framework	Jetpack Compose 1.7+
Architecture	MVVM + Clean Architecture
DI	Hilt 2.51+
DB	Room 2.6+ + SQLCipher 4.5+
Camera	CameraX 1.3+
Async	Coroutines 1.8+ + Flow
Min SDK	API 26 (Android 8.0)
Target SDK	API 34 (Android 14)
Build System	Gradle 8.x + AGP 8.x
CI/CD	GitHub Actions